

엘라스틱 스택 개발부터 운영까지

저자: 김준영, 정상운

출판사: 책만

<https://github.com/onlybooks/elasticstack/>

목차

01 엘라스틱 스택 개요

02 엘라스틱 서치 기본

03 검색

04 집계

05 Beats

06 Kibana

01

엘라스틱 스택 개요

엘라스틱 스택이란?

1 엘라스틱서치의 탄생

부인을 위한 레시피 검색을 위한 검색엔진으로 시작
Apache Lucene을 기반으로 Elasticsearch를 제작하고, 오픈 소스화 시킴

2 엘라스틱 스택으로 발전

단순한 검색엔진을 넘어 플랫폼으로 진화

- 데이터가 증가하는 상황에서 전문 검색은 데이터를 처리하는 하나의 방법에 불과했다. 다양한 사례를 지원하기 위해서 검색엔진을 넘어 플랫폼이 필요했음.
- 과거에는 빅데이터를 처리하기 위해 그림 1.1과 같은 일련의 작업을 구성하기 위해 다양한 오픈소스를 조합 하여 구축해야 했다. 엘라스틱은 빅데이터 파이프라인을 구성하고 싶은 사용자의 요구를 충족하기 위해 스택을 쌓아 올림
- 그림 1.1 파이프라인을 구축하기 위해 로그스태시, 엘라스틱서치, 키바나를 합친 엘라스틱 스택을 개발

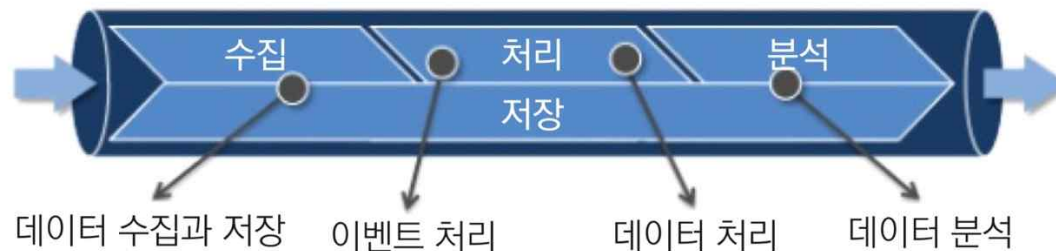


그림 1.1 일반적인 빅데이터 파이프라인¹

엘라스틱 스택이란?

3 엘라스틱 스택의 구성요소

엘라스틱 스택은 일반적인 빅데이터 파이프라인을 구성하기 위한 데이터 수집/가공/저장/분석/시각화에 필요한 모든 기능을 제공한다.

- 비츠와 로그스태시: 데이터를 수집, 가공
- 엘라스틱서치: 저장 및 분석
- 키바나: 엘라스틱서치에 저장된 데이터를 시각화 및 모니터링



그림 1.2 엘라스틱 스택 구조

엘라스틱 스택의 구성요소

1. 엘라스틱서치: 분산 검색 엔진

- 내부 document를 인덱싱하여 빠른 검색을 지원함
 - 엘라스틱서치는 text 필드의 경우 인덱싱 시점에 분석을 거쳐 용어로 분해되고 역인덱스 사전을 구축.
- 모든 레코드를 JSON 형태로 입력하고 관리하며, 일반적인 데이터베이스와 마찬가지로, 쿼리에 대한 결과에 일치하는 원본 도큐먼트를 반환.
- 텍스트, 숫자, 날짜, IP 주소, 지리(geo) 정보 등 다양한 데이터 타입을 지원
 - 숫자나 키워드 타입의 데이터는 엘라스틱서치 집계를 위해 집계에 최적화된 컬럼 기반 자료구조로 저장.
- 검색 엔진이면서 데이터베이스로 일종의 NoSQL이라 생각할 수 있다.
 - 엘라스틱서치는 분산처리나 병렬처리를 할 수 있기 때문에 다른 데이터베이스에 비해 빠른 검색과 집계가 가능.
- 엘라스틱서치 = 검색 엔진 + NoSQL
 - 엘라스틱서치는 검색어에 대한 유사도 스코어를 기반으로 한 정렬을 제공.

엘라스틱 스택의 구성요소

1. 엘라스틱서치: 분산 검색 엔진

- REST API를 이용하도록 만들어 프로그래밍 언어와 무관하게 사용자가 쉽게 접근 가능하다.
- 저장공간이 크게 압축되지 않고 시스템 리소스를 많이 사용하는 단점이 있음.
- DSL(Domain Specific Language)쿼리 사용
 - JOIN 쿼리가 어렵기 때문에 반정규화를 기본으로 모델링 해야함.
 - 인덱스가 불변의 자료구조이기 때문에 document를 수정하거나 삭제할 경우 많은 비용이 발생.

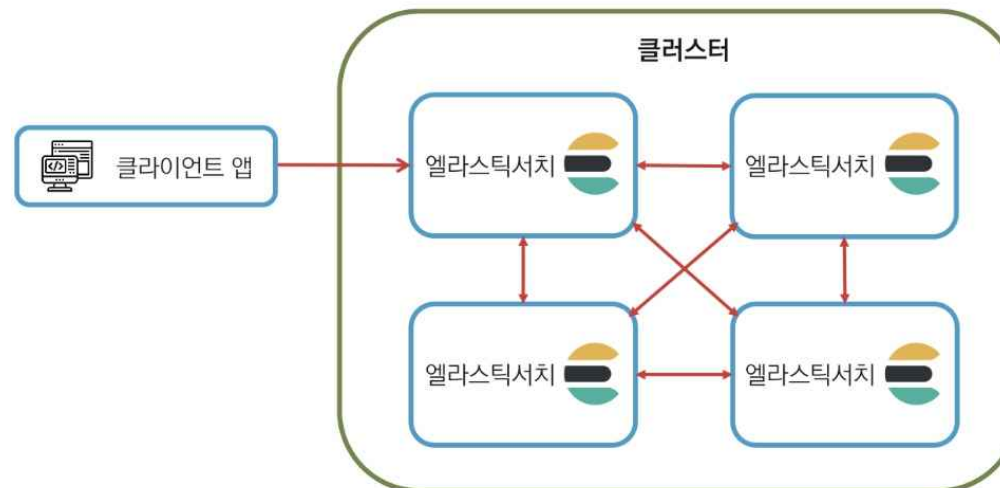


그림 1.3 엘라스틱서치 분산 시스템 구성

엘라스틱 스택의 구성요소

2. 키바나: 시각화와 엘라스틱서치 관리 도구

- 키바나는 엘라스틱서치에 대한 대부분의 관리 기능, API를 실행할 수 있는 콘솔, 솔루션 페이지들, 스택의 각 요소들을 위한 모니터링 페이지들을 담당.
- 엘라스틱서치 기반 시각화 도구.

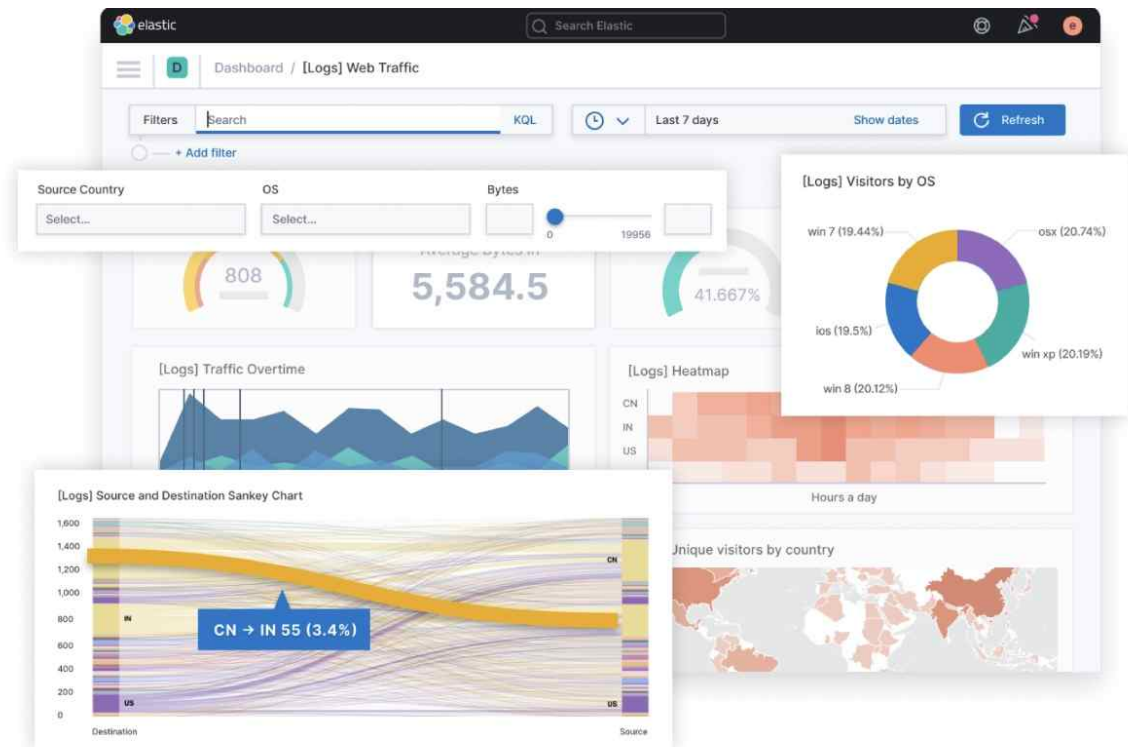


그림 1.4 키바나의 커스텀 대시보드²

엘라스틱 스택의 구성요소

3. 로그스태시: 이벤트 수집과 정제를 위한 도구

- 로그, 메트릭, 웹 애플리케이션 등 다양한 소스로부터 로그를 수집할 수 있음.
- 필터 기능을 활용 비정형이나 반정형 데이터를 분석하기 쉬운 형태로 정제할 수 있음.
- 엘라스틱서치 외에도 다양한 플랫폼으로 정제된 데이터를 내보낼 수 있음.
- 별도의 코딩 없이 설정만으로 로그를 가공 할 수 있음.
- 엘라스틱서치의 인덱싱 성능을 최적화 하기 위한 배치처리와 병렬 처리가 가능함.
- 영속적인 큐를 사용해 현재 처리 중인 이벤트의 최소 1회 전송 보장.

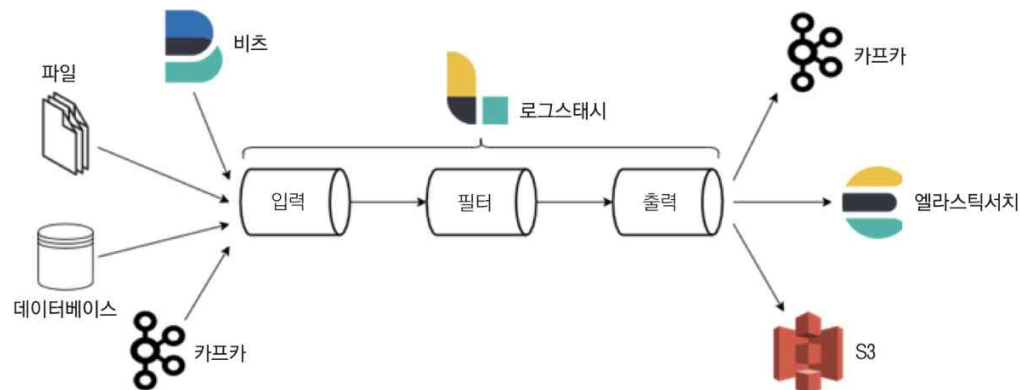


그림 1.5 로그스태시 동작 방식과 구성요소

엘라스틱 스택의 구성요소

4. 비트: 엣지단에서 동작하는 경량 수집 도구

- 실제 서비스가 동작하는 호스트에서 동작하는 수집기.
- 각 목적에 최적화된 데이터 수집기.
- 고 프로그래밍 언어로 작성

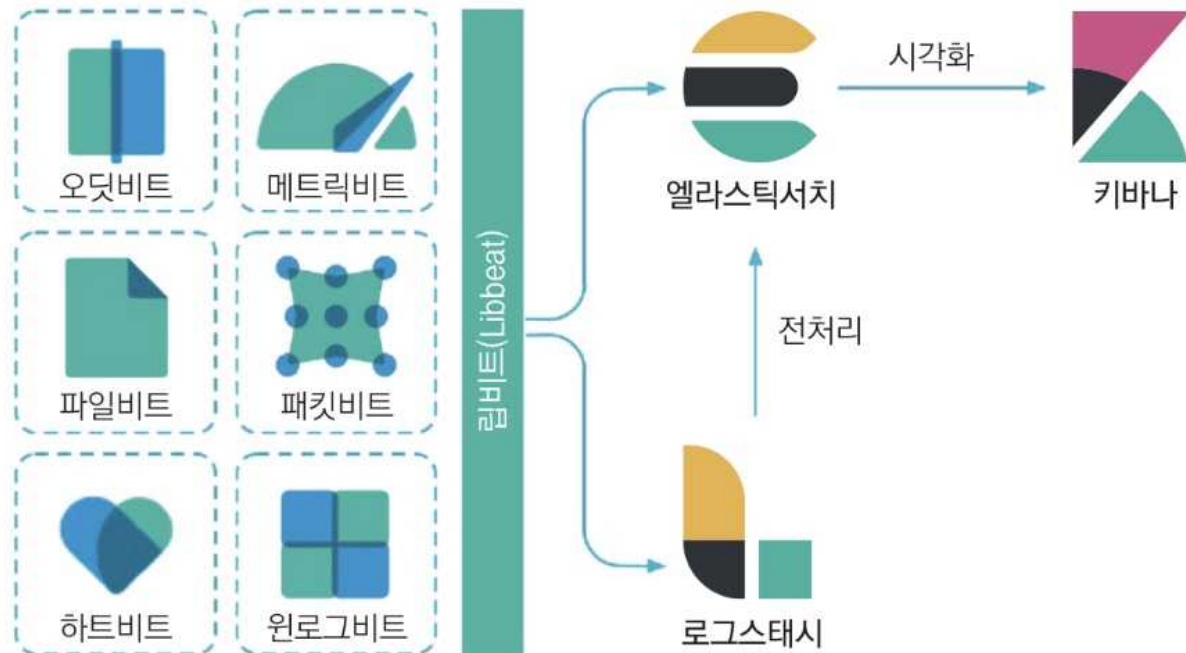


그림 1.6 비츠의 종류와 구조³

엘라스틱 스택의 구성요소

5. 기타 솔루션

- 애플리케이션 성능 모니터링을 위한 APM
- 보안 이벤트 분석을 위한 SIEM
- 컨테이너나 다수의 서비스를 모니터링 하기위한 인프라 모니터링

엘라스틱 스택이란?

4

엘라스틱 스택의 용도

1. 전문 검색 엔진

2. 로그 통합 분석

3. 보안 이벤트 분석

4. 애플리케이션 성능 분석

5. 머신 러닝

엘라스틱 스택의 용도

1. 전문 검색 엔진

- 전문(Full text)은 단순한 문장부터 뉴스 기사나 논문 등 다양한 글 전체 내용을 의미.
- 검색 대상이 많지 않다면 일반적인 관계형 데이터베이스의 LIKE 질의로 검색이 가능하지만 많은 양의 데이터가 있다면 검색에 많은 시간이 소요 됨.
- 엘라스틱서치는 전문 검색을 빠르게 하기 위해 용어 단위로 분석해 인덱싱해 두고 이를 기반으로 검색을 수행하는 역인덱싱 기법을 활용함.

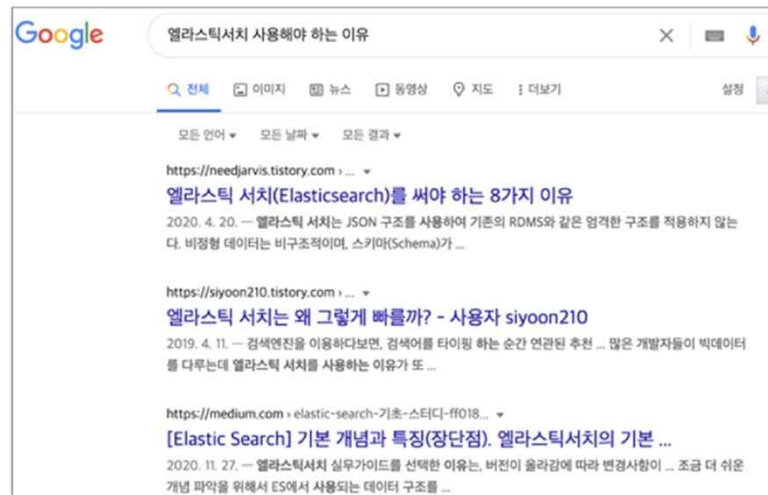


그림 1.7 검색 엔진을 활용한 구글 검색

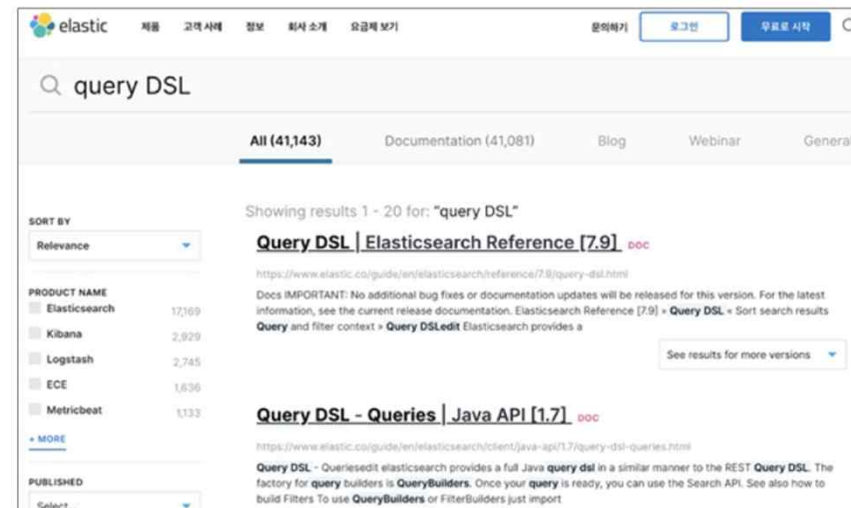


그림 1.8 검색 엔진 활용 예시

엘라스틱 스택의 용도

1. 전문 검색 엔진

- 엘라스틱 스택에는 용어 분석을 위한 다양한 언어별 분석기가 있음.
- 유사도 스코어링을 위한 다양한 방법 제공.
- 로그스태시를 활용하여 도큐먼트 수집 단계부터 도움을 주며 키바나를 이용해 별도의 UI 없이 검색 동작을 테스트 할 수 있다.
- 레거시 관계형 데이터베이스에 대한 외부 검색 엔진으로 활용.

엘라스틱 스택의 용도

2. 로그 통합 분석

- 엘라스틱 스택은 여러 장비와 서비스에서 발생하는 로그들을 통합하고, 검색하는데 최적화된 솔루션
- 시스템과 호스트, 쿠버네티스, 아파치, MySQL, 윈도우 같은 다양한 환경에서 생성되는 로그를 별도의 복잡한 구성 없이 바로 수집 가능.
- 비츠를 사용하면 적은 리소스로 각 장비의 로그들을 빠르게 수집할 수 있고, 로그 스테시는 다양한 필터를 통해 일원화된 형태로 가공을 할 수 있다.
- 엘라스틱서치의 대용량 로그에 대한 빠른 인덱싱 성능과 텍스트 검색은 여러 곳에 흩어진 서비스 로그들을 통합해서 연관 분석을 지원함.
- 키바나를 통해 직관적으로 모니터링이 가능

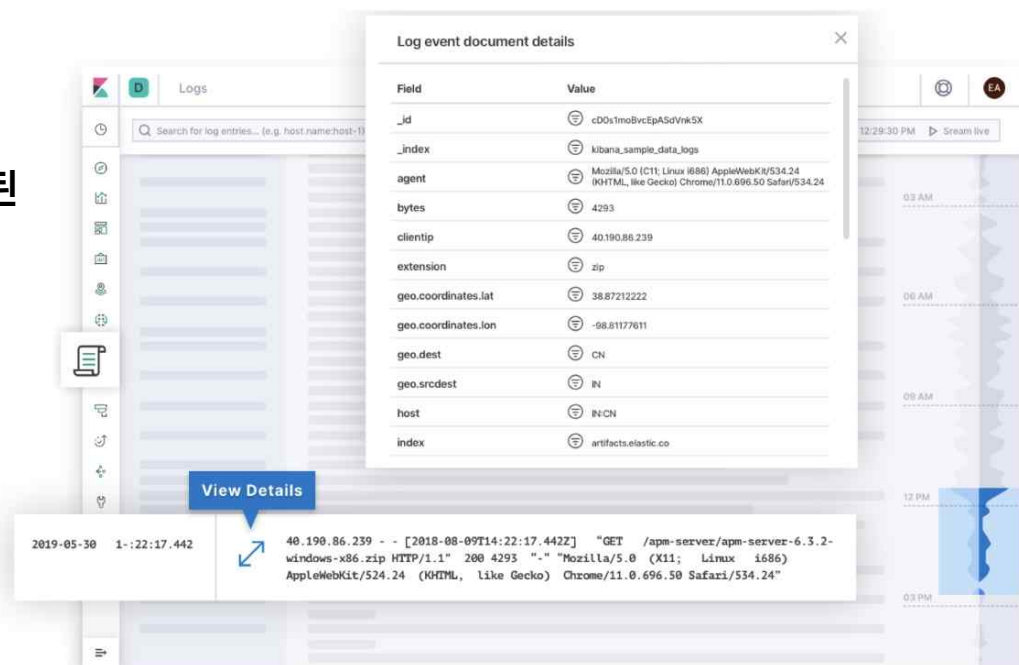


그림 1.9 키바나 로그 UI 기능⁵

엘라스틱 스택의 용도

2. 로그 통합 분석

- 로그 모니터링을 위해 엘라스틱 스택에서 할 수 있는 세부 업무
 - 파일비트는 내장된 모듈을 이용해 별도의 설정 없이 빠르게 로그를 수집할 수 있게 해준다.
 - 파일비트 모듈은 알려진 서비스들을 정제하기 위한 파이프라인 설정, 샘플 대시보드 등을 포함.
 - 커스텀 애플리케이션일 경우 로그스테시의 필터기능이나 엘라스틱서치의 수집 파이프라인 기능으로 정제하여 사용.
 - 엘라스틱서치에 저장된 데이터들은 교차 분석이나 연관 분석이 가능.
 - 관계형 데이터베이스와 다르게 엘라스틱서치는 인덱스 패턴을 이용해 한 번에 여러 인덱스에서 동시 조회가 가능.

엘라스틱 스택의 용도

3. 보안 이벤트 분석

- SIEM(Security Information and Event Management)이나 ESM(Enterprise Security Management) 등으로 불리는 솔루션은 조직 내 다양한 장비들로 부터 보안 이벤트를 수집하고 분석할 수 있게 하려는 목적으로 만들어 짐.
- 엘라스틱에서 제공하는 SIEM은 비츠의 모듈을 이용해 애플리케이션, 엔드포인트, 인증 로그, DNS 트래픽, 네트워크 플로우에서 이상 징후, 불법 접속시도 등 문제를 빠르게 찾아낼 수 있다.
- SIME UI에서는 분석을 비롯한 고유한 탐지 규칙 관리 또한 가능.
- 미리 빌드된 머신러닝 모델과 즉시 사용가능한 알고리즘을 통해 알려지지 않은 위협도 밝혀낼 수 있음.

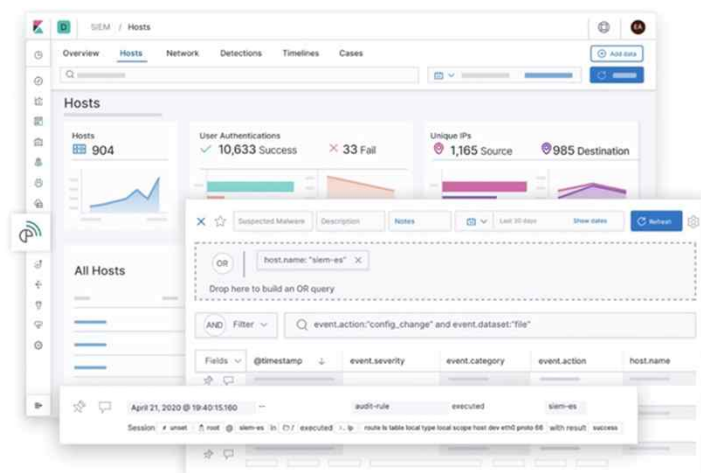


그림 1.10 엘라스틱 이벤트 분석⁶

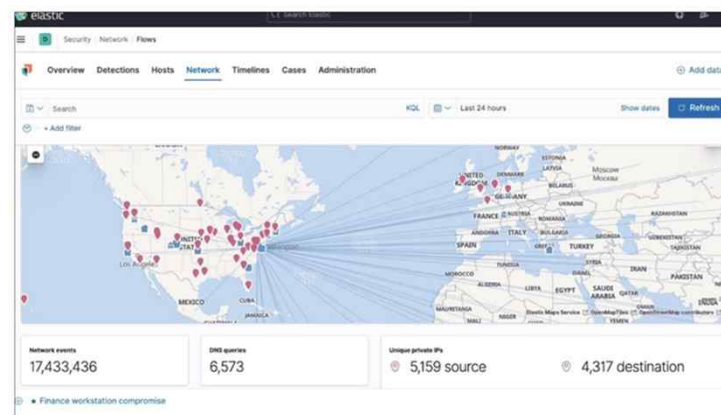


그림 1.11 엘라스틱 SIEM 시각화⁷

엘라스틱 스택의 용도

4. 애플리케이션 성능 분석

- 애플리케이션 운영에 있어 가장 중요한 것은 안정적인 서비스 유지.
 - 안정적인 서비스를 제공하기 위해 애플리케이션 상태 뿐만 아니라 연계된 모든 서비스의 성능 정보를 모니터링해야함.
- 엘라스틱 스택 애플리케이션 APM은 프로그래밍 언어별 에이전트를 통해 성능 지표 수집 및 분석을 위한 UI제공.
- 매트릭비트와 패킷비트를 사용하면 시스템을 비롯해 연계된 다양한 서비스의 성능 정보를 수집 할 수 있음.

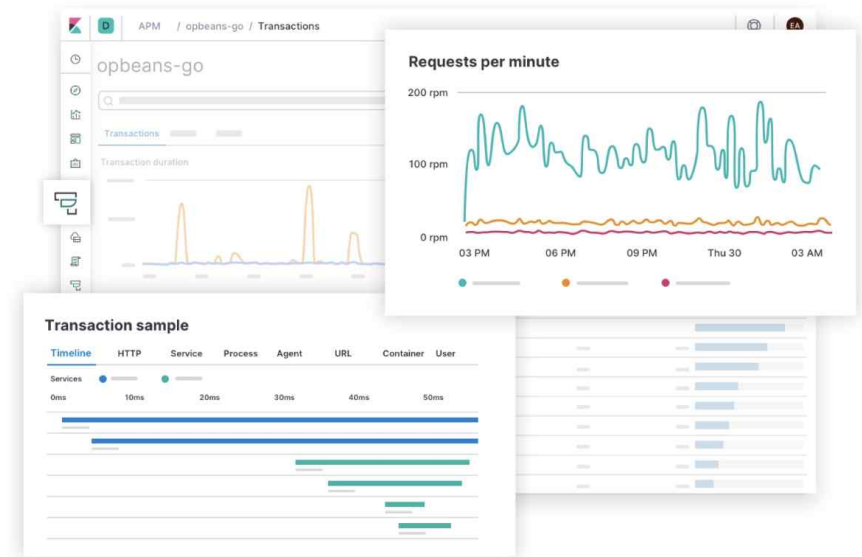


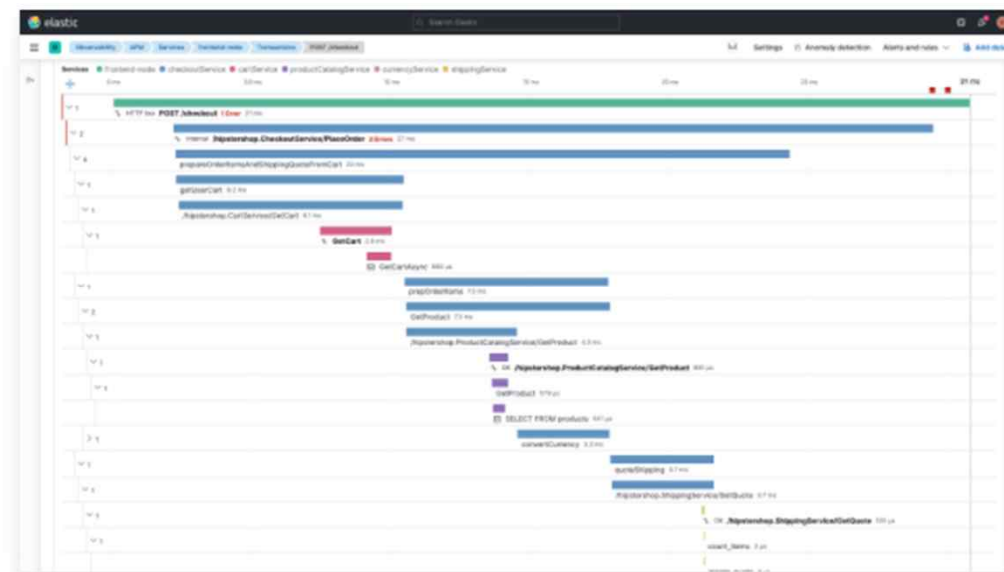
그림 1.12 애플리케이션 성능 모니터링⁸

엘라스틱 스택의 용도

4. 애플리케이션 성능 분석

엔드 투 엔드 분산 추적을 통해 코드 품질 향상

마이크로서비스 및 모놀리식 아키텍처에 걸친 분산 트랜잭션을 캡처하고 분석하세요. 배포 마커를 포함한 보강된 트랜잭션 메타데이터를 사용해 클라이언트에서 애플리케이션 서비스에 이르는 각 티어를 조사하여 가동 중단 시간을 최소화하고 고객 환경을 최적화하세요. 각 트랜잭션에 고객 메트릭/메타데이터 태그를 지정하여 귀중한 정보를 도출하고 보다 쉽게 분석하세요. 확장 가능하고 유연한 아키텍처는 필요에 따라 샘플링을 확대 또는 축소할 수 있도록 다이얼을 추가하여 완전한 충실도, 100% 트랜잭션 샘플 캡처, 스토리지, 분석 및 검색을 지원합니다.



출처: <https://www.elastic.co/kr/observability/application-performance-monitoring>

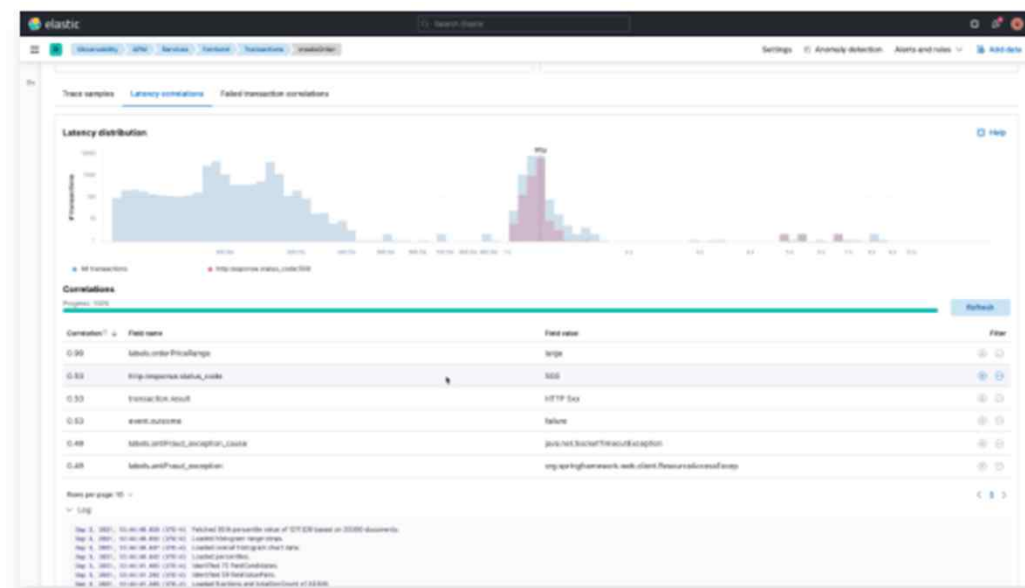
엘라스틱 스택의 용도

4. 애플리케이션 성능 분석

즉각적인 근본 원인 분석을 위한 이상 징후 탐지 자동화

마이크로서비스 및 분산 아키텍처에 대한 조사를 단순화하세요. 대기 시간 및 오류 상관 관계와 함께 머신 러닝을 이용한 자동 이상 징후 탐지를 통해 이러한 간헐적 문제라도 기존 모니터링 기법에 비해 쉽게 찾을 수 있습니다.

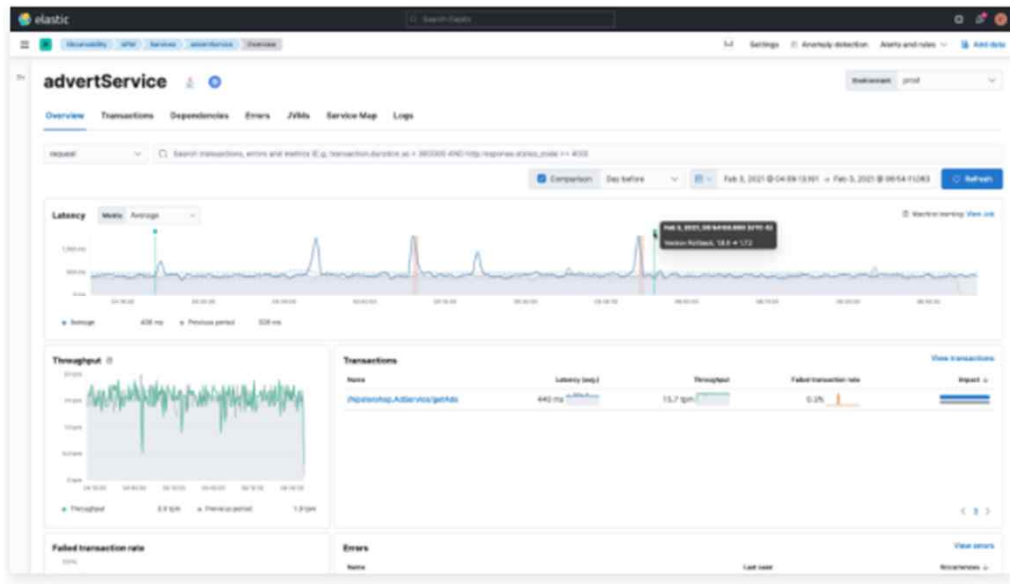
머신 러닝에 대해 알아보기 →



출처: <https://www.elastic.co/kr/observability/application-performance-monitoring>

엘라스틱 스택의 용도

4. 애플리케이션 성능 분석



자신 있게 배포하고 애플리케이션 제공 파이프라인 가속화

신규 배포 또는 "블루-그린 방식의" 배포 중에도 지속적인 가시성을 통해 애플리케이션 성능 변화를 식별하고 정량화하세요. 컨텍스트 추적, 로그 및 메트릭에 액세스하면서 릴리즈, 버전 또는 문제가 있는 노드로 범위를 좁혀 문제의 근본 원인을 찾으세요.

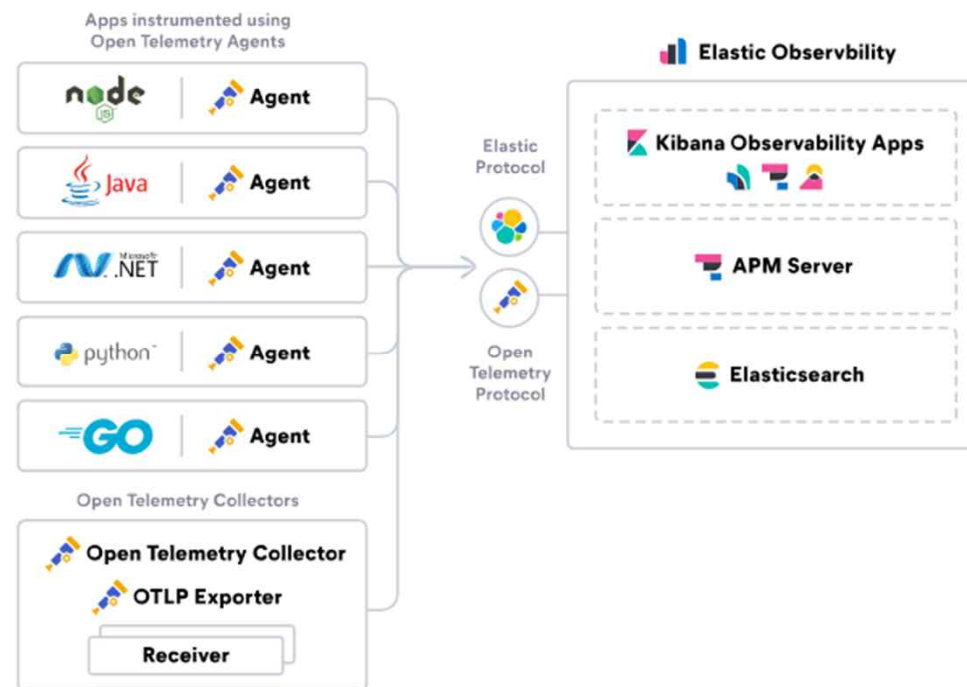
출처: <https://www.elastic.co/kr/observability/application-performance-monitoring>

엘라스틱 스택의 용도

4. 애플리케이션 성능 분석

개발자 친화적인 개방형 표준 지원

OpenTelemetry, W3C 추적 컨텍스트 및 Jaeger를 포함한 개방형 표준을 지원하여 OpenTelemetry 에이전트에서 수집한 애플리케이션 텔레메트리 데이터를 직접 수집할 수 있습니다. OpenTelemetry 및 개방형 표준 데이터 소스는 최대의 유연성을 제공하고 향후 통합 작업의 비용을 절감하여 Observability 투자의 미래를 대비합니다.



출처: <https://www.elastic.co/kr/observability/application-performance-monitoring>

엘라스틱 스택의 용도

5. 머신러닝

- 유료 라이선스를 구매할 경우 사용할 수 있음
- 데이터를 엘라스틱서치에 넣은 다음 비지도 머신러닝 기법을 활용해 데이터에서 패턴을 발견
- 시계열 모니터링은 시간과 관련된 데이터에서 이상징후 탐지 및 과거 데이터를 기반으로 동향예측 가능

빅데이터 플랫폼의 일부로 동작하는 엘라스틱 스택

5 빅데이터 플랫폼의 일부로 동작하는 엘라스틱 스택

- 우버(Uber)는 대규모 실시간 데이터 통찰 플랫폼인 가이로스(Gairos)를 구축해 가격 책정, 최대 ETA(도착 예정 시간) 계산, 수요/공급 예측과 같이 정보에 입각한 결정을 내리고 있다.
- 가이로스는 아파치 카프카와 아파치 하둡 생태계(HDFS/하이프/프레스토)를 연계해 실시간 분석과 장기 분석을 지원하며, 동적인 가격 정책과 환경 친화적인 운전 경로를 운전자에게 제공.

개요

Apache Kafka는 실시간으로 기록 스트림을 게시, 구독, 저장 및 처리할 수 있는 분산 데이터 스트리밍 플랫폼입니다. 이는 여러 소스에서 데이터 스트림을 처리하고 여러 사용자에게 전달하도록 설계되었습니다. 간단히 말해 A지점에서 B지점까지 이동하는 것뿐만 아니라 A지점에서 Z지점을 비롯해 필요한 모든 곳에서 대규모 데이터를 동시에 이동할 수 있습니다.

- Kafka: <https://www.redhat.com/ko/topics/integration/what-is-apache-kafka>

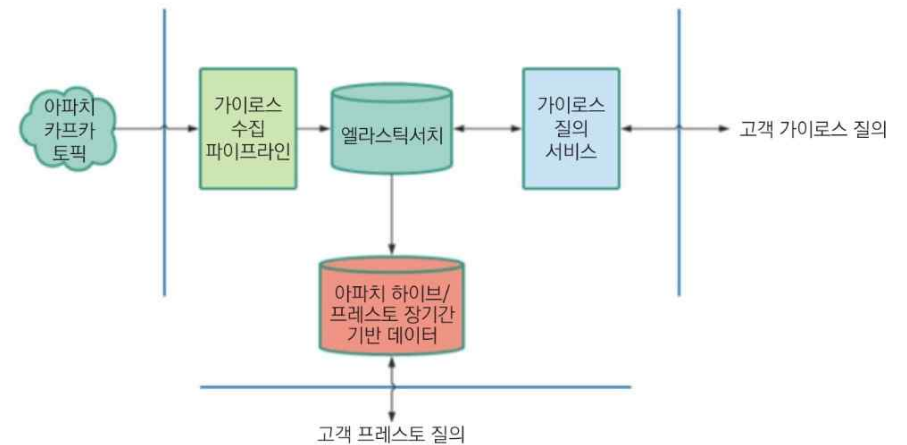


그림 1.13 가이로스의 단순화된 구성요소⁹

빅데이터 플랫폼의 일부로 동작하는 엘라스틱 스택

1. 엔터프라이즈 데이터 버스인 카프카와 연동

- 카프카는 분산 데이터 스트리밍 플랫폼, 대량의 데이터를 실시간으로 배포하는데 최적화되어 있음.
- 카프카와 엘라스틱 스택은 데이터 수집 시점에서 많이 연계되어 사용.

- 비츠에서 수집한 각 장비의 이벤트를 카프카로 전송하고 이를 로그스태시로 다시 읽어들이거나 카프카에 저장된 다른 시스템의 이벤트를 엘라스틱서치로 읽어 들이는 구성으로 사용.
- 카프카를 사용하면 엘라스틱서치나 로그스태시의 불안정으로 인덱싱이 불가능할 때 데이터 유실이 방지됨.
- 타 시스템과 데이터 연계가 손쉬워진다.

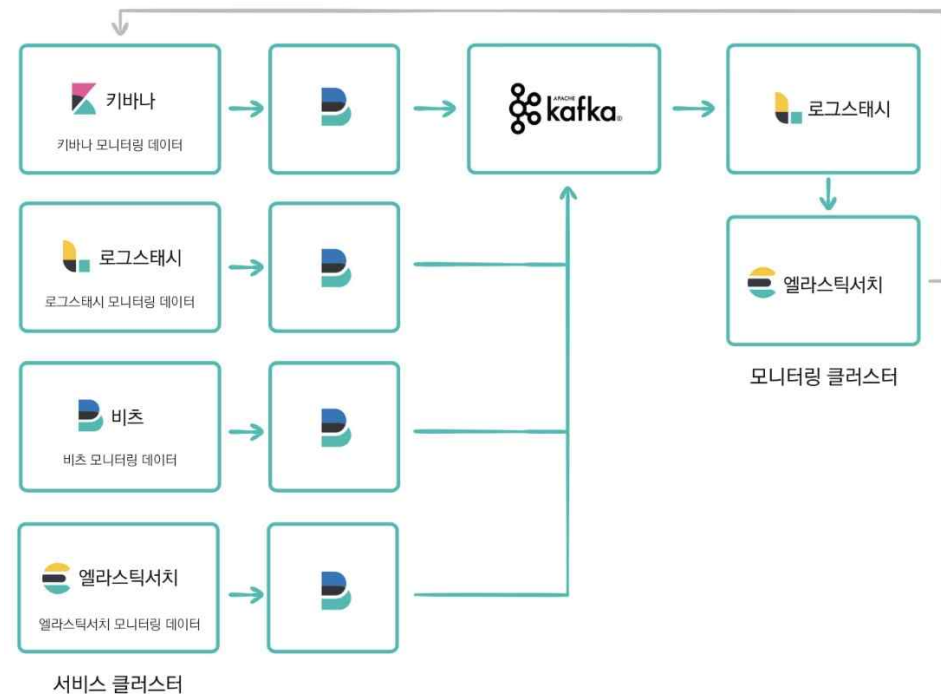


그림 1.14 카프카를 이용한 안정적인 시스템 구성¹⁰

빅데이터 플랫폼의 일부로 동작하는 엘라스틱 스택

2. 하둡 생태계와 연동

- 우버 가이로스는 HDFS/하이프/프레스토뿐만 아니라 빠른 처리를 위해 아파치 스파크도 도입.
- 엘라스틱 스택에서 제공하는 es-Hadoop 모듈을 활용해 아파치 스파크에서 엘라스틱서치 API를 사용해 문서를 읽어 들이고 반대로 인덱싱하는 등 서로 상호작용을 수행 할 수 있다.
- 엘라스틱서치는 이미 인덱싱된 결과에 대해 빠른 검색을 제공하고 스파크는 기존에 인덱싱되지 않은 데이터에 대한 빠른 일괄 처리를 제공하기 때문에 엘라스틱서치를 중간 집계 엔진이나 결과를 저장해 재활용하는 용도 등으로 활용 할 수 있다.

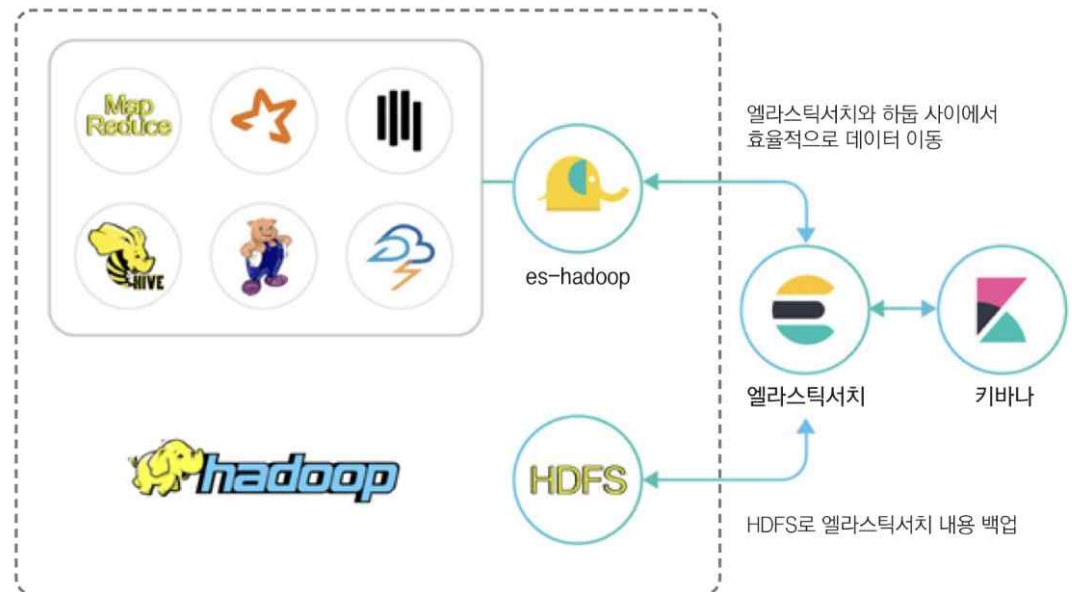


그림 1.15 es-hadoop을 활용한 대화형 데이터 분석¹¹

빅데이터 플랫폼의 일부로 동작하는 엘라스틱 스택

3. 관계형 데이터베이스와 연동

- 데이터베이스에는 레거시 데이터나 신뢰성이 중요한 데이터들이 포함되어 있다.
- 로그스테시는 JDBC input 플러그인, 필터 등 RDB와 연계할 수 있는 다양한 방법 제공.
- 기존 RDB에 저장된 데이터를 인덱싱하거나 입력 받는 이벤트에 정보를 주입하는 등 여러 용도로 사용.
- JDBC input 플러그인을 통해 RDB의 데이터를 엘라스틱서치로 이관할 경우, 엘라스틱서치의 집계 기능 사용 가능.
- 텍스트 데이터의 경우 엘라스틱서치에서 색인만 수행하여(원문 저장x) 검색 엔진의 전문검색 기능만 사용 가능.

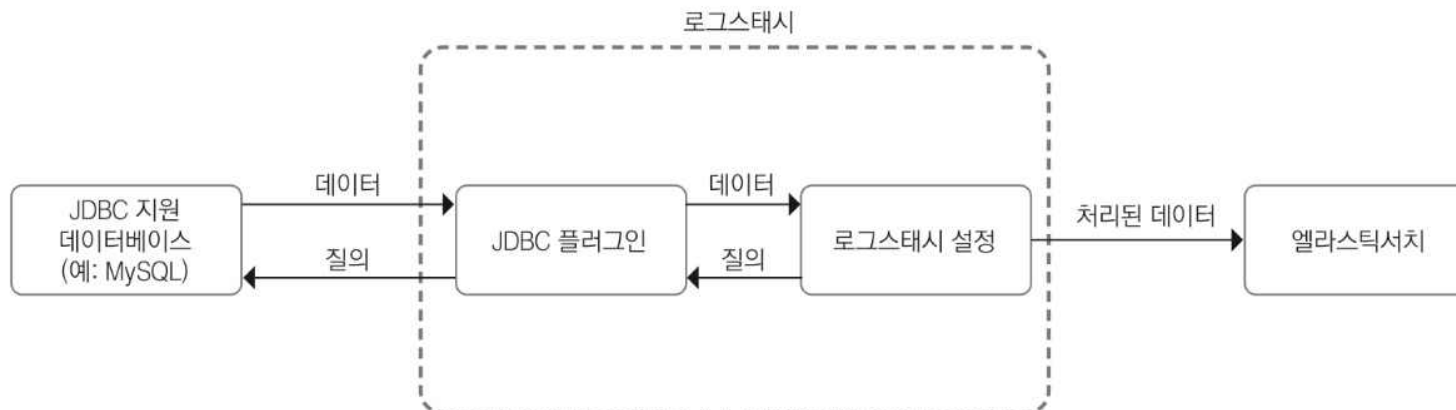


그림 1.16 로그스테시를 활용한 데이터베이스와 엘라스틱서치 연동¹²

유사 제품군 비교

6 유사 제품과의 비교

- 엘라스틱 스택에는 빅데이터 파이프라인을 구성하는 데이터 수집기, 데이터 변환기, 데이터 저장소, 시각화 도구가 많이 포함되어 있음. 각각 유사제품과 비교해 차이점과 엘라스틱의 장점을 살펴본다.

유사 제품군 비교

1. 엘라스틱서치의 유사 제품군

- 엘라스틱서치는 검색 엔진이자, NoSQL DB로 볼 수 있다.

표 1.1 엘라스틱서치와 그 밖의 데이터베이스 비교

	엘라스틱서치	몽고디비	MySQL
분류	검색 엔진	문서 저장소	관계형 데이터베이스
최초 릴리스	2010년	2009년	1995년
스키마	자동 생성	자유	필요
인터페이스	REST API	전용 프로토콜	JDBC/ODBC
분산 저장	샤딩	샤딩	별도 제품으로 지원
트랜잭션	미지원	부분 지원	지원
JOIN	미지원	최근 문서 단위로 지원 시작	지원
특징	다양한 데이터 유형에 대한 빠른 검색/집계	높은 활용성/범용성	뛰어난 데이터 무결성

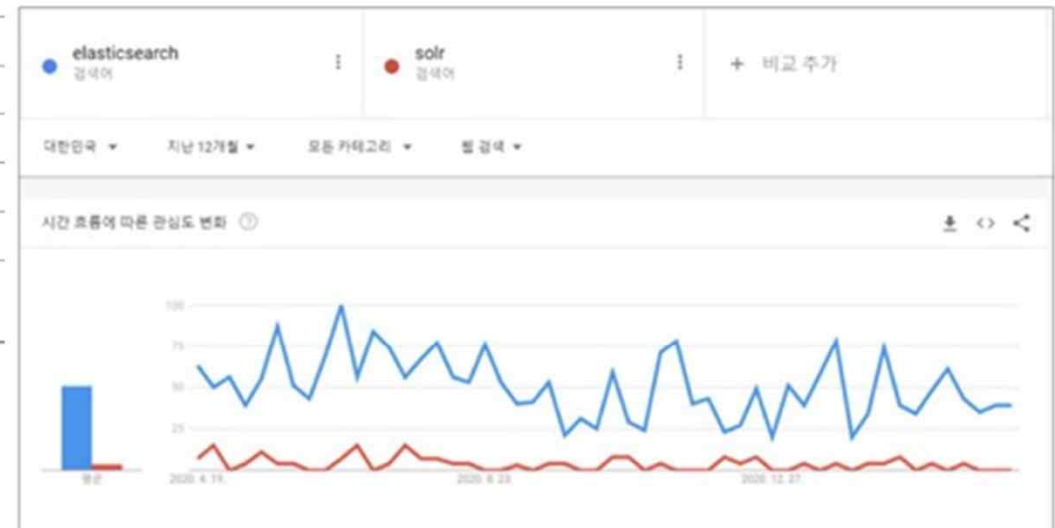


그림 1.17 구글 트렌드로 비교한 엘라스틱과 솔라 검색 추이

유사 제품군 비교

2. 로그스태시/비츠 유사 제품군

- 엘라스틱 스택에는 로그스태시, 비츠 2개의 수집기가 있다.
 - 로그스태시는 입력, 출력, 필터 플러그인을 갖춰 설정만으로 모든 형태의 가공을 수행 가능.
 - 비츠는 기능은 제한 적이나 본연의 기능에 충실하고 빠르고 가볍게 이벤트를 수집할 수 있음.
- 로그스태시, 비츠의 유사 제품으로 트레저헌터사에서 개발한 플루언트디가 있으나 엘라스틱 스택에서는 지원이 불가함.
- 플루언트디는 CNCF 스택이며 쿠버네티스나 오픈트레이싱을 사용하는 경우 필수 구성요소.

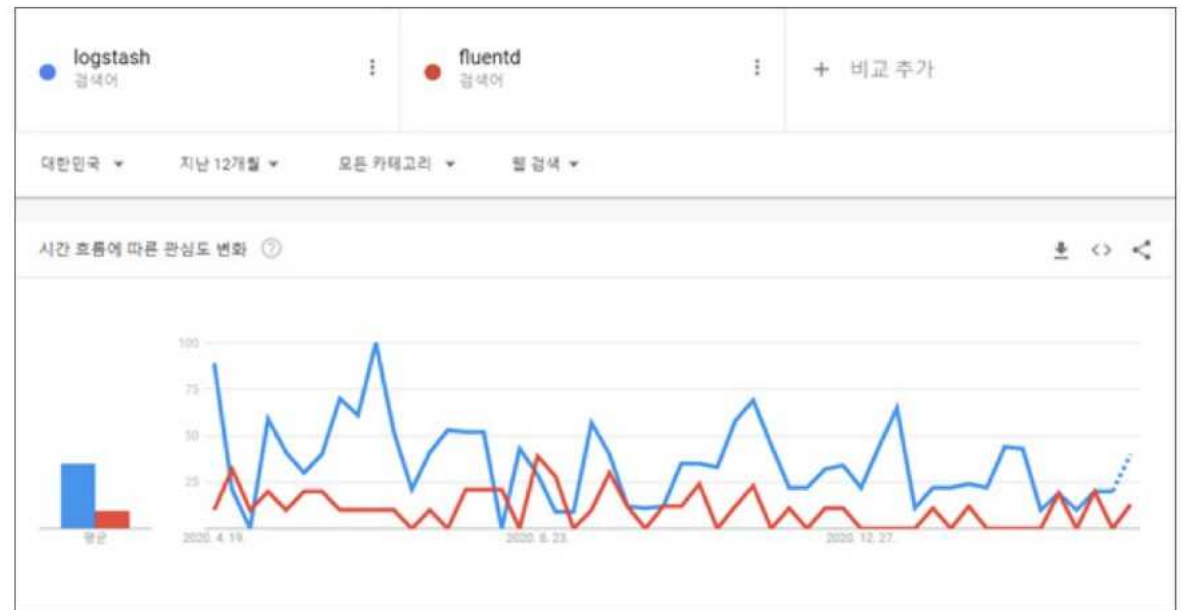


그림 1.18 구글 트렌드로 비교한 로그스태시와 플루언트디 검색 추이

유사 제품군 비교

3. 키바나의 유사 제품군

표 1.2 시각화 도구 비교

	키바나	그라파나	태블로
실행 환경	웹 서비스	웹 서비스	설치/웹 서비스 모두 제공
데이터 소스	엘라스틱서치	엘라스틱서치, 그라파이트(Graphite), 몽고디비, 프로메테우스(Prometheus) 등	관계형 데이터베이스, 엑셀 파일, JSON 파일, 드롭박스 등
특징	실시간 대시보드 외에도 캔버스, 지도, 비츠/로그스테시와 연계되어 제공되는 솔루션 등 엘라스틱서치의 기능을 십분 활용한 다양한 시각화 세트 제공	라인, 바 차트 등 기본적인 것만 인프라 모니터링에 부족함이 없는 시각화 요소들을 포함한 실시간 대시보드 제공	실시간보다는 다양한 시각화 도구를 이용한 분석에 특화되어 있음. 다양한 데이터 소스를 활용한 시각화 가능
기타 기능 지원	스택 관리, 모니터링 등 엘라스틱스택 특화 기능 지원. 알림이나 머신러닝 등 유료 기능 존재	알림 기능 지원	

유사 제품군 비교

4. 엘라스틱 스택의 유사 제품군

- 스플링크는 엘라스틱 스택 자체와 대응되는 제품.
- 스플링크는 대용량 로그분석을 위한 엔터프라이즈 제품으로, 전용 로그 수집기, 고유한 인덱싱 기술, 대시보드, 각 벤더들에서 제공하는 수많은 앱을 통해 이기종의 로그를 쉽게 수집하고 시각화 할 수 있다.
- 유료 라이선스이다.

02

엘라스틱서치 기본

엘라스틱서치 기본

- ① 엘라스틱서치와 키바나에서 데이터 로드
- ② 인덱스와 도큐먼트 개념 이해
- ③ 도큐먼트 생성/읽기/수정/삭제 방법
- ④ 복수 인덱스 관리
- ⑤ 문자열 나누는 분석기

엘라스틱서치 기본

1 준비 작업

- 엘라스틱서치는 REST API 형태 REST란 웹상의 모든 리소스에 URL을 부여하고, 활동하는 아키텍처
- REST를 도입하면 모든 리소스를 일관된 규칙으로 접근할 수 있어 애플리케이션을 모듈이나 기능별로 분리하기 쉬워진다.

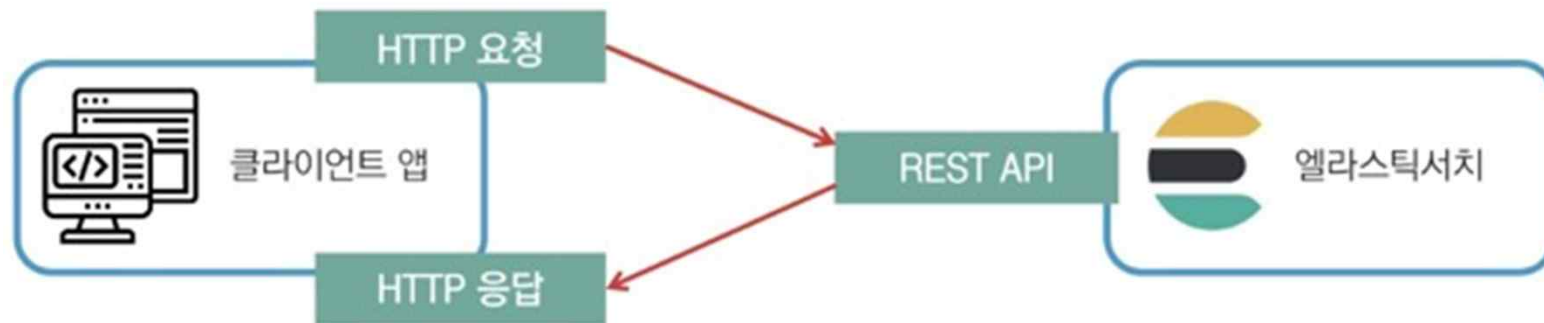


그림 3.2 엘라스틱서치의 REST API 구조

준비 작업

1. 엘라스틱서치 요청과 응답

참고 REST API

REST는 'Representational State Transfer'의 약자로 웹(HTTP)의 장점을 이용해 리소스를 주고받는 형태며, REST API는 REST 기반으로 API를 서비스하는 것을 의미한다. REST API는 메소드(method)와 경로(URI)가 합쳐진 형태다. 먼저 메소드는 네 가지로 표현할 수 있다.

메소드	설명	메소드	설명
POST	해당 리소스를 추가한다.	GET	해당 리소스를 조회한다.
PUT	해당 리소스를 수정한다.	DELETE	해당 리소스를 삭제한다.

REST API는 네 가지 메소드 타입을 가지고 리소스의 CRUD(생성/읽기/수정/삭제) 작업을 진행한다. URI(Uniform Resource Identifier)는 리소스를 명시하는 방법이다. URI로 리소스를 정의하고 메소드와 함께

서버에 요청하고 응답을 받는 구조다.

```
POST es_index/_doc/1 { "name": "kim", "age": 20 } ❶  
GET es_index ❷  
GET es_index/_doc/1 ❸  
DELETE es_index ❹
```

❶은 es_index라는 인덱스를 만들고 1번 문서를 생성한다. ❷은 es_index를 조회한다. ❸은 es_index 내부의 1번 문서만 조회한다. ❹은 es_index를 삭제한다. URI는 정보의 리소스를 표현한 것으로, 엘라스틱서치에서 설계하고 정의한 규격에 맞춰야 한다. 엘라스틱서치 REST API 사용법은 책을 따라 하다 보면 자연스럽게 익힐 수 있으니 지금은 너무 걱정하지 않아도 된다. 공식 문서¹에서 엘라스틱이 제공하는 모든 REST API를 확인할 수 있다.

Elasticsearch Reference [7.10] » REST APIs

« Glossary of terms

API conventions »

REST APIs

Elasticsearch exposes REST APIs that are used by the UI components and can be called directly to configure and access Elasticsearch features.



We are working on including more Elasticsearch APIs in this section. Some content might not be included yet.

- API conventions
- cat APIs
- Cluster APIs
- Cross-cluster replication APIs
- Data stream APIs

그림 3.1 엘라스틱서치에서 제공하는 API 문서

준비 작업

2. 키바나 콘솔 사용법

- 키바나 Dev Tools에 있는 콘솔을 이용해 REST API를 호출할 것.
- 키바나 왼쪽 상단 토글 메뉴를 클릭하면 키바나 메뉴를 확인 할 수 있다.

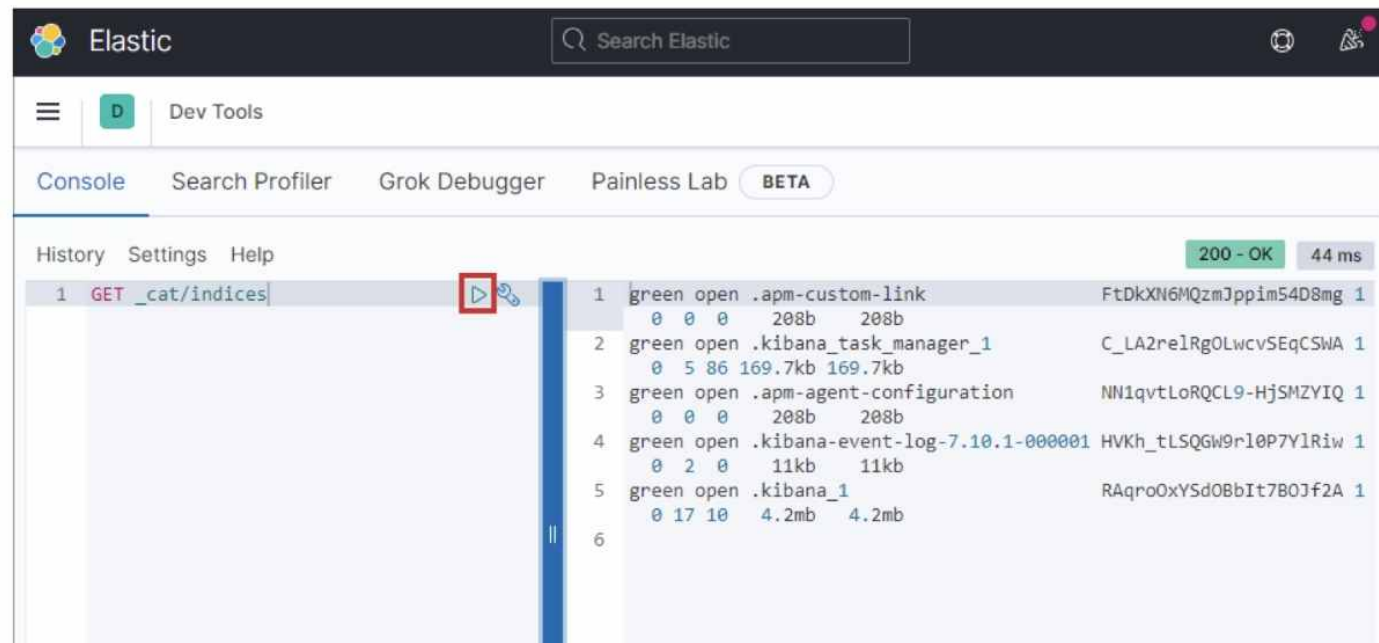
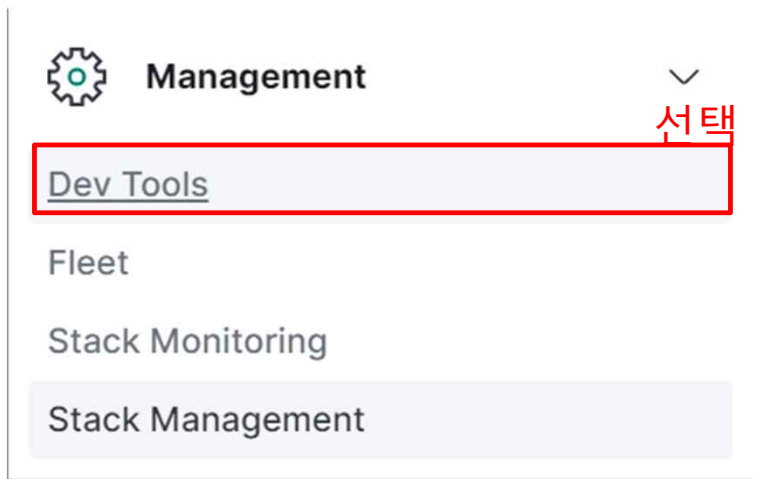
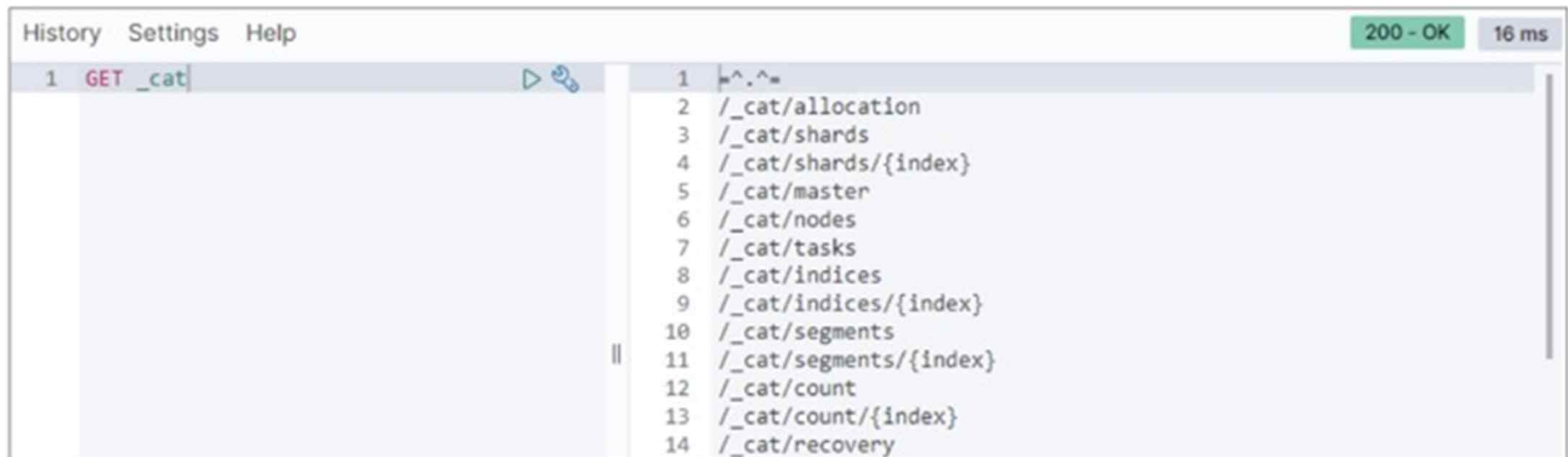


그림 3.3 키바나 콘솔로 REST API를 호출한 화면

준비 작업

3. 시스템 상태 확인

- 엘라스틱서치의 현재 상태를 빠르게 확인할 수 있는 방법으로 일반적으로 cat API를 사용.
- cat(compact and aligned text)은 콘솔에서 시스템 상태를 확인할 때 가독성을 높일 목적의 API.
- 키바나 콘솔에서 **GET _cat**을 요청하면 cat API가 지원하는 목록을 확인 가능



The screenshot shows the Kibana console interface. At the top, there are tabs for 'History', 'Settings', and 'Help'. On the right, a status bar indicates '200 - OK' and '16 ms'. The main area is split into two panes. The left pane shows the command '1 GET _cat'. The right pane shows the response, which is a list of 14 endpoints supported by the cat API:

```
1 | ^.^=
2 | /_cat/allocation
3 | /_cat/shards
4 | /_cat/shards/{index}
5 | /_cat/master
6 | /_cat/nodes
7 | /_cat/tasks
8 | /_cat/indices
9 | /_cat/indices/{index}
10 | /_cat/segments
11 | /_cat/segments/{index}
12 | /_cat/count
13 | /_cat/count/{index}
14 | /_cat/recovery
```

그림 3.4 cat API가 지원하는 목록

준비 작업

3. 시스템 상태 확인

- cat API를 통해 노드(node), 샤드(Shard), 템플릿(Template)등의 상태 정보나 통계 정보를 확인 할 수 있다.
- 그림 3.5와 같이 키바나 콘솔창에서 **GET _cat/indices?v**를 요청.
- ? 기호 뒤에는 몇 가지 파라미터를 사용할 수 있는데 v 파라미터는 컬럼의 이름을 확인 할 수 있음.
 - 이밖에 s(정렬), h(헤더) 있다.

Elastic

Search Elastic

≡

D

Dev Tools

Console

Search Profiler

Grok Debugger

Painless Lab

BETA

History

Settings

Help

200 - OK

17 ms

1 GET _cat/indices?v

1 health status index uuid pri rep

docs.count docs.deleted store.size pri.store.size

2 green open .apm-custom-link FtDkXN6MQzmJppim54D8mg 1 0

0 0 208b 208b

3 green open .kibana_task_manager_1 C_LA2relRgOLwcvSEqCSWA 1 0

5 205 158.9kb 158.9kb

4 green open .apm-agent-configuration NN1qvtLoRQCL9-HjSMZYIQ 1 0

0 0 208b 208b

5 green open .kibana-event-log-7.10.1-000001 HVKh_tLSQGW9r10P7Y1Riw 1 0

2 0 11kb 11kb

6 green open .kibana_1 RAqroOxYSd08bIt7B0Jf2A 1 0

24 4 2.1mb 2.1mb

7

그림 3.5 엘라스틱서치 인덱스 목록 확인

준비 작업

3. 시스템 상태 확인

- cat API는 키바나 콘솔 뿐 아니라 명령프롬포트에서도 curl을 통해 사용 가능.
 - > **Curl -X GET "localhost:9200/_cat/indices?v"**
- cat API를 통해 가독성 좋은 형태로 상태확인 가능함.

```
C:\kibana-7.10.1>curl -X GET "localhost:9200/_cat/indices?v"
health status index      uuid                                pri rep docs.count docs.deleted store.size pri.store.size
green  open   .apm-custom-link      FtDkXN6MQzmJppim54D8mg          1   0         0           0        208b         208b
green  open   .kibana_task_manager_1 C_LA2reIRg0LwcvSEqCSWA          1   0         5        214       137.4kb       137.4kb
green  open   .apm-agent-configuration NN1qvtLoRQCL9-HJSMZYIQ          1   0         0           0        208b         208b
green  open   .kibana-event-log-7.10.1-000001 HYKh_tLSQGw9r10P7YIRiw          1   0         2          0         11kb         11kb
green  open   .kibana_1             RAqro0xYSd0BbIt7BOJf2A          1   0        26          4         2.1mb         2.1mb
```

그림 3.6 curl을 이용한 인덱스 목록 확인

준비 작업

4. 샘플데이터 불러오기

- 키바나 홈 화면에서 'Try our sample data' 클릭
- 총 3개의 샘플데이터가 있음. Add data 버튼을 클릭해 샘플을 추가할 수 있음.
- Remove 버튼을 눌러 샘플데이터 삭제 가능.
- 샘플데이터를 추가하면 키바나의 Visualize와 Dashboard등에 샘플들이 함께 추가 됨

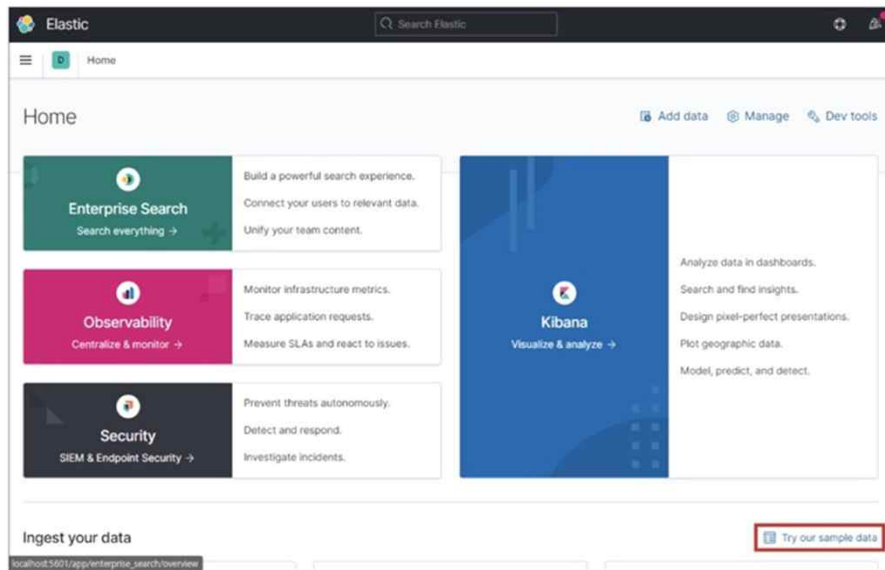


그림 3.7 키바나 홈 화면

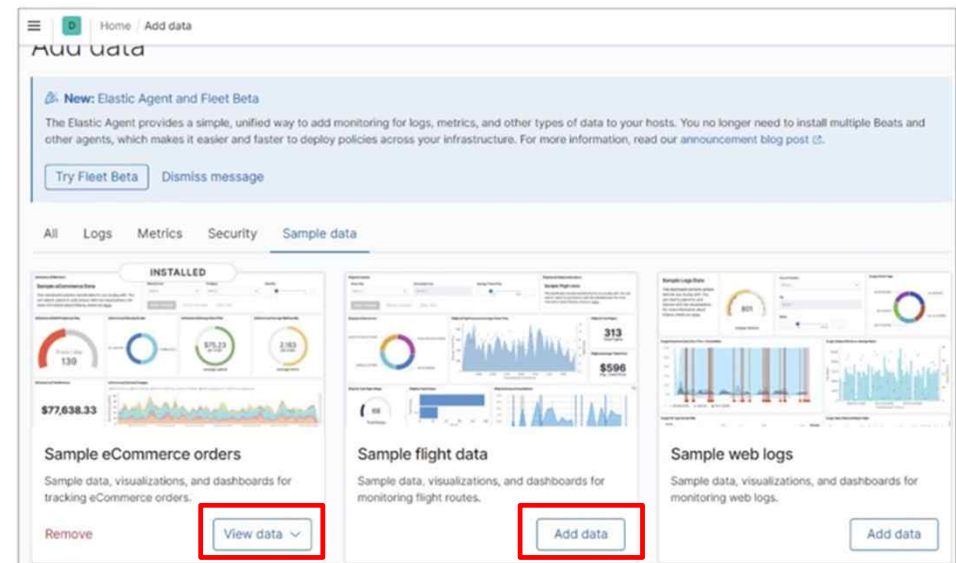


그림 3.8 키바나 샘플 데이터 화면

엘라스틱서치 기본

2 인덱스와 도큐먼트

- 엘라스틱서치를 이해하기 위해 인덱스와 도큐먼트가 무척 중요하다.
 - 인덱스: 도큐먼트를 저장하는 논리적 구분자
 - 도큐먼트: 실제 데이터를 저장하는 단위
- 그림 3.9에서 엘라스틱서치 클러스터, 인덱스와 도큐먼트, 그리고 필드 간에 어떤 포함 관계가 있는지 보여준다.

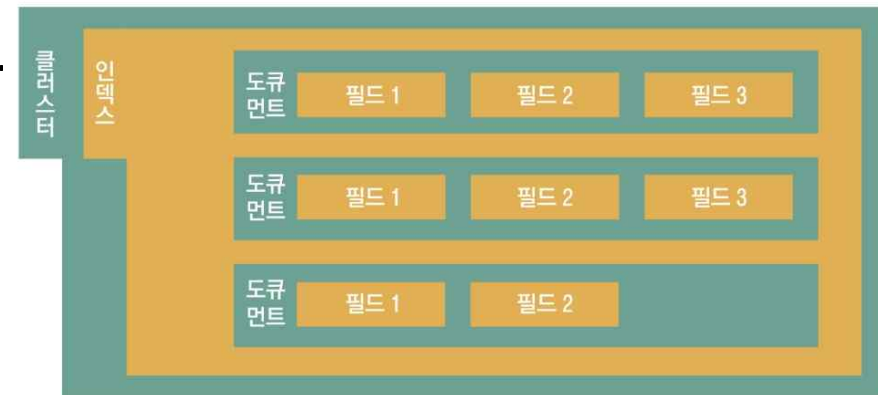


그림 3.9 인덱스, 도큐먼트, 필드의 구조

- 엘라스틱을 이용해 시스템을 개발하면 하나의 프로젝트에서 하나의 클러스터를 생성.
- 클러스터 내부에는 데이터 성격에 따라 여러 개의 인덱스를 생성.
- 인덱스 내부에는 JSON 형태로 된 다수의 도큐먼트가 존재.
- 도큐먼트는 복수의 필드가 있음.

인덱스와 도큐먼트

1. 도큐먼트

- 도큐먼트는 엘라스틱서치에서 데이터가 저장되는 기본단위로 JSON 형태이며, 하나의 도큐먼트는 여러 필드와 값을 갖는다.

리스트 3.1 MySQL에서 테이블 생성과 데이터 저장

```
CREATE TABLE member (  
  uid int not null AUTO_INCREMENT PRIMARY KEY,  
  name varchar(50) not null,  
  age int not null,  
  gender varchar(6) not null  
);
```

```
INSERT INTO member (name, age, gender) values ("mike", 25, "male");
```

리스트 3.2 JSON 형식의 도큐먼트 파일

```
{  
  "name": "mike",  
  "age": 25,  
  "gender": "male"  
}
```

↑ 필드 ↑ 값

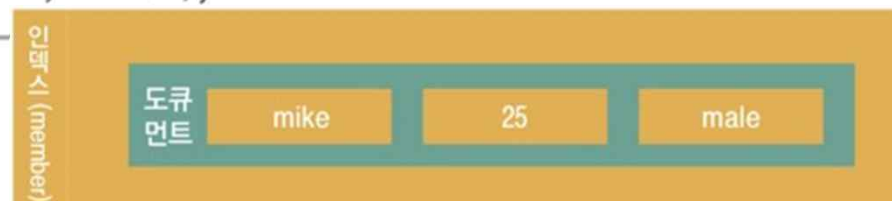


그림 3.10 도큐먼트 저장

인덱스와 도큐먼트

1. 도큐먼트

- MySQL과 엘라스틱서치의 비교

표 3.1 MySQL과 엘라스틱서치 비교

MySQL	엘라스틱서치
테이블	인덱스
레코드	도큐먼트
컬럼	필드
스키마	매핑

인덱스와 도큐먼트

2. 인덱스

- 인덱스는 도큐먼트를 저장하는 논리적 단위로 RDB의 테이블과 유사한 개념.
- 하나의 인덱스에 다수의 도큐먼트가 포함되는 구조 동일한 인덱스에 있는 도큐먼트는 동일한 스키마를 갖는다.
- 인덱스 이름은 영어 대/소문자 \, /, *, ?, ", <, >, |, #, 공백문자, 쉼표 등을 제외한 특수문자를 사용할 수 있으며 255바이트를 넘을 수 없다.

| 스키마에 따른 그룹핑

일반적으로 스키마에 따라 인덱스를 구분.

예) 회원 정보 도큐먼트와 장바구니 도큐먼트는 성격이 다르기 때문에 데이터 스키마도 다르다. 회원 정보에는 회원 이름이나 나이 같은 필드들이 들어가고 장바구니에는 상품명이나 날짜 같은 필드가 들어간다. 이렇게 다른 스키마를 도큐먼트를 하나의 인덱스에 저장하는 방법은 옳지 않음.

| 관리 목적의 그룹핑

인덱스에는 용량이나 숫자 제한 없이 무한대의 도큐먼트를 포함 할 수 있음. 하지만 인덱스가 커지면 검색시 많은 도큐먼트를 참조해야 하기 때문에 성능이 나빠짐. 그래서 엘라스틱서치를 운용할 때 인덱스 용량 제한을 둔다.

엘라스틱서치 기본

3 도큐먼트 CRUD

- 도큐먼트를 엘라스틱서치 인덱스에 저장(Create), 읽고(Read), 수정(Update), 삭제>Delete)하는 일련의 CRUD 동작 해 본다.
- 이 작업을 통해 엘라스틱서치가 어떻게 도큐먼트를 처리하는지 확인하고 인덱스와 도큐먼트가 어떻게 사용되는지 알아 본다.

1. 인덱스 생성/확인/삭제

작업을 위해 키바나 Dev Tools를 실행

- 인덱스 생성 API:> **PUT index1**
PUT 는 생성이나 수정을 위한 HTTP 메소드. PUT 대신 POST를 사용 해도 된다.
- 인덱스 확인 API: > **GET index1**
인덱스의 설정이나 매핑값을 확인 할 수 있음.
- 인덱스 삭제 API: > **DELETE index1**
삭제 시 항상 주의해야 함.

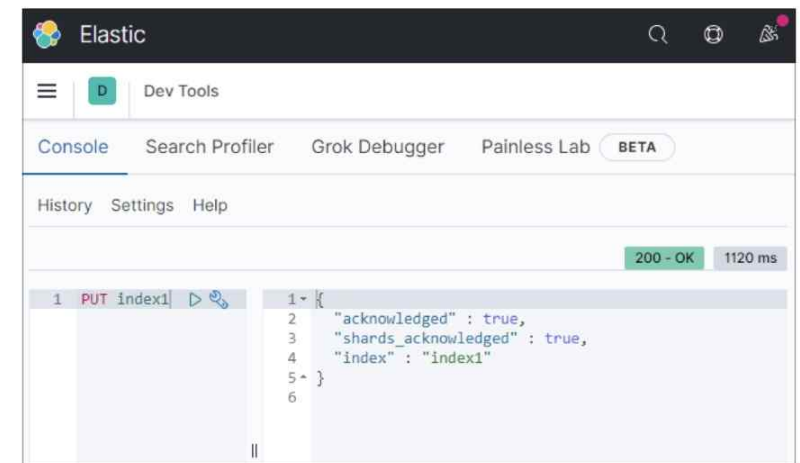


그림 3.11 키바나 콘솔 사용 방법

도큐먼트 CRUD

2. 도큐먼트 생성

- 도큐먼트는 하나에 인덱스에 반드시 포함 되어야 한다.
- 도큐먼트를 인덱스에 포함하는 것을 인덱싱이라 한다.

```
PUT index2/_doc/1
{
    "name":"mike",
    "age":25,
    "gender":"male"
}
```

리스트 3.3 도큐먼트 인덱싱

Index2 인덱스 확인:
> GET index2

존재하지 않았던 index2라는 인덱스를 생성하면서 index2 인덱스에 도큐먼트를 인덱싱. Index2는 인덱스 이름, _doc은 엔드포인트 구분을 위한 예약어, 숫자 1은 인덱싱할 도큐먼트의 고유 아이디. 도큐먼트에 age, gender, name라는 필드가 있고, 각 필드에는 값이 있음.

리스트 3.4 index2 인덱스 확인

```
{
  "index2" : {
    "aliases" : { },
    "mappings" : {
      "properties" : {
        "age" : { "type" : "long", ... },
        "gender" : { "type" : "text", ... },
        "name" : { "type" : "text", ... },
        ...
      }
    }
  }
}
```

다이나믹 매핑

도큐먼트 CRUD

2. 도큐먼트 생성

- Index2 인덱스에 country라는 이름의 새로운 필드 추가

```
PUT index2/_doc/2
{
  "name": "jane",
  "country": "france"
}
```



- GET index2 명령을 활용해 인덱스를 확인해 보면 country 필드가 추가됨을 확인.

```
{
  "country" : {
    "type" : "text",
    "fields" : {
      "keyword" : {
        "type" : "keyword",
        "ignore_above" : 256
      }
    }
  }
},
```

리스트 3.5 index2 인덱스에 새로운 필드 추가된 도큐먼트 인덱싱

- Index2 인덱스에 데이터 타입이 잘못된 도큐먼트 인덱싱

```
PUT index2/_doc/3
{
  "name": "kim",
  "age": "20",
  "gender": "female"
}
```



- 리스트 3.4에서 age를 long형으로 매핑했는데 3번 도큐먼트는 텍스트 타입으로 입력. RDB와 달리 스키마에 유연하게 대응하는 엘라스틱서치는 타입을 변환해 저장한다.

```
"age" : {
  "type" : "long"
},
```

리스트 3.6 index2 인덱스에 데이터 타입이 잘못된 도큐먼트 인덱싱

도큐먼트 CRUD

2. 도큐먼트 생성

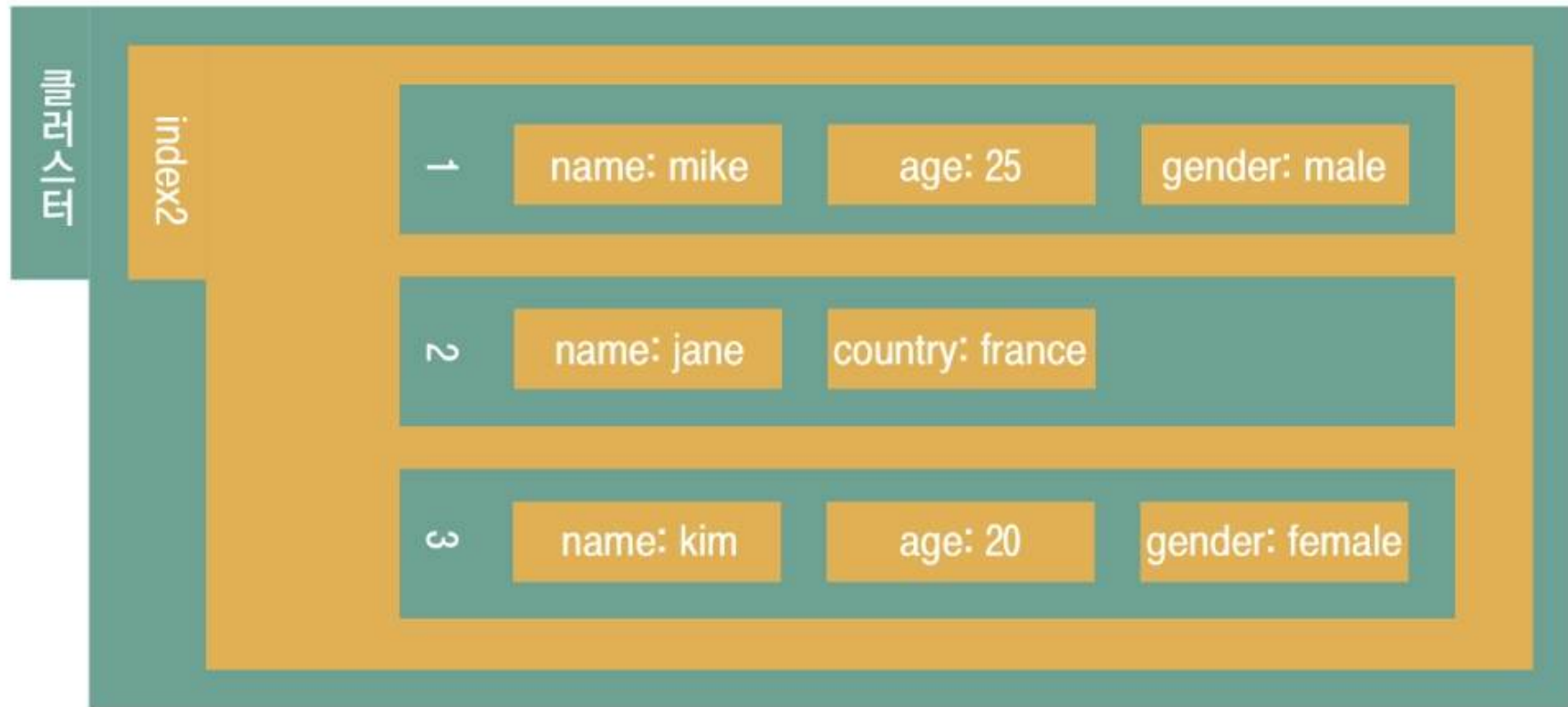


그림 3.12 index2 인덱스 도큐먼트

도큐먼트 CRUD

3. 도큐먼트 읽기

- 도큐먼트를 읽는 방법

- 아이디를 이용해 조회하는 방법

- DSL(Domain Specific Language)이라는 엘라스틱서치 쿼리문을 이용하는 방법.

- | 아이디를 이용해 조회하는 방법

- **GET index2/_doc/1** : 인덱스명과 도큐먼트아이디를 입력, refresh_interval 없이 바로 조회 가능

- | DSL을 이용해 조회하는 방법

- **GET index2/_search** : search DSL 쿼리를 이용해 읽어 올 수 있다 , refresh_interval 이후에만 검색 가능

도큐먼트 CRUD

4. 도큐먼트 수정

- Index2 인덱스의 1번 도큐먼트의 특정 필드값을 변경

```
PUT index2/_doc/1
{
  "name":"park",
  "age":"45",
  "gender":"male"
}
```

리스트 3.7 1번 도큐먼트 업데이트

- 도큐먼트를 인덱싱 하는 과정에서 같은 도큐먼트 아이디가 있으면 덮어쓰기가 자동으로 수행

```
"_id" : "1",
"_version" : 2,
"result" : "updated",
```

- Update API를 사용한 도큐먼트 업데이트

```
POST index2/_update/1
{
  "doc":{
    "name":"lee"
  }
}
```

리스트 3.9 update API를 사용한 도큐먼트 업데이트

- _update라는 엔드포인트를 추가해 특정 필드 값만 업데이트 할 수 있다.
- 엘라스틱에서 수정은 권장하지 않는다. 리소스가 많이 들어가기 때문
- 엘라스틱서치를 로그수집 용도로 사용한다면 도큐먼트를 사용하는 일이 없음.
- 개별 도큐먼트 수정이 많은 작업은 다른 DB 사용 권장.

도큐먼트 CRUD

5. 도큐먼트 삭제

- 특정 도큐먼트를 삭제하기 위해 인덱스명과 도큐먼트 아이디를 알고 있어야 한다.
- 키바나 콘솔에서 도큐먼트를 삭제하는 REST API
 - > **DELETE** index2/_doc/2
- GET index2/_search를 통해 확인하면 도큐먼트 id가 2인 것이 삭제됨을 알 수 있음

```
hits : [
  {
    "_index" : "index2",
    "_type" : "_doc",
    "_id" : "3",
    "_score" : 1.0,
    "_source" : {
      "name" : "kim",
      "age" : "20",
      "gender" : "female"
    }
  },
  {
    "_index" : "index2",
    "_type" : "_doc",
    "_id" : "1",
    "_score" : 1.0,
    "_source" : {
      "name" : "lee",
      "age" : "45",
      "gender" : "male"
    }
  }
]
```

엘라스틱서치 기본

4 응답 메시지

• REST API의 응답메시지 구조

표 3.3 엘라스틱서치 REST API 응답 코드

코드	상태	해결 방법
200, 201	정상적으로 수행함	
4xx	클라이언트 오류	클라이언트에서 문제점 수정
404	요청한 리소스가 없음	인덱스나 도큐먼트가 존재하는지 체크
405	요청 메소드(GET, POST 등)를 지원하지 않음	API 사용법 다시 확인
429	요청 과부화(busy)	재전송, 노드 추가 같은 조치
5xx	서버 오류	엘라스틱서치 로그 확인 후 조치

> GET index5

명령을 통해 인덱스 호출

```
19 ^ },
20   "status" : 404
21 ^ }
```

엘라스틱서치 기본

5 벌크 데이터

- 데이터 CRUD 동작을 할 경우 REST API를 호출해 하나하나 문서를 요청하는 것보다 벌크로 한 번에 요청하는 것이 효율적.
- 엘라스틱서치는 bulk API를 지원하는데 REST API 콜 횟수를 줄여 성능을 높인다.

```
POST _bulk
{"index":{"_index":"test","_id":"1"}}
{"_source":"value1"}
{"create":{"_index":"test","_id":"3"}}
{"_source":"value3"}
{"update":{"_id":"1","_index":"test"}}
{"doc":{"_source":"value2"}}
{"delete":{"_index":"test","_id":"2"}}
```

리스트 3.10 bulk API 포맷



- 'test', 'field1', 'field2', 'value1', 'value2', 'value3', '1', '2', '3'은 사용자가 입력해야 하는 값
- 'index', '_index', '_id', 'create', 'update', 'doc', 'delete'는 키워드
- Bulk API는 문서 읽기는 지원하지 않음.
- 문서 생성/수정/삭제만 지원
- 삭제를 제외한 index, create, update는 두 줄로 작성
- 각 줄 사이에는 쉼표와 같은 구분자가 없고, 라인 사이 공백을 허용하지 않는 NDJSON 형태이며 쉼표 사용에 주의해야 한다.

엘라스틱서치 기본

5 벌크 데이터

- 앞서 만들었던 index2에 2개의 도큐먼트 추가

```
POST _bulk
{"index":{"_index":"index2","_id":"4"}}
{"name":"park","age":30,"gender":"female"}
{"index":{"_index":"index2","_id":"5"}}
{"name":"jung","age":50,"gender":"male"}
```

리스트 3.11 벌크 데이터 요청



- 벌크로 2개의 데이터를 한 번에 인덱싱 한다.

> GET index2/_search

```
39 {
40   "_index" : "index2",
41   "_type" : "_doc",
42   "_id" : "4",
43   "_score" : 1.0,
44   "_source" : {
45     "name" : "park",
46     "age" : 30,
47     "gender" : "female"
48 }
```

```
51   "_index" : "index2",
52   "_type" : "_doc",
53   "_id" : "5",
54   "_score" : 1.0,
55   "_source" : {
56     "name" : "jung",
57     "age" : 50,
58     "gender" : "male"
59 }
```

엘라스틱서치 기본

6 매핑

- 엘라스틱서치에서 RDB의 스키마와 같은 역할을 하는 매핑이있다.
 - JSON형태의 데이터를 루씬이 이해할 수 있도록 바꾸는 작업
 - 엘라스틱서치가 검색엔진으로 전문 검색과 대용량 데이터를 빠르게 실시간 검색할 수 있는 이유는 매핑이 있기 때문
 - 엘라스틱서치가 자동으로 매핑하면 다이나믹 매핑, 사용자가 지정하면 명시적 매핑.

매핑

1. 다이내믹 매핑

- 엘라스틱서치의 모든 인덱스는 매핑 정보를 갖고 있지만 유연한 활용을 위해 인덱스 생성시 매핑을 강요하지 않음.
- 데이터 타입이나 스키마를 엘라스틱서치가 JSON 도큐먼트 타입에 맞춰 엘라스틱서치가 자동으로 인덱스를 매핑

표 3.4 다이내믹 매핑 기준

원본 소스 데이터 타입	다이내믹 매핑으로 변환된 데이터 타입
null	필드를 추가하지 않음
boolean	boolean
float	float
integer	long
object	object
string	string 데이터 형태에 따라 date, text/keyword 필드

```
70 GET index2/_mapping
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
00
01
02
03
04
05
06
07
08
09
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
1 {
2   "index2" : {
3     "mappings" : {
4       "properties" : {
5         "age" : {
6           "type" : "long"
7         },
8         "country" : {
9           "type" : "text",
10          "fields" : {
11            "keyword" : {
12              "type" : "keyword",
13              "ignore_above" : 256
14            }
15          }
16        },
17        "gender" : {
18          "type" : "text",
19          "fields" : {
20            "keyword" : {
21              "type" : "keyword",
22              "ignore_above" : 256
23            }
24          }
25        },
26        "name" : {
27          "type" : "text",
28          "fields" : {
29            "keyword" : {
30              "type" : "keyword",
31              "ignore_above" : 256
32            }
33          }
34        }
35      }
36    }
37  }
38 }
39
```

매핑

2. 명시적 매핑

- 인덱스 매핑을 직접 정의하는 것을 명시적 매핑이라 한다.
- 인덱스를 생성할 때 mapping 정의를 설정하거나 mapping API를 이용해 매핑을 지정할 수 있다.
 - 인덱스를 생성할 때 mappings, properties 아래 필드명과 필드 타입을 지정하면 된다.

리스트 3.15 인덱스 생성 시 매핑 설정 방법

```
PUT "인덱스명"
{
  "mappings": {
    "properties": {
      "필드명": "필드 타입"
      ...
    }
  }
}
```

매핑

2. 명시적 매핑

- 인덱스 매핑을 직접 정의하는 것을 명시적 매핑이라 한다.
- 인덱스를 생성할 때 mapping 정의를 설정하거나 mapping API를 이용해 매핑을 지정할 수 있다.
 - 인덱스를 생성할 때 mappings, properties 아래 필드명과 필드 타입을 지정하면 된다.

```
PUT index3
{
  "mappings": {
    "properties": {
      "age":{"type":"short"},
      "name":{"type":"text"},
      "gender":{"type":"keyword"}
    }
  }
}
```

리스트 3.16 직접 매핑을 지정하여 인덱스 생성

```
1 GET index2/_mapping
2 {
3   "index2": {
4     "mappings": {
5       "properties": {
6         "age": {
7           "type": "long"
8         },
9         "country": {
10          "type": "text",
11          "fields": {
12            "keyword": {
13              "type": "keyword",
14              "ignore_above": 256
15            }
16          }
17        },
18        "gender": {
19          "type": "text",
20          "fields": {
21            "keyword": {
22              "type": "keyword",
23              "ignore_above": 256
24            }
25          }
26        },
27        "name": {
28          "type": "text",
29          "fields": {
30            "keyword": {
31              "type": "keyword",
32              "ignore_above": 256
33            }
34          }
35        }
36      }
37    }
38  }
```

매핑

2. 명시적 매핑

- 인덱스 매핑이 정해지면 새로운 필드를 추가할 수 있으나, 이미 정의된 필드를 수정하거나 삭제할 수 없다.
- 필드 이름을 변경하거나 데이터 타입을 변경하기 위해서는 새로운 인덱스를 만들거나 reindex API를 이용해야 한다.
- Mappings API 에서 제공하는 파라미터는 <https://www.elastic.co/guide/en/elasticsearch/reference/7.10/mapping-params.html>를 참고

3. 매핑 타입

표 3.5 엘라스틱서치 데이터 타입

데이터 형태	데이터 타입	설명
텍스트	text	전문 검색이 필요한 데이터로 텍스트 분석기가 텍스트를 작은 단위로 분리한다.
	keyword	정렬이나 집계에 사용되는 텍스트 데이터로 분석을 하지 않고 원문을 통째로 인덱싱한다.
날짜	date	날짜/시간 데이터
정수	byte, short, integer, long	- byte: 부호 있는 8비트 데이터($-128 \sim 127$) - short: 부호 있는 16비트 데이터($-32,768 \sim 32,767$) - integer: 부호 있는 32비트 데이터($-2^{31} \sim 2^{31}-1$) - long: 부호 있는 64비트 데이터($-2^{63} \sim 2^{63}-1$)
실수	scaled_float, half_float, double, float	- scaled_float: float 데이터에 특정 값을 곱해서 정수형으로 바꾼 데이터. 정확도는 떨어지나 필요에 따라 집계 등에서 효율적으로 사용 가능하다. - half_float: 16비트 부동소수점 실수 데이터 - double: 32비트 부동소수점 실수 데이터 - float: 64비트 부동소수점 실수 데이터
불린	boolean	참/거짓 데이터로 true/false만을 값으로 갖는다.
IP 주소	ip	ipv4, ipv6 타입 IP 주소를 입력할 수 있다.
위치 정보	geo-point, geo-shape	- geo-point: 위도, 경도 값을 갖는다. - geo-shape: 하나의 위치 포인트가 아닌 임의의 지형

데이터 형태	데이터 타입	설명
범위 값	integer_range, long_range, float_range, double_range, ip_range, date_range	범위를 설정할 수 있는 데이터. integer_range, long_range는 정수형 범위, float_range, double_range는 실수형 범위, ip_range는 IP 주소 범위, date_range는 날짜/시간 데이터 범위 값을 저장하고 검색할 수 있게 한다. 최솟값과 최댓값을 통해 범위를 입력한다.
객체형	object	계층 구조를 갖는 형태로 필드 안에 다른 필드들이 들어갈 수 있다. name: { "first": "kim", "last": "tony" }로 타입을 정의하면 name.first/name.last 형태로 접근할 수 있다.
배열형	nested	배열형 객체를 저장한다. 객체를 따로 인덱싱하여 객체가 하나로 합쳐지는(flattend) 것을 막고, 배열 내부의 객체에 쿼리로 접근할 수 있다.
	join	부모(parent)/자식(child) 관계를 표현할 수 있다.

4. 멀티 필드를 활용한 문자열 처리

1. 텍스트 타입

- 엘라스틱서치에서 텍스트 타입은 일반적으로 문장을 저장하는 매핑 타입으로 사용.
- 일반적으로 문장이나 여러 단어가 나열된 문자열을 텍스트 타입으로 지정.
- 텍스트 타입으로 지정된 문자열은 분석기에 의해 토큰으로 분리되고, 토큰들은 인덱싱을 진행하게 된다 이를 **역인덱싱**이라 함
- 역인덱스에 저장된 토큰들을 용어라고 함.

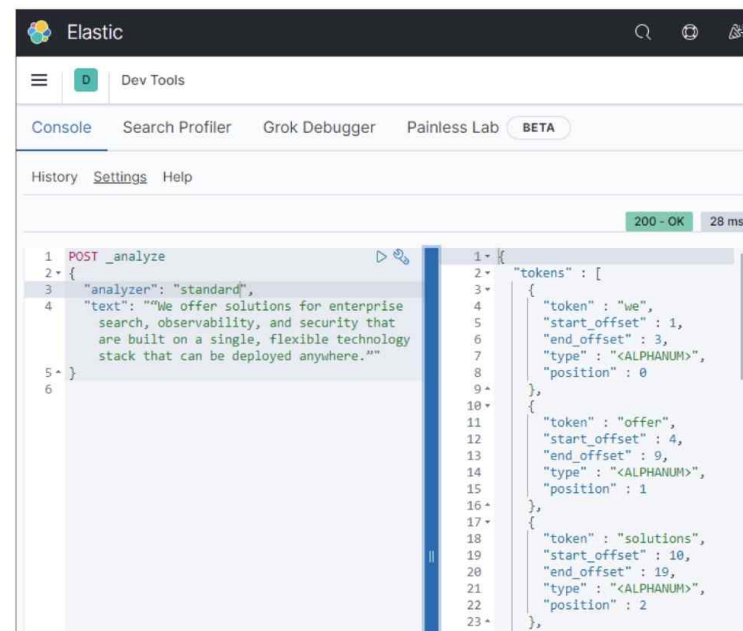


그림 3.18 텍스트 타입을 토큰으로 분리한 예

4. 멀티 필드를 활용한 문자열 처리

1. 텍스트 타입

```
PUT text_index
{
  "mappings": {
    "properties": {
      "contents": {"type": "text"}
    }
  }
}
```

리스트 3.17 텍스트 타입을 가진 text_index 인덱스 생성

```
PUT text_index/_doc/1
{
  "contents": "beautiful day"
}
```

리스트 3.18 text_index 인덱스에 documento 인덱싱

```
GET text_index/_search
{
  "query": {
    "match": {
      "contents": "day"
    }
  }
}
```

리스트 3.19 text_index 인덱스 전문 쿼리

- Match는 전문 검색을 할 수 있는 쿼리이며, contents 필드에 있는 역인덱싱 용어 중 일치하는 용어가 있는 document를 찾는 쿼리.
- 텍스트 타입은 기본적으로 집계나 정렬을 지원하지 않고, 매핑 파라미터로 집계나 정렬을 지원할 수 있으나 메모리를 많이 사용한다는 단점이 있음.

4. 멀티 필드를 활용한 문자열 처리

| 2. 키워드 타입

- 키워드 타입은 범주형 데이터에서 주로 사용.
- 키워드 타입은 분석기를 거치지 않고 문자열 전체가 하나의 용어로 인덱싱
- 완전 일치 검색과 집계, 정렬에서 사용가능.

```
PUT keyword_index
{
  "mappings": {
    "properties": {
      "contents":{"type": "keyword"}
    }
  }
}
```

리스트 3.20 키워드 타입을 가진 keyword_index 인덱스 생성

```
PUT keyword_index/_doc/1
{
  "contents":"beautiful day"
}
```

리스트 3.21 keyword_index 인덱스에 도큐먼트 인덱싱

```
GET keyword_index/_search
{
  "query":{
    "match": {
      "contents": "day"
    }
  }
}
```

리스트 3.22 keyword_index 인덱스 전문 쿼리

4. 멀티 필드를 활용한 문자열 처리

| 3. 멀티 필드

- 멀티 필드는 단일 필드 입력에 대해 여러 하위필드를 정의
- Fields라는 매핑 파라미터 사용되는데 이는 하나의 필드를 여러 용도로 사용할 수 있도록 함.
- 문자열의 경우 전문 검색이 필요하면서 정렬도 필요한 경우.
- 데이터 스키마를 잡는 시점에서 키워드 타입으로 충분했지만 데이터가 늘어나면서 전문 검색이 필요해진 경우.

4. 멀티 필드를 활용한 문자열 처리

| 3. 멀티 필드

```
PUT multifold_index
{
  "mappings": {
    "properties": {
      "message":{"type": "text"},
      "contents":{"
        "type": "text",
        "fields":{"
          "keyword":{"type":"keyword"}
        }
      }
    }
  }
}
```

리스트 3.23 멀티 필드를 갖는 multifold_index 인덱스 설정

```
PUT multifold_index/_doc/1
{
  "message":"1 document",
  "contents": "beautiful day"
}

PUT multifold_index/_doc/2
{
  "message":"2 document",
  "contents": "beautiful day"
}

PUT multifold_index/_doc/3
{
  "message":"3 document",
  "contents": "wonderful day"
}
```

리스트 3.24 multifold_index 인덱스에 도큐먼트 인덱싱

```
GET multifold_index/_search
{
  "query": {
    "match": {
      "contents": "day"
    }
  }
}
```

리스트 3.25 multifold_index 인덱스 전문 쿼리

4. 멀티 필드를 활용한 문자열 처리

| 3. 멀티 필드

```
GET multifold_index/_search
{
  "query": {
    "term": {
      "contents.keyword": "day"
    }
  }
}
```

리스트 3.26 multifold_index 인덱스 용어 쿼리



```
1 {
2   "took" : 0,
3   "timed_out" : false,
4   "_shards" : {
5     "total" : 1,
6     "successful" : 1,
7     "skipped" : 0,
8     "failed" : 0
9   },
10  "hits" : {
11    "total" : {
12      "value" : 0,
13      "relation" : "eq"
14    },
15    "max_score" : null,
16    "hits" : [ ]
17  }
18 }
19
```

- Contents.keyword 타입은 키워드 타입임으로 용어분리가 안됨.
- Beautiful day 또는 wonderful day로 검색해야 함.

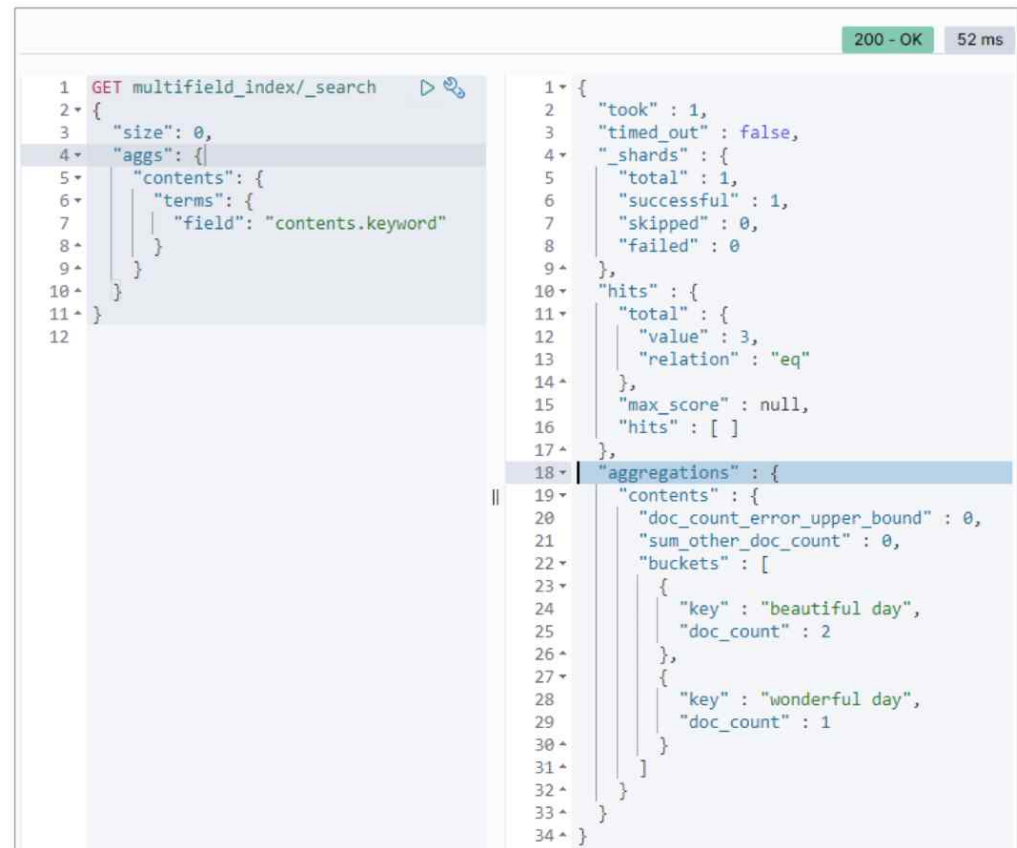
4. 멀티 필드를 활용한 문자열 처리

3. 멀티 필드

GET multifield_index/_search

```
{
  "size":0,
  "aggs": {
    "contents": {
      "terms": {
        "field": "contents.keyword"
      }
    }
  }
}
```

리스트 3.26 multifield_index 인덱스 집계 쿼리



```
1 GET multifield_index/_search
2 {
3   "size": 0,
4   "aggs": {
5     "contents": {
6       "terms": {
7         "field": "contents.keyword"
8       }
9     }
10  }
11 }
12
```

```
1 {
2   "took" : 1,
3   "timed_out" : false,
4   "_shards" : {
5     "total" : 1,
6     "successful" : 1,
7     "skipped" : 0,
8     "failed" : 0
9   },
10  "hits" : {
11    "total" : {
12      "value" : 3,
13      "relation" : "eq"
14    },
15    "max_score" : null,
16    "hits" : [ ]
17  },
18  "aggregations" : {
19    "contents" : {
20      "doc_count_error_upper_bound" : 0,
21      "sum_other_doc_count" : 0,
22      "buckets" : [
23        {
24          "key" : "beautiful day",
25          "doc_count" : 2
26        },
27        {
28          "key" : "wonderful day",
29          "doc_count" : 1
30        }
31      ]
32    }
33  }
34 }
```

200 - OK 52 ms

그림 3.21 집계 쿼리 결과

엘라스틱서치 기본

7 인덱스 템플릿

- 인덱스 템플릿은 주로 설정인 동일한 복수의 인덱스를 만들 때 사용.
- 관리 편의성, 성능 등을 위해 인덱스를 파티셔닝하는 일이 많은데 이때 파티셔닝되는 인덱스들은 같아야 한다.

1. 템플릿 확인

- 전체 인덱스 템플릿 검색: > **GET _index_template**
- 특정 인덱스 템플릿 확인:
 - > **GET _index_template/ilm-history**
 - > **GET _index_template/ilm***



```
1 GET _index_template/ilm* 200 - OK 32 ms
1 {
2   "index_templates" : [
3     {
4       "name" : "ilm-history",
5       "index_template" : {
6         "index_patterns" : ["/"],
7         "template" : {},
8         "composed_of" : [ ],
9         "priority" : 2147483647,
10        "version" : 3,
11        "_meta" : {}
12      }
13    ]
14  }
```

그림 3.22 인덱스 템플릿 확인하기

인덱스 템플릿

2. 템플릿 설정

- 인덱스 템플릿을 생성할 때 다양한 설정을 할 수 있지만, 일반적으로 매핑과 세팅 설정을 가장 많이 함.

1. 템플릿 생성

```
PUT _index_template/test_template
{
  "index_patterns": ["test_*"],
  "priority": 1,
  "template": {
    "settings": {
      "number_of_shards": 3,
      "number_of_replicas": 1
    },

```

```
"mappings": {
  "properties": {
    "name": {"type": "text"},
    "age": {"type": "short"},
    "gender": {"type": "keyword"}
  }
}
}
```

표 3.6 자주 사용하는 템플릿 파라미터 종류

파라미터	설명
index_patterns	새로 만들어지는 인덱스 중에 인덱스 이름이 인덱스 패턴과 매칭되는 경우 이 템플릿이 적용된다. 여기서는 test_로 시작되는 이름을 가진 인덱스들은 모두 test_template에 있는 매핑, 세팅 등이 적용된다.
priority	인덱스 생성 시 이름에 매칭되는 템플릿이 둘 이상일 때 템플릿이 적용되는 우선순위를 정할 수 있다. 숫자가 가장 높은 템플릿이 먼저 적용된다.
template	새로 생성되는 인덱스에 적용되는 settings, mappings 같은 인덱스 설정을 정의한다.

리스트 3.28 test_template 인덱스 템플릿 설정

인덱스 템플릿

2. 템플릿 설정

2. 템플릿 적용

- 템플릿을 만들기전 이미 존재하던 인덱스는 새로 만든 템플릿 패턴과 일치하더라도 템플릿이 적용되지 않음.
- 템플릿을 만든 이후에 새로 만들어지는 인덱스들만 템플릿 영향을 받음.

```
PUT test_index1/_doc/1
{
  "name":"kim",
  "age":10,
  "gender":"male"
}
```

리스트 3.29 test_template 템플릿이 적용된 test_index1 인덱스 생성

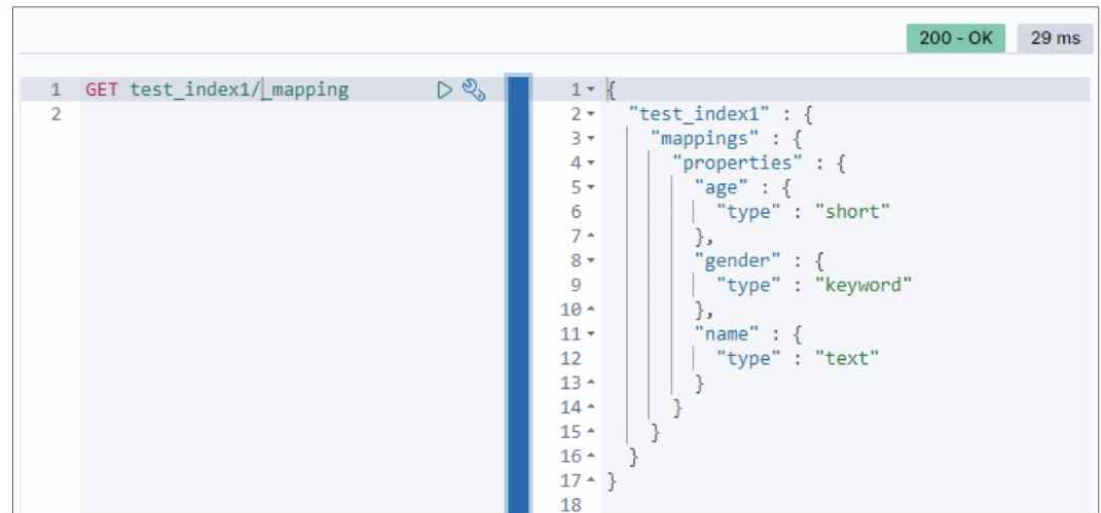


그림 3.23 test_template 템플릿이 적용된 test_index1 인덱스 확인

인덱스 템플릿

2. 템플릿 설정

2. 템플릿 적용

```
PUT train_index1/_doc/1
{
  "name":"kim",
  "age":10,
  "gender":"male"
}
```

리스트 3.30 test_template 템플릿이 적용되지 않은 train_index1 인덱스 생성

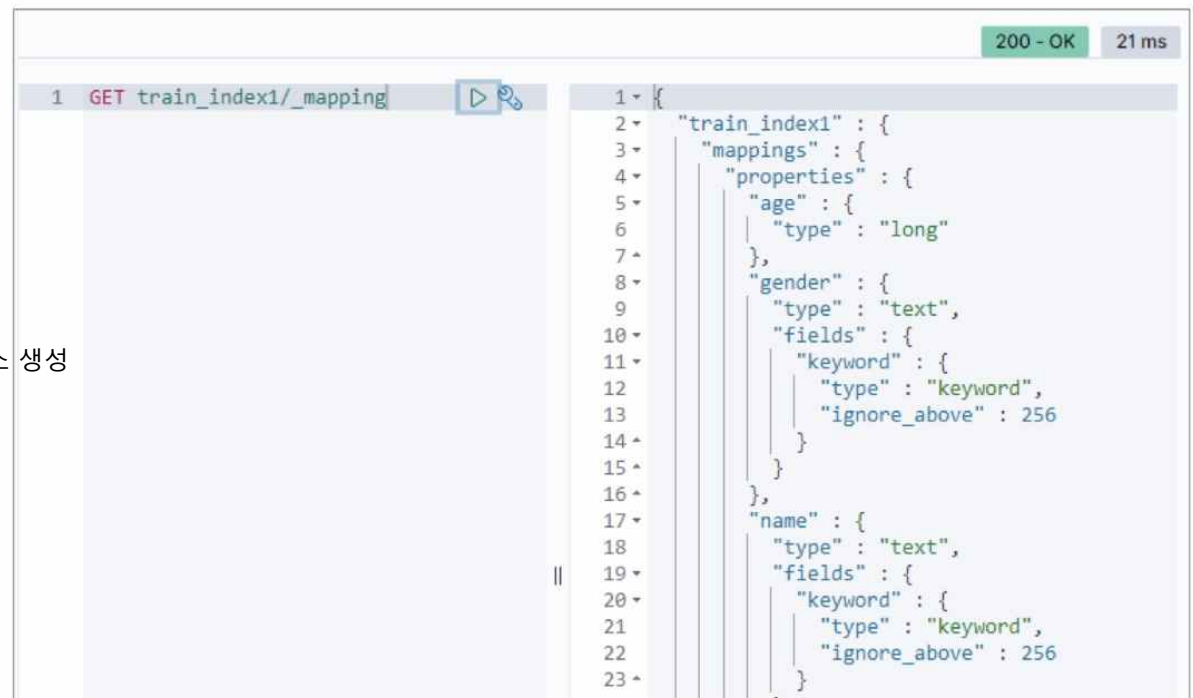


그림 3.24 test_template 템플릿이 적용되지 않은 train_index1 인덱스 확인

인덱스 템플릿

2. 템플릿 설정

2. 템플릿 적용

- 템플릿에 작성된 매핑 타입과 다르게 입력할 경우 오류발생

```
PUT train_index1/_doc/1
{
  "name": "lee",
  "age": "19 years"
}
```

리스트 3.31 템플릿 매핑값과 다른 문서 인덱싱



그림 3.25 test_template 템플릿과 매핑되지 않는 문서가 인덱싱되는 경우

인덱스 템플릿

2. 템플릿 설정

| 3. 템플릿 삭제

- 템플릿에 삭제 시 기존인덱스들을 영향을 받지 않음.
- DELETE 메소드를 사용해 삭제
 - > `DELETE _index_template/test_template`

인덱스 템플릿

3. 템플릿 우선순위

- **Priority**를 이용해 복수의 템플릿이 매칭될 경우 우선순위를 정할 수 있음.
숫자가 클 수록 우선순위가 높음.

```
PUT _index_template/multi_template1
{
  "index_patterns":"multi_*",
  "priority":1,
  "template":{
    "mappings":{
      "properties":{
        "age":{"type":"integer"},
        "name":{"type":"text"}
      }
    }
  }
}
```

리스트 3.32 multi_* 패턴을 가진 multi_template1 템플릿 생성

```
PUT _index_template/multi_template2
{
  "index_patterns":"multi_data_*",
  "priority":2,
  "template":{
    "mappings":{
      "properties":{
        "name":{"type":"keyword"}
      }
    }
  }
}
```

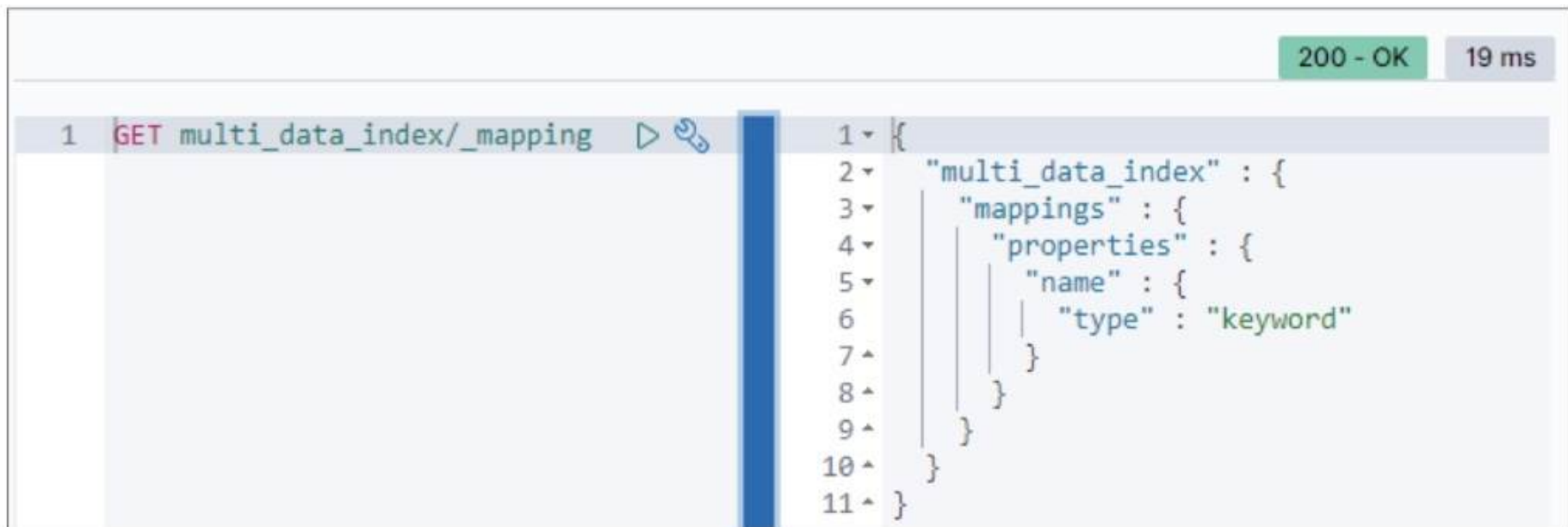
리스트 3.33 multi_data* 패턴을 가진 multi_template2 템플릿 생성

인덱스 생성

> **PUT multi_data_index**

인덱스 템플릿

3. 템플릿 우선순위



The screenshot shows a REST client interface. At the top right, there is a status bar indicating a 200 - OK response and a 19 ms execution time. The main area is divided into two panes. The left pane shows the HTTP request: `1 GET multi_data_index/_mapping`. The right pane shows the JSON response, which is a mapping definition for the `multi_data_index`. The response is a JSON object with the following structure:

```
1 {  
2   "multi_data_index" : {  
3     "mappings" : {  
4       "properties" : {  
5         "name" : {  
6           "type" : "keyword"  
7         }  
8       }  
9     }  
10  }  
11 }
```

그림 3.26 멀티 템플릿이 적용된 경우의 결과

인덱스 템플릿

4. 다이내믹 템플릿

- 매핑을 다이내믹하게 저장
- 다이내믹 템플릿은 매핑을 정확하게 정할 수 없거나 대략적인 데이터 구조만 알고 있을 때 사용.
- 인덱스를 만들때 `dynamic_templates`를 추가하면 됨.

```
PUT dynamic_index1
{
  "mappings": {
    "dynamic_templates":[
      {
        "my_string_fields":{
          "match_mapping_type":"string",
          "mapping":{"type":"keyword"}
        }
      }
    ]
  }
}
```

리스트 3.34 다이내믹 매핑을 적용한 인덱스 생성

인덱스 템플릿

4. 다이내믹 템플릿

- 다이내믹 매핑이 적용된 `dynamic_index1` 인덱스에 문서 인덱싱

```
PUT dynamic_index1/_doc/1
{
  "name":"mr.kim",
  "age":40
}
```

리스트 3.35 다이내믹 매핑이 적용된 `dynamic_index1` 인덱스에 문서 인덱싱

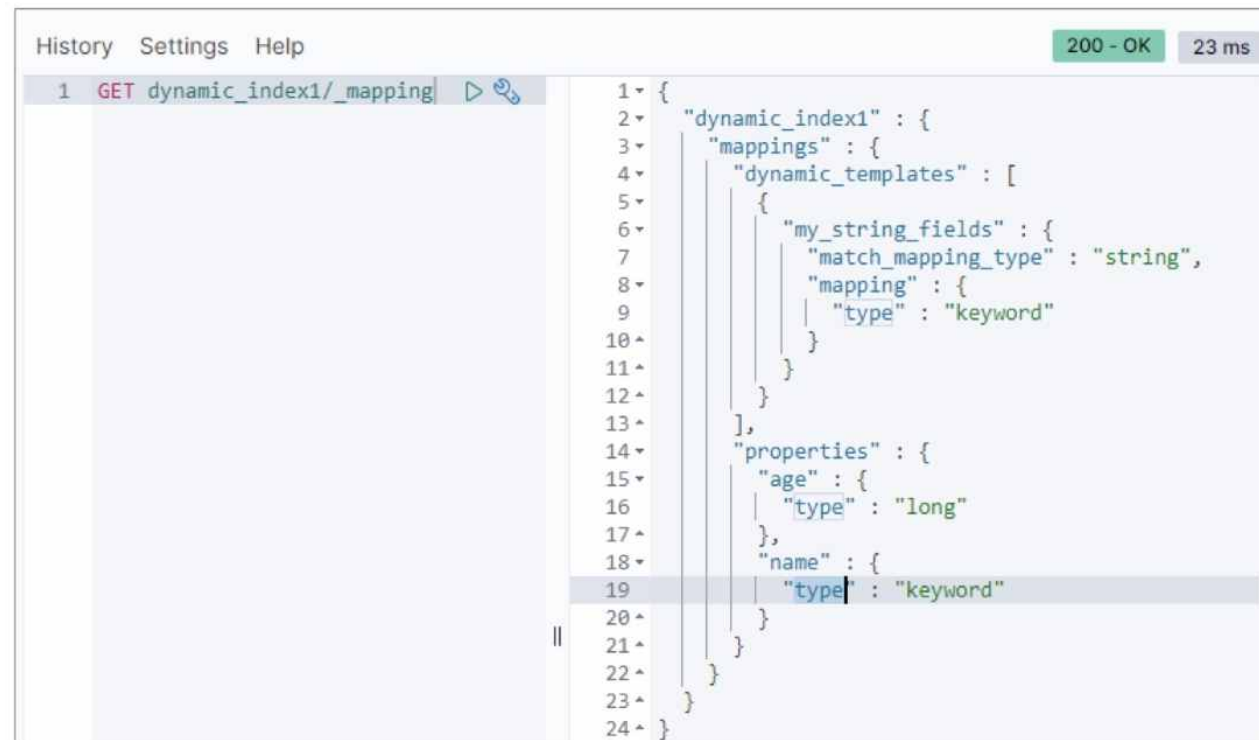


그림 3.27 다이내믹 템플릿이 적용된 결과

인덱스 템플릿

4. 다이내믹 템플릿

- Match, unmatched 조건문을 활용한 다이내믹 템플릿

```
PUT dynamic_index2
{
  "mappings": {
    "dynamic_templates": [
      {
        "my_long_fields": {
          "match": "long_*",
          "unmatch": "*_text",
          "mapping": { "type": "long" }
        }
      ]
    }
  }
}
```

리스트 3.36 다이내믹 매핑을 적용한 인덱스 생성

```
PUT dynamic_index2/_doc/1
{
  "long_num": "5",
  "long_text": "170"
}
```

리스트 3.37 다이내믹 매핑이 적용된 dynamic

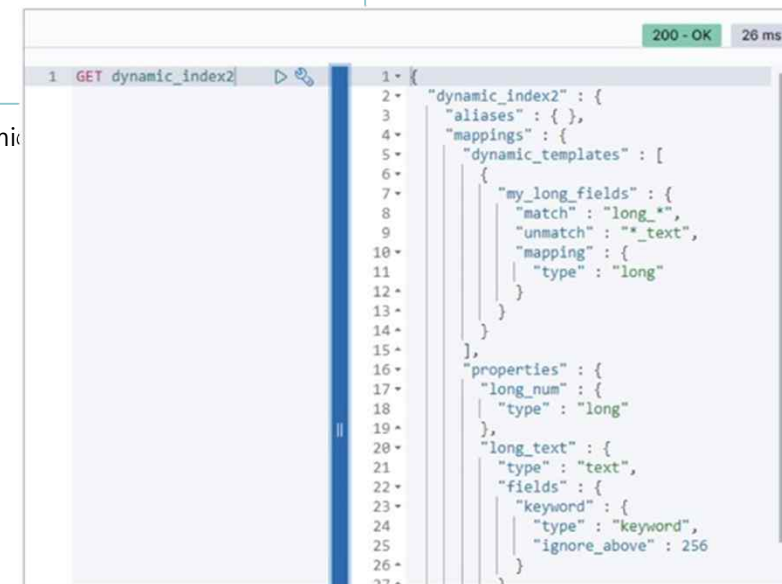


그림 3.28 다이내믹 템플릿이 적용된 결과

인덱스 템플릿

4. 다이내믹 템플릿

- Match, unmatched 조건문을 활용한 다이내믹 템플릿

표 3.7 다이내믹 템플릿 조건문

조건문	설명
match_mapping_type	데이터 타입을 확인하고 타입(boolean, date, double, long, object, string)들 중 일부를 지정한 매핑 타입으로 변경한다.
match, unmatched	match: 필드명이 패턴과 일치할 경우 매핑 타입으로 변경한다. unmatch: match 패턴과 일치하는 경우에 제외할 패턴을 설정할 수 있다.
match_pattern	match 패턴에서 사용할 수 있는 파라미터를 조정한다. 예를 들어, 정규식(regex)이나 와일드 패턴 등을 지정한다.
path_match, path_unmatch	match, unmatched와 비슷하지만 점(.)이 들어가는 필드명에서 사용한다.

엘라스틱서치 기본

8 분석기

- 엘라스틱서치는 전문검색을 위해 역인덱싱을 활용.
- 역인덱싱은 문자열을 분석해 작은 단위로 쪼개어 인덱싱 하는 기술
- 다음 그림 3.39는 엘라스틱서치의 분석기 모듈 구조.



그림 3.29 분석기 구성

표 3.8 분석기 구성요소

구성요소	설명
캐릭터 필터	입력받은 문자열을 변경하거나 불필요한 문자들을 제거한다.
토크나이저	문자열을 토큰으로 분리한다. 분리할 때 토큰의 순서나 시작, 끝 위치도 기록한다.
토큰 필터	분리된 토큰들의 필터 작업을 한다. 대소문자 구분, 형태소 분석 등의 작업이 가능하다.

엘라스틱서치 기본

8 분석기

참고 토큰과 용어

앞으로 토큰(token)과 용어(term)라는 단어가 많이 사용되는데 두 단어가 헷갈리기 쉽다. 간단히 토큰과 용어를 정의하고 넘어가자.

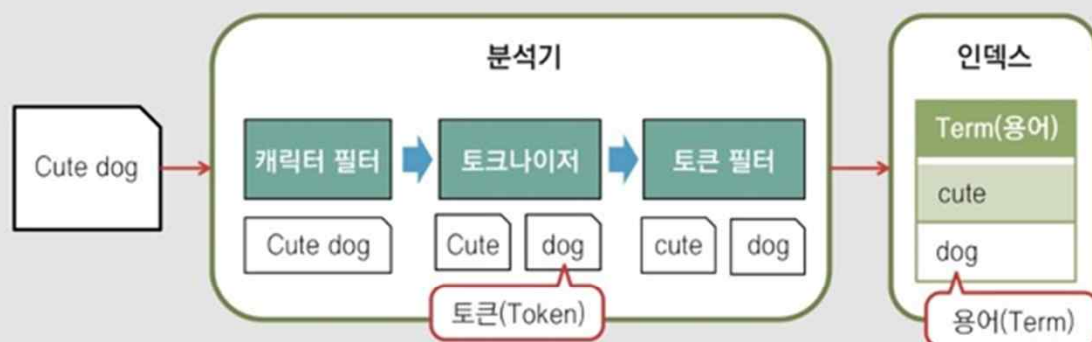


그림 3.30 토큰과 용어

'Cute dog'라는 문자열이 분석기를 거쳐 인덱스에 저장된다고 가정해보자(그림 3.30 참고). 분석기는 먼저 캐릭터 필터를 통해 원문에서 불필요한 문자들을 제거한다. 이 과정까지는 문자열 자체가 분리되지 않기 때문에 그저 필터링된 문자열 정도로 볼 수 있다. 이후 분석기는 토크나이저를 이용해 필터링된 문자열을 자르게 되는데, 이때 잘린 단위를 토큰이라고 지칭한다. 이러한 토큰들은 복수의 토큰 필터를 거치며 정제되는데, 정제 후 최종으로 역 인덱스에 저장되는 상태의 토큰들을 용어라고 한다. 토큰은 분석기 내부에서 일시적으로 존재하는 상태이고, 인덱싱되어 있는 단위, 또 검색에 사용되는 단위는 모두 용어라고 할 수 있다.

분석기

1. 분석기 구성

1. 역인덱싱

- 분석기는 문자열을 토큰화하고 이를 인덱싱하는데 이를 역인덱싱이라 한다.

찾아보기	
가	나
가상 머신 461	라시드 칸 37
가이로스 51	라우팅 512
가중치 161	라이선스 73
개발 모드 548	레플리카 542
검색 결과 내에서의 집계 191	레플리카 샤드 513
검색 엔진 38	로그 레벨 변경 534
고가용성 481, 514	로그스테시 37, 41, 213
고 프로그래밍 언어 43	로그 내 46
공공데이터 포털 421	로드밸런싱 498
관계형 데이터베이스 54	루비 필터 플러그인 381
관측성 271	루신 35, 39
그라파나 58	리눅스 패키지 매니저 472
그리드(맵스) 355	리인덱싱 411
	리포지토리 507
	립비트 271
다	
날짜/시간 데이터 타입 163	
날짜 히스토그램 집계 324	
노드 481	마
	마스터 노드 483

그림 3.31 책의 색인



그림 3.32 엘라스틱서치의 역인덱싱 과정

분석기

1. 분석기 구성

| 2. 분석기 API

- 엘라스틱서치는 필터와 토크나이즈를 테스트해볼 수 있는 analyze라는 이름의 REST API를 제공.

```
POST _analyze
{
  "analyzer": "stop",
  "text": "The 10 most loving dog breeds"
}
```

리스트 3.38 스톱분석기 테스트

리스트 3.39 스톱 분석기 결과

```
{
  "tokens" : [
    { "token" : "most", ... },
    { "token" : "loving", ... },
    { "token" : "dog", ... },
    { "token" : "breeds", ... }
  ]
}
```

분석기

1. 분석기 구성

3. 분석기 종류

- 엘라스틱서치는 다양한 분석기를 제공.
- 스톱(stop), 스탠다드(standard), 심플(simple), 공백문자(whitespace) 분석기 등.

표 3.9 자주 사용되는 분석기

분석기	설명
standard	특별한 설정이 없으면 엘라스틱서치가 기본적으로 사용하는 분석기다. 영문법을 기준으로 한 스탠다드 토큰라이저와 소문자 변경 필터, 스톱 필터가 포함되어 있다(소문자 변경 필터와 스톱 필터는 3.8.3절 '필터'에서 다시 다루겠다). [the, 10, most, loving, dog, breeds]
simple	문자만 토큰화한다. 공백, 숫자, 하이픈(-)이나 작은따옴표(') 같은 문자는 토큰화하지 않는다. [the, most, loving, dog, breeds]
whitespace	공백을 기준으로 구분하여 토큰화한다. [The, 10, most, loving, dog, breeds.]
stop	simple 분석기와 비슷하지만 스톱 필터가 포함되었다. 스톱 필터에 의해 'the'가 제거되었다. [most, loving, dog, breeds]

분석기

2. 토큰나이저

- 토큰나이저는 문자열을 분리해 토큰화하는 역할.
- 분석기에 반드시 포함되어야 함.

표 3.10 대표적인 토큰나이저의 예

토큰나이저	설명
standard	스탠다드 분석기가 사용하는 토큰나이저로, 특별한 설정이 없으면 기본 토큰나이저로 사용된다. 쉼표(.)나 점(.) 같은 기호를 제거하며 텍스트 기반으로 토큰화한다.
lowercase	텍스트 기반으로 토큰화하며 모든 문자를 소문자로 변경해 토큰화한다.
ngram	원문으로부터 N 개의 연속된 글자 단위를 모두 토큰화한다. 예를 들어, '엘라스틱서치'라는 원문을 2gram으로 토큰화한다면 [엘라, 라스, 스틱, 틱서, 서치]와 같이 연속된 두 글자를 모두 추출한다. 사실상 원문으로부터 검색할 수 있는 거의 모든 조합을 얻어낼 수 있기 때문에 정밀한 부분 검색에 강점이 있지만, 토큰나이징을 수행한 N 개 이하의 글자 수로는 검색이 불가능하며 모든 조합을 추출하기 때문에 저장공간을 많이 차지한다는 단점이 있다.
uax_url_email	스탠다드 분석기와 비슷하지만 URL이나 이메일을 토큰화하는 데 강점이 있다.

분석기

2. 토큰나이저

- 토큰나이저는 문자열을 분리해 토큰화하는 역할.
- 분석기에 반드시 포함되어야 함.

```
POST _analyze
{
  "tokenizer": "uax_url_email",
  "text": "email: elastic@elk-company.com"
}
```

리스트 3.40 uax_url_email 토큰나이저 테스트

리스트 3.41 uax_url_email 토큰나이저 결과

```
{
  "tokens" : [
    { "token" : "email", ... },
    { "token" : "elastic@elk-company.com", ... }
  ]
}
```

분석기

3. 필터

- 분석기는 하나의 토큰라이저와 다수의 필터가 있음
- 분석기에서 필터를 옵션으로 하나 이상을 포함할 수 있지만 없어도 분석기는 실행됨.

```
POST _analyze
{
  "tokenizer": "standard",
  "filter": ["uppercase"],
  "text": "The 10 most loving dog breeds."
}
```

리스트 3.42 uppercase 필터 테스트

리스트 3.43 uppercase 필터 결과

```
{
  "tokens" : [
    { "token" : "THE", ... },
    { "token" : "10", ... },
    { "token" : "MOST", ... },
    { "token" : "LOVING", ... },
    { "token" : "DOG", ... },
    { "token" : "BREEDS", ... }
  ]
}
```

분석기

3. 필터

1. 캐릭터 필터

- 캐릭터 필터는 토크나이저 전에 위치하며 문자들을 전처리.
- HTML 문법을 제거/변경하거나 특정 문자가 왔을 때 다른 문자로 대체()

2. 토큰 필터

- 토큰화되어있는 문자들에 필터를 적용.
- 토큰들을 변경하거나 삭제하고 추가하는 작업 가능.

표 3.11 자주 사용하는 토큰 필터

필터	설명
lowercase	모든 문자를 소문자로 변환한다. 반대로 모든 문자를 대문자로 변환하는 uppercase 필터가 있다.
stemmer	영어 문법을 분석하는 필터다. 언어마다 고유한 문법이 있어서 필터 하나로 모든 언어에 대응하기는 힘들다. 한글의 경우 아리랑, 노리 같은 오픈소스가 있다. 10.4절에서 한글 형태소 분석기를 사용해 보겠다.
stop	기본 필터에서 제거하지 못하는 특정한 단어를 제거할 수 있다.

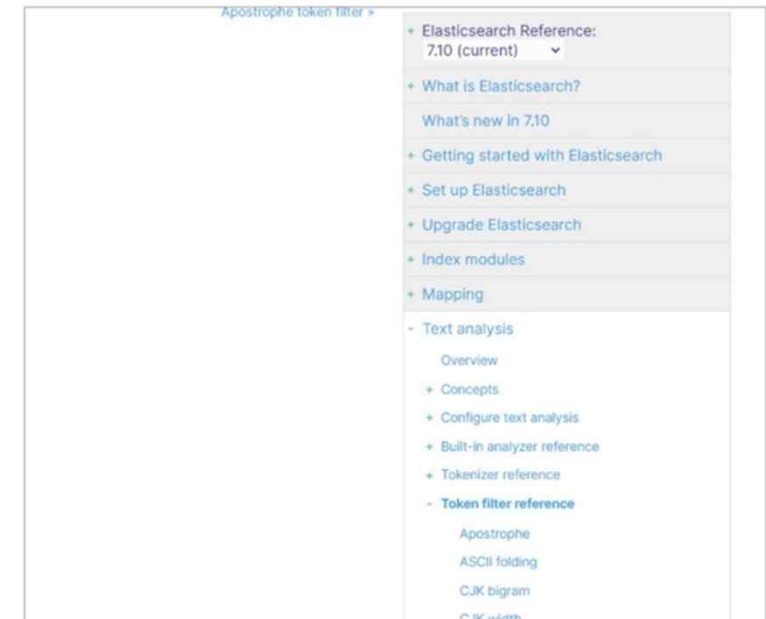


그림 3.33 온라인 문서에서 토큰 필터 찾기

분석기

3. 필터

2. 토큰 필터



그림 3.34 lowercase 필터 동작

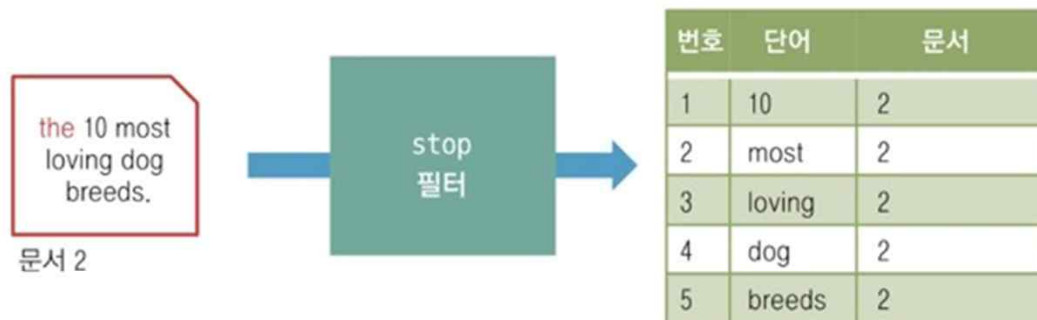


그림 3.35 stop 필터 동작

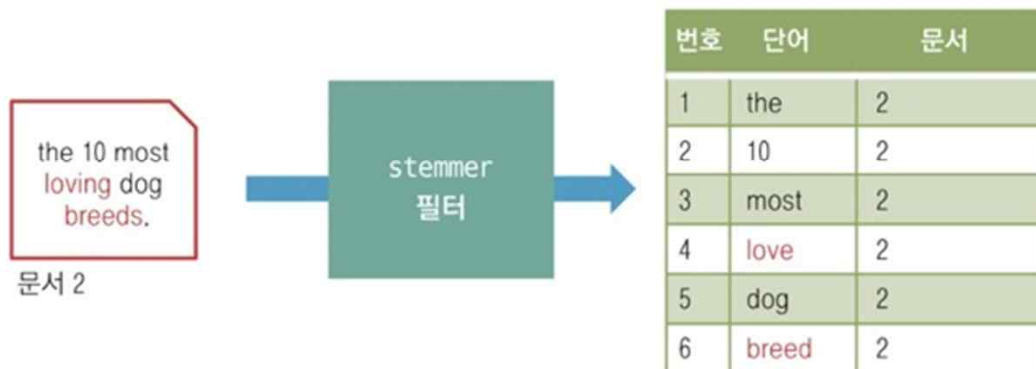


그림 3.36 stemmer 필터 동작

분석기

4. 커스텀 분석기

- 엘라스틱서치에서 기본적으로 제공하는 분석기 중 원하는 분석기가 없을 경우 사용자가 직접 만들어 사용

| 1. 커스텀 분석기 설정

```
PUT customer_analyzer
{
  "settings": {
    "analysis": {
      "filter": {
        "my_stopwords": {
          "type": "stop",
          "stopwords": ["lions"]
        }
      }
    }
  },
}
```

리스트 3.44 커스텀 분석기를 적용한 customer_analyzer인덱스 생성

```
"analyzer": {
  "my_analyzer": {
    "type": "custom",
    "char_filter": [],
    "tokenizer": "standard",
    "filter": ["lowercase", "my_stopwords"]
  }
}
```

분석기

4. 커스텀 분석기

| 1. 커스텀 분석기 설정

```
GET customer_analyzer/_analyze
{
  "analyzer": "my_analyzer",
  "text": "Cats Lions Dogs"
}
```

리스트 3.45 my_analyzer 커스텀 분석기 테스트

리스트 3.46 my_analyzer 커스텀 분석기 결과

```
{
  "tokens" : [
    { "token" : "cats", ... },
    { "token" : "dogs", ... }
  ]
}
```

분석기

4. 커스텀 분석기

| 2. 필터 적용 순서

- 여러 개의 필터를 사용한다면 필터의 순서에도 주의를 해야함.
- 배열의 순차적으로 수행

```
GET customer_analyzer/_analyze
{
  "tokenizer": "standard",
  "filter": ["lowercase", "my_stopwords"],
  "text": "Cats Lions Dogs"
}
```

리스트 3.47 lowercase, my_stopwords 순서로 필터를 적용하여 분석기 테스트

```
GET customer_analyzer/_analyze
{
  "tokenizer": "standard",
  "filter": ["my_stopwords", "lowercase"],
  "text": "Cats Lions Dogs"
}
```

리스트 3.49 my_stopwords, lowercase 순서로 필터를 적용하여 분석기 테스트



리스트 3.48 lowercase, my_stopwords 순서로 필터를 적용한 분석기 결과

```
{
  "tokens" : [
    { "token" : "cats", ... },
    { "token" : "dogs", ... }
  ]
}
```



리스트 3.50 my_stopwords, lowercase 순서로 필터를 적용한 분석기 결과

```
{
  "tokens" : [
    { "token" : "cats", ... },
    { "token" : "lions", ... },
    { "token" : "dogs", ... }
  ]
}
```



03

엘라스틱서치 검색

엘라스틱서치: 검색

1 쿼리 컨텍스트와 필터 컨텍스트

- 쿼리 컨텍스트: 질의에 대한 유사도를 계산해 이를 기준으로 더 정확한 결과 출력
- 필터 컨텍스트: 유사도를 계산하지 않고 일치 여부에 따른 결과만을 출력

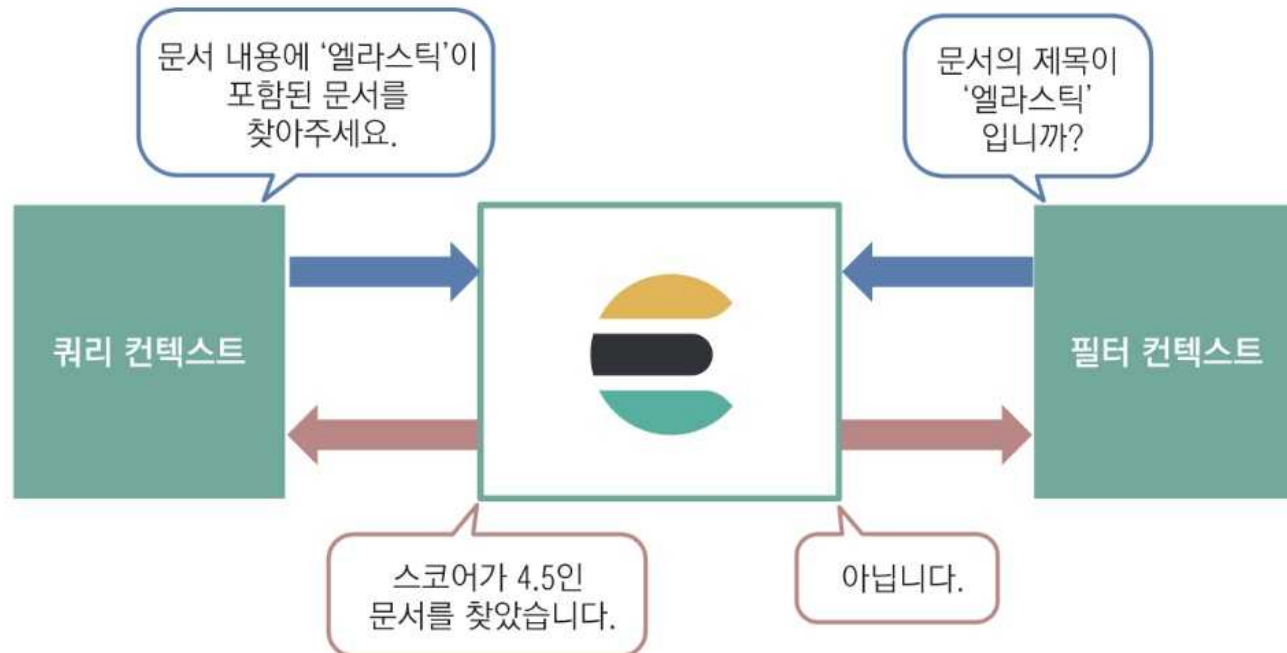
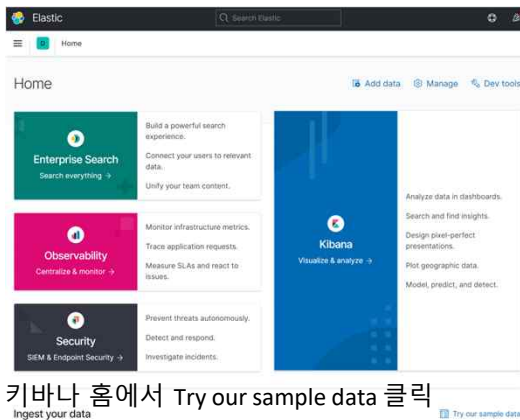


그림 4.1 쿼리와 필터 검색의 차이점

엘라스틱서치: 검색

• 쿼리 컨텍스트 실행



키바나 홈에서 Try our sample data 클릭

GET kibana_sample_data_ecommerce/_search

```
{
  "query": {
    "match": {
      "category": "clothing"
    }
  }
}
```

리스트 4.1 쿼리 컨텍스트 실행

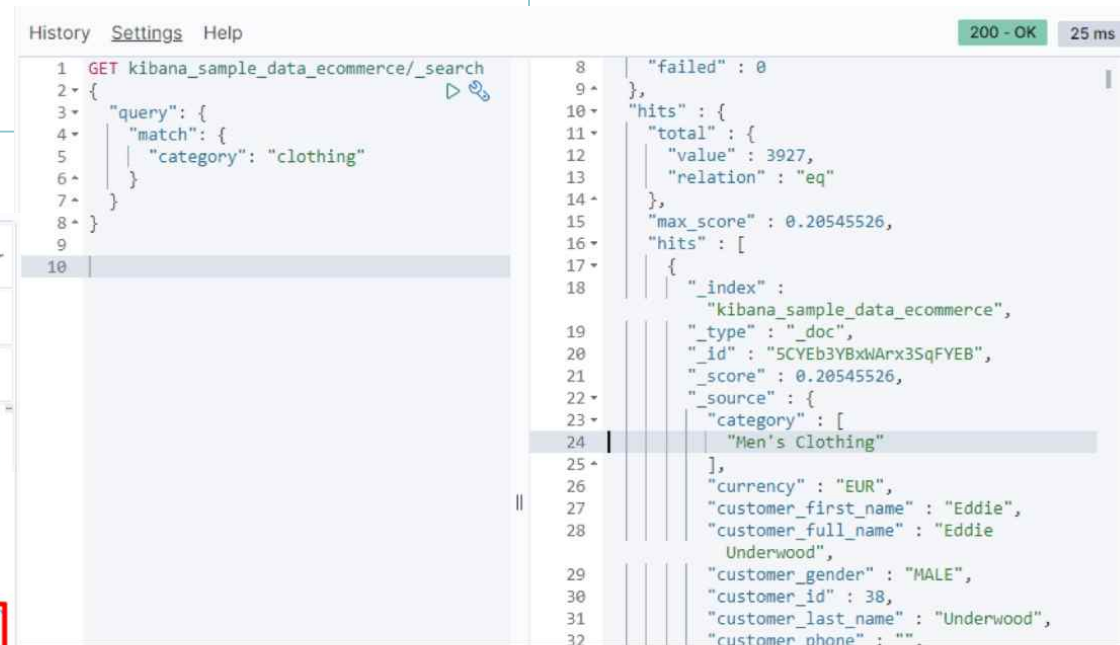


그림 4.2 쿼리 컨텍스트의 응답 결과

Sample eCommerce orders와 Sample flight data의 Add data 클릭

엘라스틱서치: 검색

필터 컨텍스트 실행

GET kibana_sample_data_ecommerce/_search

```
{
  "query": {
    "bool": {
      "filter": {
        "term": {
          "day_of_week": "Friday"
        }
      }
    }
  }
}
```

리스트 4.2 필터 컨텍스트 실행

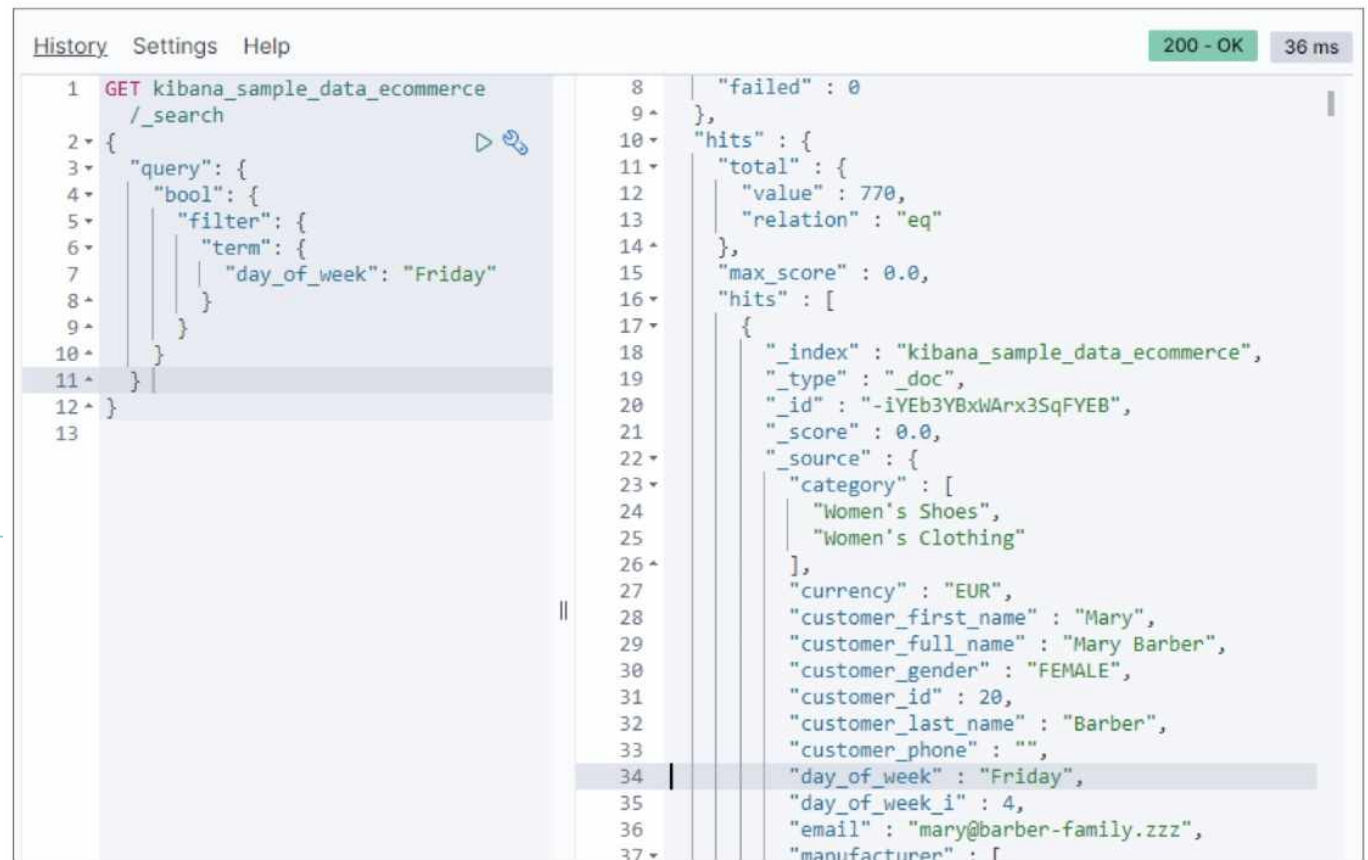


그림 4.3 필터 컨텍스트의 응답 결과

엘라스틱서치: 검색

2 쿼리 스트링과 쿼리 DSL

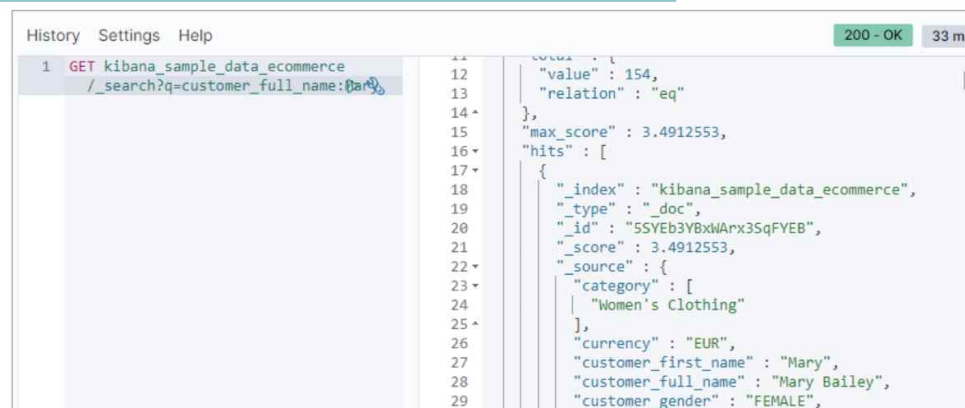
- 쿼리 스트링: 한 줄 정도의 간단한 쿼리에서 사용.
- 쿼리 DSL: 한 줄에 넣기 힘든 복잡한 쿼리에 사용.

1. 쿼리 스트링

- REST API의 URI 주소에 쿼리문을 작성하는 방식으로 사용하기 쉽다.
- 복잡한 논리 조건인 경우 괄호를 사용하는데 가독성이 좋지 않아 오류발생 가능성이 높음.

GET kibana_sample_data_ecommerce/_search?q=customer_full_name:Mary

리스트 4.3 쿼리 스트링 예시



```
History Settings Help
1 GET kibana_sample_data_ecommerce/_search?q=customer_full_name:Mary
200 - OK 33 ms

{
  "value": 154,
  "relation": "eq",
  "max_score": 3.4912553,
  "hits": [
    {
      "_index": "kibana_sample_data_ecommerce",
      "_type": "_doc",
      "_id": "5SYEb3Y8xWArx3SqFYEB",
      "_score": 3.4912553,
      "_source": {
        "category": [
          "Women's Clothing"
        ],
        "currency": "EUR",
        "customer_first_name": "Mary",
        "customer_full_name": "Mary Bailey",
        "customer_gender": "FEMALE",

```

그림 4.4 쿼리 스트링 결과

쿼리 스트링과 쿼리 DSL

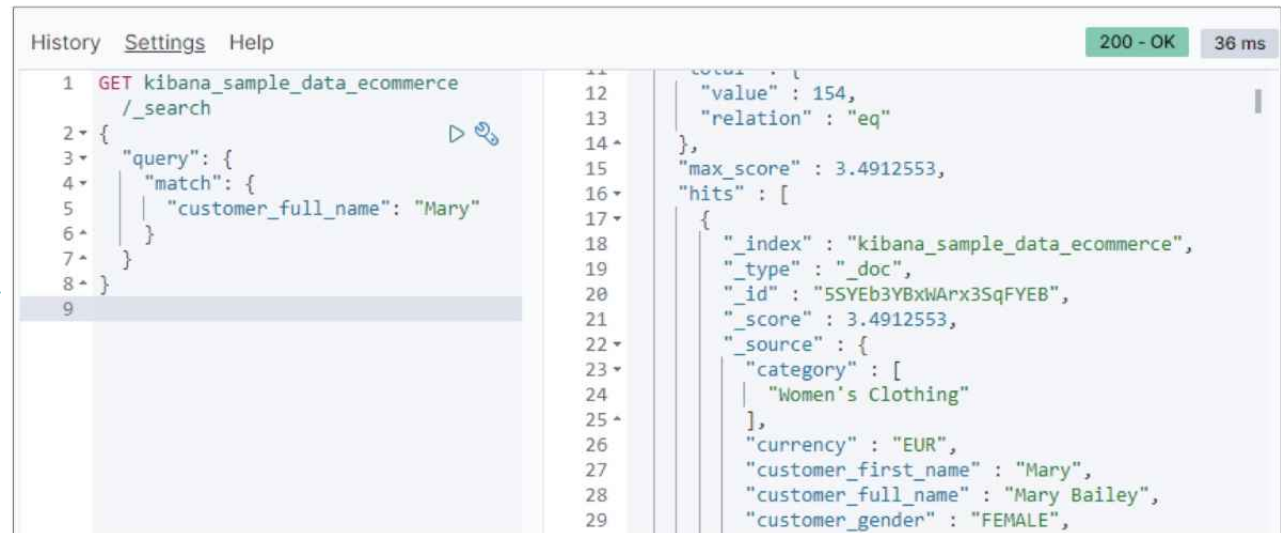
2. 쿼리 DSL

- 쿼리 DSL은 REST API의 요청 본문 안에 JSON 형태로 쿼리를 입력.
- 복잡한 쿼리를 구현하는데 용이.

GET kibana_sample_data_ecommerce/_search

```
{
  "query": {
    "match": {
      "customer_full_name": "Mary"
    }
  }
}
```

리스트 4.4 쿼리 DSL 예시



The screenshot shows the Kibana search interface. The left pane displays the query: `GET kibana_sample_data_ecommerce/_search` with a JSON body: `{ "query": { "match": { "customer_full_name": "Mary" } } }`. The right pane shows the results in a table format. The first result is a document from the `kibana_sample_data_ecommerce` index with ID `5SYEb3YBxWArx3SqFVEB`. It has a score of `3.4912553` and a source object containing `category: "Women's Clothing", currency: "EUR", customer_first_name: "Mary", customer_full_name: "Mary Bailey", customer_gender: "FEMALE"`.

```
1 GET kibana_sample_data_ecommerce
2 /_search
3 {
4   "query": {
5     "match": {
6       "customer_full_name": "Mary"
7     }
8   }
9 }
11
12   "value" : 154,
13   "relation" : "eq"
14 },
15   "max_score" : 3.4912553,
16   "hits" : [
17     {
18       "_index" : "kibana_sample_data_ecommerce",
19       "_type" : "_doc",
20       "_id" : "5SYEb3YBxWArx3SqFVEB",
21       "_score" : 3.4912553,
22       "_source" : {
23         "category" : [
24           "Women's Clothing"
25         ],
26         "currency" : "EUR",
27         "customer_first_name" : "Mary",
28         "customer_full_name" : "Mary Bailey",
29         "customer_gender" : "FEMALE",
```

그림 4.5 쿼리 DSL 결과

엘라스틱서치: 검색

3 유사도 스코어

- 쿼리 컨텍스트는 엘라스틱서치에서 제공하는 다양한 유사도 스코어 알고리즘을 사용.
- 기본적으로 BM25 알고리즘 사용.
- 유사도 스코어는 질문과 도큐먼트의 유사도를 표현하는 값으로 스코어가 높을수록 찾고자 하는 도큐먼트에 가깝다는 사실을 의미.
- 쿼리를 요청하고 스코어가 어떤 식으로 계산되었는지 알기 위해 `explain` 옵션 사용.

```
GET kibana_sample_data_ecommerce/_search
{
  "query": {
    "match": {
      "products.product_name": "Pants"
    }
  },
  "explain": true
}
```

리스트 4.5 설명이 포함된 쿼리 컨텍스트 실행

유사도 스코어

리스트 4.6 설명이 추가된 리스트 4.5의 응답 결과

```
{
  ...
  "hits" : {
    "total" : {
      "value" : 3,
      "relation" : "eq"
    },
    "max_score" : 8.268259,
    "hits" : [
      {
        "_id" : "NiYeb3YBxWArx3SqGIQX",
        "_score" : 8.268259,
        "_source" : {
          "products" : [
            {
              "product_name" : "Boots - tan",
            },
            {
              "product_name" : "Casual Cuffed Pants",
            },
            ...
          ]
        },
        "_explanation" : {
          "value" : 8.268259,
          "description" : "weight(products.product_name:pant in 94) [PerFieldSimilarity],
result of:",
          "details" : [
            {
              "value" : 8.268259,
              "description" : "score(freq=1.0), computed as boost * idf * tf from:",
              "details" : [
                {
```

```
          "value" : 2.2,
          "description" : "boost",
          "details" : [ ]
        },
        {
          "value" : 7.1974354,
          "description" : "idf, computed as log(1 + (N - n + 0.5) / (n + 0.5))
from:",
        },
        {
          "value" : 0.52217203,
          "description" : "tf, computed as freq / (freq + k1 * (1 - b + b * dl /
avgdl)) from:",
        }
      ]
    },
    ...
  }
}
```

유사도 스코어

1. 스코어 알고리즘(BM25) 이해하기

- 스코어는 문서와 쿼리 간의 연관성 수치로, 값이 클수록 연관성이 높다는 것을 의미
- 엘라스틱서치는 5.x 버전 부터 BM25알고리즘을 사용.
- BM25알고리즘은 검색, 추천에 많이 사용되는 알고리즘으로 TF-IDF 개념에 문서 길이를 고려한 알고리즘.
- 검색어가 문서에서 얼마나 자주 나타나는지, 검색어가 문서 내에서 중요한 용어인지를 판단.
- 루씬에서 스코어 알고리즘을 BM25알고리즘으로 변경하면서 엘라스틱서치에도 반영됨.

유사도 스코어

2. IDF 계산

- 도큐먼트 내에 단어의 발생 빈도가 적을 수록 가중치를 높게 주는데 이를 문서 빈도의 역수(Inverse Document Frequency:IDF)라 한다.
 - 전체 문서에서 자주 발생하는 단어일 수록 중요하지 않은 단어로 인식하고 가중치를 낮춘다.

"idf, computed as $\log(1 + (N - n + 0.5) / (n + 0.5))$ from:"

리스트 4.7 IDF 계산에 필요한 변수값

```
{
  "value" : 7.1974354,
  "description" : "idf, computed as log(1 + (N - n + 0.5) / (n + 0.5)) from:",
  "details" : [
    {
      "value" : 3,
      "description" : "n, number of documents containing term",
      "details" : [ ]
    },
    {
      "value" : 4675,
      "description" : "N, total number of documents with field",
      "details" : [ ]
    }
  ]
},
```

n: 검색어가 몇 개의 도큐먼트에 있는지 알려주는 값
N: 인덱스 전체의 도큐먼트 수

유사도 스코어

3. TF 계산

- 용어 빈도(Term Frequency :TF)는 특정 용어가 하나의 도큐먼트에 얼마나 많이 등장했는지를 의미하는 지표.
- 특정 용어가 도큐먼트에서 많이 반복되었으면 그 용어는 도큐먼트의 주제와 연관성이 높음.

"tf, computed as $\text{freq} / (\text{freq} + k1 * (1 - b + b * dl / \text{avgdl}))$ from:"

리스트 4.8 TF 계산에 필요한 변수값

```
{
  "value" : 0.52217283,
  "description" : "tf, computed as freq / (freq + k1 * (1 - b + b * dl / avgdl)) from:",
  "details" : [
    {
      "value" : 1.0,
      "description" : "freq, occurrences of term within document",
      "details" : [ ]
    },
    {
      "value" : 1.2,
      "description" : "k1, term saturation parameter",
      "details" : [ ]
    },
    {
      "value" : 0.75,
      "description" : "b, length normalization parameter",
      "details" : [ ]
    },
    {
      "value" : 5.0,
      "description" : "dl, length of field",
      "details" : [ ]
    },
    {
      "value" : 7.3161497,
      "description" : "avgdl, average length of field",
      "details" : [ ]
    }
  ]
}
```

freq: 도큐먼트에서 용어가 나온 횟수

k1: 알고리즘을 정규화하기 위한 가중치(디폴트로 상수)

b: 알고리즘을 정규화하기 위한 가중치(디폴트로 상수)

dl: 필드 길이(현재 도큐먼트를 토큰화했을 때 토큰의 수)

Avgdl: 평균 필드 길이(전체 문서의 도큐먼트당 토큰의 평균)

리스트 4.9 NiYeb3YBxWArx3SqGIQX 도큐먼트

```
{
  "_id" : "NiYeb3YBxWArx3SqGIQX",
  ...
  "products" : [
    { "product_name" : "Boots - tan", },
    { "product_name" : "Casual Cuffed Pants", }
  ]
}
```

유사도 스코어

3. TF 계산

리스트 4.10 최종 스코어 계산식

```
"description" : "score(freq=1.0), computed as boost * idf * tf from:",  
"details" : [  
  {  
    "value" : 2.2,  
    "description" : "boost",  
    "details" : [ ]  
  },
```

- 최종 스코어는 IDF 와 TF를 곱하고, boost 변수를 곱한다.
 - Boost는 엘라스틱이 지정한 고유값 2.2

엘라스틱서치: 검색

4 쿼리

- 엘라스틱서치는 검색을 위해 리프 쿼리, 복합 쿼리를 제공
 - 리프 쿼리: 특정 필드에서 용어를 찾는 쿼리.
 - 매치(match), 용어(term), 범위(range) 쿼리 등이 있음.
 - 복합 쿼리: 쿼리를 조합해 사용하는 쿼리.
 - 논리(bool) 쿼리 등이 있다.

쿼리

1. 전문 쿼리와 용어 수준 쿼리

- 전문 쿼리: 전문 검색을 하기 위한 쿼리, 전문 검색을 할 필드는 인덱스 매핑 시 텍스트 타입으로 매핑되어야 함.
- 용어 수준 쿼리: 정확히 일치하는 용어를 찾기 위해 사용, 인덱스 매핑 시 필드를 키워드 타입으로 매핑해야 함.

쿼리

1. 전문 쿼리와 용어 수준 쿼리

| 전문 쿼리

- qindex 인덱스를 생성하고 contents 필드를 갖는 문서 인덱싱.
- Contents 필드는 텍스트 타입으로 매핑, 분석기에 의해 토큰화
 - [I, love, elastic, stack]
- 전문 쿼리에서 contents가 토큰화
 - [elastic, world]
- 전문 쿼리 에서 토큰화 된 단어들과 문서 용어들이 매칭되어 스코어 계산.
- 구글, 네이버 검색 엔진과 동일한 방식.

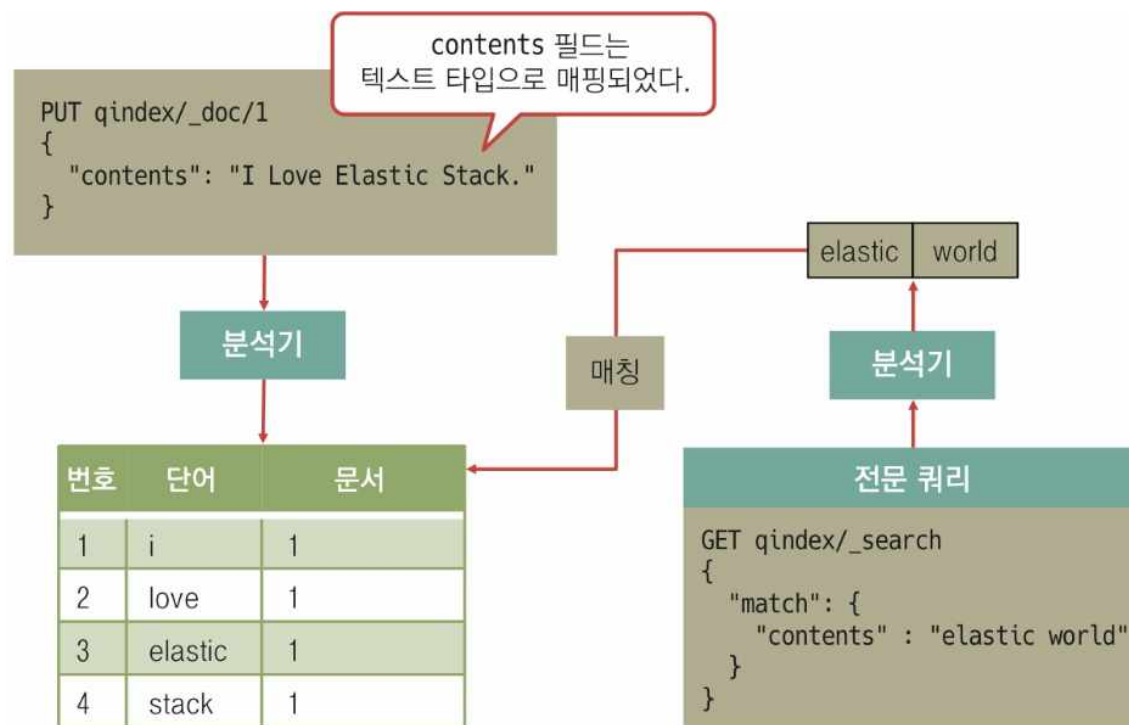


그림 4.6 전문 쿼리 동작 과정

쿼리

1. 전문 쿼리와 용어 수준 쿼리

| 용어 수준 쿼리

- qindex 인덱스를 생성하고 contents 필드를 갖는 도큐먼트 인덱싱.
- Category 필드는 키워드 타입으로 매핑, 분석기 사용 하지 않음.
- 검색은 용어 쿼리를 사용하는데 검색어 'tech' 또한 분석기를 사용하지 않음.
- 그림 4.7 과 같은 경우 'Tech'와 'tech'는 문자 하나의 대소문자 차이로 매칭이 되지 않음.
- 데이터베이스의 WHERE절과 비슷한 역할.

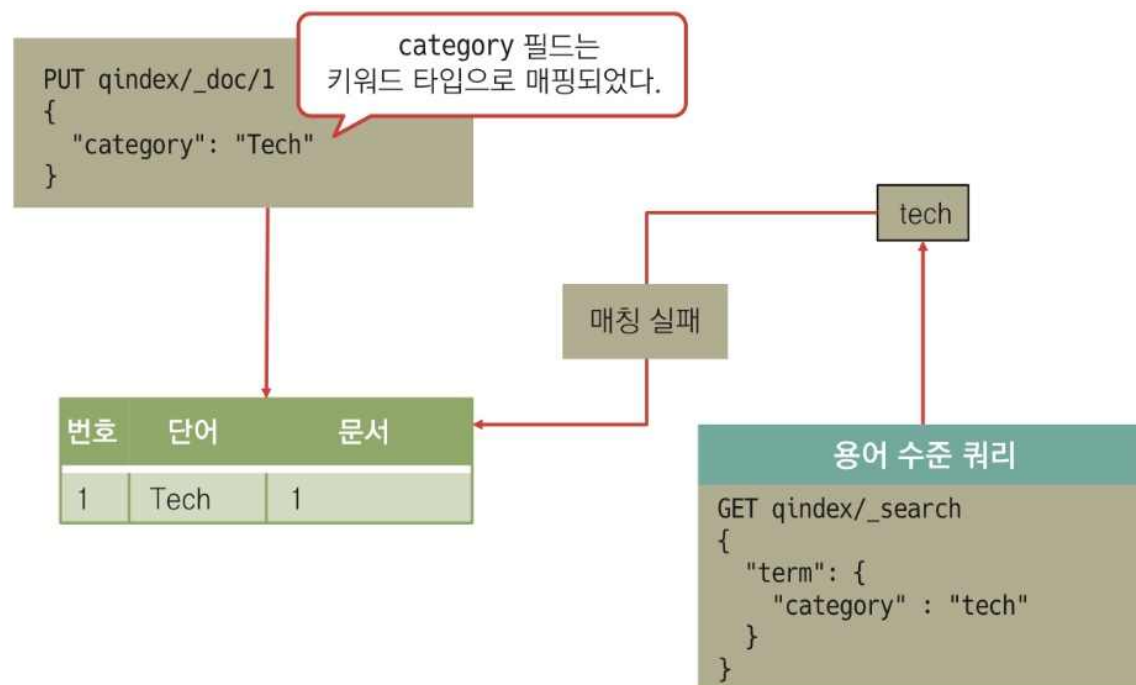


그림 4.7 용어 수준 쿼리 동작 과정

쿼리

2. 매치 쿼리

- 매치 쿼리는 대표적인 전문 쿼리
- 전체 텍스트에서 특정 용어나 용어들을 검색할 때 사용. (※용어 = 단어)
- 매치 쿼리를 사용하기 위해서는 검색하고자 하는 필드명을 알아야 함
- 필드명을 모를 경우 `_mapping` 매소드를 활용하여 인덱스에 포함된 필드를 확인 해야함.

> GET kibana_sample_data_ecommerce/_mapping

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_full_name"],
  "query": {
    "match": {
      "customer_full_name": "Mary"
    }
  }
}
```

리스트 4.11 하나의 용어를 검색하는 매치 쿼리

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_full_name"],
  "query": {
    "match": {
      "customer_full_name": "Mary bailey"
    }
  }
}
```

리스트 4.12 복수 개의 용어를 검색하는 매치 쿼리

쿼리

2. 매치 쿼리

- Mary와 bailey가 모두 포함 된 문서 찾기
- Operator 파라미터 수정

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_full_name"],
  "query": {
    "match": {
      "customer_full_name": {
        "query": "mary bailey",
        "operator": "and"
      }
    }
  }
}
```

리스트 4.14 operator를 and로 설정한 매치 검색

리스트 4.15 'mary' and 'bailey' 검색 시 응답 결과

```
{
  "hits" : {
    "total" : 3,
    "max_score" : 9.155506,
    "hits" : [
      {
        ...
        "_source" : {
          ...
          "customer_full_name" : "Mary Bailey",
          ...
          "customer_full_name" : "Mary Bailey",
          ...
          "customer_full_name" : "Mary Bailey",

```

쿼리

3. 매치 프레이즈 쿼리

- 매치 프레이즈 쿼리는 구(phrase)를 검색 할 때 사용하는 쿼리.
- 구는 동사가 아닌 2개 이상의 단어가 연결되어 만들어지는 단어. Ex) 빨간색 바지, 65인치 텔레비전
- 단어의 순서도 중요.

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_full_name"],
  "query": {
    "match_phrase": {
      "customer_full_name": "mary bailey"
    }
  }
}
```

리스트 4.16 매치 프레이즈 쿼리

리스트 4.17 매치 프레이즈 쿼리 결과

```
{
  "hits" : {
    "total" : 3,
    "max_score" : 9.155506,
    "hits" : [
      {
        "_id" : "5SYEb3YBxWArx3SqFYEB",
        "_source" : {
          "customer_full_name" : "Mary Bailey",
        }
      },
      {
        "_id" : "gCYEb3YBxWArx3SqFYIB",
        "_source" : {
          "customer_full_name" : "Mary Bailey",
        }
      },
      {
        "_id" : "PyYEB3YBxWArx3SqJZEc",
        "_source" : {
          "customer_full_name" : "Mary Bailey",
        }
      }
    ]
  },
  ...
}
```

쿼리

4. 용어 쿼리

- 용어 쿼리는 용어 수준 쿼리의 대표적인 쿼리
- 매치 쿼리와 차이는 매치 쿼리는 전문 쿼리에 속하기 때문에 검색어가 분석기에 의해 토근화 되고 토근화 된 용어가 있는 경우 매칭이 된다.
- 용어 쿼리는 검색어가 분석기를 통하지 않기 때문에 대소문자 및 띄어쓰기가 동일하게 일치해야 검색이 된다.

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_full_name"],
  "query": {
    "term": {
      "customer_full_name": "Mary Bailey"
    }
  }
}
```

리스트 4.18 텍스트 타입 필드에 대한 용어 쿼리

```
1 {
2   "took" : 0,
3   "timed_out" : false,
4   "_shards" : {
5     "total" : 1,
6     "successful" : 1,
7     "skipped" : 0,
8     "failed" : 0
9   },
10  "hits" : {
11    "total" : {
12      "value" : 0,
13      "relation" : "eq"
14    },
15    "max_score" : null,
16    "hits" : [ ]
17  }
18 }
```

쿼리

4. 용어 쿼리

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_full_name"],
  "query": {
    "term": {
      "customer_full_name": "Mary"
    }
  }
}
```

리스트 4.19 텍스트 타입 필드에 대한 용어 쿼리

- customer_full_name는 텍스트 타입으로 분석기에 의해 소문자로 변경되어 [mary, bailey]로 매핑되어 있지만 용어 쿼리는 [Mary]를 찾기 때문에 매칭이 되지 않는다.
- 용어 쿼리는 키워드 타입의로 매핑된 필드에서 사용.

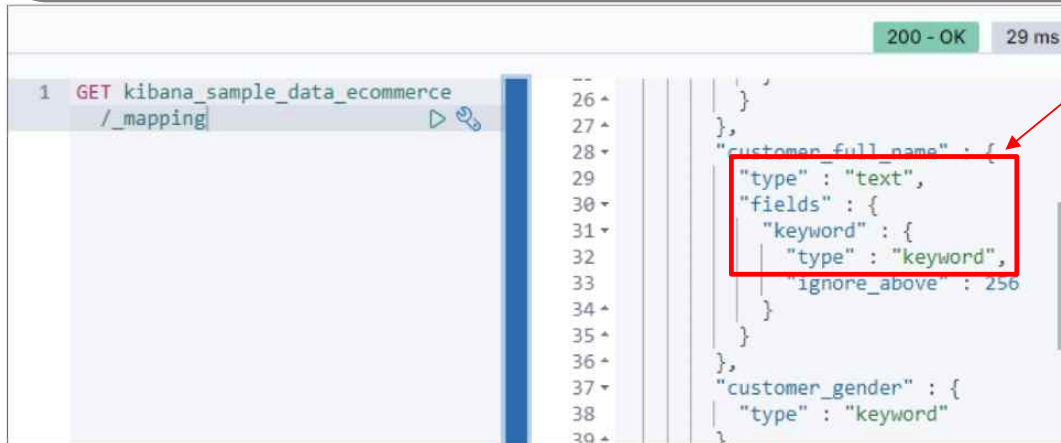
```
1 {
2   "took" : 0,
3   "timed_out" : false,
4   "_shards" : {
5     "total" : 1,
6     "successful" : 1,
7     "skipped" : 0,
8     "failed" : 0
9   },
10  "hits" : {
11    "total" : {
12      "value" : 0,
13      "relation" : "eq"
14    },
15    "max_score" : null,
16    "hits" : [ ]
17  }
18 }
```



그림 4.8 customer_full_name 필드 매핑 확인

쿼리

4. 용어 쿼리



멀티 필드를 갖음

리스트 4.21 리스트 4.20의 결과

```
{
  "hits": {
    "total": 3,
    "max_score": 7.1974354,
    "hits": [
      {
        "_id": "5SYEb3YBxWArx3SqFYEB",
        "_source": {
          "customer_full_name": "Mary Bailey",
        }
      },
      {
        "_id": "gCYEb3YBxWArx3SqFYIB",
        "_source": {
          "customer_full_name": "Mary Bailey",
        }
      },
      {
        "_id": "PyYEB3YBxWArx3SqJZEc",
        "_source": {
          "customer_full_name": "Mary Bailey",
        }
      }
    ]
  },
  ...
}
```

그림 4.8 customer_full_name 필드 매핑 확인

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_full_name"],
  "query": {
    "term": {
      "customer_full_name.keyword": "Mary Bailey"
    }
  }
}
```

리스트 4.20 키워드 타입 필드에 대한 용어 쿼리

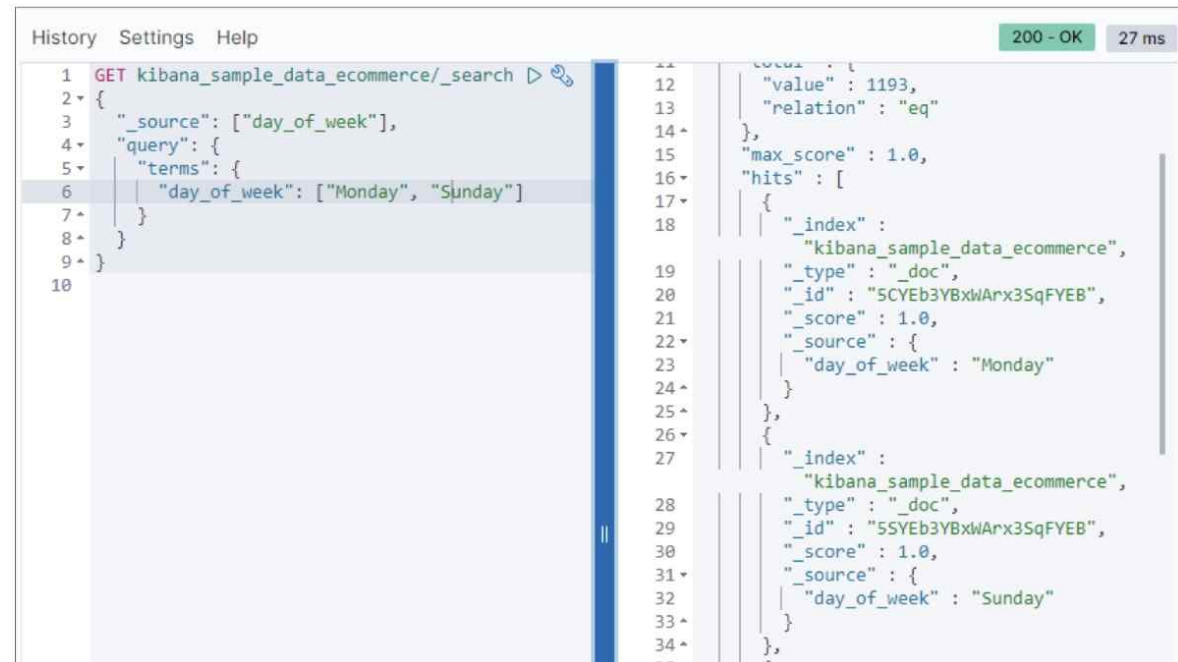
쿼리

5. 용어들 쿼리

- 용어들을 검색 하는 쿼리
- 키워드 타입으로 매핑 된 필드에서 사용.

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["day_of_week"],
  "query": {
    "terms": {
      "day_of_week": ["Monday", "Sunday"]
    }
  }
}
```

리스트 4.22 용어들 쿼리



The screenshot shows a Kibana search console interface. The left pane displays the search query: `GET kibana_sample_data_ecommerce/_search` with a JSON body containing a terms query for the `day_of_week` field, filtering for `Monday` and `Sunday`. The right pane shows the search results in JSON format. The status bar at the top right indicates `200 - OK` and `27 ms`. The results show two hits, each representing a document from the `kibana_sample_data_ecommerce` index with a score of 1.0. The first hit has a `day_of_week` of `Monday`, and the second has `Sunday`.

```
1 GET kibana_sample_data_ecommerce/_search
2 {
3   "_source": ["day_of_week"],
4   "query": {
5     "terms": {
6       "day_of_week": ["Monday", "Sunday"]
7     }
8   }
9 }
10
11
12   "value" : 1193,
13   "relation" : "eq"
14 },
15   "max_score" : 1.0,
16   "hits" : [
17     {
18       "_index" :
19         "kibana_sample_data_ecommerce",
20       "_type" : "_doc",
21       "_id" : "5CYEb3YBxWArx3SqFYEB",
22       "_score" : 1.0,
23       "_source" : {
24         "day_of_week" : "Monday"
25       }
26     },
27     {
28       "_index" :
29         "kibana_sample_data_ecommerce",
30       "_type" : "_doc",
31       "_id" : "5SYEb3YBxWArx3SqFYEB",
32       "_score" : 1.0,
33       "_source" : {
34         "day_of_week" : "Sunday"
35       }
36     }
37   ]
38 }
```

그림 4.9 리스트 4.22 실행 결과

6. 멀티 매치 쿼리

- 엘라스틱서치는 필드를 기준으로 찾으려는 용어나 구절을 검색하기 때문에 검색을 위해 필드명을 입력해야 했음.
- 하지만 전문검색 서비스를 하거나 여러 상황에 따라 용어나 구절이 어떤 필드에 있는지 모르는 경우가 발생한다.
- 멀티 매치 쿼리는 여러 개의 필드에서 검색을 지원.
- 텍스트 타입으로 매핑 된 필드에서 주로 사용.

6. 멀티 매치 쿼리

```
GET kibana_sample_data_ecommerce/_search?explain=true
{
  "_source": ["customer_first_name","customer_last_name","customer_full_name"],
  "query": {
    "multi_match": {
      "query": "mary",
      "fields": [
        "customer_full_name","customer_first_name","customer_last_name"
      ]
    }
  }
}
```

리스트 4.23 여러 필드에 쿼리 요청하기

- "customer_first_name","customer_last_name","customer_full_name" 3개의 필드에 대해 'mary'라는 용어로 매치쿼리를 하고 3개의 필드에서 개별 스코어를 구한 다음 가장 큰 값을 대표 스코어로 구한다.

쿼리

6. 멀티 매치 쿼리

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_first_name","customer_last_name","customer_full_name"],
  "query": {
    "multi_match": {
      "query": "mary",
      "fields": [
        "customer_*_name"
      ]
    }
  }
}
```

리스트 4.24 와일드카드를 이용한 멀티 필드에 쿼리 요청하기

- 검색하려는 필드가 많을 때는 필드명에 * 같은 와일드카드를 사용해 이름이 유사한 복수의 필드를 선택할 수 있음.

6. 멀티 매치 쿼리

| 필드에 가중치 두기

- 여러 개의 필드 중 특정 필드에 가중치를 두는 방법을 부스팅(boosting)기법이라 하는데 멀티 매치 쿼리에서 자주 사용됨.
- 특정 필드에 스코어 값을 n배 해주는 효과

```
GET kibana_sample_data_ecommerce/_search?explain=true
{
  "_source": ["customer_first_name","customer_last_name","customer_full_name"],
  "query": {
    "multi_match": {
      "query": "mary",
      "fields": [
        "customer_full_name^2","customer_first_name","customer_last_name"
      ]
    }
  }
}
```

리스트 4.23 여러 필드에 쿼리 요청하기

쿼리

7. 범위 쿼리

- 특정 날짜나 숫자 범위를 지정해 범위 안에 포함된 데이터들을 검색할 때 사용.
- 날짜/숫자/IP 타입의 데이터는 범위 쿼리가 가능하지만 문자형, 키워드 타입의 데이터에서 범위 쿼리를 사용할 수 없다.

```
GET kibana_sample_data_flights/_search
{
  "query": {
    "range": {
      "timestamp": {
        "gte": "2022-02-21",
        "lt": "2022-02-22"
      }
    }
  }
}
```

리스트 4.26 날짜/시간 범위 쿼리

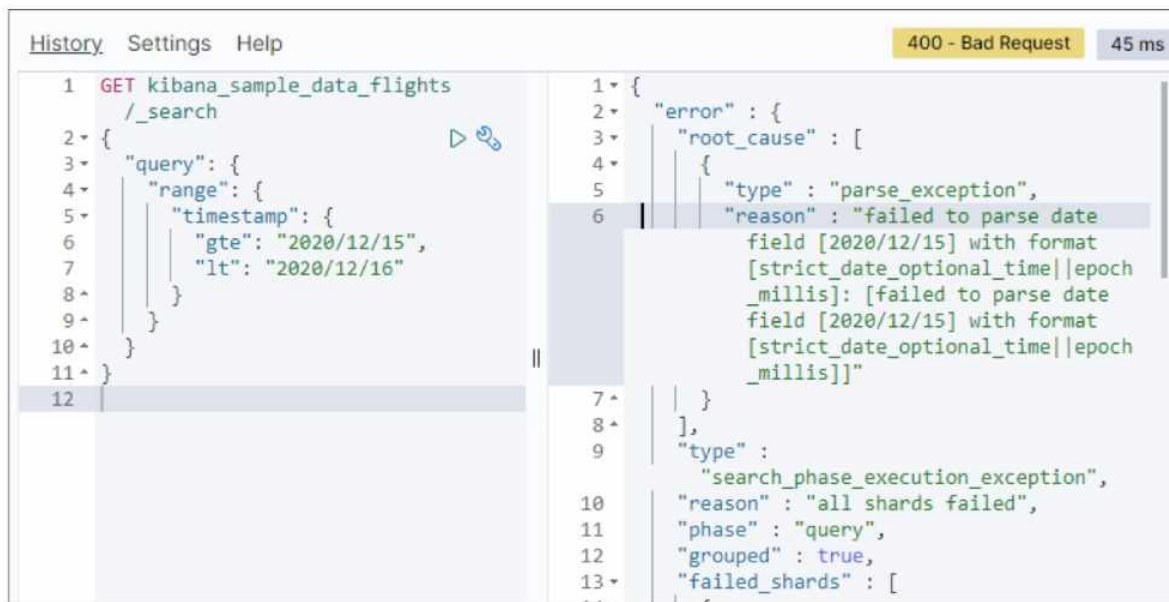


그림 4.10 잘못된 날짜/시간 포맷을 사용하는 경우

쿼리

7. 범위 쿼리

표 4.1 검색 범위를 지정하는 파라미터

파라미터	설명
gte	(gte: 10) 10과 같거나 10보다 큰 값
	(gte: 2021-01-21) 2021년 1월 21일이거나 그 이후의 날짜
gt	(gt: 10) 10보다 큰 값
	(gt: 2021-01-21) 2021년 1월 21일 이후의 날짜(2021년 1월 21일은 포함되지 않음)
lte	(lte: 20) 20과 같거나 20보다 작은 값
	(lte: 2021-01-21) 2021년 1월 21일이거나 그 이전의 날짜
lt	(lt: 20) 20보다 작은 값
	(lt: 2021-01-21) 2021년 1월 21일 이전의 날짜(2021년 1월 21일은 포함되지 않음)

쿼리

7. 범위 쿼리

| 날짜/시간 데이터 타입

- 날짜/시간 검색은 현재 날짜를 기준으로 하는 경우가 많음.
- 기준 날짜로 하루 전, 일주일 전 문서만 골라서 진행 할 때 편리하게 검색 가능

```
GET kibana_sample_data_flights/_search
{
  "query": {
    "range": {
      "timestamp": {
        "gte": "now-1M"
      }
    }
  }
}
```

리스트 4.27 날짜/시간 범위 쿼리

표 4.2 날짜/시간 관련 범위 표현식

표현식	설명
now	현재 시각(2021-01-21T15:20:33)
now+1d	현재 시각 + 1일(2021-01-22T15:20:33)
now+1h+30m+10s	현재 시각 + 1시, 30분, 10초(2021-01-21T16:50:43)
2021-01-21 +1M	2021-01-21 + 1달(2021-02-21T00:00:00)

표 4.3 날짜/시간 단위 표기법

시간 단위	의미	시간 단위	의미	시간 단위	의미
y(year)	연	M(month)	월	w(weeks)	주
d(days)	일	H 또는 h(hours)	시	m(minutes)	분
s(seconds)	초				

쿼리

7. 범위 쿼리

| 범위 데이터 타입

- 엘라스틱서치에서 범위 데이터 타입을 제공.
 - `integer_range`, `float_range`, `long_range`, `double_range`, `date_range`, `ip_range`

```
PUT range_test_index
{
  "mappings": {
    "properties": {
      "test_date": {
        "type": "date_range"
      }
    }
  }
}
```

리스트 4.28 날짜/시간 범위 타입을 갖는 인덱스 생성

```
PUT range_test_index/_doc/1
{
  "test_date": {
    "gte": "2021-01-21",
    "lt": "2021-01-25"
  }
}
```

리스트 4.29 날짜/시간 범위를 갖는 문서 인덱싱

```
PUT range_test_index/_doc/2
{
  "test_date": "2021-01-21"
}
```

리스트 4.30 날짜/시간 범위데이터 필드에 날짜/시간 데이터 인덱싱

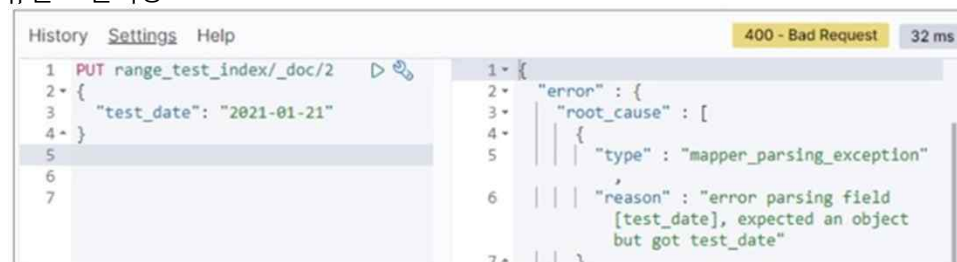


그림 4.11 타입이 잘못되었다는 응답 결과

쿼리

7. 범위 쿼리

| 범위 데이터 타입

- 날짜/시간 범위 타입과 날짜/시간 타입은 다르다.
- 범위 타입은 시작(gte)과 끝(lte)를 지정해야 한다.
- Relation 파라미터를 이용해 어떤 범위를 포함할지 결정

표 4.4 relation에 들어갈 수 있는 값들

값	설명
intersects(기본값)	쿼리 범위 값이 도큐먼트의 범위 데이터를 일부라도 포함하기만 하면 된다.
contains	도큐먼트의 범위 데이터가 쿼리 범위 값을 모두 포함해야 한다.
within	도큐먼트의 범위 데이터가 쿼리 범위 값 내에 전부 속해야 한다.

문서	숫자 범위	쿼리 범위 값	relation	찾은 문서
1	10 ~ 19	15 ~ 45	intersects	1, 2, 3, 4
2	20 ~ 29	15 ~ 45	within	2, 3
3	30 ~ 39	25 ~ 29	contains	2
4	40 ~ 49	25 ~ 35	contains	X

그림 4.12 relation 값에 따른 검색 결과 비교

```
GET range_test_index/_search
{
  "query": {
    "range": {
      "test_date": {
        "gte": "2021-01-21",
        "lte": "2021-01-28",
        "relation": "within"
      }
    }
  }
}
```

리스트 4.31 relation 파라미터를 이용한 범위 설정

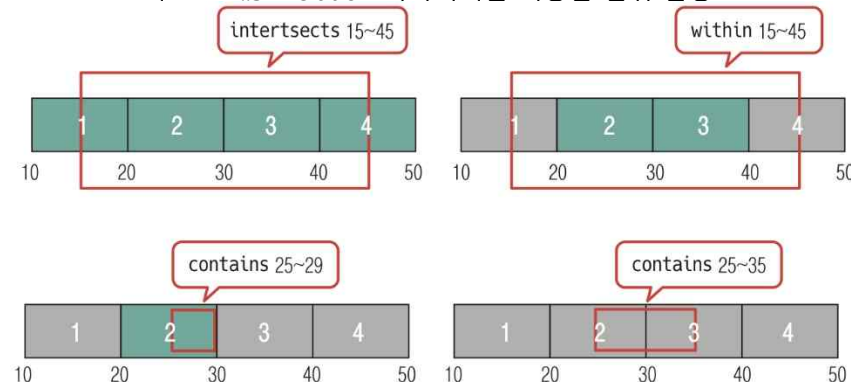


그림 4.13 범위 검색에서 relation의 영향

8. 논리 쿼리

- 논리 쿼리는 복합 쿼리로 앞서 배운 쿼리를 조합할 수 있음.

리스트 4.32 논리 쿼리 작성 포맷과 지원하는 타입들

```
GET <index>/_search
{
  "query": {
    "bool": {
      "must": [
        { 쿼리문 }, ...
      ],
      "must_not": [
        { 쿼리문 }, ...
      ],
      "should": [
        { 쿼리문 }, ...
      ],
      "filter": [
        { 쿼리문 }, ...
      ]
    }
  }
}
```

표 4.5 논리 쿼리 타입

타입	설명
must	쿼리를 실행하여 참인 문서를 찾는다.
	복수의 쿼리를 실행하면 AND 연산을 한다.
must_not	쿼리를 실행하여 거짓인 문서를 찾는다.
	다른 타입과 같이 사용할 경우 문서에서 제외한다.
should	단독으로 사용 시 쿼리를 실행하여 참인 문서를 찾는다.
	복수의 쿼리를 실행하면 OR 연산을 한다.
	다른 타입과 같이 사용할 경우 스코어에만 활용된다.
filter	쿼리를 실행하여 '예/아니오' 형식의 필터 컨텍스트를 수행한다.

8. 논리 쿼리

| must 타입

- must 타입은 쿼리를 실행하고 참인 도큐먼트를 찾음.

```
GET kibana_sample_data_ecommerce/_search
{
  "query": {
    "bool": {
      "must": {
        "match": {"customer_first_name": "mary"}
      }
    }
  }
}
```

리스트 4.33 하나의 쿼리를 사용하는 must 타입

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["day_of_week", "customer_full_name"],
  "query": {
    "bool": {
      "must": [
        {"term": {"day_of_week": "Sunday"}},
        {"match": {"customer_full_name": "mary"}}
      ]
    }
  }
}
```

리스트 4.34 복수개의 쿼리를 사용하는 must 타입

- must 타입으로 복수 개의 쿼리를 실행하면 AND 연산의 효과를 얻음

8. 논리 쿼리

| must_not 타입

- must_not 타입은 도큐먼트에서 제외할 쿼리를 실행

```
GET kibana_sample_data_ecommerce/_search
{
  "query": {
    "bool": {
      "must_not": {
        "match": {"customer_full_name": "mary"}
      }
    }
  }
}
```

리스트 4.35 하나의 쿼리를 사용하는 must_not 타입

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_first_name"],
  "query": {
    "bool": {
      "must": {
        "match": {"customer_full_name": "mary"}
      },
      "must_not": {
        "term": {"customer_last_name": "bailey"}
      }
    }
  }
}
```

리스트 4.36 다른 타입과 must_not 타입을 함께 사용하는 경우

쿼리

8. 논리 쿼리

| must_not 타입



```
History Settings Help 200 - OK 20 ms
1 GET kibana_sample_data_ecommerce
2 /_search
3 {"_source": ["customer_full_name"],
4  "query": {
5    "bool": {
6      "must": {
7        "match": {
8          "customer_first_name":
9            "mary" }
10     },
11     "must_not": {
12       "term": {
13         "customer_last_name":
14           "bailey" }
15     }
16   }
17 }
18
19 "total" : {
20   "value" : 151,
21   "relation" : "eq"
22 },
23 "max_score" : 3.5671005,
24 "hits" : [
25   {
26     "_index" :
27       "kibana_sample_data_ecommerce",
28     "_type" : "_doc",
29     "_id" : "Nq9x9ncBGRwSyFwFv6aT",
30     "_score" : 3.5671005,
31     "_source" : {
32       "customer_full_name" : "Mary
33         Barber"
34     }
35   }
36 ]
```

그림 4.15 리스트 4.36 실행 결과

8. 논리 쿼리

| should 타입

- should 타입에 하나의 쿼리를 사용한다면 must 타입과 같은 결과를 얻음

```
GET kibana_sample_data_ecommerce/_search
{
  "query": {
    "bool": {
      "should": {
        "match": {"customer_first_name": "mary"}
      }
    }
  }
}
```

리스트 4.37 하나의 쿼리를 사용하는 should 타입

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["day_of_week", "customer_first_name"],
  "query": {
    "bool": {
      "should": [
        {"term": {"day_of_week": "Sunday"}},
        {"match": {"customer_full_name": "mary"}}
      ]
    }
  }
}
```

리스트 4.38 복수 개의 쿼리를 사용하는 should 타입

쿼리

8. 논리 쿼리

| should 타입

- should 타입에서 복수 개의 쿼리를 사용하면 OR 조건이 수행.



```
History Settings Help 200 - OK 26 ms
1 GET kibana_sample_data_ecommerce
2 /_search
3 {
4   "_source": ["day_of_week",
5               "customer_full_name"],
6   "query": {
7     "bool": {
8       "should": [
9         { "term": { "day_of_week": "Sunday" } },
10        { "match": { "customer_full_name": "mary" } }
11      ]
12    }
13  }
14
15  "total": {
16    "value": 753,
17    "relation": "eq"
18  },
19  "max_score": 5.5206447,
20  "hits": [
21    {
22      "_index": "kibana_sample_data_ecommerce",
23      "_type": "_doc",
24      "_id": "Ia9x9ncBGRwSyFwFv6aT",
25      "_score": 5.5206447,
26      "_source": {
27        "customer_full_name": "Mary Bailey",
28        "day_of_week": "Sunday"
29      }
30    }
31  ]
32}
```

그림 4.16 리스트 4.38 실행 결과

쿼리

8. 논리 쿼리

| should 타입

- should 타입을 다른 타입과 함께 사용하는 경우
 - Should 타입에 적은 쿼리는 검색에 영향을 주지 않고 스코어에만 영향을 준다.

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_full_name", "day_of_week"],
  "query": {
    "bool": {
      "must": {
        "match": { "customer_full_name": "mary" }
      }
    }
  }
}
```

리스트 4.39 must 타입만 단독으로 사용하는 쿼리



그림 4.17 리스트 4.39 실행 결과

쿼리

8. 논리 쿼리

| should 타입

- should 타입과 must 타입을 같이 사용하는 경우

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["customer_full_name", "day_of_week"],
  "query": {
    "bool": {
      "must": {
        "match": {"customer_full_name": "mary"}
      },
      "should": {
        "term": {"day_of_week": "Monday"}
      }
    }
  }
}
```

리스트 4.40 must와 should 타입을 같이 사용하는 쿼리

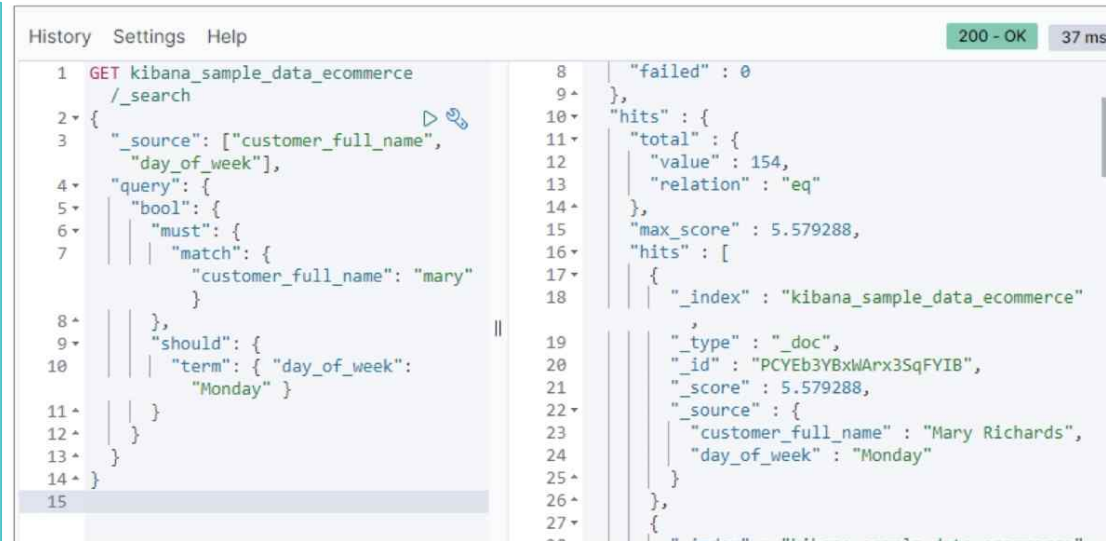


그림 4.18 리스트 4.40 실행 결과

customer_full_name에 'mary' 가 들어간 모든 도큐먼트 중 day_of_week가 'Monday'인 도큐먼트의 우선순위를 높임.

8. 논리 쿼리

| filter 타입

- filter 타입은 must와 같은 동작을 하지만 필터 컨텍스트로 동작하기 때문에 유사도 스코어에 영향을 미치지 않음.
 - YES/NO 두 가지 결과만 제공할 뿐 유사도를 고려하지 않음

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["products.base_price"],
  "query": {
    "bool": {
      "filter": {
        "range": {
          "products.base_price": {
            "gte": 30,
            "lte": 60
          }
        }
      }
    }
  }
}
```

리스트 4.41 하나의 쿼리를 사용하는 filter 타입

쿼리

8. 논리 쿼리

| filter 타입

- 스코어는 0으로 표시, products.base_price가 30 이상 60이하인 모든 도큐먼트 검색 됨.



The screenshot shows a Kibana search console interface. The left pane displays the search query, and the right pane displays the search results. The query is a GET request to the kibana_sample_data_ecommerce index, using the /_search endpoint. It includes a filter query that searches for documents where the products.base_price is between 30 and 60. The results show a total of 2336 hits, with a max_score of 0.0. The first two hits are displayed, showing the _index, _type, _id, _score, and _source fields. The _source field contains the products object, which includes the base_price field.

```
History Settings Help 200 - OK 20 ms

1 GET kibana_sample_data_ecommerce
2 /_search
3 {
4   "_source": ["products.base_price"],
5   "query": {
6     "bool": {
7       "filter": {
8         "range": {
9           "products.base_price": {
10            "gte": 30,
11            "lte": 60
12          }
13        }
14      }
15    }
16  }
17
18

11- "total": {
12-   "value": 2336,
13-   "relation": "eq"
14- },
15- "max_score": 0.0,
16- "hits": [
17-   {
18-     "_index":
19-       "kibana_sample_data_ecommerce",
20-     "_type": "_doc",
21-     "_id": "JK9x9ncBGRwSyFwFv6aT",
22-     "_score": 0.0,
23-     "_source": {
24-       "products": [
25-         {
26-           "base_price": 59.99
27-         },
28-         {
29-           "base_price": 20.99
30-         }
31-       ]
32-     }
33-   }
34- ]
35- }
```

그림 4.19 리스트 4.41 실행 결과

쿼리

8. 논리 쿼리

filter 타입

- filter 타입을 다른 타입과 같이 사용해 불필요한 문서 필터링

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": ["day_of_week", "customer_full_name"],
  "query": {
    "bool": {
      "filter": {
        "filter": {
          "term": {"day_of_week": "Sunday"}
        },
      },
      "must": {
        "match": {"customer_full_name": "mary"}
      }
    }
  }
}
```

리스트 4.42 filter와 must 타입을 같이 사용하는 쿼리



그림 4.20 리스트 4.42 실행 결과

day_of_week가 Sunday인 경우를 필터링 하고,
그 결과에서 customer_full_name가 mary인 문서를 검색.
day_of_week 필터 절에서 불필요한 스코어 계산을 줄일 수 있음.

9. 패턴 검색

- 검색어가 길거나 검색어를 정확히 알지 못하는 경우가 있다.
- 대략적인 키워드나 몇 개의 알파벳만 알고 있다면 패턴을 이용해 검색을 할 수 있다.
- 패턴을 이용한 검색은 와일드카드를 사용하는 와일드카드 쿼리(Wildcard query)와 정규표현식을 사용하는 정규식 쿼리(regex query)가 있다.
- 두 방법 모두 용어 수준 쿼리에 해당됨으로 분석기에 의해 분리된 용어를 찾기 위한 쿼리이다.

9. 패턴 검색

| 와일드카드 쿼리

- 와일드카드 쿼리는 용어 검색 시 * (별표)기호와 ?(물음표)기호를 사용.
 - *(별표): 공백까지 포함하여 글자 수에 상관 없이 모든 문자 매칭 가능
 - ?(물음표): 오직 한 문자만 매칭 가능
- 검색하려는 용어 앞에 * (별표)기호와 ?(물음표)를 사용하면 속도가 매우 느려진다.

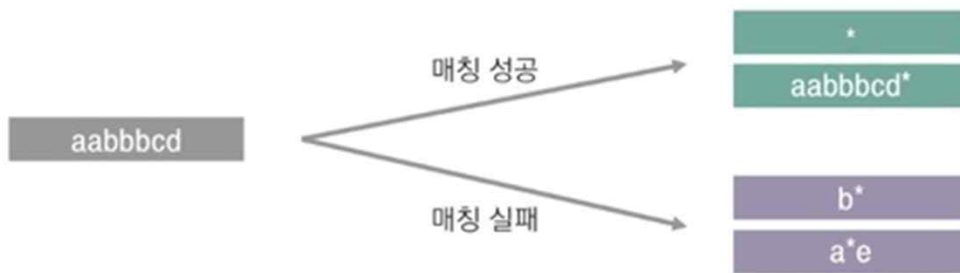


그림 4.21 '*' 매칭 성공과 실패 케이스

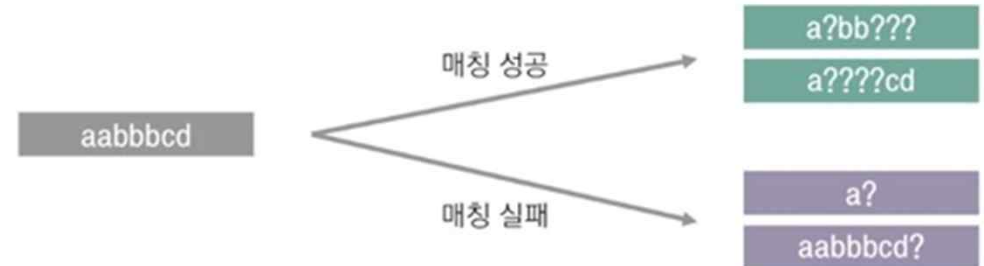


그림 4.22 '?' 매칭 성공과 실패 케이스

9. 패턴 검색

| 와일드카드 쿼리

- 와일드카드 쿼리는 용어 검색 시 * (별표)기호와 ?(물음표)기호를 사용.
 - *(별표): 공백까지 포함하여 글자 수에 상관 없이 모든 문자 매칭 가능
 - ?(물음표): 오직 한 문자만 매칭 가능
- *(별표)기호와 ?(물음표)를 사용하면 속도가 매우 느려진다.

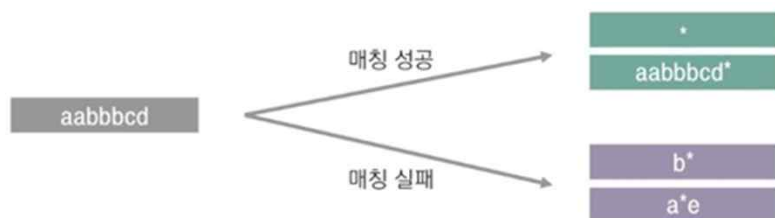


그림 4.21 '*' 매칭 성공과 실패 케이스

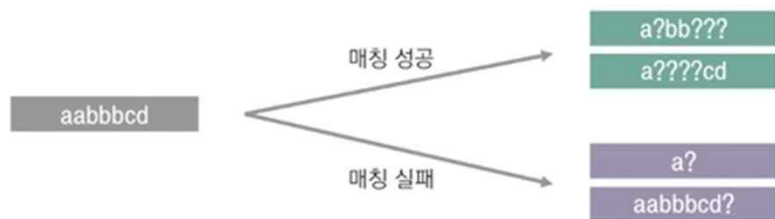


그림 4.22 '?' 매칭 성공과 실패 케이스

```
GET kibana_sample_data_ecommerce/_search
{
  "_source": "customer_full_name",
  "query": {
    "wildcard": {
      "customer_full_name.keyword": "M?r*"
    }
  }
}
```

리스트 4.43 와일드카드 패턴 검색

쿼리

9. 패턴 검색

| 정규식 쿼리

- 정규식: 특정한 패턴을 가진 문자열을 표현하기 위한 형식 언어

메타문자	설명	
.	점(.)은 하나의 문자를 의미	
+	+기호는 +기호 앞 문자와 같은 문자가 한번 이상 반복되면 매칭	
*	*기호 앞 문자와 같은 문자가 0번 혹은 여러 번 반복되면 매칭	
?	?기호 앞 문자와 같은 문자가 0번 혹은 한번 나타나면 매칭	
()	()기호는 문자들을 그룹핑하여 반복되는 문자들을 매칭(+,*,? 기호와 혼합사용)	
[]	[] 기호는 문자를 클래스화하여 특정 범위의 문자들을 매칭	

9. 패턴 검색

| 정규식 쿼리

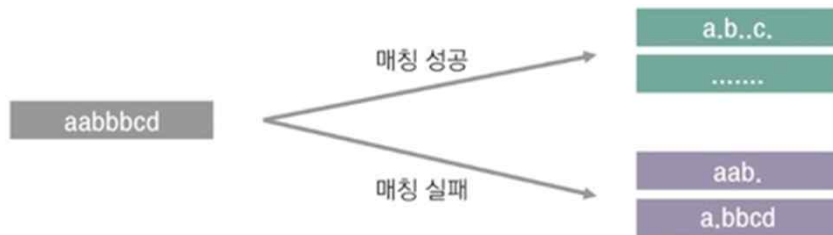


그림 4.23 '.' 정규식 매칭 성공과 실패 케이스

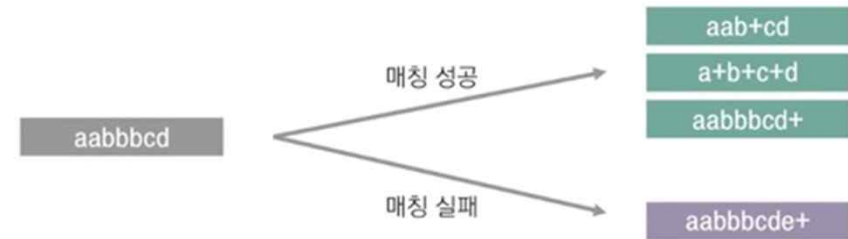


그림 4.24 '+' 정규식 매칭 성공과 실패 케이스

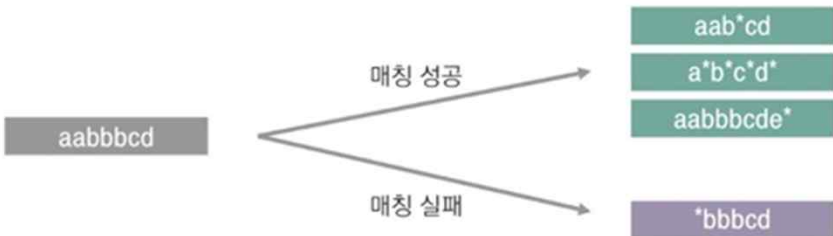


그림 4.25 '*' 정규식 매칭 성공과 실패 케이스

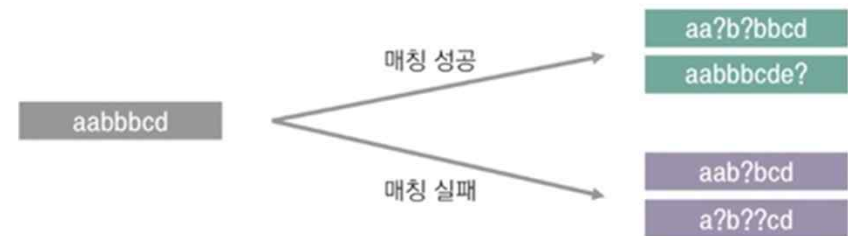


그림 4.26 '?' 정규식 매칭 성공과 실패 케이스

9. 패턴 검색

| 정규식 쿼리

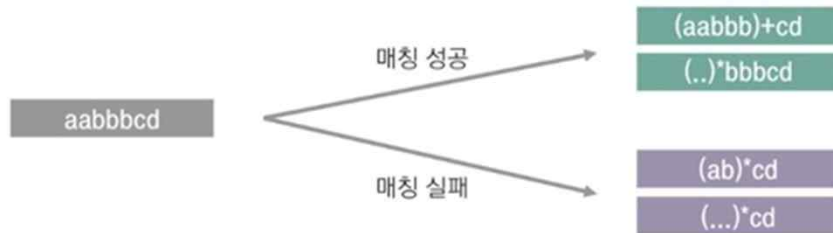


그림 4.27 () 정규식 매칭 성공과 실패 케이스

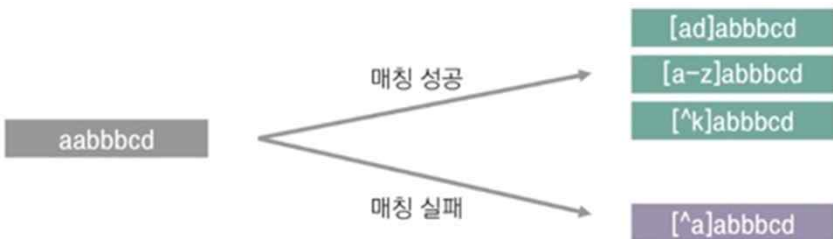


그림 4.28 [] 정규식 매칭 성공과 실패 케이스

```
GET kibana_sample_data_ecommerce/_search
{
  "query": {
    "regexp": {
      "customer_first_name.keyword": "Mar."
    }
  }
}
```

리스트 4.44 정규식을 이용한 패턴 검색

04

엘라스틱서치: 집계

엘라스틱서치 집계

- 엘라스틱서치에서 집계(aggregation)는 데이터를 그룹핑하고 통계값을 얻는 기능 SQL의 GROUP BY 와 통계 함수를 포함 하는 개념.
- 집계 기능은 강력한 검색 성능과 맞물려 엘라스틱서치를 고성능 집계 엔진으로 사용 할 수 있게 한다.
- 키바나의 데이터 시각화 대시보드는 대부분 집계 기능을 기반으로 동작.

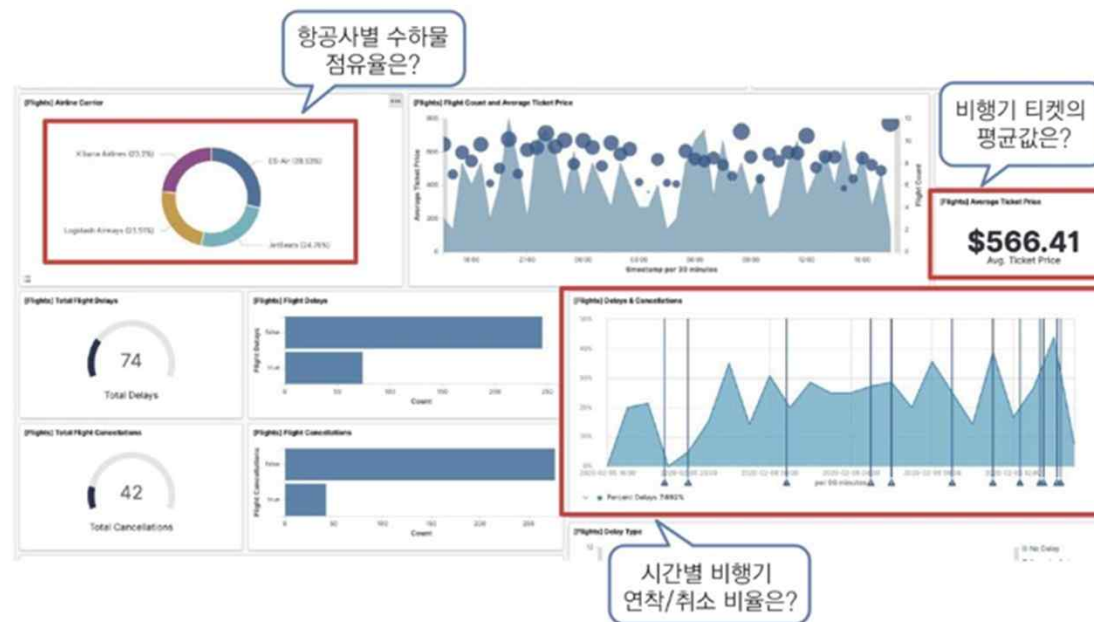


그림 5.1 집계를 통해 얻을 수 있는 결과들

엘라스틱서치: 집계

1 집계 요청-응답 형태

- 집계를 사용할 경우 search API의 요청 본문에 aggs 파라미터를 이용하여 집계를 요청 할 수 있음.

리스트 5.1 집계를 위한 기본 형태

```
GET <인덱스>/_search
{
  "aggs": {
    "my_aggs": {
      "agg_type": {
        ...
      }
    }
  }
}
```

- aggs: 집계를 요청한다는 의미.
- my_agg: 사용자가 지정하는 집계 이름
- agg_type: 집계 타입
 - 메트릭 집계: 통계나 계산에 사용
 - 버킷 집계: 문서를 그룹핑

엘라스틱서치: 집계

1 집계 요청-응답 형태

리스트 5.2 집계 응답 기본 형태

```
{
  "hits" : {
    "total" : {
      ...
    },
    "aggregations" : {
      "my_aggs" : {
        "value" :
      }
    }
  }
}
```

- **aggregations**: 해당 응답 메시지가 집계요청에 대한 결과임을 표시.
- **my_agg**: 사용자가 지정한 집계 이름

엘라스틱서치: 집계

2 메트릭 집계

- 최소/최대/합계/평균/중간값 같은 통계 결과를 반환
- 텍스트 타입필드는 합계나 평균같은 수치연산을 계산할 수 없음
- 지리 정보가 있는 필드를 위한 집계도 존재

표 5.1 메트릭 집계 종류

메트릭 집계	설명
avg	필드의 평균값을 계산한다.
min	필드의 최솟값을 계산한다.
max	필드의 최댓값을 계산한다.
sum	필드의 총합을 계산한다.
percentiles	필드의 백분위값을 계산한다.
stats	필드의 min, max, sum, avg, count(도큐먼트 개수)를 한 번에 볼 수 있다.
cardinality	필드의 유니크한 값 개수를 보여준다.
geo-centroid	필드 내부의 위치 정보의 중심점을 계산한다.

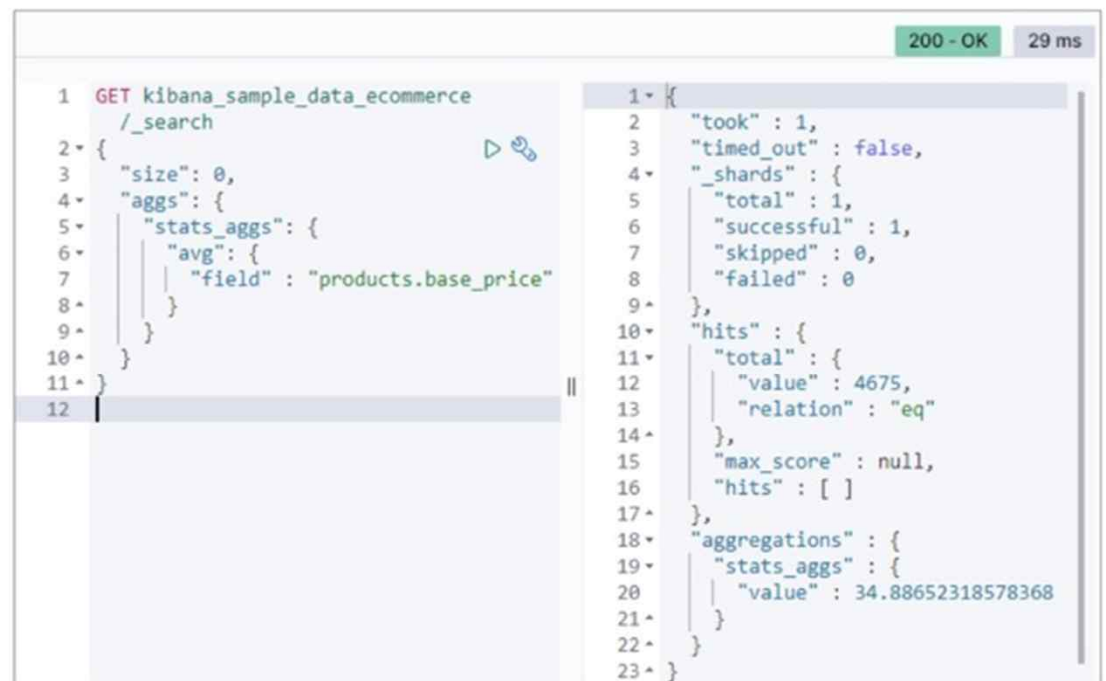
메트릭 집계

1. 평균값/중앙값 구하기

- 평균값을 구하는 평균 집계와, 중간값을 구할 수 있는 백분위 집계

```
GET kibana_sample_data_ecommerce/_search
{
  "size": 0,
  "aggs": {
    "stats_aggs": {
      "avg": {
        "field": "products.base_price"
      }
    }
  }
}
```

리스트 5.3 평균값을 구하는 집계 요청



The screenshot shows a Kibana search console interface. On the left, the query is entered: `GET kibana_sample_data_ecommerce/_search` with an aggregation configuration for `stats_aggs` to calculate the average of `products.base_price`. On the right, the results are displayed in a JSON format. The response includes metadata like `took`, `timed_out`, and `_shards`. The `hits` section shows a total of 1 document. The `aggregations` section contains the `stats_aggs` result, which reports an average value of `34.88652318578368`.

```
1 GET kibana_sample_data_ecommerce
2 /_search
3 {
4   "size": 0,
5   "aggs": {
6     "stats_aggs": {
7       "avg": {
8         "field": "products.base_price"
9       }
10    }
11  }
12 }

1 {
2   "took": 1,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 4675,
13      "relation": "eq"
14    },
15    "max_score": null,
16    "hits": [ ]
17  },
18  "aggregations": {
19    "stats_aggs": {
20      "value": 34.88652318578368
21    }
22  }
23 }
```

그림 5.2 평균 집계 결과

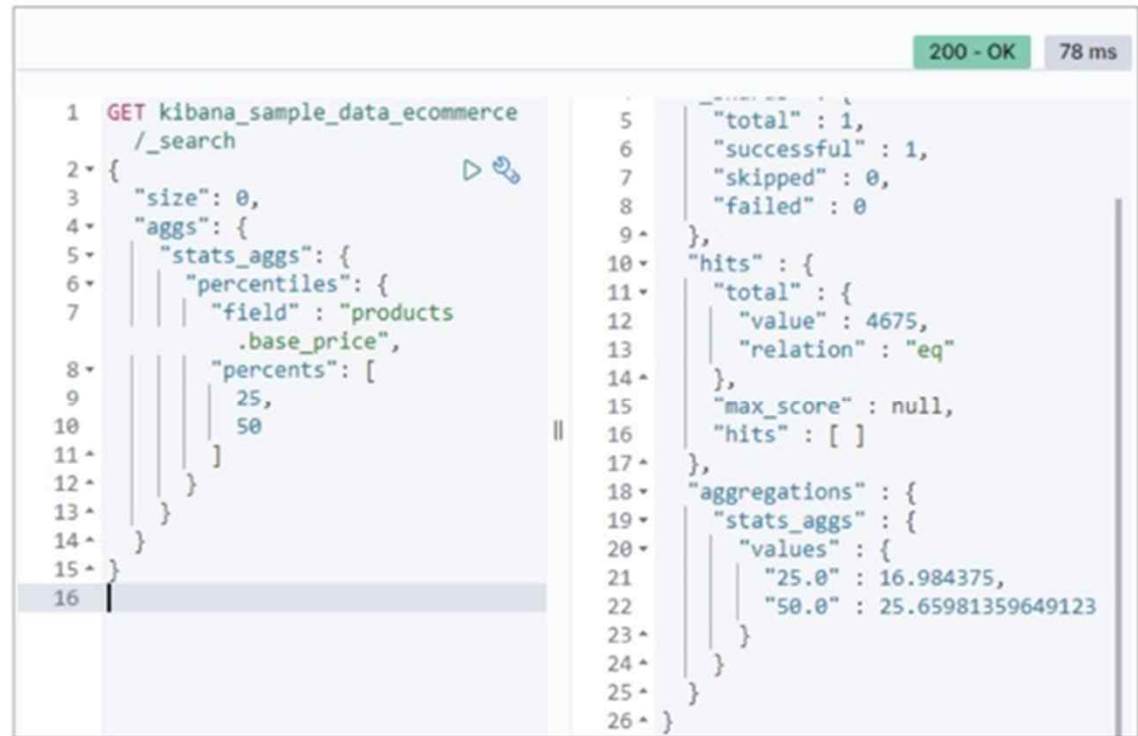
메트릭 집계

1. 평균값/중앙값 구하기

GET kibana_sample_data_ecommerce/_search

```
{
  "size": 0,
  "aggs": {
    "stats_aggs": {
      "percentiles": {
        "field": "products.base_price",
        "percents": [
          25,
          50
        ]
      }
    }
  }
}
```

리스트 5.4 백분위를 구하는 집계 요청



```
1 GET kibana_sample_data_ecommerce
2 /_search
3 {
4   "size": 0,
5   "aggs": {
6     "stats_aggs": {
7       "percentiles": {
8         "field": "products
9         .base_price",
10        "percents": [
11          25,
12          50
13        ]
14      }
15    }
16  }
17 }
18
19 {
20   "total": 1,
21   "successful": 1,
22   "skipped": 0,
23   "failed": 0
24 },
25 {
26   "hits": {
27     "total": {
28       "value": 4675,
29       "relation": "eq"
30     },
31     "max_score": null,
32     "hits": [ ]
33   },
34   "aggregations": {
35     "stats_aggs": {
36       "values": {
37         "25.0" : 16.984375,
38         "50.0" : 25.65981359649123
39       }
40     }
41   }
42 }
```

그림 5.3 백분위 집계 결과

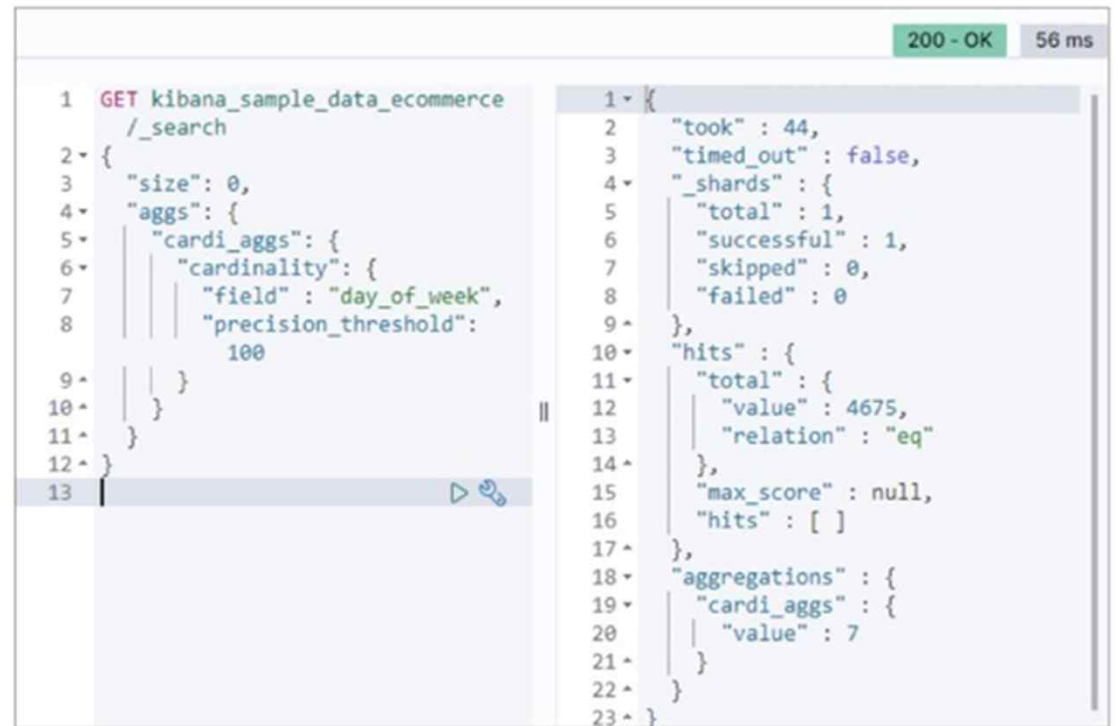
메트릭 집계

2. 필드의 유니크한 값 개수 확인

```
GET kibana_sample_data_ecommerce/_search
{
  "size": 0,
  "aggs": {
    "card_agg": {
      "cardinality": {
        "field": "day_of_week",
        "precision_threshold": 100
      }
    }
  }
}
```

리스트 5.5 카디널리티 집계 요청

precision_threshold: 정확도 수치



```
1 GET kibana_sample_data_ecommerce
  /_search
2 {
3   "size": 0,
4   "aggs": {
5     "card_agg": {
6       "cardinality": {
7         "field": "day_of_week",
8         "precision_threshold":
9           100
10      }
11    }
12  }
13 }

1 {
2   "took": 44,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 4675,
13      "relation": "eq"
14    },
15    "max_score": null,
16    "hits": [ ]
17  },
18  "aggregations": {
19    "card_agg": {
20      "value": 7
21    }
22  }
23 }
```

그림 5.4 카디널리티 집계 결과

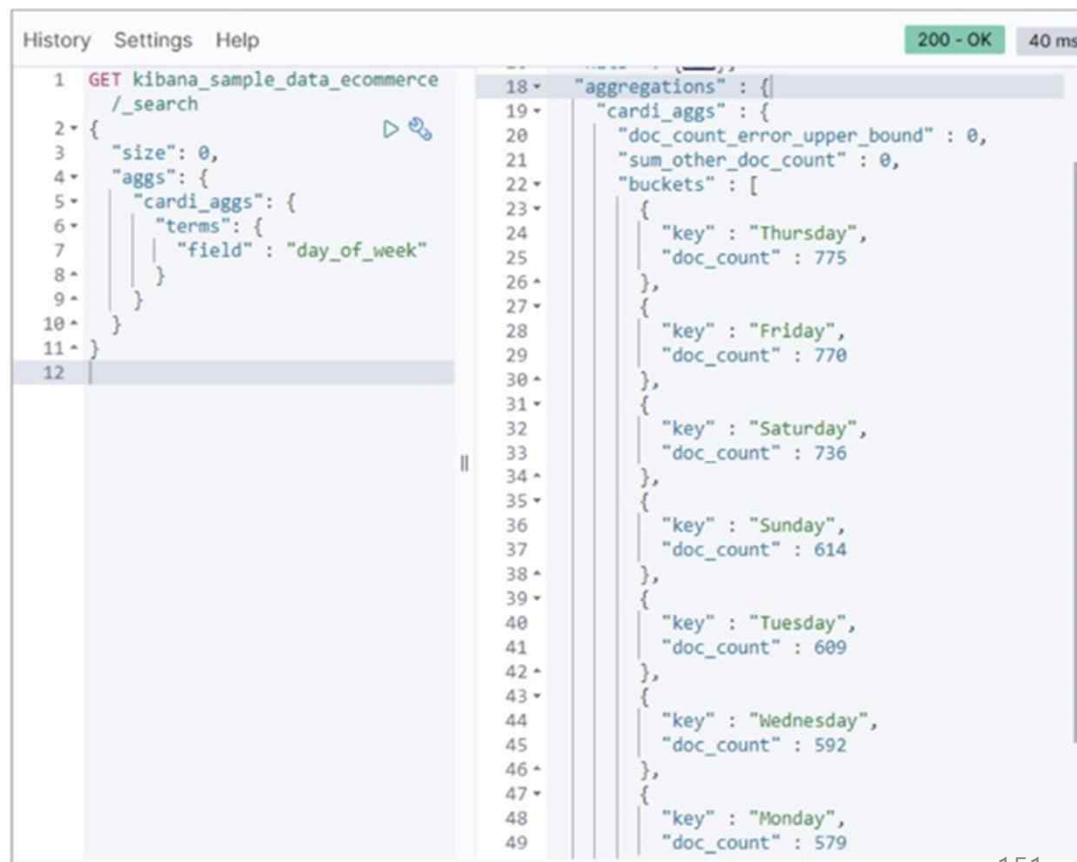
메트릭 집계

2. 필드의 유니크한 값 개수 확인

- 버킷 집계와 함께 용어 집계를 활용하면 유니크한 필드 개수와 함께 필드값들을 확인할 수 있음

```
GET kibana_sample_data_ecommerce/_search
{
  "size": 0,
  "aggs": {
    "card_agg": {
      "terms": {
        "field": "day_of_week"
      }
    }
  }
}
```

리스트 5.6 용어 집계 요청



The screenshot shows the Kibana search interface. The left pane displays the search query, and the right pane shows the results of a terms aggregation on the 'day_of_week' field. The results are presented as a list of buckets, each containing a key (the day of the week) and a doc_count (the number of documents for that day).

key	doc_count
Thursday	775
Friday	770
Saturday	736
Sunday	614
Tuesday	609
Wednesday	592
Monday	579

그림 5.6 용어 집계 결과

메트릭 집계

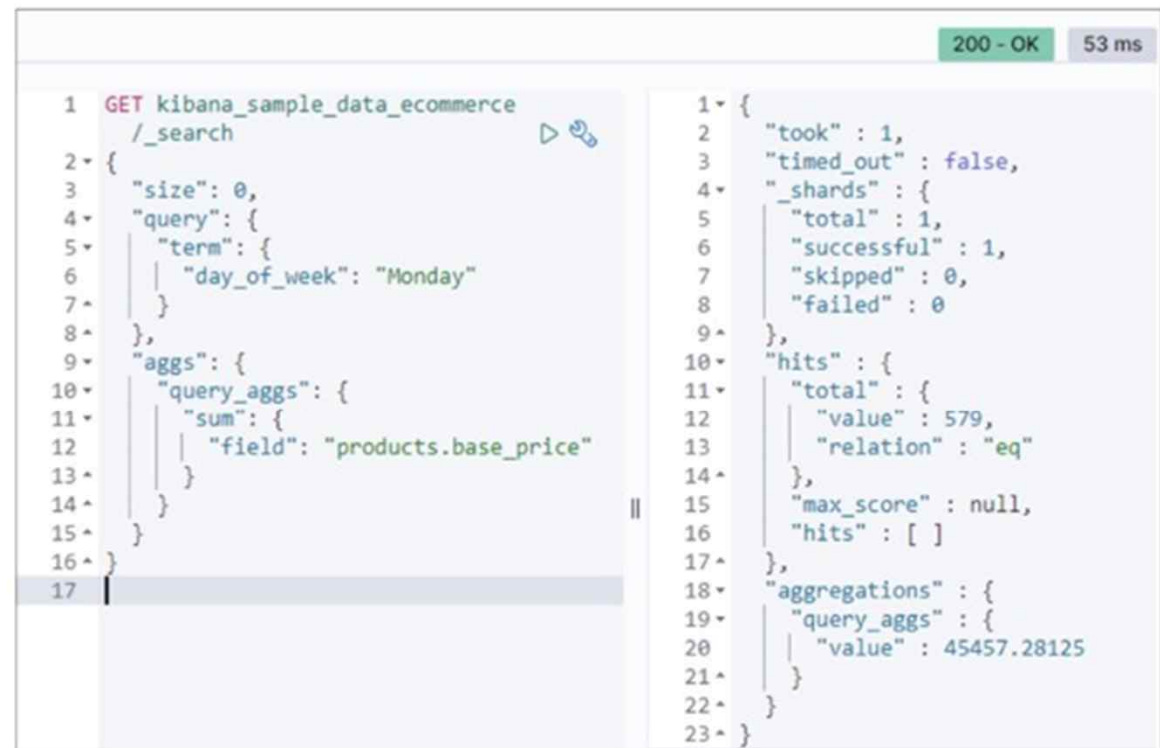
3. 검색 결과 내에서의 집계

- 검색 쿼리와 집계 쿼리를 같이 사용

GET kibana_sample_data_ecommerce/_search

```
{
  "size": 0,
  "query": {
    "term": {
      "day_of_week": "Monday"
    }
  },
  "aggs": {
    "query_aggs": {
      "sum": {
        "field": "products.base_price"
      }
    }
  }
}
```

리스트 5.7 쿼리를 이용해 집계 범위 지정



The screenshot shows a Kibana search console interface. On the left, the search query is entered: `GET kibana_sample_data_ecommerce/_search` with a JSON body: `{ "size": 0, "query": { "term": { "day_of_week": "Monday" } }, "aggs": { "query_aggs": { "sum": { "field": "products.base_price" } } } }`. On the right, the search results are displayed in a JSON format. The status bar at the top right indicates "200 - OK" and "53 ms". The results show a single aggregation: `{ "took": 1, "timed_out": false, "_shards": { "total": 1, "successful": 1, "skipped": 0, "failed": 0 }, "hits": { "total": { "value": 579, "relation": "eq" }, "max_score": null, "hits": [ ] }, "aggregations": { "query_aggs": { "value": 45457.28125 } } }`.

그림 5.7 리스트 5.7 실행 결과

엘라스틱서치: 집계

3 버킷 집계

- 특정 기준에 맞춰 도큐먼트를 그룹핑하는 역할
- 버킷은 도큐먼트가 분할되는 단위로 나뉜 각 그룹을 의미



그림 5.8 버킷 집계

표 5.2 버킷 집계 종류

버킷 집계	설명
histogram	숫자 타입 필드를 일정 간격으로 분류한다.
date_histogram	날짜/시간 타입 필드를 일정 날짜/시간 간격으로 분류한다.
range	숫자 타입 필드를 사용자가 지정하는 범위 간격으로 분류한다.
date_range	날짜/시간 타입 필드를 사용자가 지정하는 날짜/시간 간격으로 분류한다.
terms	필드에 많이 나타나는 용어(값)들을 기준으로 분류한다.
significant_terms	terms 버킷과 유사하나, 모든 값을 대상으로 하지 않고 인덱스 내 전체 문서 대비 현재 검색 조건에서 통계적으로 유의미한 값들을 기준으로 분류한다.
filters	각 그룹에 포함시킬 문서의 조건을 직접 지정한다. 이때 조건은 일반적으로 검색에 사용 되는 쿼리와 동일하다.

버킷 집계

1. 히스토그램 집계

- 숫자 타입 필드를 일정 간격 기준으로 구분해 주는 집계

```
GET kibana_sample_data_ecommerce/_search
{
  "size": 0,
  "aggs": {
    "histogram_aggs": {
      "histogram": {
        "field": "products.base_price",
        "interval": 100
      }
    }
  }
}
```

리스트 5.8 히스토그램 집계 요청

리스트 5.9 히스토그램 집계 결과

```
"aggregations" : {
  "histogram_aggs" : {
    "buckets" : [
      { "key" : 0.0, "doc_count" : 4672 },
      { "key" : 100.0, "doc_count" : 263 },
      { "key" : 200.0, "doc_count" : 12 },
      { "key" : 300.0, "doc_count" : 1 },
      { "key" : 400.0, "doc_count" : 1 },
      { "key" : 500.0, "doc_count" : 0 },
      { "key" : 600.0, "doc_count" : 0 },
      { "key" : 700.0, "doc_count" : 0 },
      { "key" : 800.0, "doc_count" : 0 },
      { "key" : 900.0, "doc_count" : 0 },
      { "key" : 1000.0, "doc_count" : 1 },
    ]
  }
}
```

버킷 집계

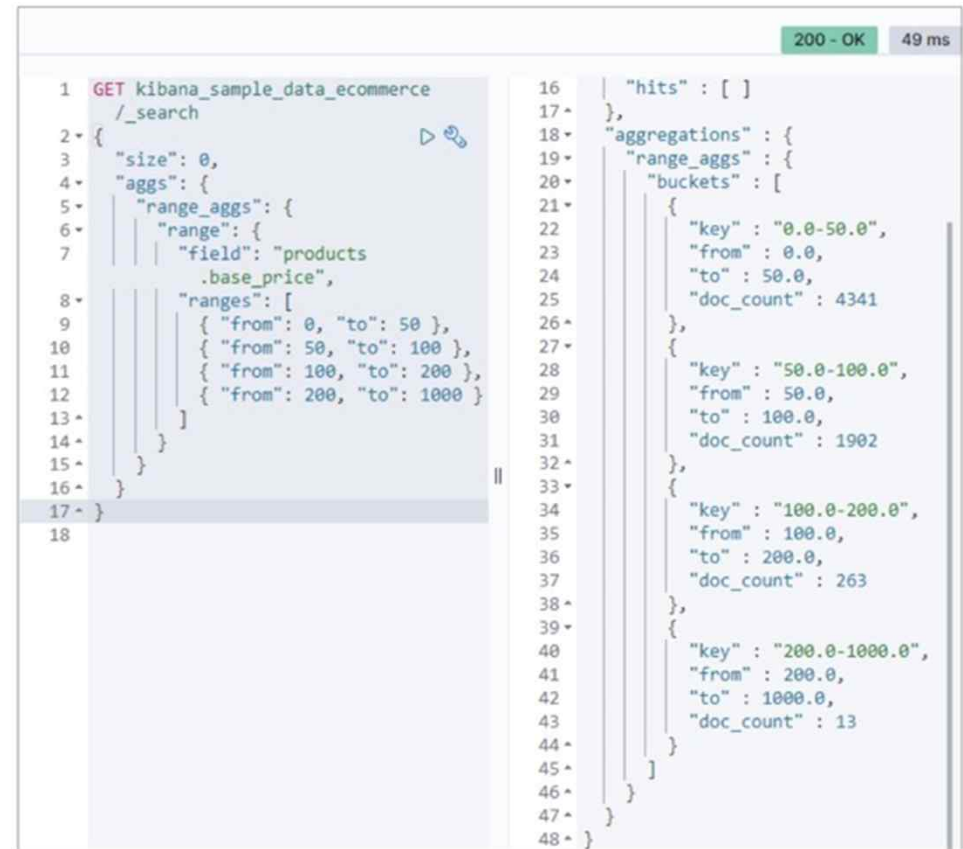
2. 범위 집계

- 각 버킷의 범위를 사용자가 직접 지정

GET kibana_sample_data_ecommerce/_search

```
{
  "size": 0,
  "aggs": {
    "range_aggs": {
      "range": {
        "field": "products.base_price",
        "ranges": [
          {"from": 0, "to": 50},
          {"from": 50, "to": 100},
          {"from": 100, "to": 200},
          {"from": 200, "to": 1000}
        ]
      }
    }
  }
}
```

리스트 5.9 범위 집계 요청



```
1 GET kibana_sample_data_ecommerce
2 /_search
3 {
4   "size": 0,
5   "aggs": {
6     "range_aggs": {
7       "range": {
8         "field": "products
9         .base_price",
10        "ranges": [
11          { "from": 0, "to": 50 },
12          { "from": 50, "to": 100 },
13          { "from": 100, "to": 200 },
14          { "from": 200, "to": 1000 }
15        ]
16      }
17    }
18  }
19 }
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
16 | "hits" : [ ]
17 },
18 "aggregations" : {
19   "range_aggs" : {
20     "buckets" : [
21       {
22         "key" : "0.0-50.0",
23         "from" : 0.0,
24         "to" : 50.0,
25         "doc_count" : 4341
26       },
27       {
28         "key" : "50.0-100.0",
29         "from" : 50.0,
30         "to" : 100.0,
31         "doc_count" : 1902
32       },
33       {
34         "key" : "100.0-200.0",
35         "from" : 100.0,
36         "to" : 200.0,
37         "doc_count" : 263
38       },
39       {
40         "key" : "200.0-1000.0",
41         "from" : 200.0,
42         "to" : 1000.0,
43         "doc_count" : 13
44       }
45     ]
46   }
47 }
48 }
```

그림 5.9 범위 집계 결과

버킷 집계

2. 범위 집계

- 범위에 따라 문서의 총합이 다른 이유

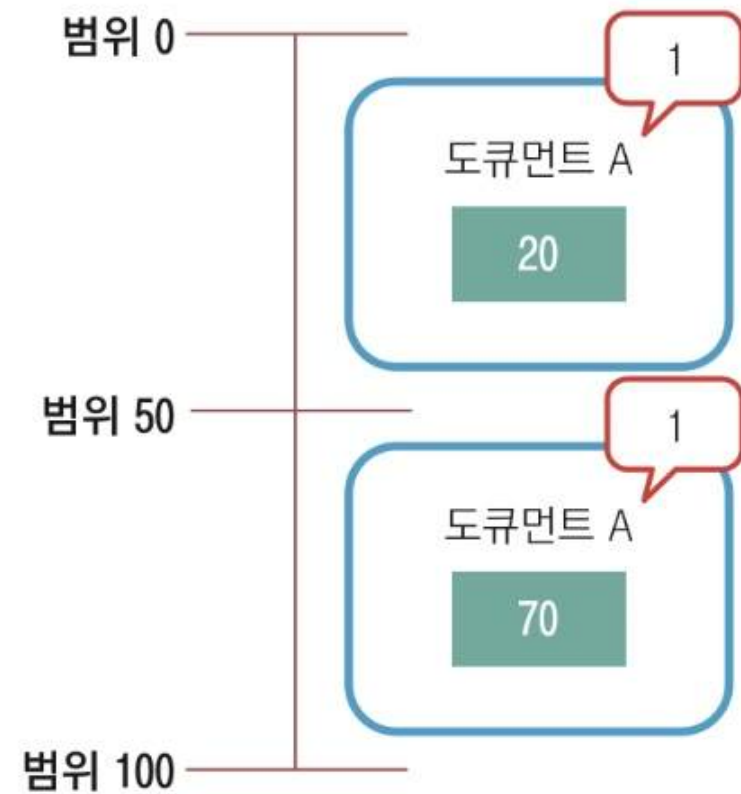
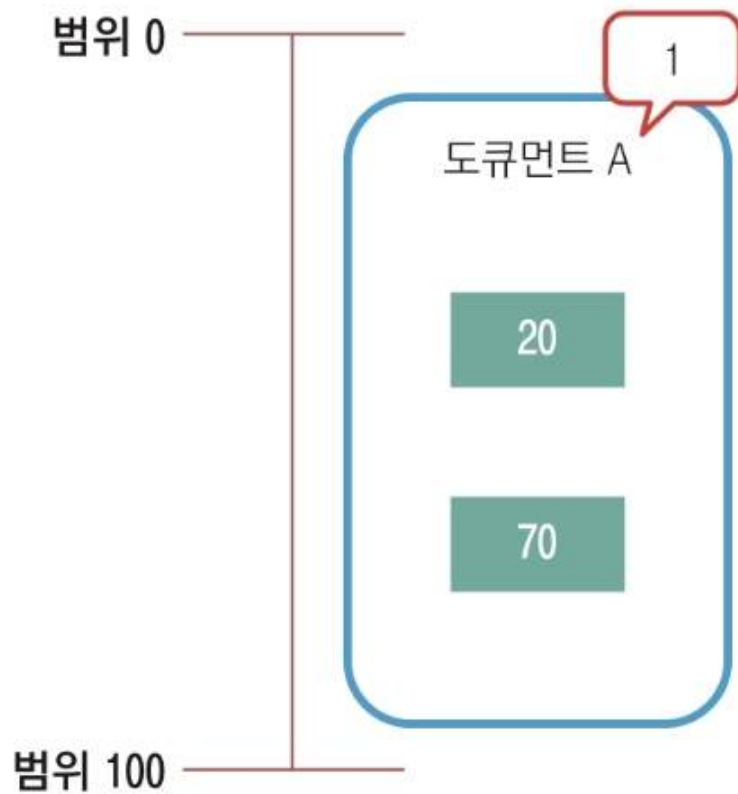


그림 5.10 범위에 따라 문서 총합이 다른 이유

버킷 집계

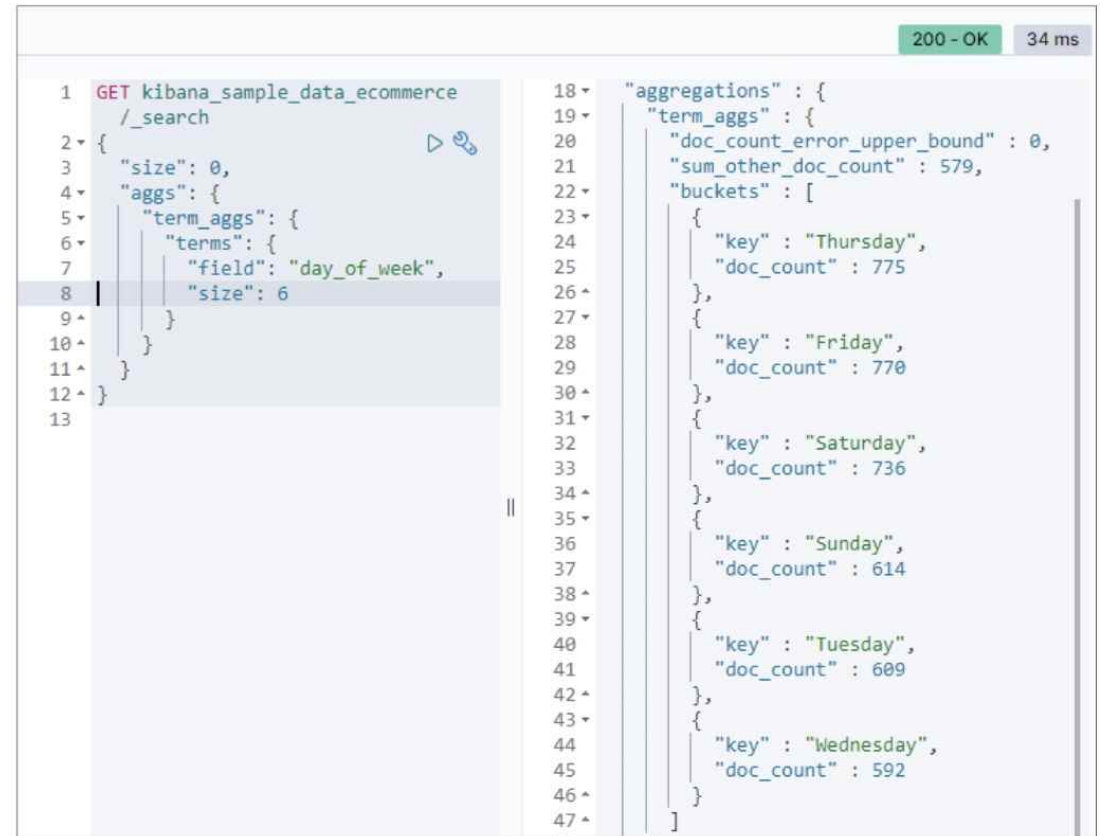
3. 용어 집계

- 필드의 유니크한 값을 기준으로 나눌 때 사용

GET kibana_sample_data_ecommerce/_search

```
{
  "size": 0,
  "aggs": {
    "term_aggs": {
      "terms": {
        "field": "day_of_week",
        "size": 6
      }
    }
  }
}
```

리스트 5.11 용어 집계 요청



```
1 GET kibana_sample_data_ecommerce
2 /_search
3 {
4   "size": 0,
5   "aggs": {
6     "term_aggs": {
7       "terms": {
8         "field": "day_of_week",
9         "size": 6
10      }
11    }
12  }
13 }

18 "aggregations" : {
19   "term_aggs" : {
20     "doc_count_error_upper_bound" : 0,
21     "sum_other_doc_count" : 579,
22     "buckets" : [
23       {
24         "key" : "Thursday",
25         "doc_count" : 775
26       },
27       {
28         "key" : "Friday",
29         "doc_count" : 770
30       },
31       {
32         "key" : "Saturday",
33         "doc_count" : 736
34       },
35       {
36         "key" : "Sunday",
37         "doc_count" : 614
38       },
39       {
40         "key" : "Tuesday",
41         "doc_count" : 609
42       },
43       {
44         "key" : "Wednesday",
45         "doc_count" : 592
46       }
47     ]
48   }
49 }
```

그림 5.11 용어 집계 결과

버킷 집계

3. 용어 집계

| 용어 집계가 정확하지 않은 이유

- 분산 시스템의 집계 과정에서 발생할 수 있는 잠재적인 오류 가능성 때문
- 데이터를 여러 노드에 분산하고 취합하는 과정에서 오류가 발생.

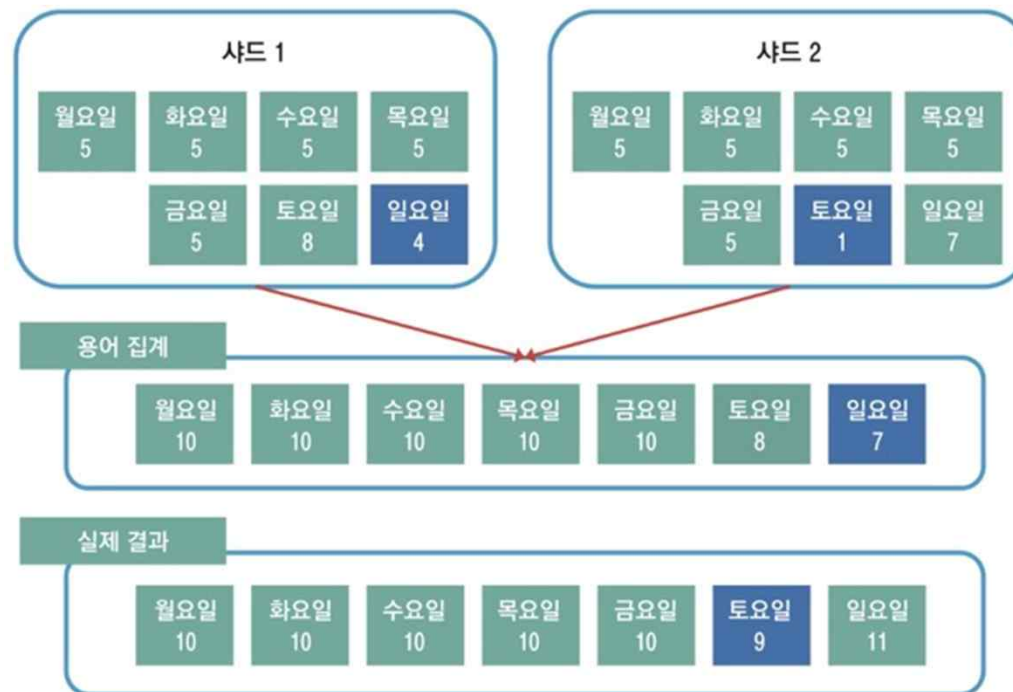


그림 5.12 용어 집계 과정에서 발생할 수 있는 오류

버킷 집계

3. 용어 집계

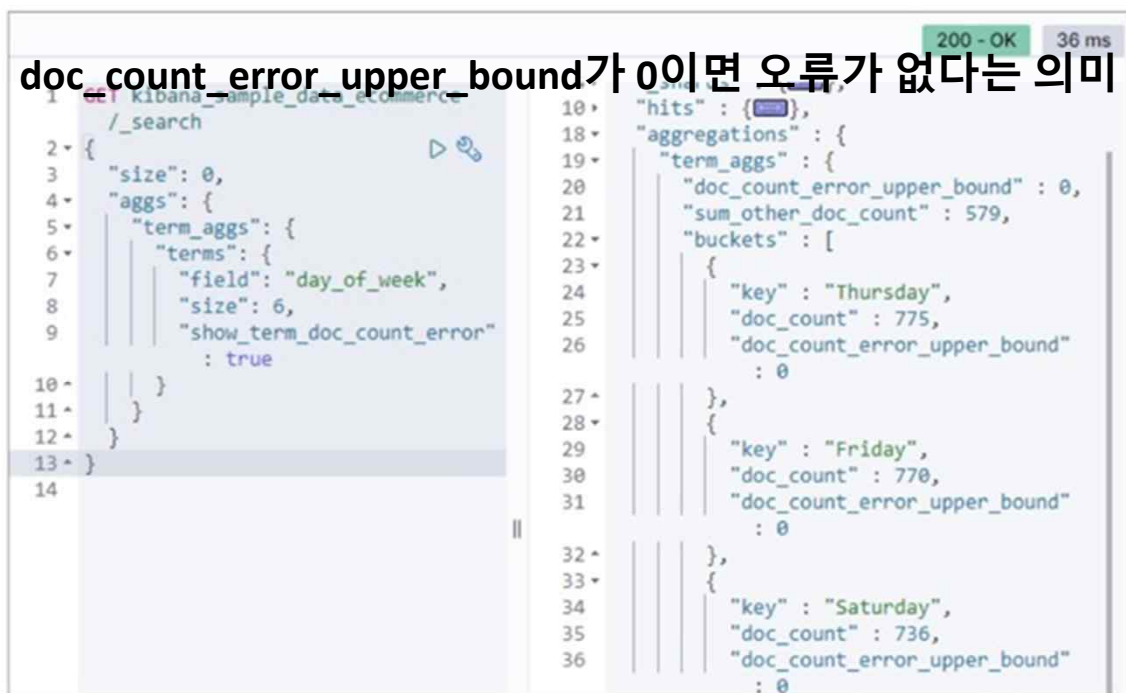
| 용어 집계의 정확성 높이기

- 분산처리 환경에서 발생하는 오류로 인해 성능이 저하됨으로, 고속 처리를 위한 리소스와 속도 간 트레이드오프 일환으로 리소스 소비량을 늘려 정확도를 높일 수 있다.

```
GET kibana_sample_data_ecommerce/_search
{
  "size": 0,
  "aggs": {
    "term_aggs": {
      "terms": {
        "field": "day_of_week",
        "size": 6,
        "show_term_doc_count_error": true
      }
    }
  }
}
```

리스트 5.12 용어 집계 오류 확인 요청

doc_count_error_upper_bound가 0이면 오류가 없다는 의미



```
1 GET kibana_sample_data_ecommerce/_search
2 {
3   "size": 0,
4   "aggs": {
5     "term_aggs": {
6       "terms": {
7         "field": "day_of_week",
8         "size": 6,
9         "show_term_doc_count_error": true
10      }
11    }
12  }
13 }
14
15 200 - OK 36 ms
16
17 "hits": {
18   "aggregations": {
19     "term_aggs": {
20       "doc_count_error_upper_bound": 0,
21       "sum_other_doc_count": 579,
22       "buckets": [
23         {
24           "key": "Thursday",
25           "doc_count": 775,
26           "doc_count_error_upper_bound": 0
27         },
28         {
29           "key": "Friday",
30           "doc_count": 770,
31           "doc_count_error_upper_bound": 0
32         },
33         {
34           "key": "Saturday",
35           "doc_count": 736,
36           "doc_count_error_upper_bound": 0
37         }
38       ]
39     }
40   }
41 }
```

그림 5.13 리스트 5.12 실행 결과

버킷 집계

3. 용어 집계

| 용어 집계의 정확성 높이기

- `doc_count_error_upper_bound`에 이상값이 존재하면 샤드 크기 파라미터 조절 할 필요 있음

```
GET kibana_sample_data_ecommerce/_search
{
  "size": 0,
  "aggs": {
    "term_aggs": {
      "terms": {
        "field": "day_of_week",
        "size": 6,
        "shard_size": 100
      }
    }
  }
}
```

`shard_size`는 용어 집계 시 개별 샤드에서 집계를 위해 처리하는 개수를 의미. 샤드 크기가 커지면 정확도가 올라가지만 리소스 사용량 또한 올라간다.

리스트 5.13 용어 집계 시 샤드 크기를 늘린 요청

엘라스틱서치: 집계

4 집계 조합

- RDB에서 GROUP BY절로 그룹핑 한 다음 통계함수를 사용하는 것처럼 버킷 집계와 메트릭 집계를 활용하면 그룹별 통계를 계산 할 수 있음.

1. 버킷 집계와 메트릭 집계

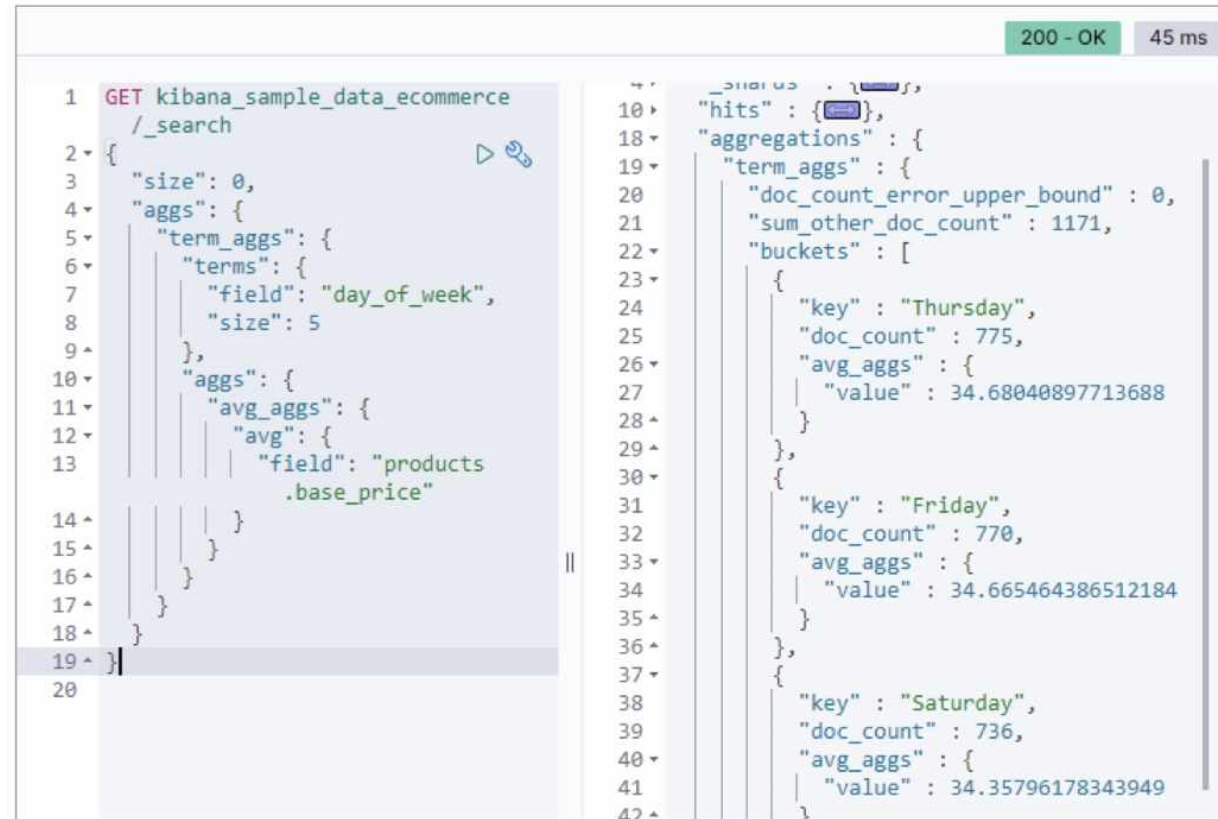
- 버킷 집계로 문서를 그룹핑하고 각 버킷 집계별 메트릭 집계를 사용

집계 조합

1. 버킷 집계와 메트릭 집계

GET kibana_sample_data_ecommerce/_search

```
{
  "size": 0,
  "aggs": {
    "term_aggs": {
      "terms": {
        "field": "day_of_week",
        "size": 5
      },
      "aggs": {
        "avg_aggs": {
          "avg": {
            "field": "products.base_price"
          }
        }
      }
    }
  }
}
```



```
1 GET kibana_sample_data_ecommerce
  /_search
2 {
3   "size": 0,
4   "aggs": {
5     "term_aggs": {
6       "terms": {
7         "field": "day_of_week",
8         "size": 5
9       },
10      "aggs": {
11        "avg_aggs": {
12          "avg": {
13            "field": "products
14              .base_price"
15          }
16        }
17      }
18    }
19  }
20 }

21 "hits" : {},
22 "aggregations" : {
23   "term_aggs" : {
24     "doc_count_error_upper_bound" : 0,
25     "sum_other_doc_count" : 1171,
26     "buckets" : [
27       {
28         "key" : "Thursday",
29         "doc_count" : 775,
30         "avg_aggs" : {
31           "value" : 34.68040897713688
32         }
33       },
34       {
35         "key" : "Friday",
36         "doc_count" : 770,
37         "avg_aggs" : {
38           "value" : 34.665464386512184
39         }
40       },
41       {
42         "key" : "Saturday",
43         "doc_count" : 736,
44         "avg_aggs" : {
45           "value" : 34.35796178343949
46         }
47       }
48     ]
49   }
50 }
```

그림 5.14 리스트 5.14 실행 결과

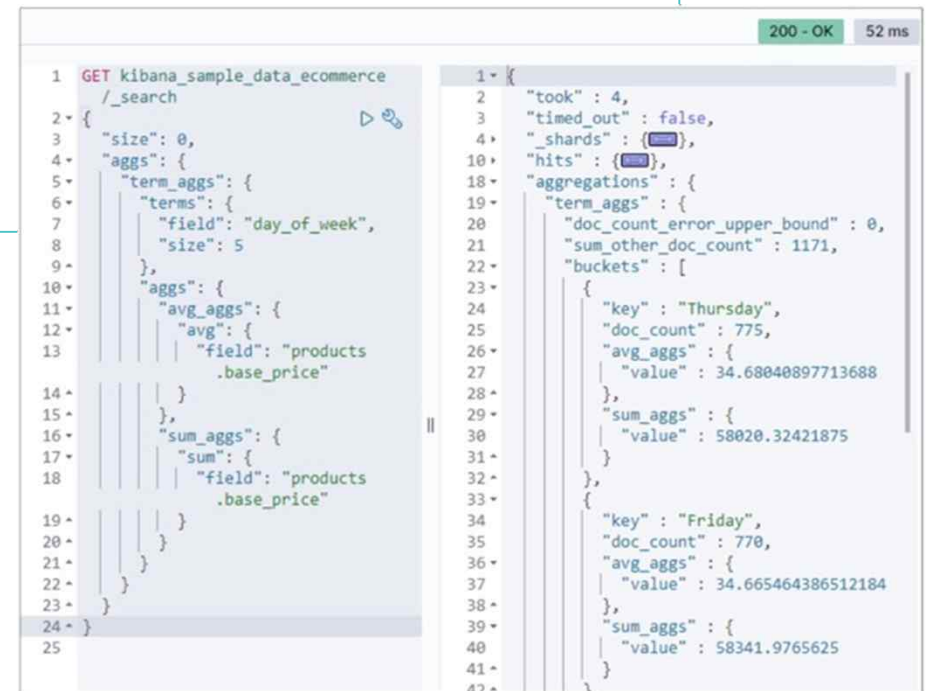
집계 조합

1. 버킷 집계와 메트릭 집계

```
GET kibana_sample_data_ecommerce/_search
{
  "size": 0,
  "aggs": {
    "term_aggs": {
      "terms": {
        "field": "day_of_week",
        "size": 5
      },
      "aggs": {
        "avg_aggs": {
          "avg": {
            "field": "products.base_price"
          }
        },
        "sum_aggs": {
          "sum": {
            "field": "products.base_price"
          }
        }
      }
    }
  }
}
```

리스트 5.15 버킷 집계 후 다수의 메트릭 집계 요청

```
"sum_aggs":{
  "sum": {
    "field": "products.base_price"
  }
}
```



```
1 GET kibana_sample_data_ecommerce
2 /_search
3 {
4   "size": 0,
5   "aggs": {
6     "term_aggs": {
7       "terms": {
8         "field": "day_of_week",
9         "size": 5
10      },
11      "aggs": {
12        "avg_aggs": {
13          "avg": {
14            "field": "products
15              .base_price"
16          }
17        },
18        "sum_aggs": {
19          "sum": {
20            "field": "products
21              .base_price"
22          }
23        }
24      }
25    }
26  }
27 }

1 {
2   "took": 4,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": 0,
12    "max_score": null,
13    "hits": []
14  },
15  "aggregations": {
16    "term_aggs": {
17      "doc_count_error_upper_bound": 0,
18      "sum_other_doc_count": 1171,
19      "buckets": [
20        {
21          "key": "Thursday",
22          "doc_count": 775,
23          "avg_aggs": {
24            "value": 34.68040897713688
25          },
26          "sum_aggs": {
27            "value": 58020.32421875
28          }
29        },
30        {
31          "key": "Friday",
32          "doc_count": 770,
33          "avg_aggs": {
34            "value": 34.665464386512184
35          },
36          "sum_aggs": {
37            "value": 58341.9765625
38          }
39        }
40      ]
41    }
42  }
43 }
```

그림 5.15 리스트 5.15 실행 결과

집계 조합

2. 서브 버킷 집계

- 버킷 안에서 다시 버킷 집계를 요청하는 집계

GET kibana_sample_data_ecommerce/_search

```
{
  "size": 0,
  "aggs": {
    "histogram_aggs": {
      "histogram": {
        "field": "products.base_price",
        "interval": 100
      },
      "aggs": {
        "term_aggs": {
          "terms": {
            "field": "day_of_week",
            "size": 2
          }
        }
      }
    }
  }
}
```

리스트 5.16 서브 버킷 생성 요청

```
1 GET kibana_sample_data_ecommerce
2 /_search
3 {
4   "size": 0,
5   "aggs": {
6     "histogram_aggs": {
7       "histogram": {
8         "field": "products
9         .base_price",
10        "interval": 100
11      },
12      "aggs": {
13        "term_aggs": {
14          "terms": {
15            "field": "day_of_week",
16            "size": 2
17          }
18        }
19      }
20    }
21  }
22 }
23
24 "aggregations" : {
25   "histogram_aggs" : {
26     "buckets" : [
27       {
28         "key" : 0.0,
29         "doc_count" : 4672,
30         "term_aggs" : {
31           "doc_count_error_upper_bound" : 0,
32           "sum_other_doc_count" : 3128,
33           "buckets" : [
34             {
35               "key" : "Thursday",
36               "doc_count" : 775
37             },
38             {
39               "key" : "Friday",
40               "doc_count" : 769
41             }
42           ]
43         }
44       },
45       {
46         "key" : 100.0,
47         "doc_count" : 263,
48         "term_aggs" : {
49           "doc_count_error_upper_bound" : 0,
50           "sum_other_doc_count" : 176,
51           "buckets" : [
52             {
53               "key" : "Friday",
54               "doc_count" : 44
55             },
56             {
57               "key" : "Thursday",
58               "doc_count" : 43
59             }
60           ]
61         }
62       }
63     ]
64   }
65 }
```

그림 5.16 리스트 5.16 실행 결과

엘라스틱서치: 집계

5 파이프라인 집계

- 이전 결과를 다음 단계에서 이용하는 파이프라인 개념을 차용하는 집계.
- 이전 집계로 만들어진 결과를 다시 입력으로 삼아 집계하는 방식.
- 부모 집계와 형제 집계 두 가지 유형이 존재.

표 5.3 부모/형제 집계

집계 종류	설명
부모 집계	<pre>{ "aggs": { ... "aggs": { ... "부모 집계" } } }</pre> 기존 집계 결과를 이용해 새로운 집계를 생성한다. 결과는 기존 집계 내부에서 나온다.
형제 집계	<pre>{ "aggs": { ... "aggs": { ... }, "형제 집계" } }</pre> 기존 집계를 참고해 집계를 수행한다. 결과는 기존 집계와 동일선상에서 나온다.

엘라스틱서치: 집계

5 파이프라인 집계

· 자주 사용되는 파이프라인 집계

표 5.4 파이프라인 집계 종류

형제/부모 집계	집계 종류	설명
형제 집계	min_bucket	기존 집계 중 최소값을 구한다.
	max_bucket	기존 집계 중 최대값을 구한다.
	avg_bucket	기존 집계의 평균값을 구한다.
	sum_bucket	기존 집계의 총합을 구한다.
	stat_bucket	기존 집계의 min, max, sum, count, avg를 구한다.
	percentile_bucket	기존 집계의 백분위값을 구한다.
	moving_avg	기존 집계의 이동 평균을 구한다. 단, 기존 집계는 순차적인 데이터 구조여야 한다.
부모 집계	derivative	기존 집계의 미분을 구한다.
	cumulative_sum	기존 집계의 누적합을 구한다.

파이프라인 집계

1. 부모 집계

- 부모 집계는 단독으로 사용할 수 없고 먼저 다른 집계기가 있어야 한다.
- `products.base_price`를 100의 기준으로 버킷을 나눈 후 각 버킷에서 `taxful_total_price`의 합을 구하는 집계.
- 부모 집계는 합계 집계를 입력으로 받아 최종적으로 각 버킷의 누적합을 계산.

```
GET kibana_sample_data_ecommerce/_search
{
  "size": 0,
  "aggs": {
    "histogram_aggs": {
      "histogram": {
        "field": "products.base_price",
        "interval": 100
      },

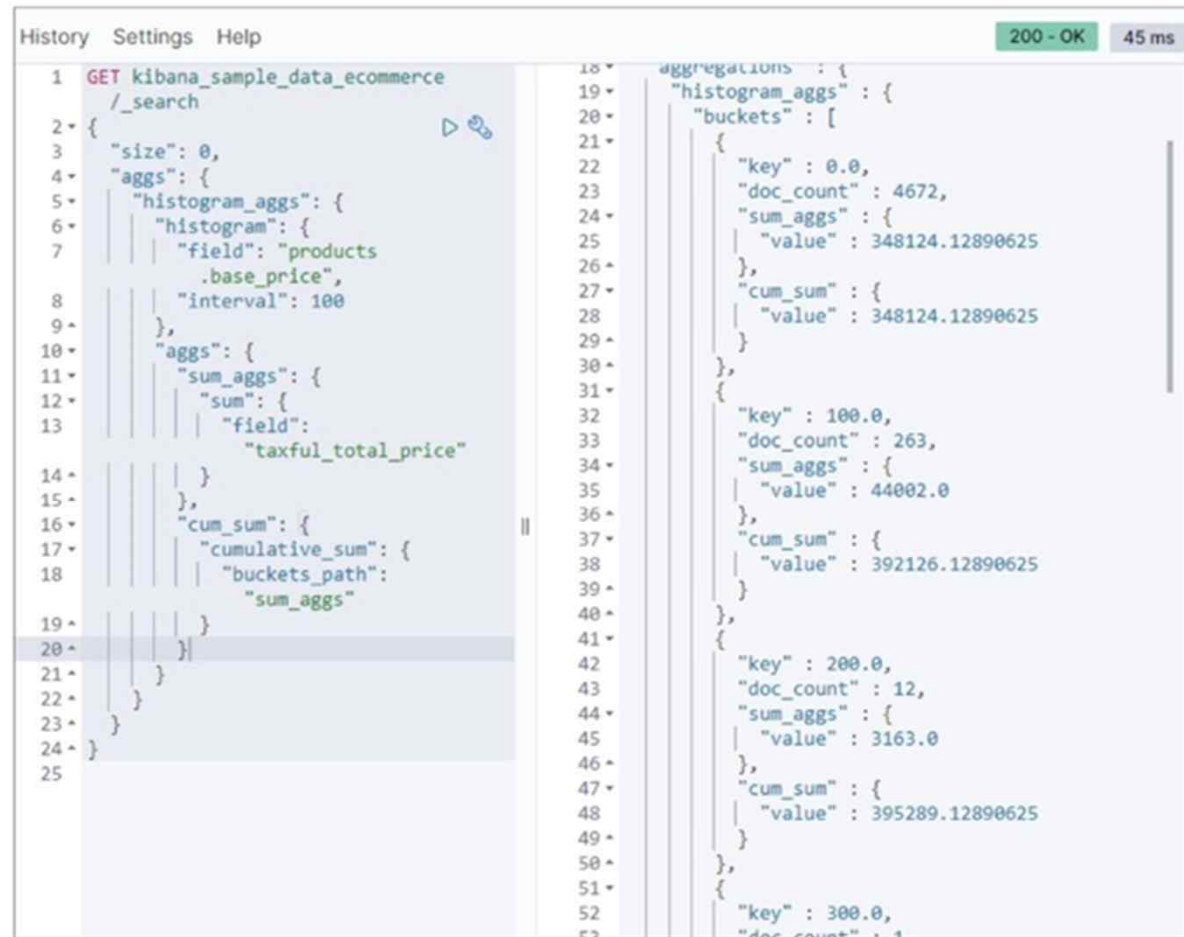
```

```
    "aggs": {
      "sum_aggs": {
        "sum": {
          "field": "taxful_total_price"
        }
      },
      "cum_sum": {
        "cumulative_sum": {
          "buckets_path": "sum_aggs"
        }
      }
    }
  }
}
```

리스트 5.17 누적합을 구하는 부모집계 요청

파이프라인 집계

1. 부모 집계



```
History Settings Help 200 - OK 45 ms
1 GET kibana_sample_data_ecommerce
  /_search
2 {
3   "size": 0,
4   "aggs": {
5     "histogram_aggs": {
6       "histogram": {
7         "field": "products
          .base_price",
8         "interval": 100
9       },
10      "aggs": {
11        "sum_aggs": {
12          "sum": {
13            "field":
              "taxful_total_price"
14          }
15        },
16        "cum_sum": {
17          "cumulative_sum": {
18            "buckets_path":
              "sum_aggs"
19          }
20        }
21      }
22    }
23  }
24 }
25
18 aggregations : {
19   "histogram_aggs" : {
20     "buckets" : [
21       {
22         "key" : 0.0,
23         "doc_count" : 4672,
24         "sum_aggs" : {
25           "value" : 348124.12890625
26         },
27         "cum_sum" : {
28           "value" : 348124.12890625
29         }
30       },
31       {
32         "key" : 100.0,
33         "doc_count" : 263,
34         "sum_aggs" : {
35           "value" : 44002.0
36         },
37         "cum_sum" : {
38           "value" : 392126.12890625
39         }
40       },
41       {
42         "key" : 200.0,
43         "doc_count" : 12,
44         "sum_aggs" : {
45           "value" : 3163.0
46         },
47         "cum_sum" : {
48           "value" : 395289.12890625
49         }
50       },
51       {
52         "key" : 300.0,
53         "doc_count" : 1
```

그림 5.17 리스트 5.17 실행 결과

파이프라인 집계

2. 형제 집계

- 부모 집계는 기존 집계 내부에서 집계 작업을 했다.
- 형제 집계는 기존 집계 내부가 아닌 외부에서 기존 집계를 이용해 집계 작업을 함.
- day_of_week 기준으로 상위 2개의 버킷을 생성하고, sum_aggs에서 products.base_price 필드의 총합을 구한 후 sum_bucket 형제 집계를 이용해 기존 버킷별 합을 구한 집계를 다시 합친다.
- 버킷 경로에서 '>'는 하위 집계 경로를 나타냄.

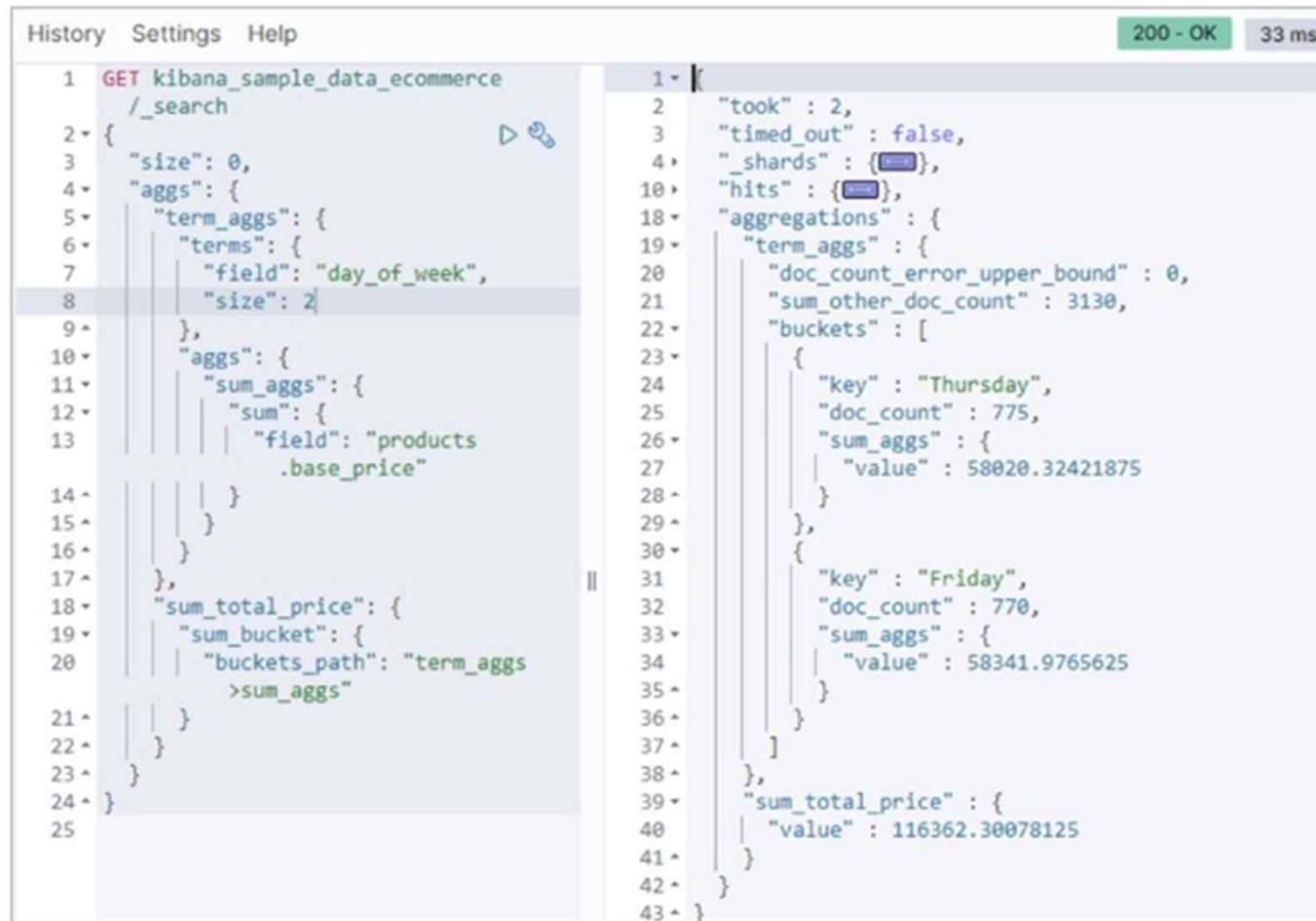
```
GET kibana_sample_data_ecommerce/_search
{
  "size": 0,
  "aggs": {
    "term_aggs": {
      "terms": {
        "field": "day_of_week",
        "size": 2
      },

```

```
    "aggs": {
      "sum_aggs": {
        "sum": {
          "field": "products.base_price"
        }
      },
    },
    "sum_total_price": {
      "sum_bucket": {
        "buckets_path": "term_aggs>sum_aggs"
      }
    }
  }
}
```

파이프라인 집계

2. 형제 집계



```
History Settings Help 200 - OK 33 ms

1 GET kibana_sample_data_ecommerce
  /_search
2 {
3   "size": 0,
4   "aggs": {
5     "term_aggs": {
6       "terms": {
7         "field": "day_of_week",
8         "size": 2
9       },
10    "aggs": {
11      "sum_aggs": {
12        "sum": {
13          "field": "products
14            .base_price"
15        }
16      }
17    },
18    "sum_total_price": {
19      "sum_bucket": {
20        "buckets_path": "term_aggs
21          >sum_aggs"
22      }
23    }
24  }
25

1 {
2   "took": 2,
3   "timed_out": false,
4   "_shards": { "total": 1, "successful": 1, "skipped": 0, "no_shards": 0 },
10  "hits": { "total": 0, "max_score": null, "hits": [] },
18  "aggregations": {
19    "term_aggs": {
20      "doc_count_error_upper_bound": 0,
21      "sum_other_doc_count": 3130,
22      "buckets": [
23        {
24          "key": "Thursday",
25          "doc_count": 775,
26          "sum_aggs": {
27            "value": 58020.32421875
28          }
29        },
30        {
31          "key": "Friday",
32          "doc_count": 770,
33          "sum_aggs": {
34            "value": 58341.9765625
35          }
36        }
37      ]
38    },
39    "sum_total_price": {
40      "value": 116362.30078125
41    }
42  }
43 }
```

그림 5.18 리스트 5.18 실행 결과



05

비츠

비츠

- 가볍고 사용하기 쉬운 데이터 수집기.
- 로그스태시, 엘라스틱서치와 연계해 다양한 시스템 이벤트를 수집할 수 있음.
- 비츠는 오픈소스 라이선스를 따르며, 립비트(Libbeat)라는 프레임워크를 제공, 사용자들이 직접 필요한 각 비트를 구현할 수 있음.

비츠

1 비츠 소개

- 비츠는 하나의 목적만 수행.
- 목적별로 다양한 비츠가 존재.
- 로그스테시는 범용성이 높고 무거운 반면, 비츠는 범용성이 낮고 가벼움.

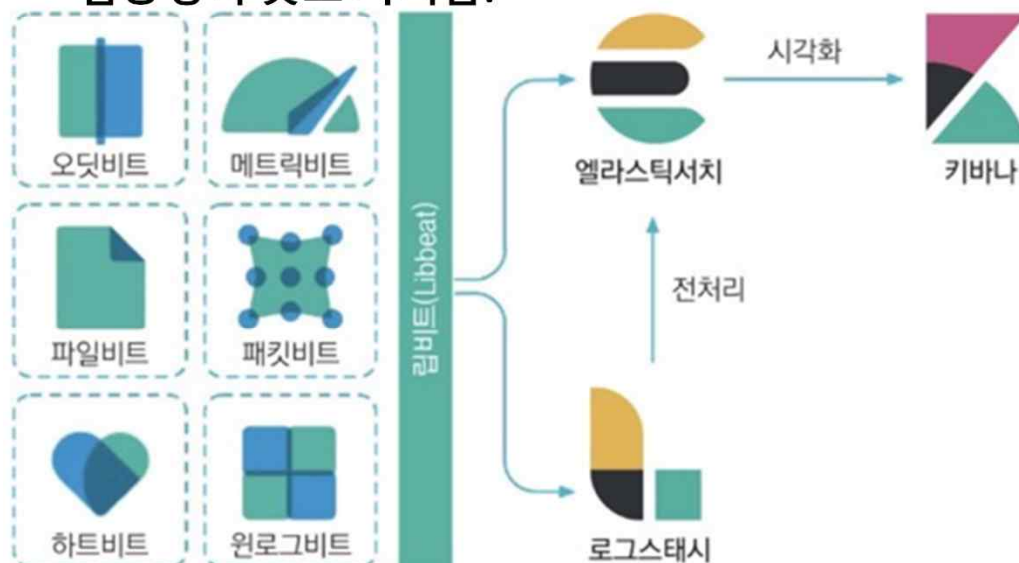


그림 7.1 엘라스틱 스택에서 비츠의 역할¹

커스텀 비츠

표 7.1 비츠 종류

비트명	설명
파일비트 (filebeat)	로그 파일을 실시간으로 읽어 들여 전송한다. 로그스테시나 엘라스틱서치에 데이터 전달 시 부하 방지 기능을 지원한다. 가장 많이 사용되는 비츠다.
하트비트 (heartbeat)	서비스가 살아 있는지 확인하고 모니터링한다. 핑(ping), HTTP 서비스 체크 등 서비스 상태를 파악하기 위한 여러 방법들을 지원하며 크론(cron) 스케줄 표현식을 기반으로 주기적으로 서비스 활성 여부를 판단한다.
메트릭비트 (metricbeat)	운영체제를 포함하여 서비스로부터 주기적으로 통계 지표를 수집한다. 시스템의 CPU나 메모리 같은 정보나 데이터베이스, 프록시 같은 특정 서비스의 통계 데이터도 수집할 수 있다.
패킷비트 (packetbeat)	네트워크 패킷 분석기로 네트워크 데이터를 수집한다. 트래픽, 응답 시간, 사용자 패턴 분석 같은 작업 등을 패킷비트를 이용해 할 수 있다.
오딧비트 (auditbeat)	리눅스의 시스템 보안 정보를 감사하는 auditd에서 생성하는 로그를 수집한다. 사용자가 사전에 정의한 규칙에 따르지 않는 로그들을 식별하여 계정 미준수나 보안 정책에 위배되는 동작을 식별할 수 있다.
저널비트 (journalbeat)	리눅스 systemd로 실행되는 서비스들에 대한 로그를 수집한다.
펑션비트 (functionbeat)	클라우드의 서버리스 환경에서 사용할 수 있고 아마존의 큐 데이터 등을 수집한다.
윈로그비트 (winlogbeat)	윈도우에서 발생하는 이벤트 로그를 수집한다.
커스텀 비츠 (custom beats)	오픈소스 커뮤니티에서 만든 비츠다. 링크 ² 를 따라가 보면 다양한 비츠들을 확인할 수 있다. 아파치 비트(apachebeat), 카프카비트(kafkabeat), 엔진엑스비트(nginxbeat) 등 공식 지원하지 않더라도 유용한 비츠들이 이미 많이 만들어져 있으므로 필요한 비츠를 찾아보고 활용하는 것도 좋다.

2 비츠 설치

- <https://www.elastic.co/kr/downloads/beats/>에서 다운로드 가능.
- <https://www.elastic.co/guide/en/beats/libbeat/7.10/community-beats.html>

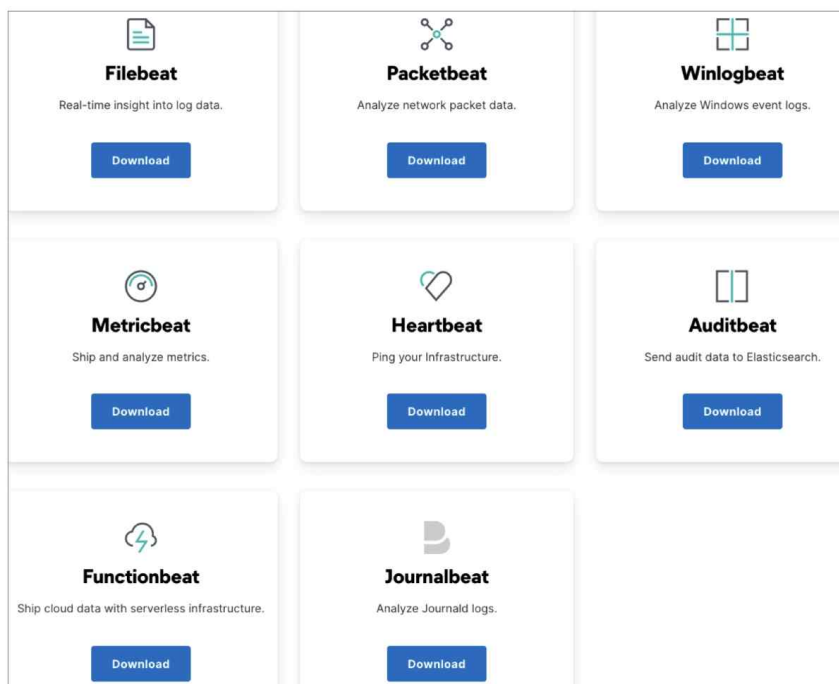


그림 7.2 비츠 다운로드 페이지

amazonbeat	Reads data from a specified Amazon product.
apachebeat	Reads status from Apache HTTPD server-status.
apexbeat	Extracts configurable contextual data and metrics from Java applications via the APEX toolkit.
browserbeat	Reads and ships browser history (Chrome, Firefox, & Safari) to an Elastic output.
cborbeat	Reads from cbor encoded files (specifically log files). More: CBOR Encoding Decoder
cloudflarebeat	Indexes log entries from the Cloudflare Enterprise Log Share API.

그림 7.3 커스텀 비츠 페이지

비츠

2 비츠 설치

· 비츠 동작 과정



그림 7.4 비트 동작 과정

3 파일 설치

- 비츠 사용 방법은 크게 어렵지 않고 비츠 별로 크게 다르지 않기 때문에 파일비트의 사용 방법을 익혀두면 다른 비츠를 사용하는데 어려움이 없다.

1. 파일비트 아키텍처

- 파일비트는 시스템 운영에서 발생하는 로그들을 파일 형태로 발생.
- 가볍기 때문에 수집할 시스템에 부담 없이 설치 가능.
- 간단한 필터 작업 가능.

표 7.2 파일비트 구성요소

구성요소	내용
입력(input)	설정 파일에서 하베스터에 대한 입력 소스를 정한다. 파일비트는 하나 혹은 여러 개의 입력을 가질 수 있다.
하베스터(harvester)	입력에 명시된 파일을 직접 수집하는 주체다. 파일은 하나의 하베스터를 가지며, 하베스터는 파일을 한 줄씩 읽고 내보내는 역할을 한다. 또한 파일을 열고 닫는 역할도 한다. 하베스터가 실행되는 동안에는 파일 디스크립터가 열려 있다.
스풀러(spooler)	하베스터가 수집한 이벤트를 엘라스틱서치나 로그스태시 같은 장소로 전달한다.

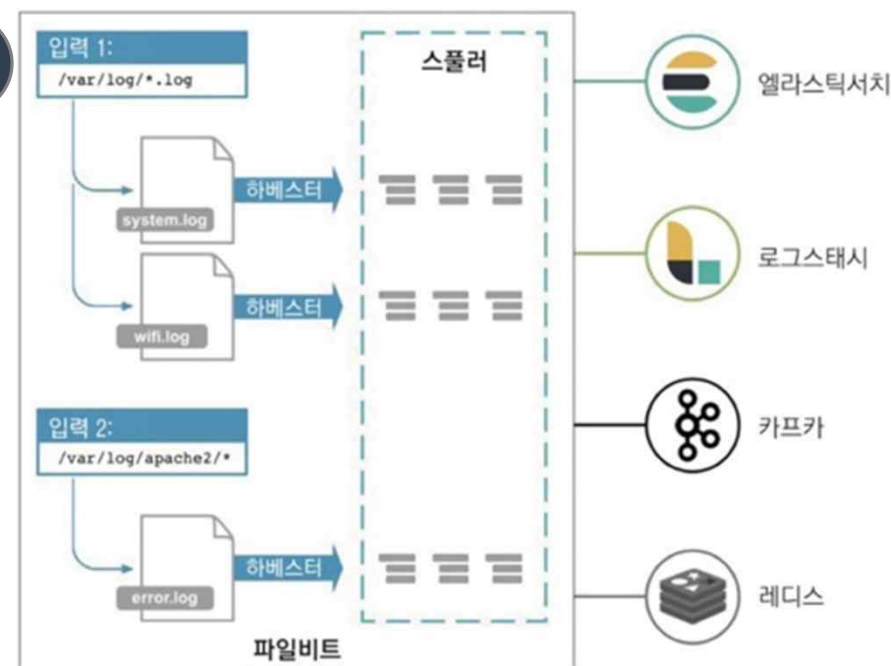


그림 7.5 파일비트 아키텍처⁵

2. 파일비트 다운로드

- 파일비츠 다운로드: <https://www.elastic.co/kr/downloads/beats/filebeat>

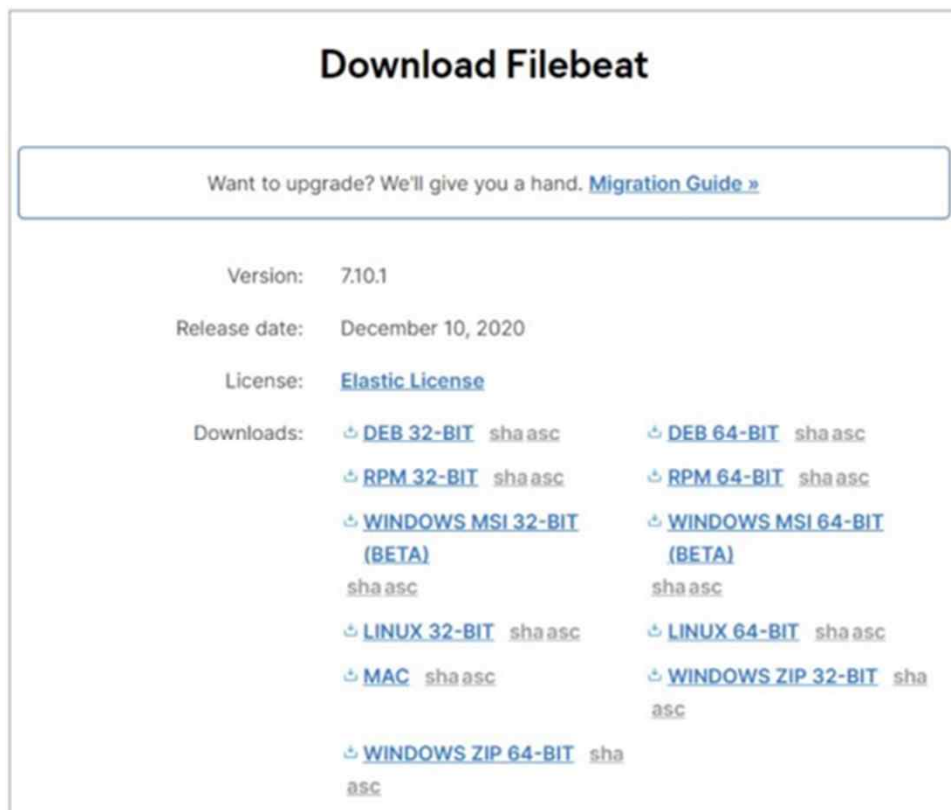


그림 7.6 파일비트 최신 버전 다운로드

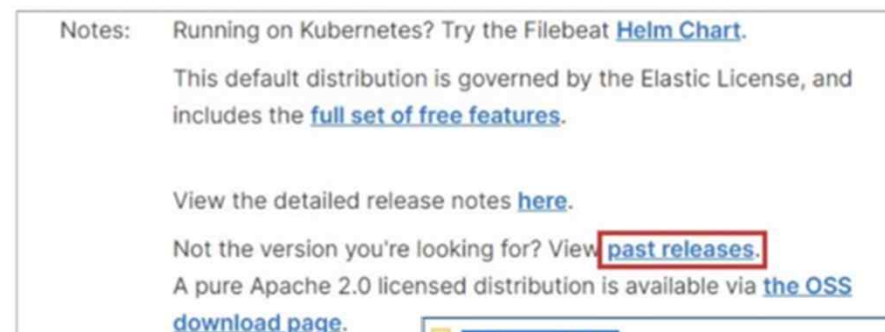


그림 7.7 파일비트 이전 버전 다운로드

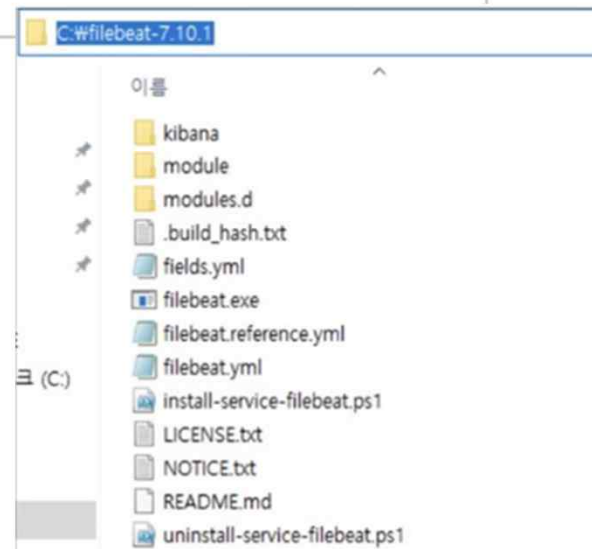


그림 7.8 파일비트 폴더 구조

3. 파일비트 실행

- 로그 파일 수집하기 위해 설정파일 수정
- filebeat.yml 설정 파일 수정

리스트 7.1 파일비트의 설정 파일

```
filebeat.inputs:  
- type: log  
  enabled: true  
  paths:  
    - c:/ELK/elasticsearch-7.10.1/logs/*.log
```

```
output.elasticsearch:  
  hosts: ["localhost:9200"]
```

```
setup.kibana:  
  host: "http://localhost:5601"
```

표 7.3 파일비트 인풋 타입

인풋 타입	설명
log	가장 기본이 되는 타입으로, 파일시스템의 지정한 경로에서 로그 파일을 읽어 들인다.
container	도커 같은 컨테이너의 로그를 수집하기 위한 입력으로, 파일을 읽어 들인다는 점에서 log와 유사하다.
s3	log 타입과 유사하나, 아마존 웹 서비스의 S3 버킷에 위치한 파일을 읽어 들인다.
kafka	다른 타입과는 다르게 파일을 읽어 들이는 대신 카프카의 토픽을 읽어 들인다.

3. 파일비트 실행

표 7.3 파일비트 인풋 타입

인풋 타입	설명
log	가장 기본이 되는 타입으로, 파일시스템의 지정한 경로에서 로그 파일을 읽어 들인다.
container	도커 같은 컨테이너의 로그를 수집하기 위한 입력으로, 파일을 읽어 들인다는 점에서 log와 유사하다.
s3	log 타입과 유사하나, 아마존 웹 서비스의 S3 버킷에 위치한 파일을 읽어 들인다.
kafka	다른 타입과는 다르게 파일을 읽어 들이는 대신 카프카의 토픽을 읽어 들인다.

표 7.4 파일비트 아웃풋 타입

아웃풋 타입	설명
elasticsearch	가장 많이 사용되는 타입으로, 수집한 이벤트를 엘라스틱서치로 직접 인덱싱한다.
logstash	다수의 비츠를 사용해 엘라스틱서치로 전송되는 인덱싱 리퀘스트의 양이 많거나, 비츠나 인제스트 노드 수준에서 처리하기 어려운 가공 작업이 필요할 때 별도의 로그스테시를 구축한 후 수집한 이벤트를 전송한다. 다수의 인덱싱 요청이 로그스테시에서 단일 벌크 리퀘스트로 묶여 인덱싱 효율의 개선을 기대할 수 있다. 이때 전송한 이벤트는 로그스테시의 beats 인풋을 이용해 입력받을 수 있다.
kafka	파일비트에서 1차적으로 수집한 이벤트를 카프카로 전송한다. 카프카는 좀 더 안정적인 수집 파이프라인을 구성할 때 신뢰할 만한 중간 저장소/큐이므로 수집 중 장애 발생 시 데이터 손실을 최소화하기 위한 방안으로 활용된다. 최종적으로 엘라스틱서치의 인덱싱을 원할 경우 이후 다시 파일비트의 카프카 인풋을 이용하거나 로그스테시의 카프카 인풋을 이용해 입력할 수 있다.
console	수집한 이벤트를 시스템 콘솔에 출력한다. 일반적으로 수집이 정상적으로 이뤄지는지 입력 설정을 테스트하기 위한 목적으로 사용된다.

[파일비트 인풋](#)
[파일비트 아웃풋](#)

3. 파일비트 실행

```

2020-12-27T10:29:05.076+0900 INFO eslegclient/connection.go:99 elasticsearch url:
http://localhost:9200
2020-12-27T10:29:05.080+0900 INFO [esclientleg] eslegclient/connection.go:314 Att
empting to connect to Elasticsearch version 7.10.1
2020-12-27T10:29:05.080+0900 INFO kibana/client.go:119 Kibana url: http://localhos
t:5601
2020-12-27T10:29:05.117+0900 WARN fileset/modules.go:421 X-Pack Machine Learning is
not enabled
Loaded machine learning job configurations
2020-12-27T10:29:05.117+0900 INFO eslegclient/c
http://localhost:9200
2020-12-27T10:29:05.121+0900 INFO [esclientleg]
empting to connect to Elasticsearch version 7.10.1
Loaded Ingest pipelines

```

그림 7.9 setup 옵션 적용 시 나오는 파일비트 로그

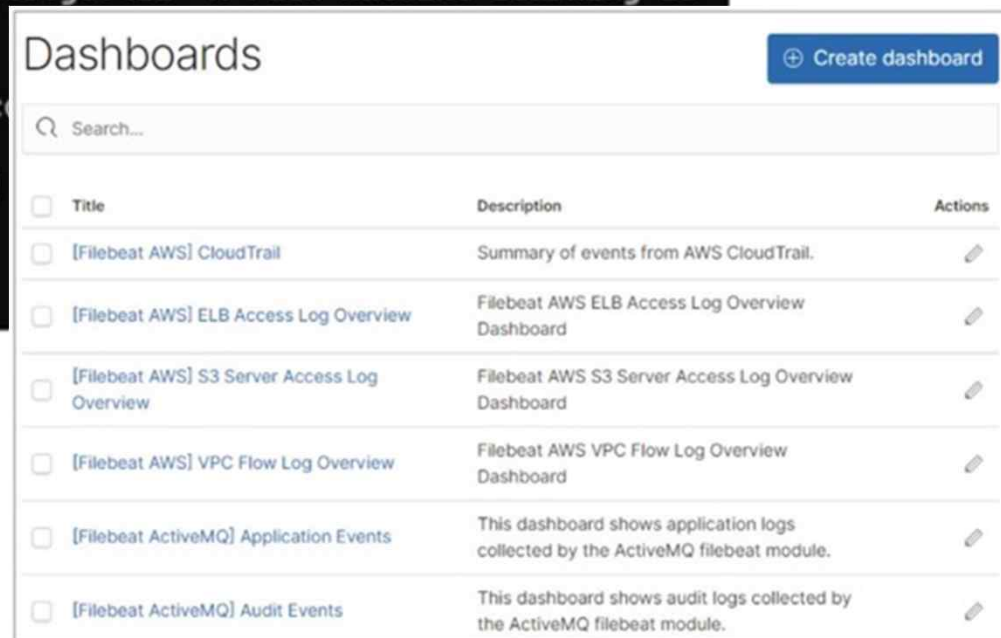


그림 7.10 키바나 대시보드에 파일비트 항목들 추가

3. 파일비트 실행

- 파일비트 실행
c:\WELKW\filebeat-7.10.1-windows-x86_64>filebeat.exe -e
- 키바나 메뉴의 Management > Stack Management > Index Management 확인.

☐	filebeat-7.10.1-2020.12.27-000001	● yellow	open	1	1	19121	4.1mb
--------------------------	-----------------------------------	----------	------	---	---	-------	-------

그림 7.11 파일비트가 filebeat-* 인덱스에 인덱싱되고 있는 키바나 화면

- Kibana > Discover에서 데이터 탐색
인덱스패턴 filebeat-*로 변경

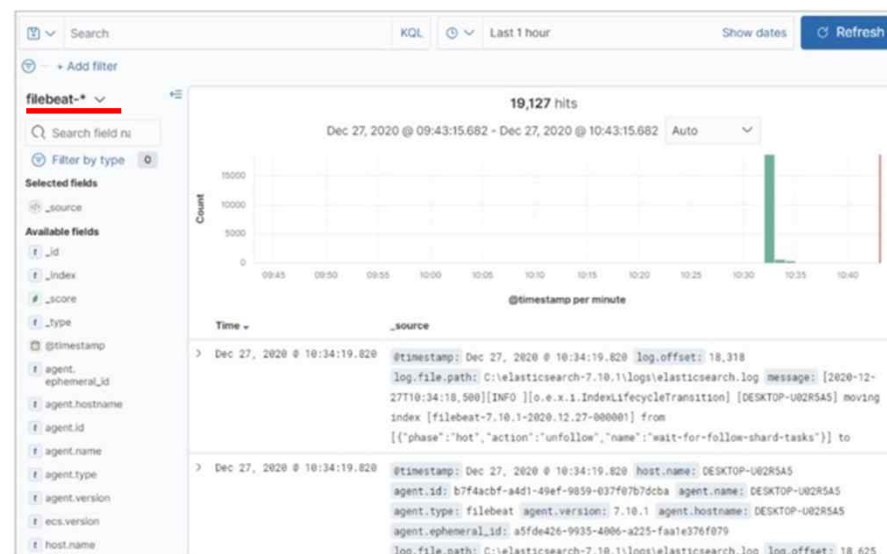


그림 7.12 파일비트를 통해 들어온 데이터의 탐색 화면

4. 파일비트 설정

- filebeat.reference.yml이라는 파일을 보면 파일비트의 모든 설정에 대한 설명과 기본값이 적혀 있다.

| 유용한 설정

- ignore_older: 새로운 파일 탐색 시 오래된 파일은 읽어 들이지 말고 무시하기 위한 설정
- 특정 라인 파일을 추가/제외
 - include_lines: 특정 라인을 정규 표현식을 이용해 필터링하고 매칭된 라인만 비츠에서 수용.
 - exclude_lines: 특정 라인을 정규 표현식을 이용해 필터링하고 매칭된 라인은 비츠에서 수집하지 않는다.
 - exclude_files: 패턴에 일치하는 파일 무시

```
C:\filebeat-7.10.1>type filebeat.yml
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - C:/elasticsearch-7.10.1/logs/*.log
  ignore_older: 24h

output.elasticsearch:
  hosts: ["localhost:9200"]

setup.kibana:
  host: "localhost:5601"
```

그림 7.13 파일비트의 인풋 log 타입에서 ignore_older 설정

```
C:\filebeat-7.10.1>type filebeat.yml
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - C:/elasticsearch-7.10.1/logs/*.log
  include_lines: ['^ERR', '^WARN']
  exclude_lines: ['^DBG']
  exclude_files: ['\.*gz$']

output.elasticsearch:
  hosts: ["localhost:9200"]

setup.kibana:
  host: "localhost:5601"
```

그림 7.14 파일비트 인풋 log 타입에서 특정 라인 처리

4. 파일비트 설정

| 멀티라인 로그 처리

- 하나의 로그가 여러 줄로 나올 경우 처리

```
java.io.IOException: 현재 연결은 원격 호스트에 의해 강제로 끊겼습니다
    at sun.nio.ch.SocketDispatcher.read0(Native Method) ~[?:?]
    at sun.nio.ch.SocketDispatcher.read(SocketDispatcher.java:43) ~[?:?]
    at sun.nio.ch.IOUtil.readIntoNativeBuffer(IOUtil.java:223) ~[?:?]
    at sun.nio.ch.IOUtil.read(IOUtil.java:192) ~[?:?]
    at sun.nio.ch.SocketChannelImpl.read(SocketChannelImpl.java:378) ~[?:?]
    at org.elasticsearch.transport.CopyBytesSocketChannel.readFromSocketChannel(CopyBytesSocketChannel.java:100)
    at org.elasticsearch.transport.CopyBytesSocketChannel.doReadBytes(CopyBytesSocketChannel.java:100)
    at io.netty.channel.nio.AbstractNioByteChannel$NioByteUnsafe.read(AbstractNioByteChannel.java:100)
    at io.netty.channel.nio.NioEventLoop.processSelectedKey(NioEventLoop.java:714) [netty-transport-4.1.49.Final]
    at io.netty.channel.nio.NioEventLoop.processSelectedKeysPlain(NioEventLoop.java:615) [netty-transport-4.1.49.Final]
    at io.netty.channel.nio.NioEventLoop.processSelectedKeys(NioEventLoop.java:578) [netty-transport-4.1.49.Final]
    at io.netty.channel.nio.NioEventLoop.run(NioEventLoop.java:493) [netty-transport-4.1.49.Final]
    at io.netty.util.concurrent.SingleThreadEventExecutor$4.run(SingleThreadEventExecutor.java:162)
    at io.netty.util.internal.ThreadExecutorMap$2.run(ThreadExecutorMap.java:74) [netty-common-4.1.49.Final]
```

그림 7.15 자바 오류 로그

4. 파일비트 설정

| 멀티라인 로그 처리

- multiline 설정
 - pattern: 정규식을 이용해 패턴을 지정, 패턴과 일치하는 라인이 나타나면 멀티라인으로 인식.
 - negate: true일 경우 패턴 일치 조건을 반전.
 - match: before와 after를 지정할 수 있다.

```
C:\filebeat-7.10.1>type filebeat.yml
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - C:/elasticsearch-7.10.1/logs/*.log
  multiline.pattern: '^[[:space:]]'
  multiline.negate: false
  multiline.match: after

output.elasticsearch:
  hosts: ["localhost:9200"]

setup.kibana:
  host: "localhost:5601"
```

그림 7.16 멀티라인 처리를 위한 설정

4. 파일비트 설정

멀티라인 로그 처리

리스트 7.2 멀티라인 로그

```
aoo
koo
boo
boo
zoo
boo
```

리스트 7.3 멀티라인 처리를 위한 설정

```
filebeat.inputs:
- type: log
...
multiline.pattern: '^b'
multiline.negate: true or false
multiline.match: before or after
```

```
multiline.pattern: '^b'
multiline.negate: false
multiline.match: after
```

패턴이 일치하는
연속된 라인을

패턴이 일치하지 않는
앞쪽 라인에 붙인다.

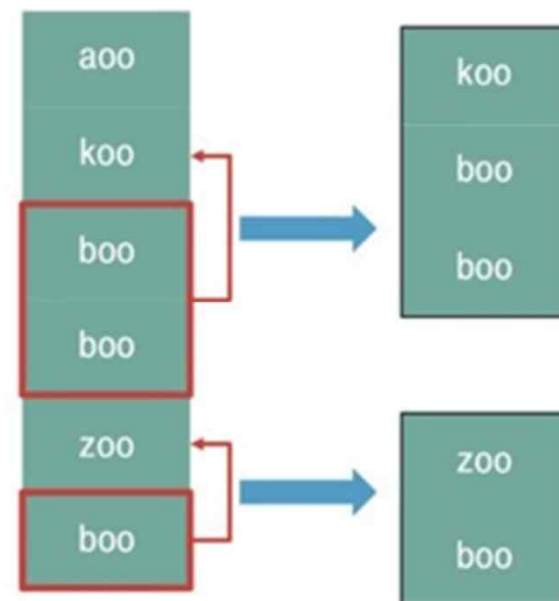


그림 7.17 negate: false, match: after인 경우

4. 파일비트 설정

| 멀티라인 로그 처리

리스트 7.2 멀티라인 로그

```
aoo
koo
boo
boo
zoo
boo
```

리스트 7.3 멀티라인 처리를 위한 설정

```
filebeat.inputs:
- type: log
...
multiline.pattern: '^b'
multiline.negate: true or false
multiline.match: before or after
```

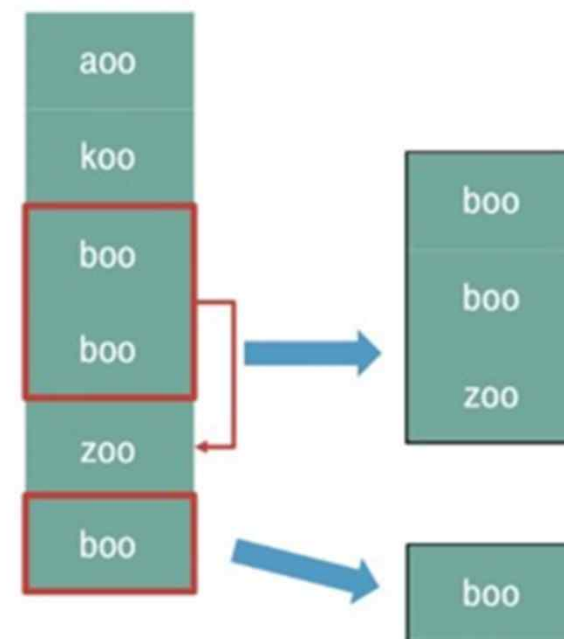
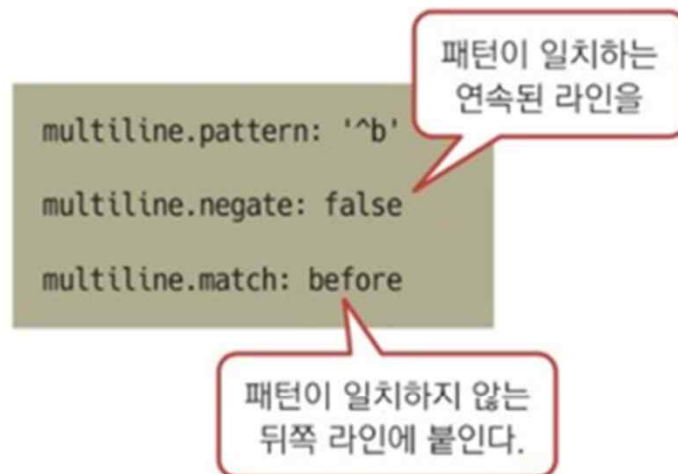


그림 7.18 negate: false, match: before인 경우

4. 파일비트 설정

멀티라인 로그 처리

리스트 7.2 멀티라인 로그

```
aoo
koo
boo
boo
zoo
boo
```

리스트 7.3 멀티라인 처리를 위한 설정

```
filebeat.inputs:
- type: log
...
multiline.pattern: '^b'
multiline.negate: true or false
multiline.match: before or after
```

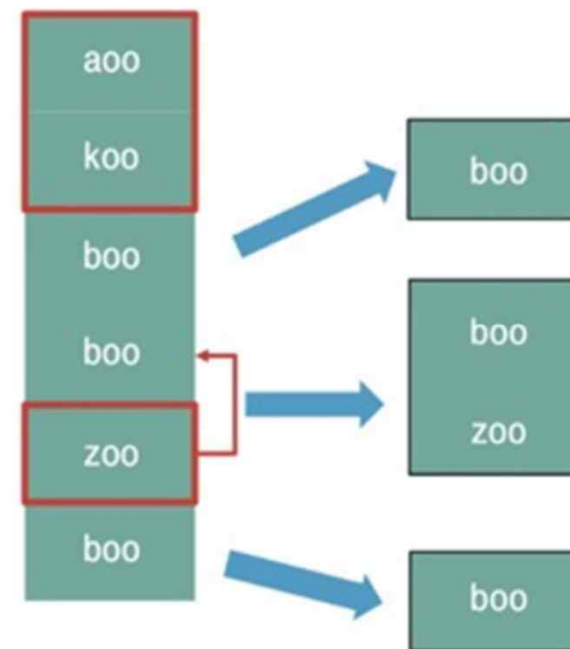
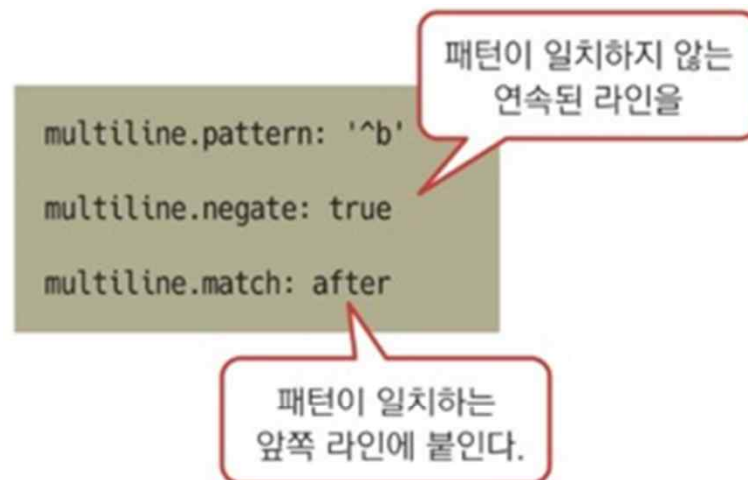


그림 7.19 negate: true, match: after인 경우

4. 파일비트 설정

멀티라인 로그 처리

리스트 7.2 멀티라인 로그

```
aoo
koo
boo
boo
zoo
boo
```

리스트 7.3 멀티라인 처리를 위한 설정

```
filebeat.inputs:
- type: log
...
multiline.pattern: '^b'
multiline.negate: true
multiline.match: before
```

```
multiline.pattern: '^b'
multiline.negate: true
multiline.match: before
```

패턴이 일치하지 않는
연속된 라인을

패턴이 일치하는
뒤쪽 라인에 붙인다.

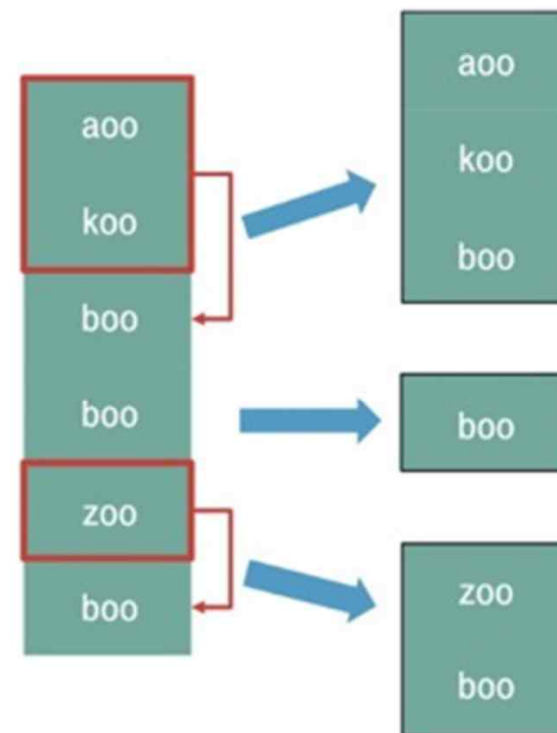


그림 7.20 negate: true, match: before인 경우

5. 모듈

- 모듈은 많이 사용되고 잘 알려진 시스템 데이터를 수집하기 위한 일반적인 설정을 사전 정의한 것.
- 모듈을 이용하면 복잡한 가공이 필요한 이벤트인 경우에도 최소한의 비츠 설정으로 손쉽게 로그 수집 가능.

표 7.5 파일비트 모듈

모듈	설명
aws	AWS의 CloudWatch, CloudTrail 같은 서비스에서 발생하는 로그들을 수집할 수 있다.
cef	시스코를 통해 CEF(Common Event Format) 이벤트를 입력받을 수 있다.
cisco	시스코사의 ASA, Nexus 등 네트워크 장비에서 발생하는 이벤트를 수집한다.
elasticsearch	엘라스틱서치의 클러스터, GC, 감사로그 등을 수집할 때 사용한다.
googlecloud	구글 클라우드 플랫폼의 VPC 플로우, 방화벽 로그 등을 수집할 수 있다.
logstash	로그스태시에서 발생한 로그들을 수집할 수 있다.

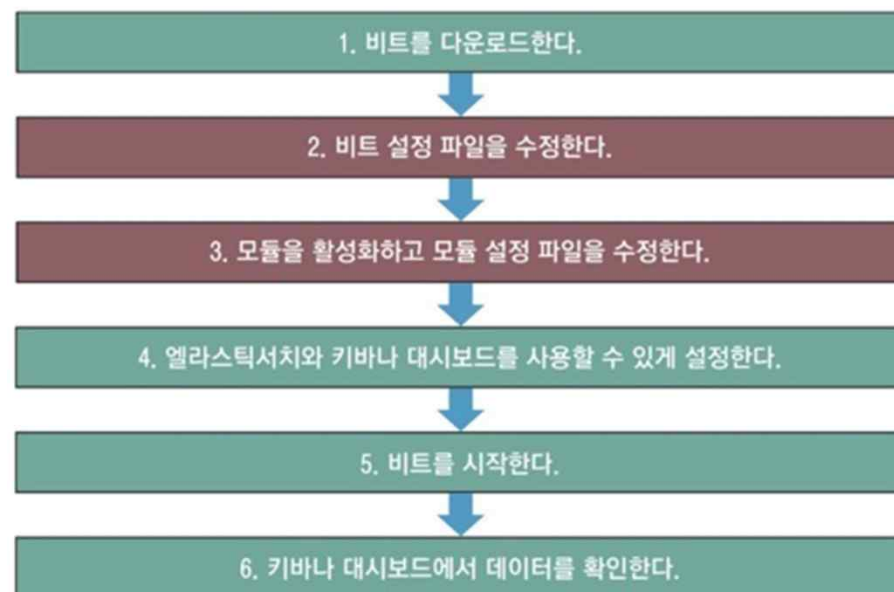


그림 7.21 모듈을 이용한 비츠 동작 과정

5. 모듈

| 비트 다운로드

- 사용하려는 모듈에 따라 다운로드 하는 비트가 달라진다.
- 모듈의 종류는 비츠 온라인 문서에서 확인 가능.

<https://www.elastic.co/guide/en/beats/filebeat/7.10/filebeat-modules.html>

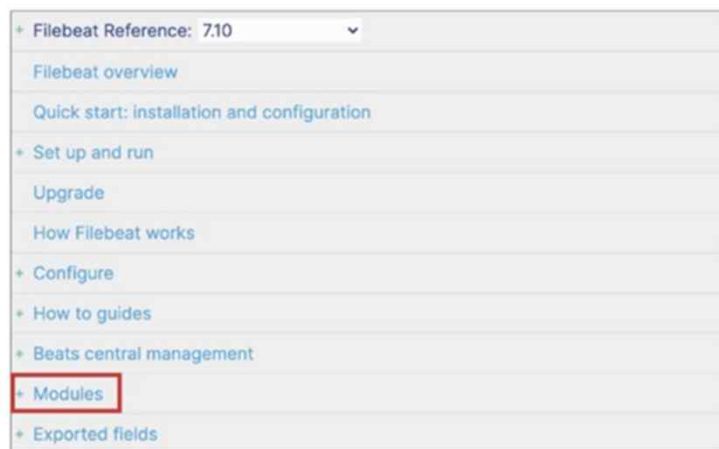


그림 7.22 파일비트 온라인 페이지: 모듈 사용 가능함

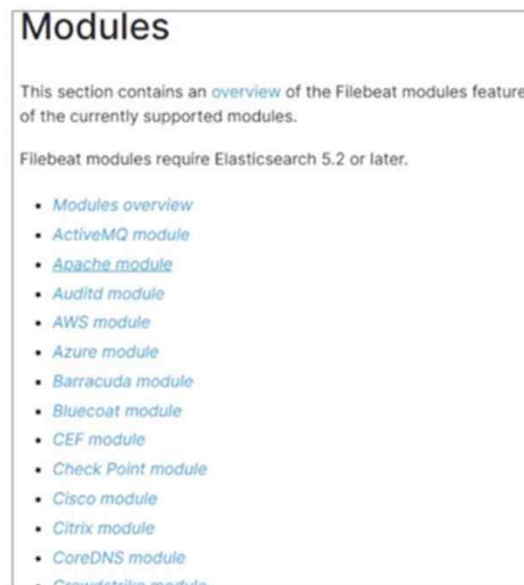


그림 7.23 파일비트 모듈 종류

5. 모듈

| 비츠 설정 파일 수정 (filebeat.yml)

리스트 7.4 모듈 사용을 위한 비츠 설정

```
filebeat.inputs:  
- type: log  
  enabled: true  
  paths:  
    - C:/ELK/elasticsearch-7.10.1/logs/*.log  
output.elasticsearch:  
  hosts: ["localhost:9200"]  
  
setup.kibana:  
  host: "http://localhost:5601"  
  
filebeat.config.modules:  
  path: ${path.config}/modules.d/*.yml
```

참고 파일비트 경로 변수

파일비트에서 자주 사용되는 경로를 변수로 정의하고 있다. 다음을 확인해보자.

경로	경로
path.home	파일비트 설치 경로(C:\filebeat-7.10.1)
path.config	파일비트 구성 파일 경로(C:\filebeat-7.10.1)
path.data	파일비트 데이터 경로(C:\filebeat-7.10.1\data)
path.logs	파일비트 로그 경로(C:\filebeat-7.10.1\logs)

5. 모듈

| 모듈 활성화

- logstash 모듈은 로그스태시 로그를 가져오는 모듈.
- **filebeat.exe modules enable logstash** -> enable/disable을 이용해 모듈을 활성화/비활성

```
C:\filebeat-7.10.1>.\filebeat.exe modules enable logstash
Enabled logstash
```

그림 7.24 로그스태시 모듈 활성화

```
C:\filebeat-7.10.1>.\filebeat.exe modules enable logstash kibana elasticsearch
Module logstash is already enabled
Enabled kibana
Enabled elasticsearch

C:\filebeat-7.10.1>.\filebeat.exe modules disable kibana elasticsearch
Disabled kibana
Disabled elasticsearch
```

그림 7.25 멀티 모듈 활성화/비활성화

```
C:\filebeat-7.10.1>.\filebeat.exe modules list
Enabled:
logstash

Disabled:
activemq
apache
auditd
aws
azure
barracuda
bluecoat
cef
checkpoint
cisco
citrix
coredns
```

그림 7.26 활성화/비활성화된 모듈 확인

5. 모듈

| 모듈 설정 파일 수정

- Logstash.yml 수정
- 모듈은 기본적으로 사람들이 많이 쓰는 기능을 사전 정의했기 때문에 수정할 필요가 없다.
- 사용자의 요구사항을 모두 맞추기 힘들기 때문에 개별적인 수정이 가능.
- 모듈 설정 파일은 파일 디렉토리 modules.d라는 디렉토리에 있다.
- 비활성된 모듈은 disabled확장자가 있고, 활성화된 모듈은 disabled확장자가 없다.

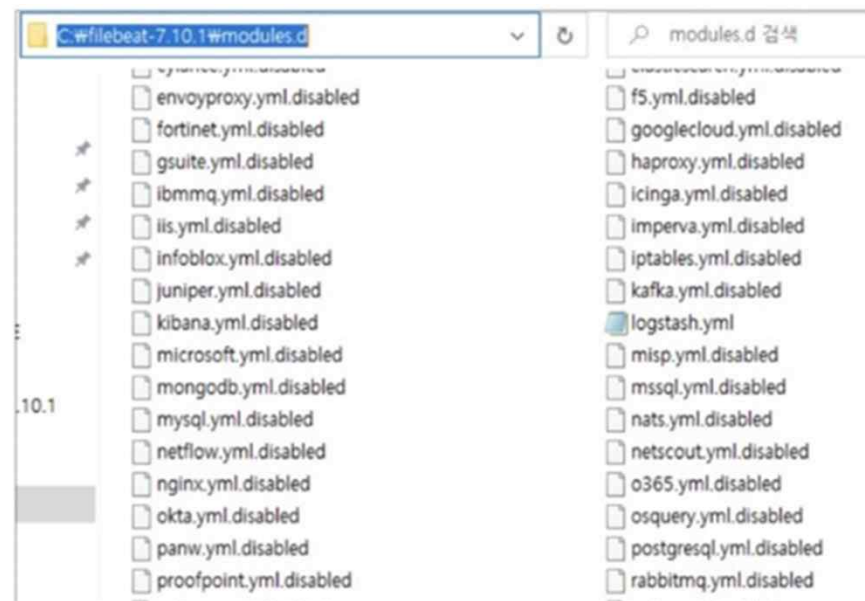


그림 7.27 로그스태시 모듈 파일 확장자 변경

5. 모듈

| 모듈 설정 파일 수정

- Logstash 모듈은 일반 로그와 슬로우 로그 수집을 지원.
 - 일반 로그: 대부분의 로그
 - 슬로우 로그: 파이프라인 내에서 과도하게 처리 시간이 지연되는 이벤트

```
C:\filebeat-7.10.1\modules.d>type logstash.yml
# Module: logstash
# Docs: https://www.elastic.co/guide/en/beats/filebeat/7.10/filebeat-module-logstash.html

- module: logstash
  # logs
  log:
    enabled: true

    # Set custom paths for the log files. If left empty,
    # Filebeat will choose the paths depending on your OS.
    var.paths: [C:\logstash-7.10.1\logs\*.log]

  # Slow logs
  slowlog:
    enabled: true
    # Set custom paths for the log files. If left empty,
    # Filebeat will choose the paths depending on your OS.
    #var.paths:
```

var.paths: [C:\ELK\logstash-7.10.1\logs*.log]

그림 7.28 시스템 모듈 수정

5. 모듈

| setup 명령과 비트 실행

- 모듈 파일 수정을 완료 후 setup 명령을 통해 엘라스틱 인덱스 템플릿을 생성하고 키바나 대시보드를 사용할 수 있게 한다.

filebeat.exe setup

```
C:\filebeat-7.10.1>.\filebeat.exe setup
Overwriting ILM policy is disabled. Set 'setup.ilm.overwrite: true' for enabling.

Index setup finished.
Loading dashboards (Kibana must be running and reachable)
Loaded dashboards
Setting up ML using setup --machine-learning is going to be removed in 8.0.0. Please use
instead.
See more: https://www.elastic.co/guide/en/machine-learning/current/index.html
Loaded machine learning job configurations
Loaded Ingest pipelines
```

그림 7.29 파일비트의 setup 명령 실행

5. 모듈

| setup 명령과 비츠 실행

- 비츠 설치가 완료된 후 비츠 실행 전 파일비트 샘플 대시보드 확인
[Dashboard](#) -> [Filebeat Logstash] Logstash Logs ECS

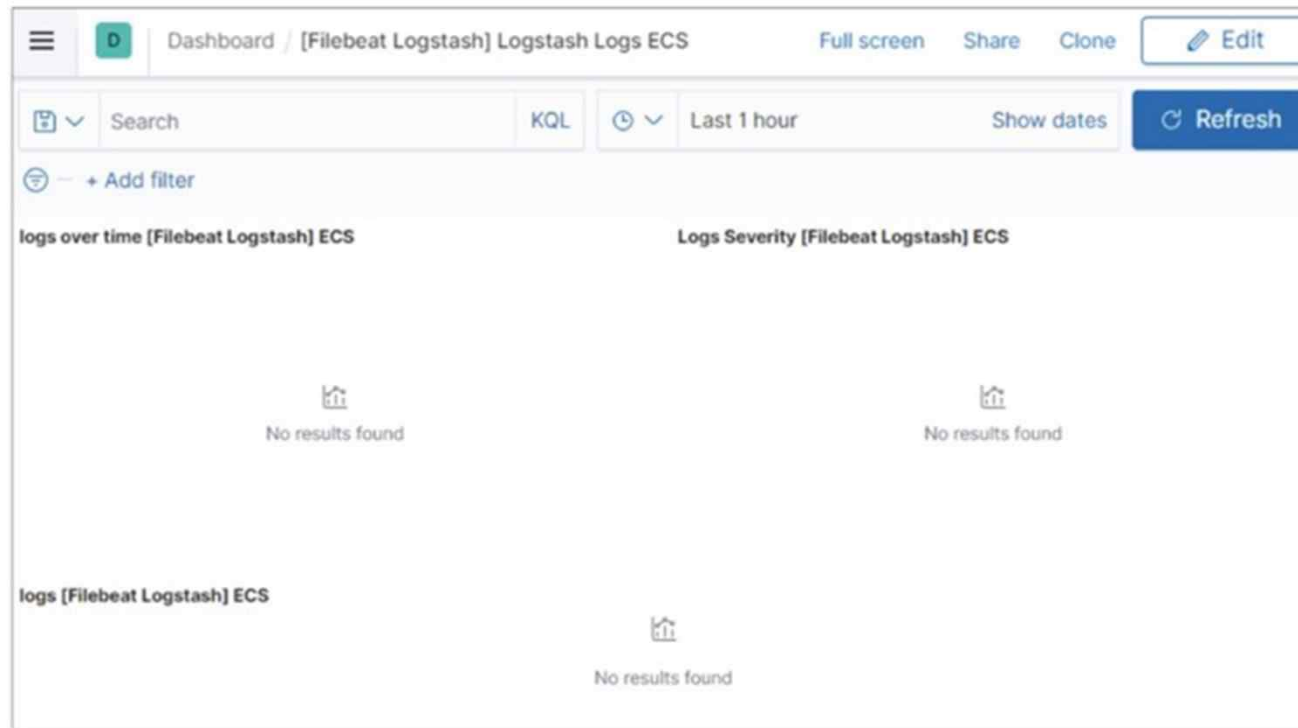


그림 7.30 대시보드에서 파일비트 확인

5. 모듈

| setup 명령과 비트 실행

- **filebeat.exe -e** 명령으로 파일비트 실행
- 새 명령프롬프트 실행 후 로그스태시 실행(**bin\logstash.bat**)

```
[2020-12-28T23:13:48,128][INFO ][logstash.javapipeline ][mypipe1] Pipeline started {"pipeline.id"=>"mypipe1"}
[2020-12-28T23:13:48,128][INFO ][logstash.javapipeline ][mypipe2] Pipeline started {"pipeline.id"=>"mypipe2"}
[2020-12-28T23:13:48,178][INFO ][filewatch.observingtail ][mypipe2][f8bf9671e4bdf362d498dbe1e66df72a89c706602d160f693a2a3f104358c358] START, creating Discoverer, Watch with file and since db collections[2020-12-28T23:13:48,180][INFO ][filewatch.observingtail ][mypipe1][707ac4a0db2e1589fd97c73f882cc83ac6345b782aaaf9029741e0714dad340b] START, creating Discoverer, Watch with file and since db collections[2020-12-28T23:13:48,183][INFO ][logstash.agent ][mypipe2] Pipelines running {:count=>3, :running_pipelines=>[:mypipe2, :".monitoring-logstash", :mypipe1], :non_running_pipelines=>[]}
[2020-12-28T23:13:49,318][INFO ][logstash.agent ][logstash.agent] Successfully started Logstash API endpoint {:port=>9600}
{
  "message" => "[2020-12-28T02:16:51,479][INFO ][o.e.x.m.a.TransportDeleteExpiredDataAction] [DESKTOP-U02R5A5] Completed deletion of expired ML data\r",
  "@timestamp" => 2020-12-28T14:13:48.647Z,
  "path" => "C:/elasticsearch-7.10.1/logs/elasticsearch.log",
  "@version" => "1",
  "host" => "DESKTOP-U02R5A5"
}
```

그림 7.31 로그스태시 실행 화면

```
2020-12-28T23:13:40.758+0900 INFO log/harvester.go:302 Harvester started for file: C:\logstash-7.10.1\logs\logstash-plain.log
```

그림 7.32 파일비트가 로그스태시 로그 파일을 가져오는 로그

5. 모듈

| 키바나에서 확인

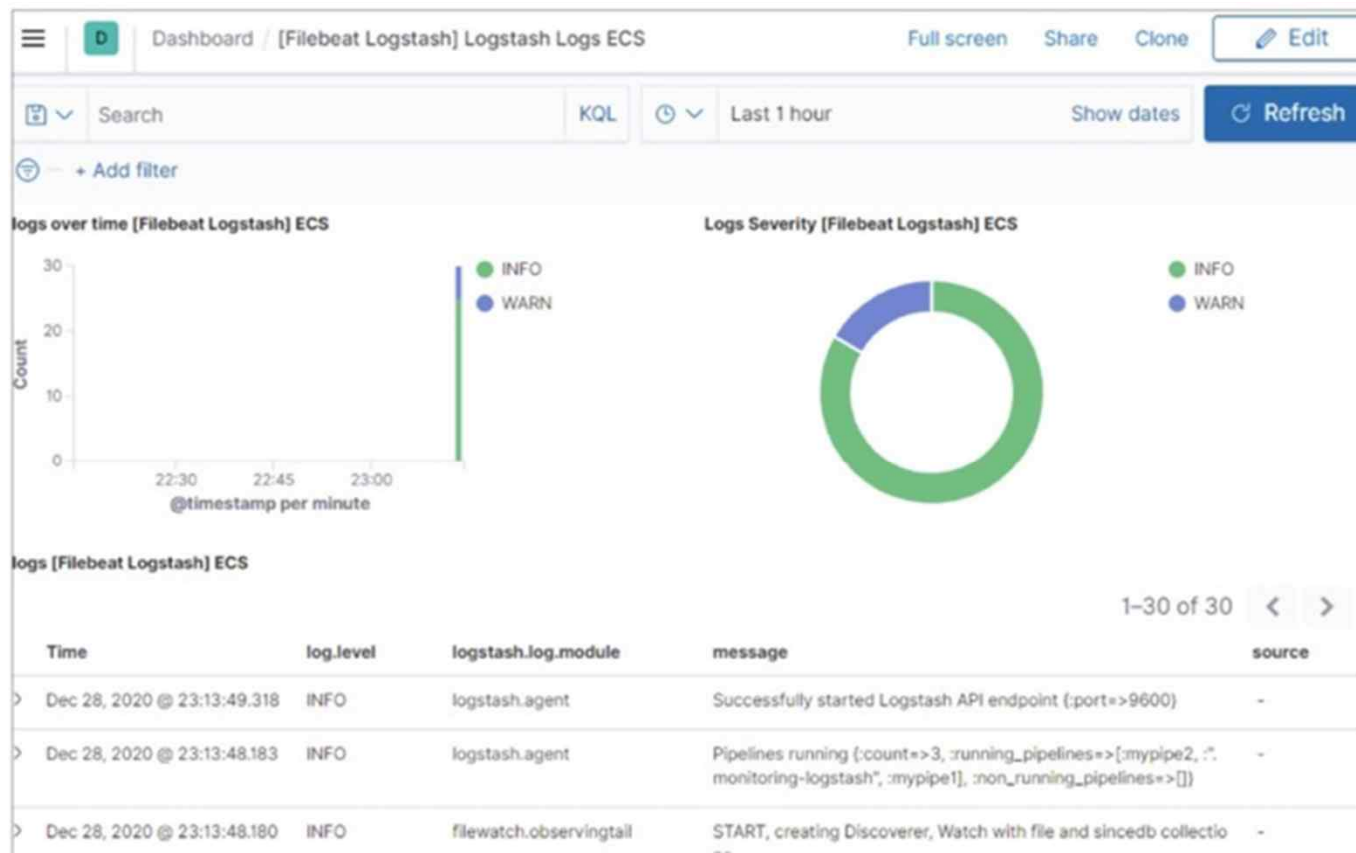


그림 7.33 파일비트용 엘라스틱서치 인덱스

비츠

4 모니터링

- 비츠 모니터링은 비츠가 데이터 입출력을 잘하고 있는지 비츠가 동작 중인 호스트의 리소스들은 문제 없는지 등을 엘라스틱서치와 키바나를 통해 확인 할 수 있는 기능

리스트 7.5 모니터링을 위한 비츠 설정 파일 변경 부분

filebeat.inputs:

- type: log

enabled: true

paths:

- C:/ELK/elasticsearch-7.10.1/logs/*.log

output.elasticsearch:

hosts: ["localhost:9200"]

setup.kibana:

host: "http://localhost:5601"

monitoring.enabled: true

#monitoring.cluster_uuid:PRODUCTION_ES_CLUSTER_UUID

monitoring.elasticsearch:

hosts: ["localhost:9200"]

```
C:\filebeat-7.10.1>curl -XGET "localhost:9200"
{
  "name" : "DESKTOP-U02R5A5",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "VlLeOSDfTtyhikERi1VMCg",
  "version" : {
    "number" : "7.10.1",
    "build_flavor" : "default",
    "build_type" : "zip",
    "build_hash" : "1c34507e66d7db1211f66f3513706fdf548736aa",
    "build_date" : "2020-12-05T01:00:33.671820Z",
    "build_snapshot" : false,
    "lucene_version" : "8.7.0",
    "minimum_wire_compatibility_version" : "6.8.0",
    "minimum_index_compatibility_version" : "6.0.0-beta1"
  },
  "tagline" : "You Know, for Search"
}
```

그림 7.34 클러스터 UUID 확인 방법

모니터링

- 비츠 모니터링은 비츠가 데이터 입출력을 잘하고 있는지 비츠가 동작 중인 호스트의 리소스들은 문제 없는지 등을 엘라스틱서치와 키바나를 통해 확인 할 수 있는 기능

C:\WELK\filebeat-7.10.1-windows-x86_64>filebeat.exe -e

```
2020-12-29T08:02:18.573+0900 INFO [monitoring] pipeline/retry.go:219 retryer: send unw...
nal to consumer
2020-12-29T08:02:18.574+0900 INFO [monitoring] pipeline/retry.go:223 done
2020-12-29T08:02:18.578+0900 INFO [esclientleg] eslegclient/connection.go:314 Attempting
nnect to Elasticsearch version 7.10.1
2020-12-29T08:02:18.635+0900 INFO [publisher_pipeline_output] pipeline/output.go:151 Co
on to backoff(monitoring(http://localhost:9200)) established
2020-12-29T08:02:38.530+0900 INFO [monitoring] log/log.go:145 Non-zero metrics in the la
{"monitoring": {"metrics": {"beat": {"cpu": {"system": {"ticks": 343, "time": {"ms": 343}}, "total
cks": 639, "time": {"ms": 639}, "value": 639}, "user": {"ticks": 296, "time": {"ms": 296}}}, "handles": {"open":
info": {"ephemeral_id": "3183aea9-487f-47f8-8261-39dd5a547bd4", "uptime": {"ms": 30524}}, "memstats": {"g
": 19250480, "memory_alloc": 12154640, "memory_total": 54049784, "rss": 54321152, "runtime": {"goroutines":
"filebeat": {"events": {"added": 126, "done": 126}, "harvester": {"open_files": 2, "running": 2, "started": 2},
beat": {"config": {"module": {"running": 0}}, "output": {"events": {"acked": 118, "batches": 6, "total": 118},
: {"bytes": 5700}, "type": "elasticsearch", "write": {"bytes": 77030}}, "pipeline": {"clients": 1, "events":
e": 0, "filtered": 8, "published": 118, "retry": 50, "total": 126, "queue": {"acked": 118}}, "registrar": {"st
{"cleanup": 1, "current": 9, "update": 126}, "writes": {"success": 11, "total": 11}}, "system": {"cpu": {"cores
}}}
```

그림 7.35 파일비트 로그 중 모니터링 관련 로그

메뉴 Management > Stack Monitoring

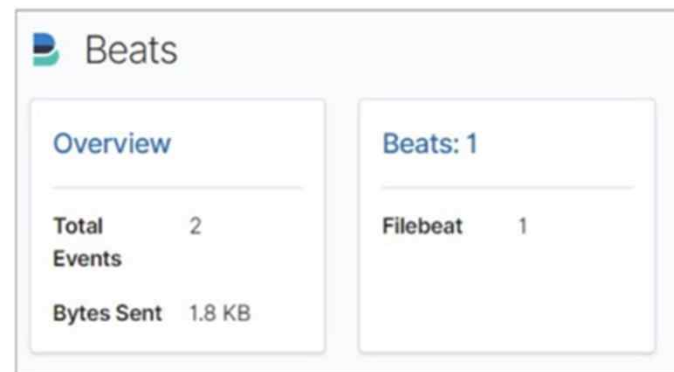


그림 7.36 비츠가 활성화된 모니터링

모니터링

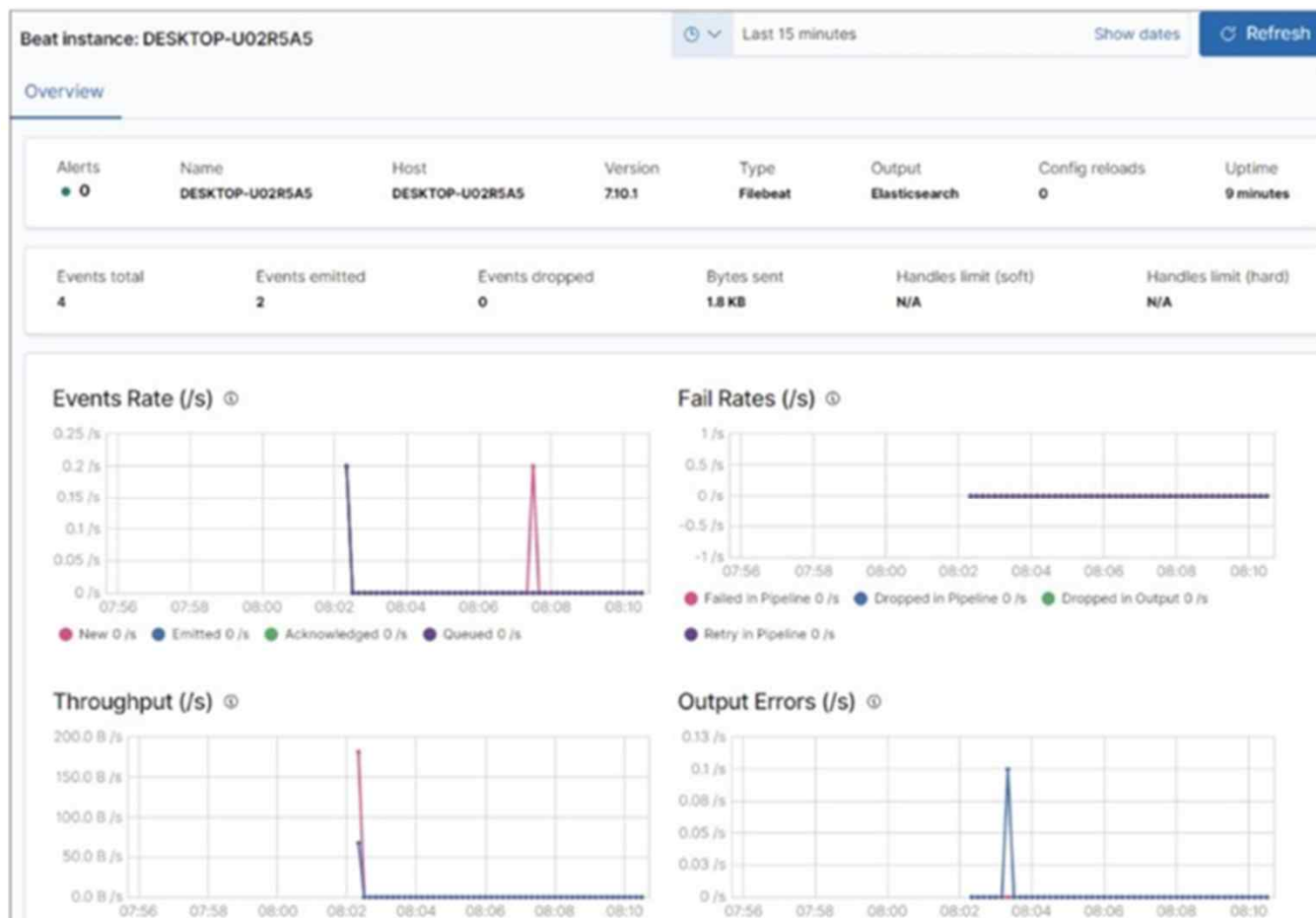


그림 7.37 세부적인 모니터링 화면

5 그 밖의 비츠 사용하기

- **교재 301p ~ 306p**

Ingest your data

 [Try our sample data](#)



Add data

Ingest data from popular apps and services.



Add Elastic Agent

Add and manage your fleet of Elastic Agents and integrations.



Upload a file

Import your own CSV, NDJSON, or log file.



06

키바나

키바나

- 엘라스틱 스택에서 직관적인 방법으로 데이터 시각화를 가능하게 하는 도구
- 텍스트 문서부터 로그파일, 메트릭, 전자상거래 트래픽, 기업의 비즈니스 트랜잭션에 이르기까지 다양한 데이터를 탐색하고 조사할 수 있음.
- 키바나 검색창에 데이터를 찾은 후 다양한 차트를 활용해 데이터를 시각화할 수 있다.
- 키바나는 엘라스틱서치의 쿼리나 집계를 활용해 데이터를 처리.

키바나

1 키바나 소개

- 단순화 시각화 툴을 넘어 엘라스틱 스택의 관리, 모니터링, 솔루션을 총괄하는 메인 UI

표 8.1 키바나의 주요 기능

키바나 기능	설명
데이터 분석과 시각화 툴	오픈소스 기반의 데이터 탐색 및 시각화 도구 제공
엘라스틱 관리	보안, 스냅샷, 인덱스 관리, 개발자 도구 등을 제공
엘라스틱 중앙 허브	모니터링을 비롯해 엘라스틱 솔루션을 탐색하기 위한 포털



그림 8.1 엘라스틱 스택에서 키바나의 역할

키바나 소개

- 유료 라이선스를 구입시 사용할 수 있는 기능도 있지만 무료 라이선스에도 많은 기능을 제공

표 8.2 키바나 시각화 기능

시각화 기능	설명
디스커버(Discover)	데이터를 도큐먼트 단위로 탐색해 구조와 관계 등을 확인할 수 있다.
시각화(Visualize)	다양한 그래프 타입으로 데이터 시각화를 할 수 있다.
대시보드(Dashboard)	그래프, 지도 등을 한곳에서 확인하면서 다양한 인사이트를 얻을 수 있다.
캔버스(Canvas)	그래프와 이미지 등을 프레젠테이션 슬라이드처럼 구성할 수 있다.
맵스(Maps)	위치 기반 데이터를 지도 위에 표현할 수 있다.

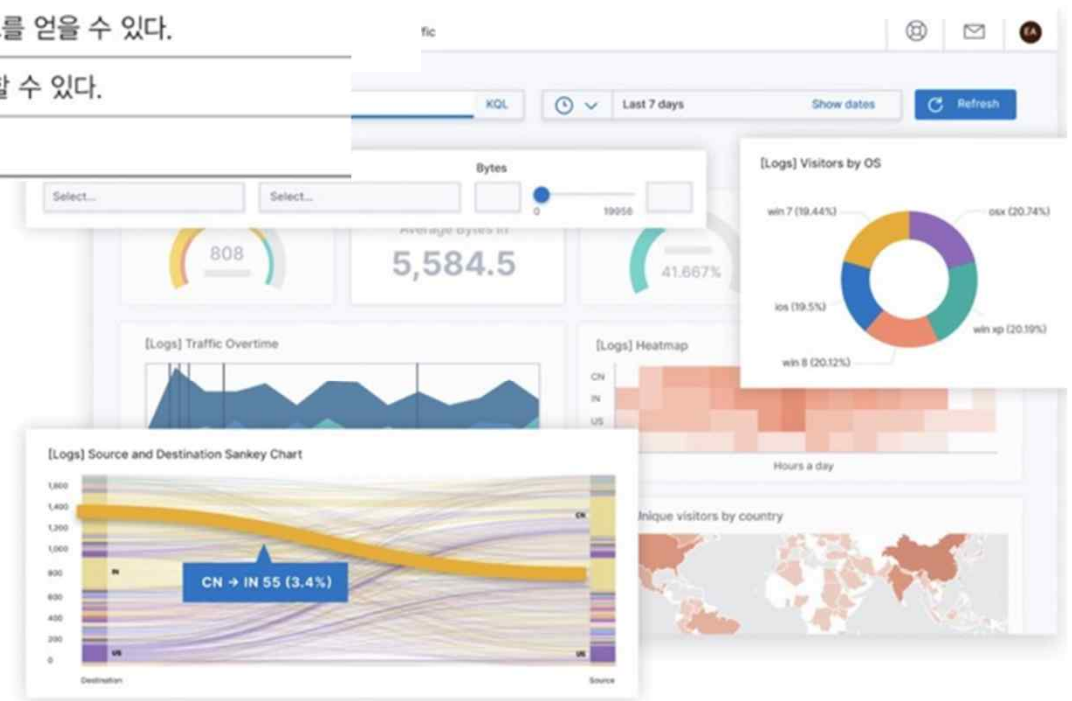


그림 8.2 키바나에서 제공하는 시각화 기능들¹

키바나 소개

1. 인덱스 패턴

- 키바나에서 시각화를 하기 위해 엘라스틱서치 인덱스에 연결되어야만 한다.
- 키바나는 데이터 소스를 엘라스틱서치 인덱스에서 가져오는데 이를 인덱스 패턴(INDEX PATTERNS)라 한다.
- 키바나 인덱스 패턴은 인덱스 매핑 정보 등을 키바나에서 사용하기 적합하게 미리 캐싱해둔 것으로 여러 개의 인덱스에 대한 메타데이터를 병합해 저장해두고 검색이나 시각화 생성 시 활용하게 됨.

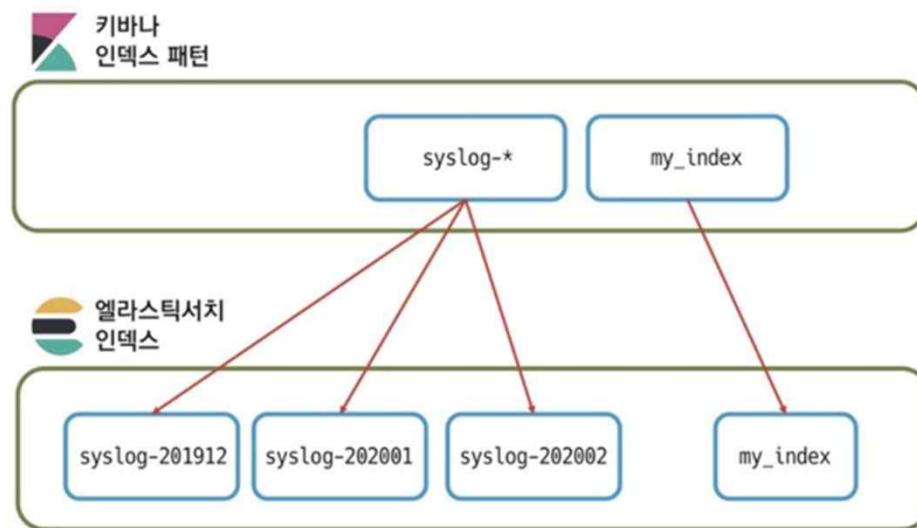


그림 8.3 인덱스와 인덱스 패턴 간의 관계

키바나 소개

1. 인덱스 패턴

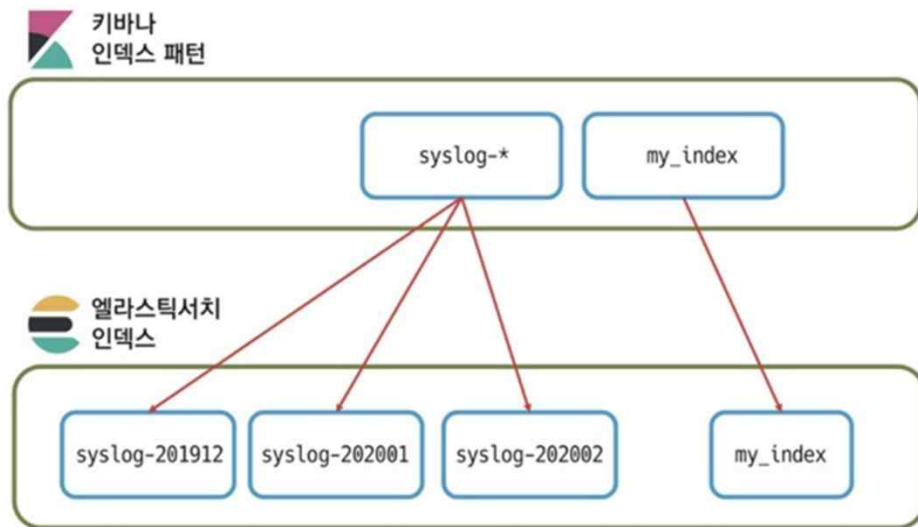


그림 8.3 인덱스와 인덱스 패턴 간의 관계

- Syslog-*는 인덱스 패턴으로, 키바나에서 활용하기 쉽도록 필드 포맷 등을 캐싱해둔 메타데이터.
- 엘라스틱 서치의 syslog-xxx 인덱스들과 연결되어 있음.

키바나 소개

1. 인덱스 패턴

```
PUT kibana_index1/_doc/1
{
  "name":"kim"
}
```

```
PUT kibana_index2/_doc/1
{
  "name":"lee"
}
```

리스트 8.1 인덱스 생성

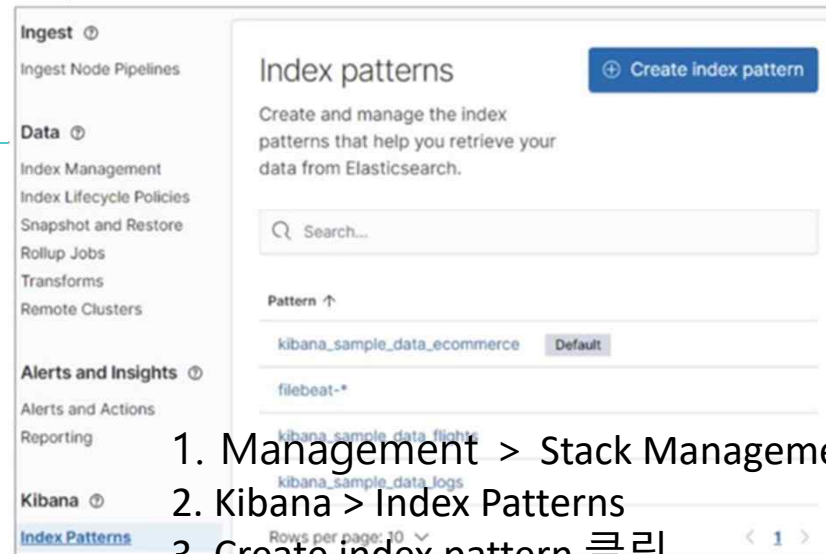


그림 8.4 키바나에서 인덱스 패턴 생성(1/2)

1. Management > Stack Management
2. Kibana > Index Patterns
3. Create index pattern 클릭

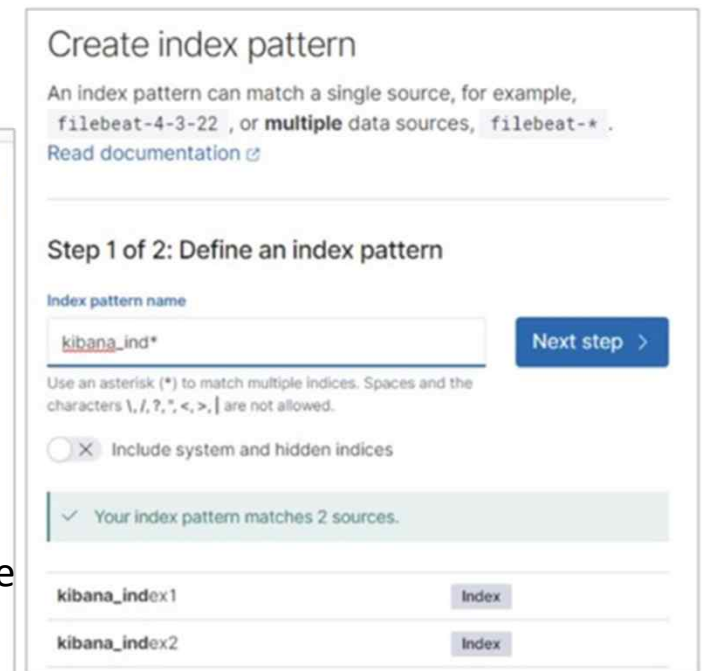


그림 8.5 키바나에서 인덱스 패턴 생성(2/2)

키바나 소개

1. 인덱스 패턴

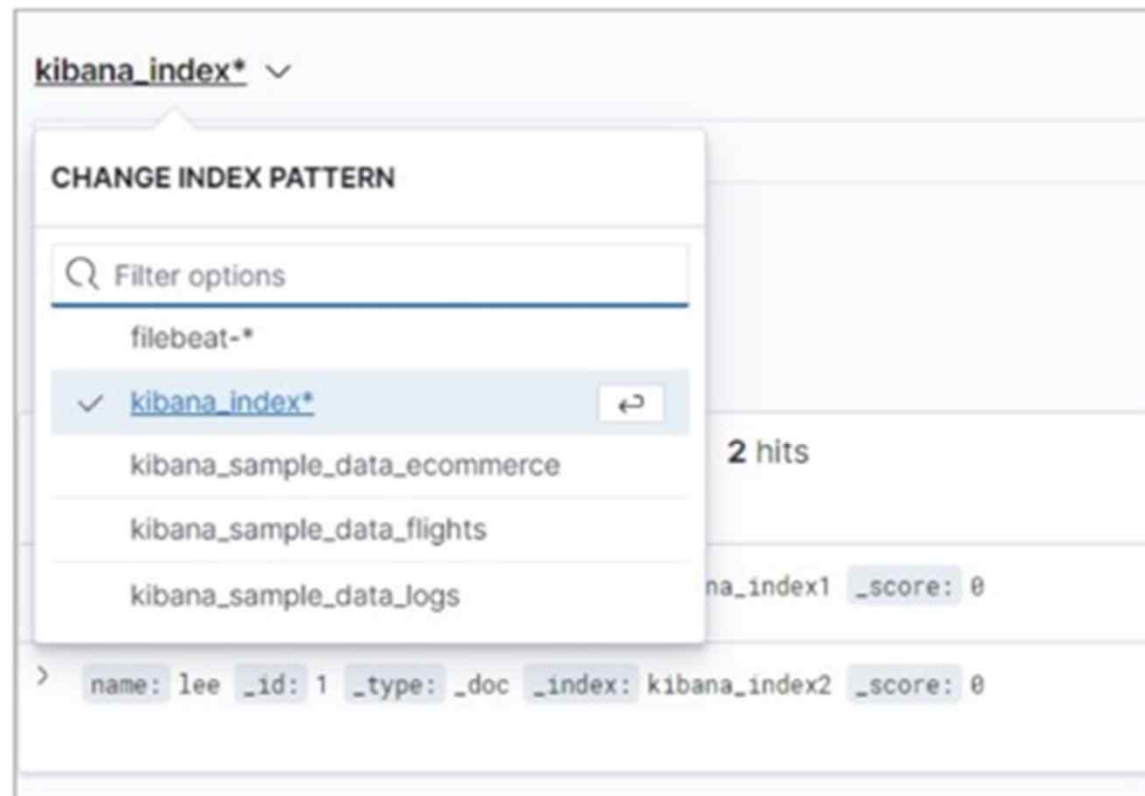


그림 8.6 Discover 메뉴에서 인덱스 패턴 확인

2 디스커버

- 3.1 절에서 로드한 샘플데이터를 활용
- 샘플 데이터를 로드하면 인덱스를 생성 함과 동시에 인덱스 패턴이 만들어진다.
- Discover은 데이터를 확인하고 탐색하기 위한 용도.
- 문서/이벤트/도큐먼트/로그의 시간에 따른 발생량을 히스토그램으로 보여주기도 하고, 데이터 구조나 필드 타입 등을 확인 할 수 있음.
- 적절한 쿼리와 필터 등을 이용해 데이터를 탐색해가면서 추후 시각화에서 사용할 수 있음.

디스커버

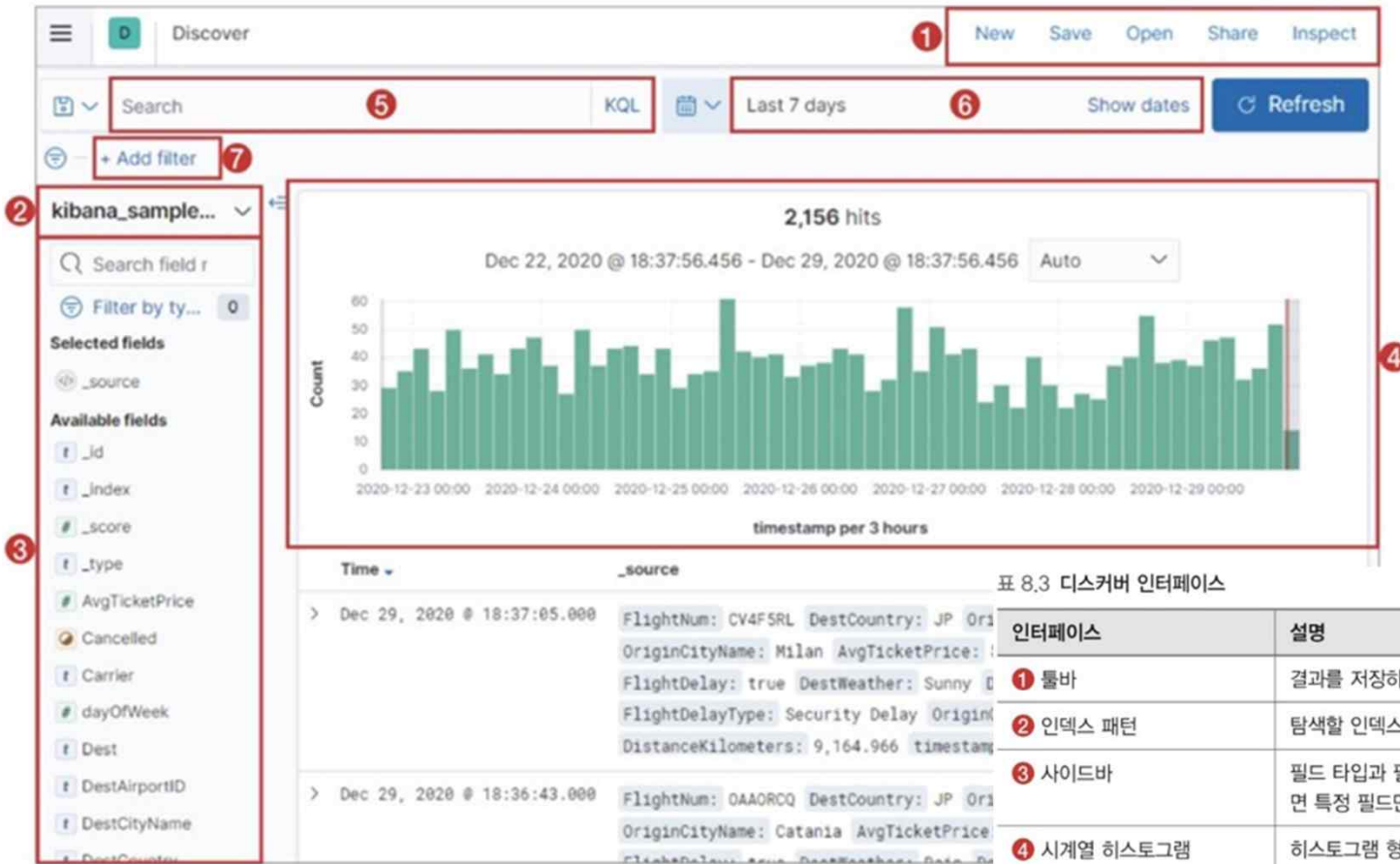


표 8.3 디스커버 인터페이스

인터페이스	설명
1 툴바	결과를 저장하고 불러오는 등의 기능 버튼이 위치한다.
2 인덱스 패턴	탐색할 인덱스 패턴을 선택할 수 있다.
3 사이드바	필드 타입과 필드명을 볼 수 있어 데이터 구조를 확인할 수 있다. 데이터가 너무 많다면 특정 필드만 선택해서 볼 수도 있다.
4 시계열 히스토그램	히스토그램 형태로 시간에 따른 발생량을 확인할 수 있다. 인덱스 패턴에 시간/날짜 필드가 없는 경우에는 보이지 않는다.
5 쿼리바	쿼리를 이용해 데이터를 검색할 수 있다.
6 타임피커	시계열 데이터의 시간 범위를 조정할 수 있다.
7 필터바	쿼리를 필터처럼 사용할 수 있다.

그림 8.7 디스커버 화면 구성

디스커버

1. 쿼리바, 필터바, 타임피커

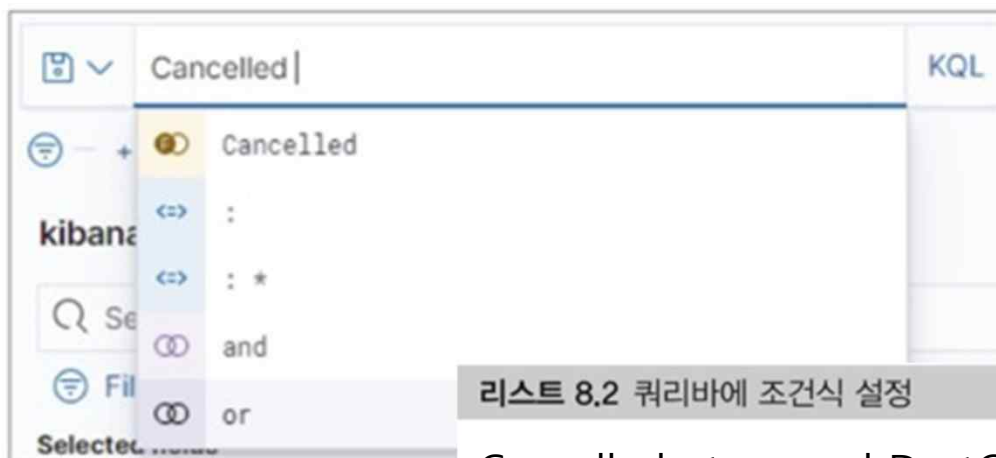
- Discover, Visualize, Dashboard 같은 메뉴에서도 사용.
- 데이터를 쉽게 조작하고 탐색할 수 있음.
- 로그 분석이나 전문 검색을 할 때 쿼리바나 필터바를 유용하게 사용.
- 타임피커는 인덱스에 타임 필드를 지정한 경우 사용.

디스커버

1. 쿼리바, 필터바, 타임피커

| 쿼리바

- 쿼리바는 KQL(kibanaQuery Language), 루씬 쿼리 스트링 두 가지 언어를 지원.
- 자동완성 기능으로 쉽게 쿼리를 구현 할 수 있음.



리스트 8.2 쿼리바에 조건식 설정

Cancelled : true and DestCountry : "US"

Cancelled가 true이고 DestCountry가 "US"인 도큐먼트 탐색

그림 8.8 쿼리바

디스커버

1. 쿼리바, 필터바, 타임피커

| 필터바

- 쿼리바와 하는 역할이 비슷하지만 필드를 개별적으로 처리할 수 있음.

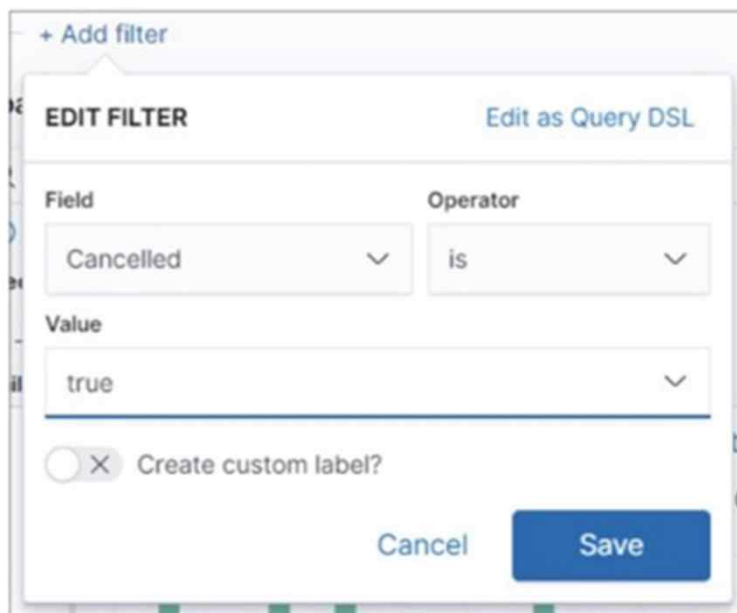


그림 8.9 필터

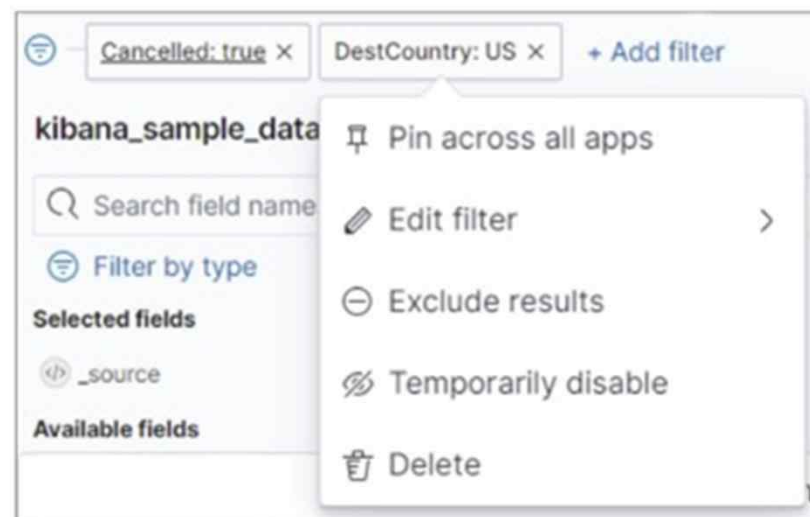


그림 8.10 필터 사용법

디스커버

1. 쿼리바, 필터바, 타임피커

| 타임파커

- 날짜/시간 정보를 조작해 데이터를 특정한 날짜/시간 범위 내에서 시각화.
- 키바나 인덱스 패턴 생성시 기본 날짜/시간 필드를 설정해야 함.

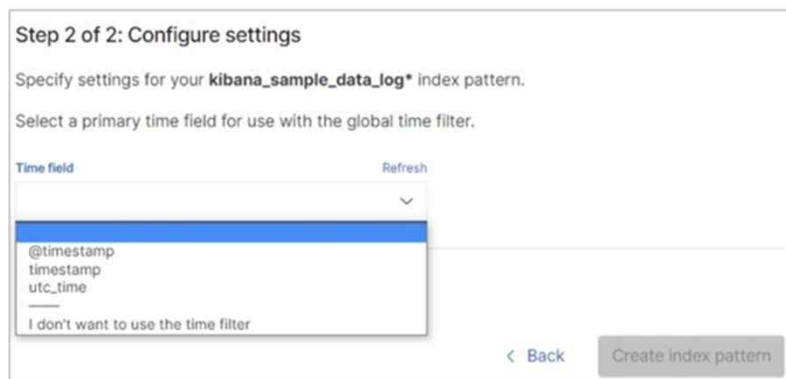


그림 8.11 인덱스 패턴 생성 시 기본 날짜/시간 필드 지정

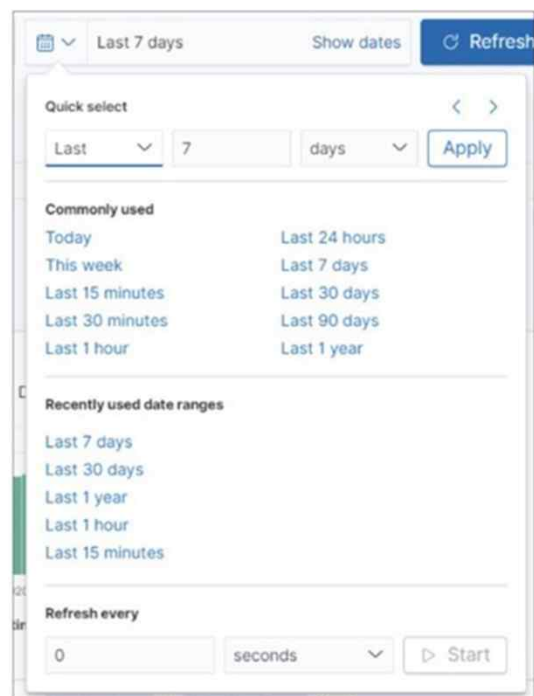


그림 8.12 타임피커 빠른 설정

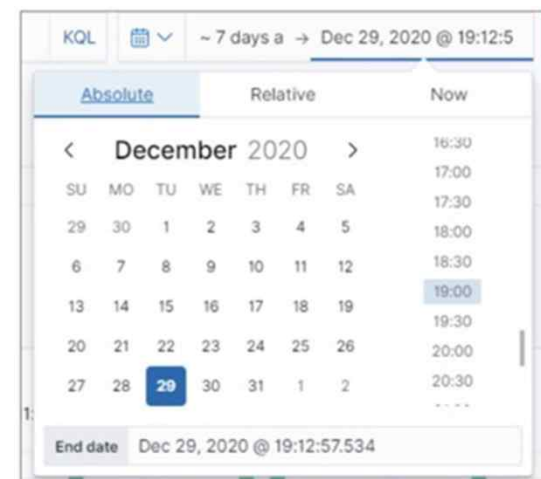


그림 8.13 타임피커 사용자 설정

3 시각화

- Visualize 메뉴는 엘라스틱서치에 저장된 데이터를 그래프나 표, 지도 등 다양한 타입으로 보여준다.
- 라인, 바, 파이 차트, 맵, 시계열 비주얼 빌더, 태그 클라우드 등 다양한 시각화 타입을 제공
- Visualize 메뉴는 사용이 어렵지 않으나 집계를 모르면 이해가 쉽지 않음

- 메트릭 집계: 평균/최소/최대 같은 수량을 계산한다.
- 버킷 집계: 특정 기준에 맞춰 데이터를 분리한다. 서브 버킷을 생성할 수 있다.
- 파이프라인 집계: 집계 결과를 입력으로 받아 다시 집계를 한다. 부모/형제 집계 유형이 있다.

시각화

- 시각화 타입은 대부분 메트릭 집계와 버킷 집계를 통해 그래프를 그리고, 복잡한 그래프는 파이프라인 집계를 이용.

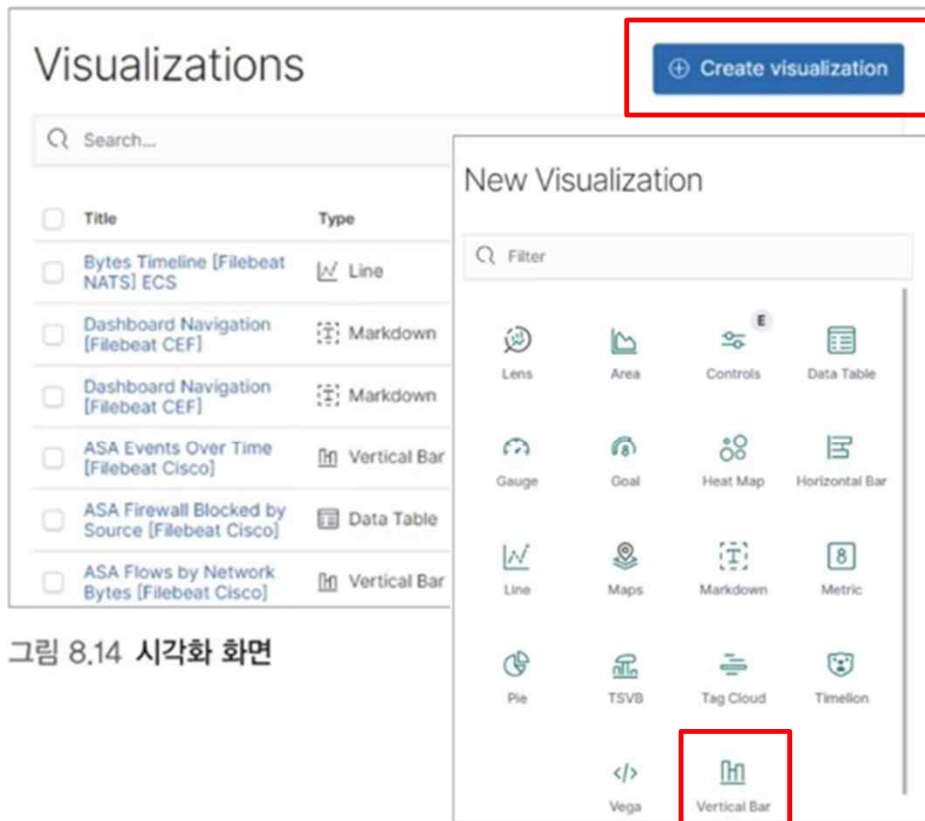


그림 8.14 시각화 화면

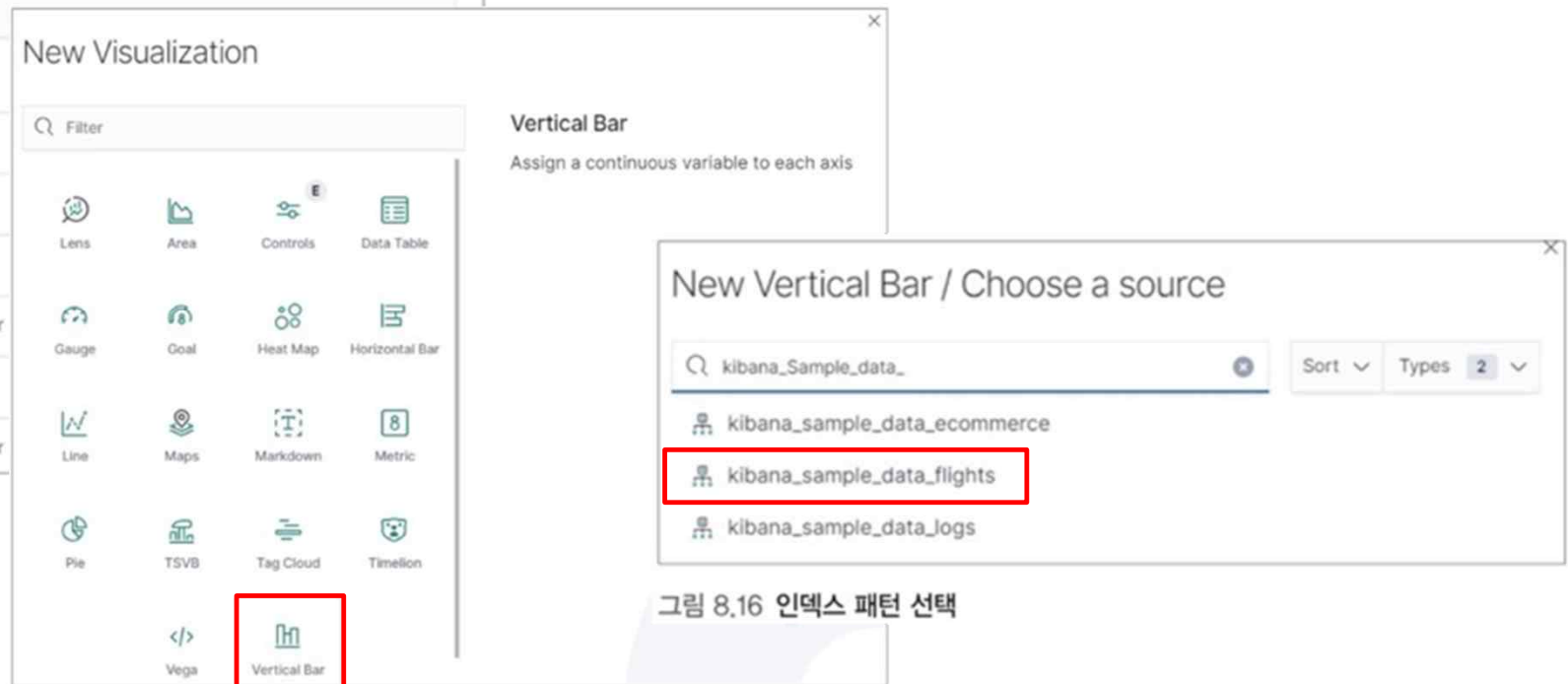


그림 8.16 인덱스 패턴 선택

그림 8.15 시각화 타입 선택

시각화

1. 막대 그래프

- Metrics과 Buckets 설정
 - Metrics 평균값/최소값/최대값 같은 통계를 보여주고 그래프 상 Y축을 담당
 - Buckets은 특정 기준으로 데이터를 나누는 역할을 하며 그래프 상 X축을 담당
- Buckets의 +Add 버튼
 - X-axis: X축을 의미
 - Split series: 서브 버킷 용도
 - Split chart: 그래프를 버킷으로 쪼개 보여줄 때 사용

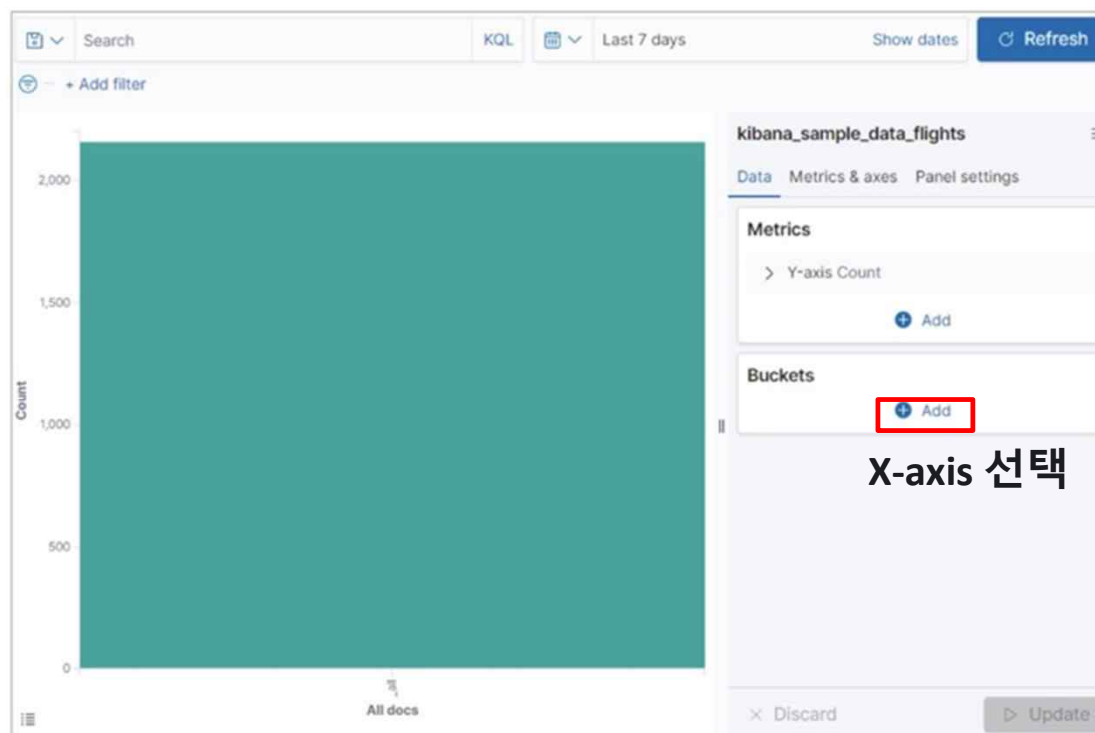


그림 8.17 막대 그래프 타입의 첫 화면

시각화

1. 막대 그래프

· x축을 나누는 기준

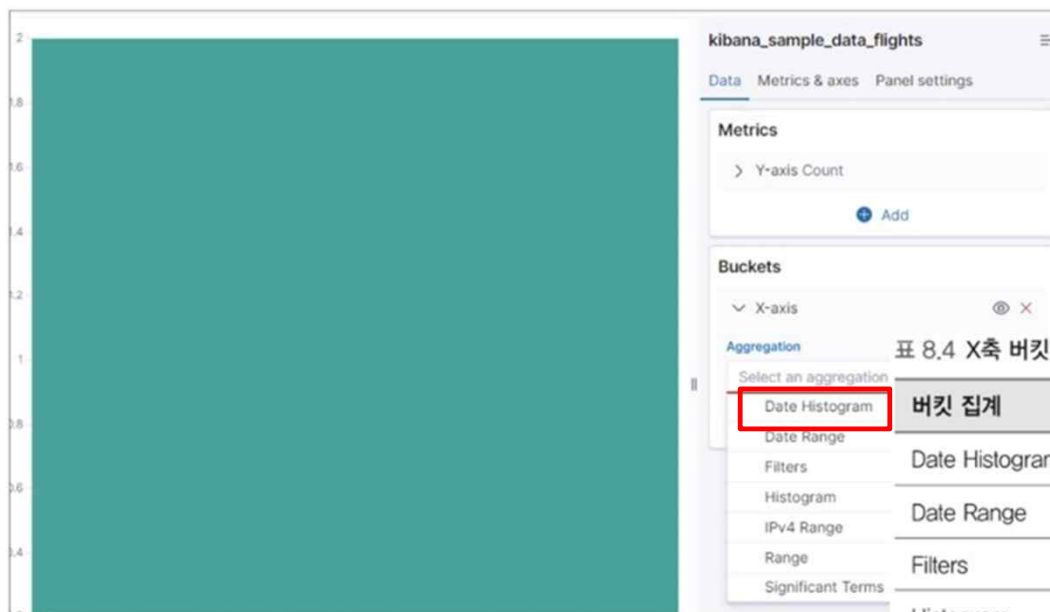


그림 8.18 버킷 선택 화면

표 8.4 X축 버킷 집계 종류

버킷 집계	설명
Date Histogram	날짜/시간 데이터 타입을 가진 필드만 사용 가능. 일정한 주기를 기준으로 버킷을 구분함
Date Range	날짜/시간 데이터 타입을 가진 필드만 사용 가능. 사용자가 임의 범위를 지정해 버킷을 구분함
Filters	필터 적용 가능
Histogram	일정한 주기를 기준으로 버킷을 구분함
IPv4 Range	ip 타입을 가진 필드만 사용 가능. ip 범위를 임의 지정해 버킷을 구분함
Range	사용자가 임의의 범위를 지정해 버킷을 구분함
Significant Terms	필드의 유니크한 값 중 통계적으로 의미 있는 용어를 기준으로 구분함
Terms	필드의 유니크한 값을 기준으로 구분함

시각화

1. 막대 그래프

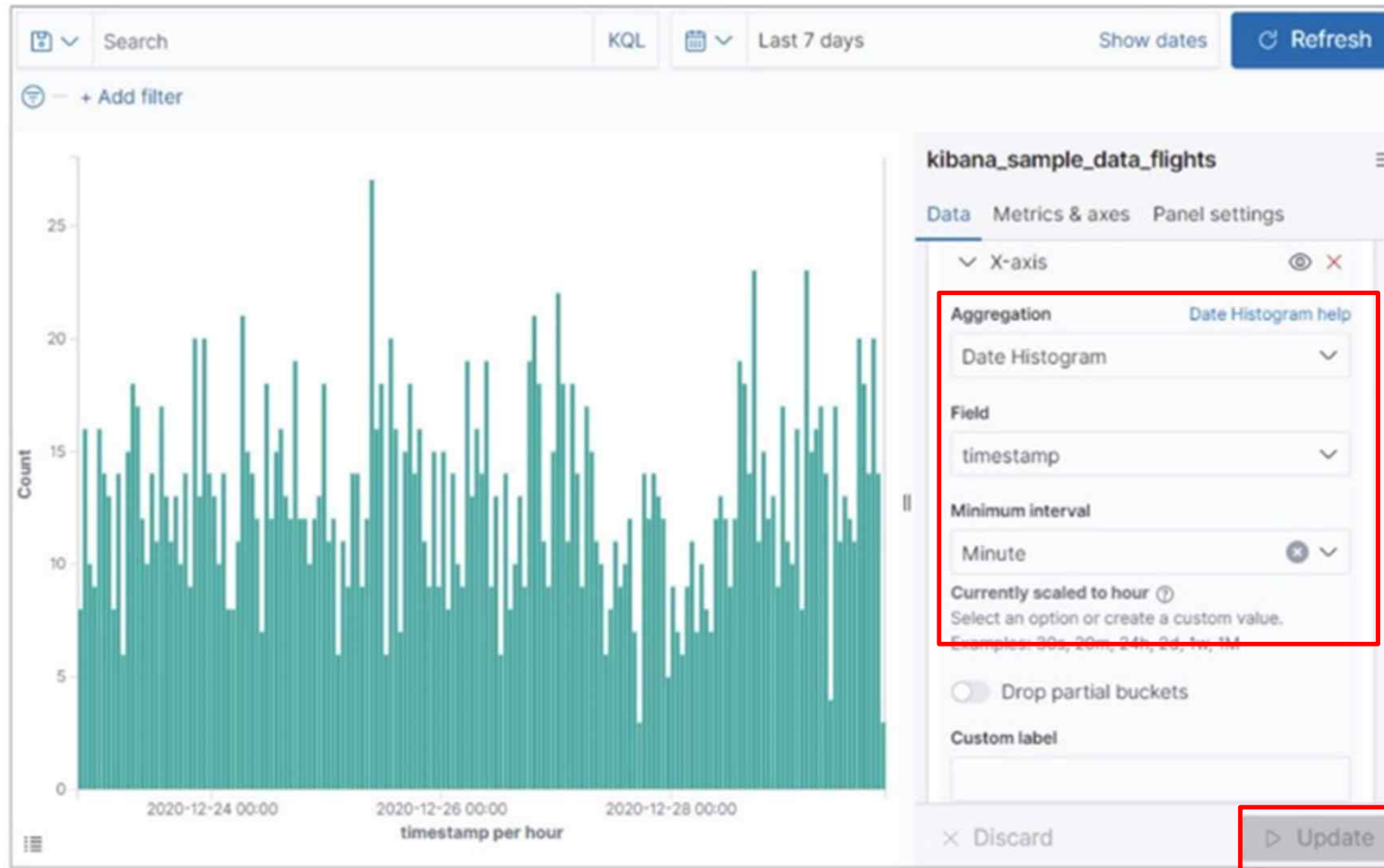


그림 8.19 Date Histogram 버킷 집계를 이용해 X축 생성

시각화

1. 막대 그래프

- Aggregation에서 range사용.
 - Range 집계는 사용자가 임의로 간격을 지정해 버킷을 생성하는 것으로, 사용자가 데이터 값의 범위를 대략적으로 알고 있어야 한다.
 - 키바나 콘솔을 통해 확인 가능

리스트 8.3 terms를 이용해 필드값 확인

```
GET kibana_sample_data_flights/_search
{
  "size": 0,
  "aggs": {
    "unique_aggs": {
      "terms": {
        "field": "dayOfWeek"
      }
    }
  }
}
```

리스트 8.4 리스트 8.3의 집계 결과

```
...
"aggregations" : {
  "unique_aggs" : {
    "doc_count_error_upper_bound" : 0,
    "sum_other_doc_count" : 0,
    "buckets" : [
      { "key" : 0, "doc_count" : 2033 },
      { "key" : 4, "doc_count" : 2001 },
      { "key" : 2, "doc_count" : 1948 },
      { "key" : 1, "doc_count" : 1934 },
      { "key" : 5, "doc_count" : 1915 },
      { "key" : 3, "doc_count" : 1914 },
      { "key" : 6, "doc_count" : 1314 }
    ]
  }
}
...
```

시각화

1. 막대 그래프

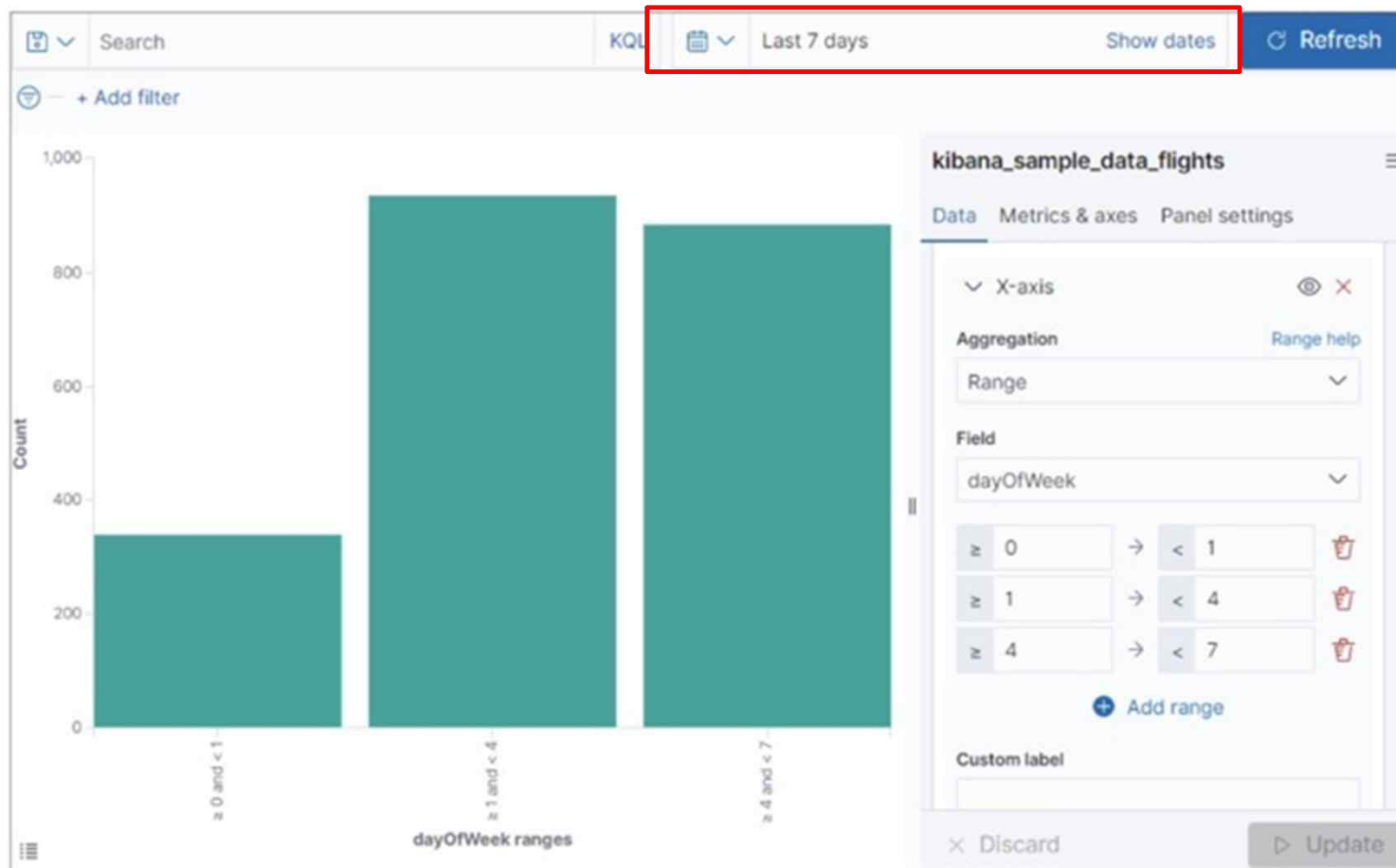


그림 8.20 범위 집계를 이용한 막대 그래프

시각화

1. 막대 그래프

- Aggregation에서 terms사용 후 정렬.
 - 용어 집계는 특정필드의 고윳값을 기준으로 데이터를 구분
 - Aggregation는 Terms, Field는 dayOfWeek 선택
 - Order by
 - Metric: 따로 지정하지 않으면 count로 지정
 - Alphabetical: 알파벳 순으로 정렬
 - Custom Metric: 사용자가 정의한 정렬에 사용하기 위한 집계
- Order
 - Ascending: 오름차순
 - Descending: 내림차순

시각화

1. 막대 그래프

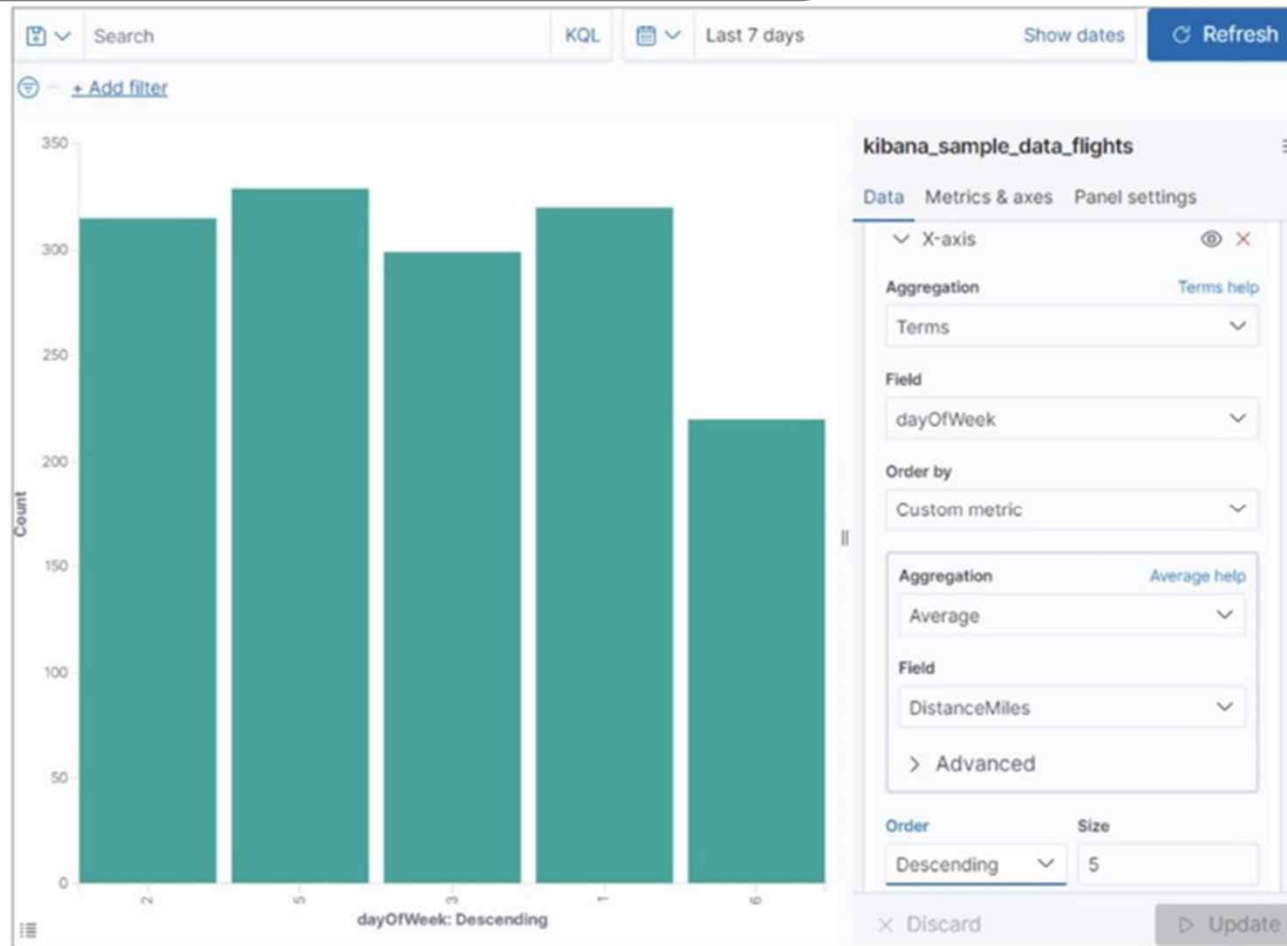
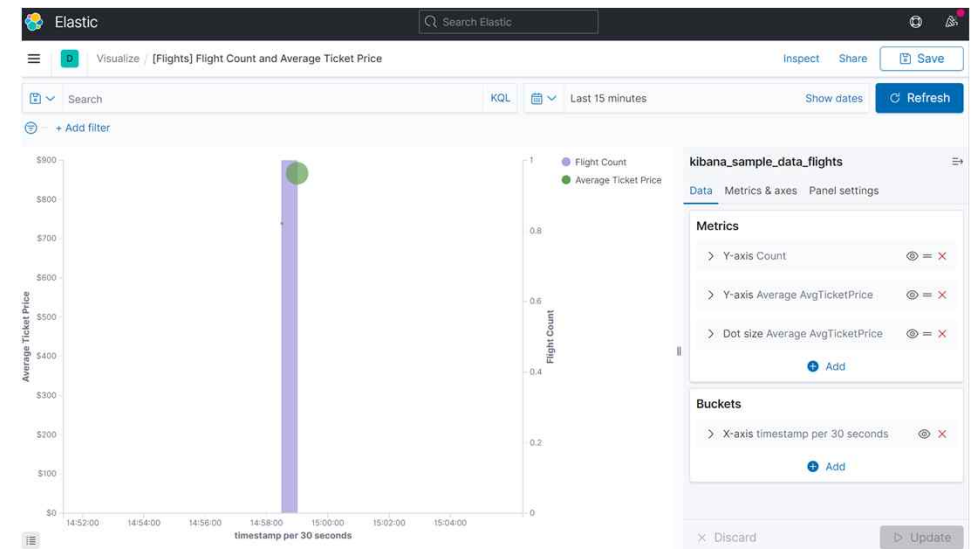


그림 8.21 용어 집계를 이용한 막대 그래프

시각화

1. 막대 그래프

- Metrics은 Y축을 설정, X축을 어떤 통계 값으로 표현할지 보여준다.
 - Buckets의 Aggregation는 Aggregation는 Histogram으로, Field는 dayOfWeek로 선택 Minimum interval은 1로 지정
- Metrics 의 +Add 버튼
 - Y-axis: Y축이 되는 메트릭 메뉴
 - Dot size: 라인을 동그라미 형태로 만들어 줌 (참고: [Flights] Flight Count and Average Ticket Price)

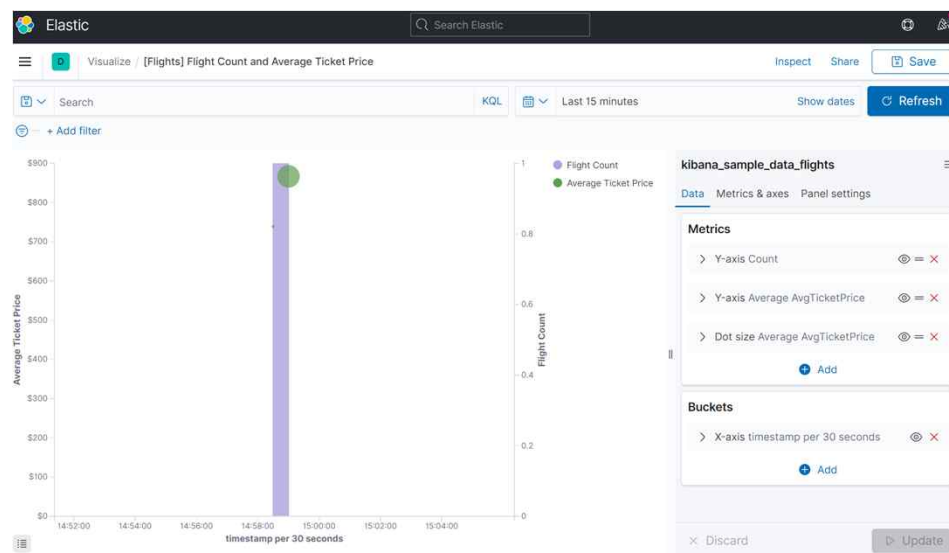


[Flights] Flight Count and Average Ticket Price

시각화

1. 막대 그래프

- Metrics은 Y축을 설정, X축을 어떤 통계 값으로 표현할지 보여준다.
 - Buckets의 Aggregation는 Aggregation는 Histogram으로, Field는 dayOfWeek로 선택 Minimum interval은 1로 지정
- Metrics 의 +Add 버튼
 - Y-axis: Y축이 되는 메트릭 메뉴
 - Aggregation는 Count가 기본 설정 즉. Buckets의 문서 수를 Y축의 문서 수를 보여준다.
 - Dot size: 라인을 동그라미 형태로 만들어 줌 (참고: [Flights] Flight Count and Average Ticket Price)



[Flights] Flight Count and Average Ticket Price

시각화

1. 막대 그래프

- Metrics > Aggregation > Average
- Field > AvgTicketPrice

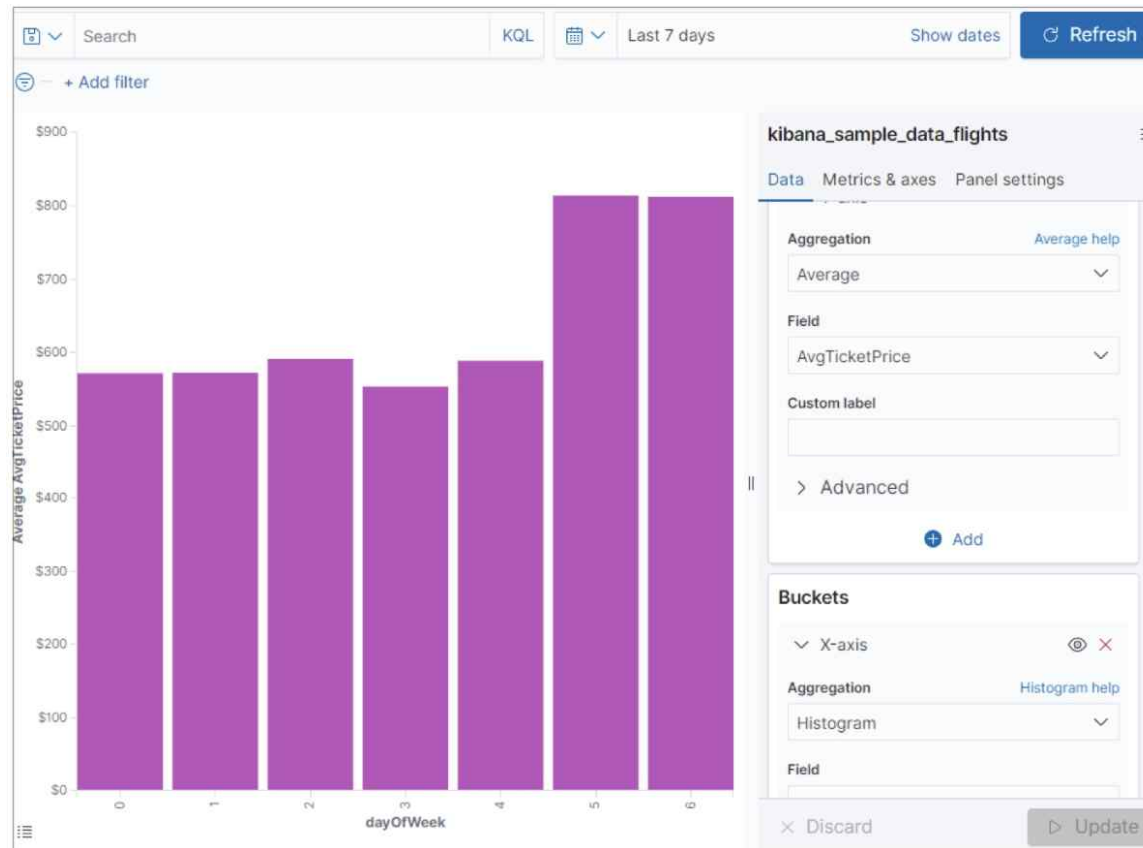


그림 8.22 Y축 기준을 변경한 막대 그래프

시각화

1. 막대 그래프

- Metric에서 파이프라인 집계 사용. 미분 집계 사용

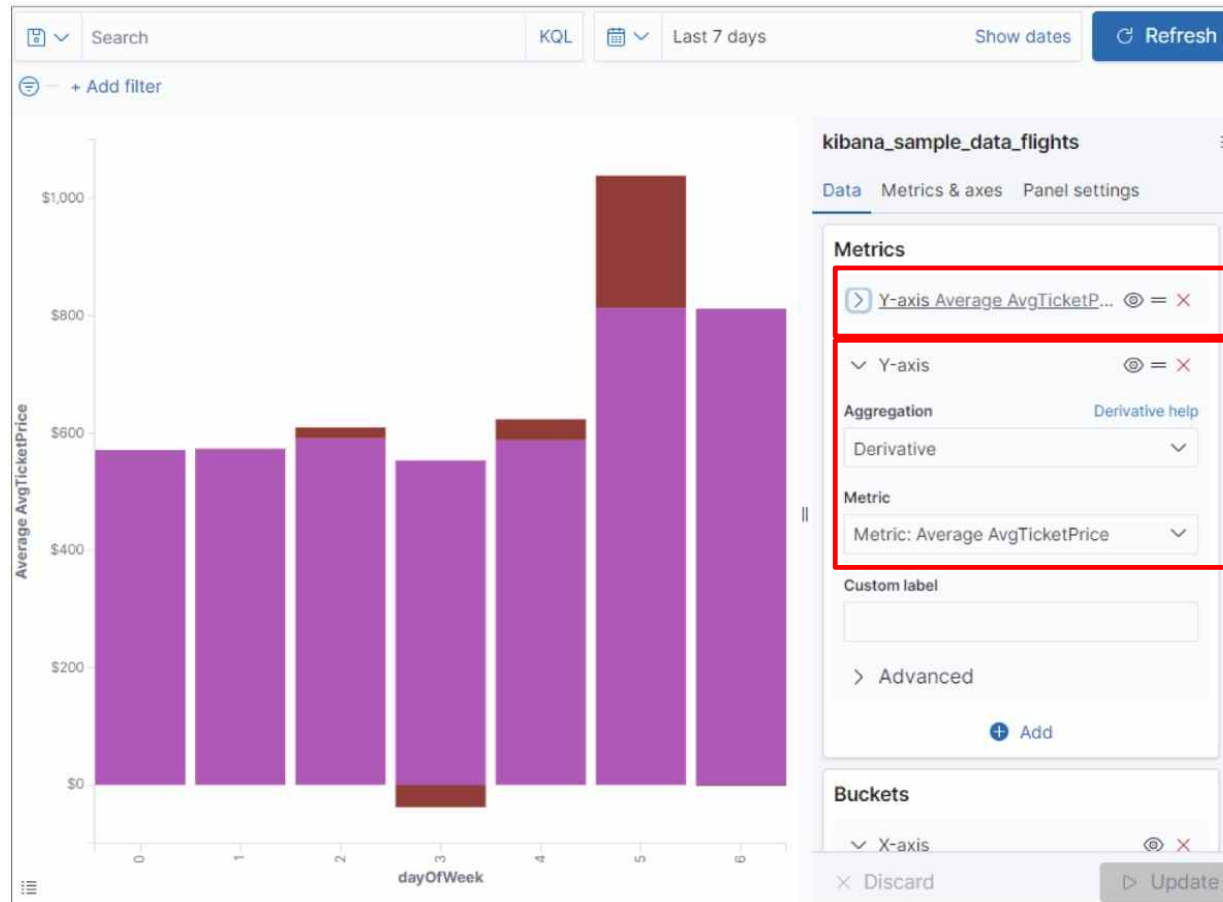


그림 8.23 파이프라인 집계

시각화

2. 히트맵

- X, Y축을 2개의 버킷으로 생성하고 메트릭 값을 색의 진하기로 표현하는 열분포 형태의 시각화 표현 방식
- 3차원 데이터 처리 가능
- 예제에 서는 요일별 비행기 목적지로 가장 많이 가는 도시가 어딘지 확인.

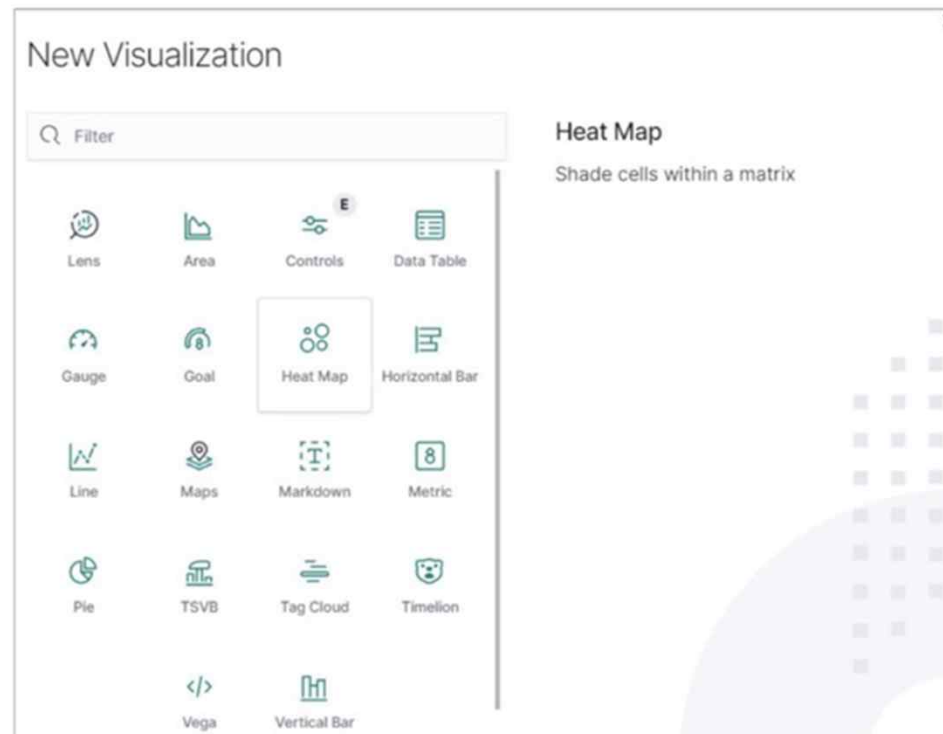


그림 8.24 히트맵 타입 선택

시각화

2. 히트맵

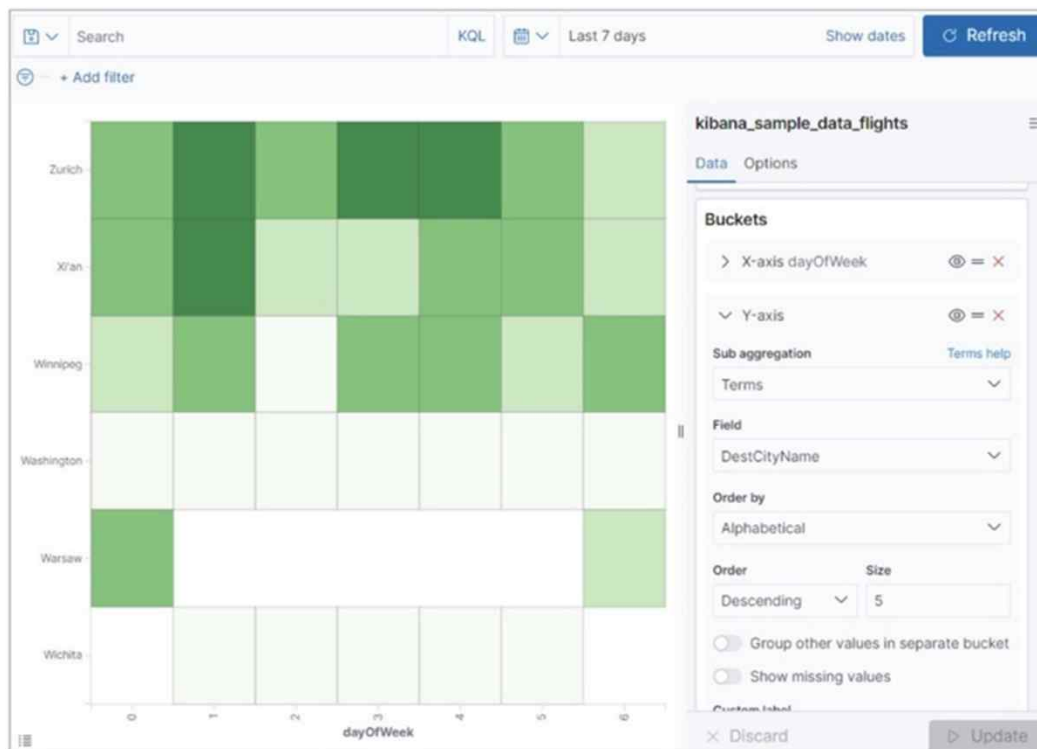


그림 8.25 히트맵 구성: Y축 고정되지 않음

- Metrics은 비행기 도착 수 즉 count이기 때문에 변경할 필요 없음.
- 기간은 지난 7일로 수정
- Buckets에서 +Add 클릭 후 X-axis, Y-axis 추가
- X-axis : Aggregation는 Histogram, Field는 dayOfWeek, Minimum interval은 1로 설정
- Y-axis: Sub aggregation는 Terms, Field는 DestCityName, Order by는 Alphabetical, 내림차순, size는 5로 설정
- 하단 Update 클릭
- Y축의 size가 설정한 5개가 아닌 6개가 보이는 데 이는 Warsaw의 항공편이 특정 요일에만 항공편이 몰려 있기 때문.
- 결과적으로 히트맵의 축이 고정되지 않고 데이터에 따라 기준이 변경됨.

시각화

2. 히트맵

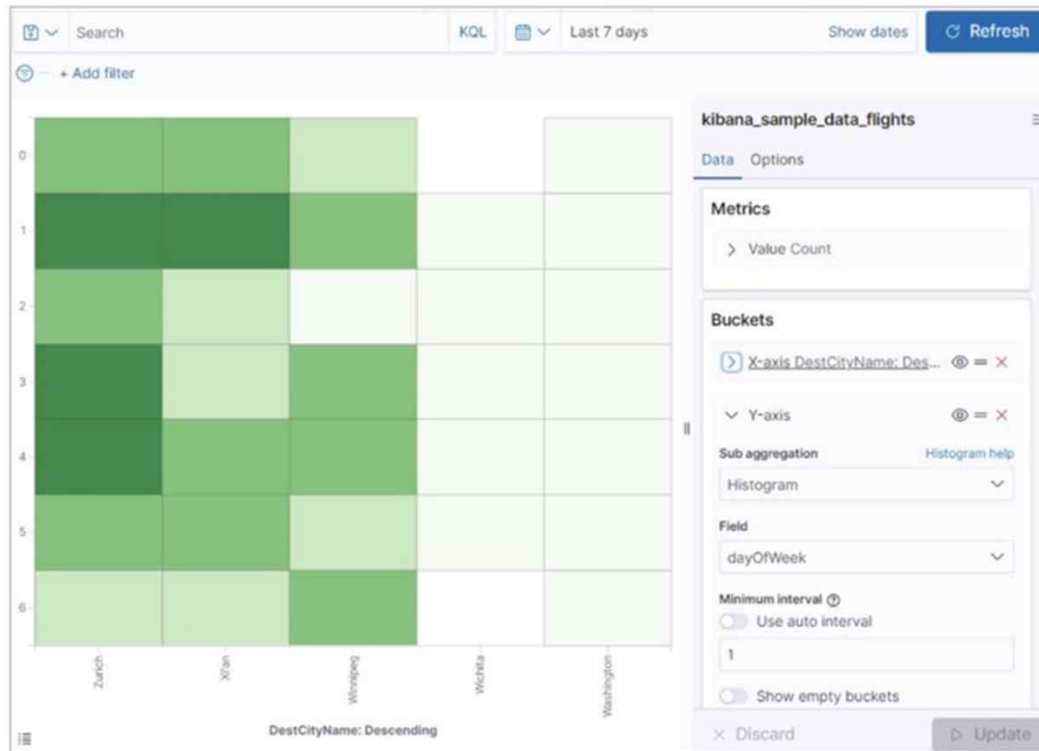


그림 8.26 히트맵 구성: Y축 고정

- x축과 y축을 바꿔보자.
- y축에 모든 데이터가 반드시 값을 갖고 있는 요일을 사용했기 때문에 모양이 들어지지 않음.

시각화

3. TSVB(Time Series Visual Builder)

- 시계열 데이터를 처리하기 위한 메뉴, 로그 모니터링이나 시간 범위 내의 특정 동작을 시각화.
- TSVB는 시계열 데이터, 통계 정보, n번째 상위 값, 게이지 등을 편하게 확인할 수 있음.

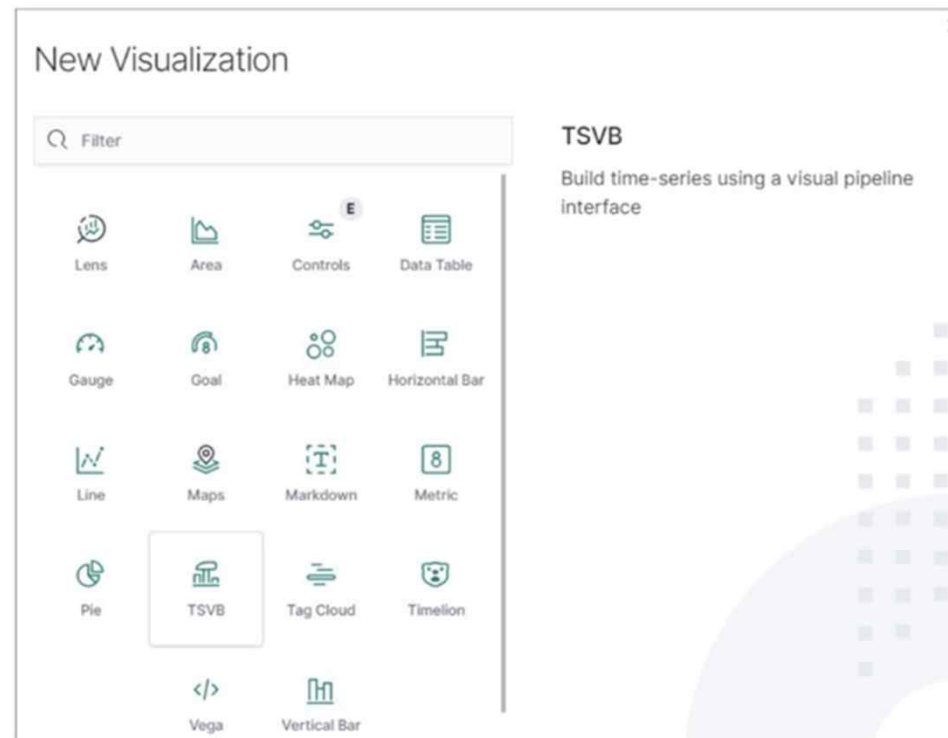


그림 8.27 TSVB 타입 선택

시각화

3. TSVB(Time Series Visual Builder)



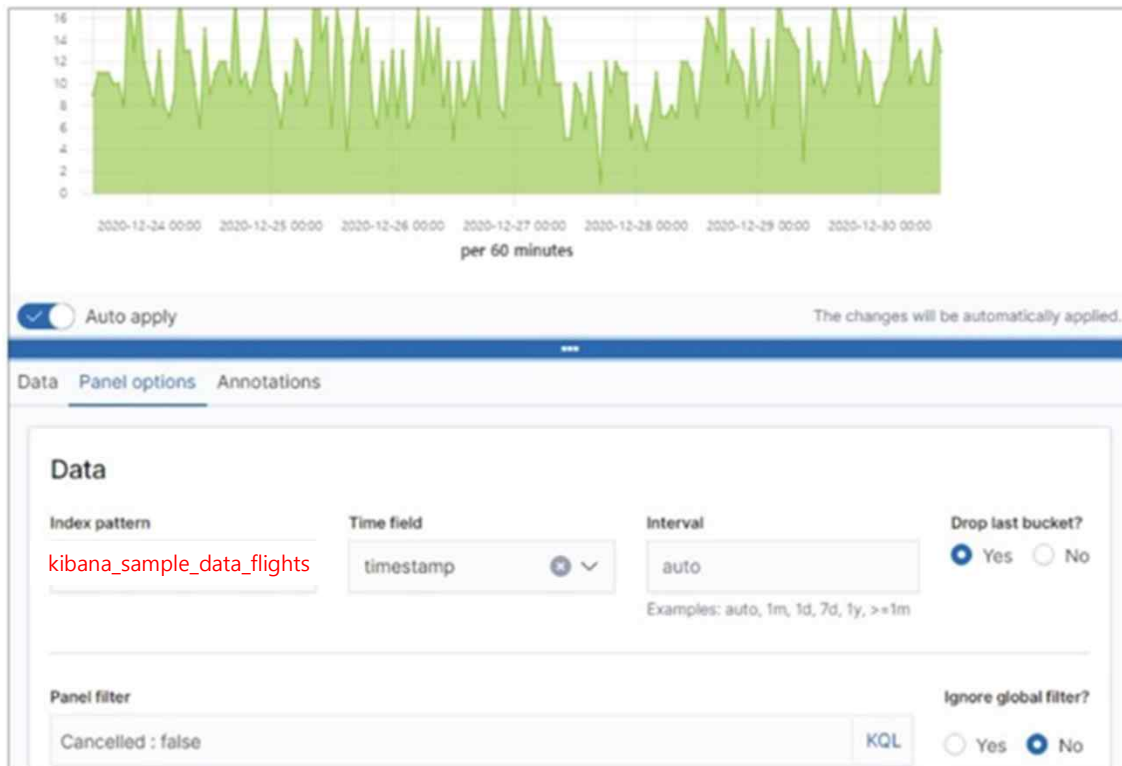
그림 8.28 TSVB 메인 화면

- 시계열 데이터를 처리하기 위한 메뉴, 로그 모니터링이나 시간 범위 내의 특정 동작을 시각화.
- TSVB는 시계열 데이터, 통계 정보, n번째 상위 값, 게이지 등을 편하게 확인할 수 있음.
- 다른 시각화와 다르게 TSVB는 인덱스 패턴을 TSVB 내부에서 진행
- Time Series: 시계열 데이터를 히스토그램 형태
- Metric: 텍스트 형태
- Top N: 수평 바 형태
- Gauge: 게이지 형태
- Markdown: 마크다운 형태
- Table: 테이블 형태
- Auto apply: 데이터 소스가 변할 경우 바로 시각화

시각화

3. TSVB(Time Series Visual Builder)

- Panel options에 인덱스 패턴을 선택할 수 있음.



- Index pattern: kibana_sample_data_flights
- Time field: timestamp
 - Time field에는 날짜/시간 타입만 입력 됨
- Interval: 타임 필드를 구분할 주기
- Panel filter: Cancelled : false

그림 8.29 TSVB 패널 옵션 설정

시각화

3. TSVB(Time Series Visual Builder)

- Data메뉴 수정
 - Data에서 메트릭 집계나 파이프라인 집계를 추가 할 수 있음.

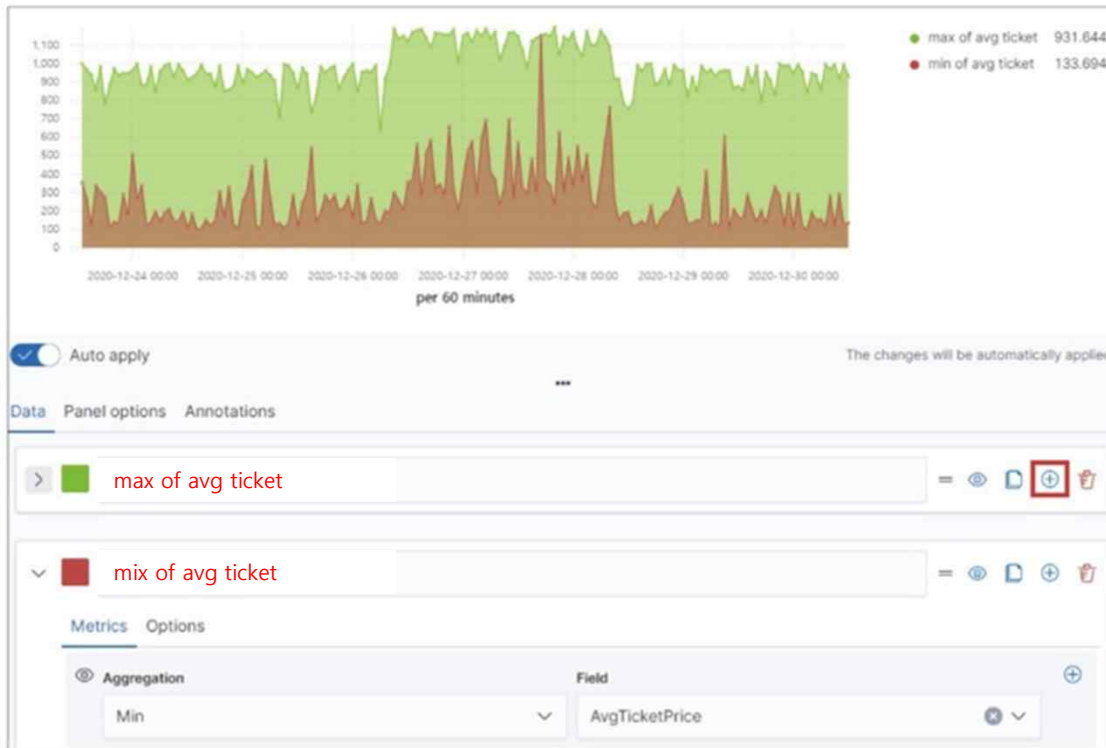


그림 8.30 TSVB Data 옵션 설정

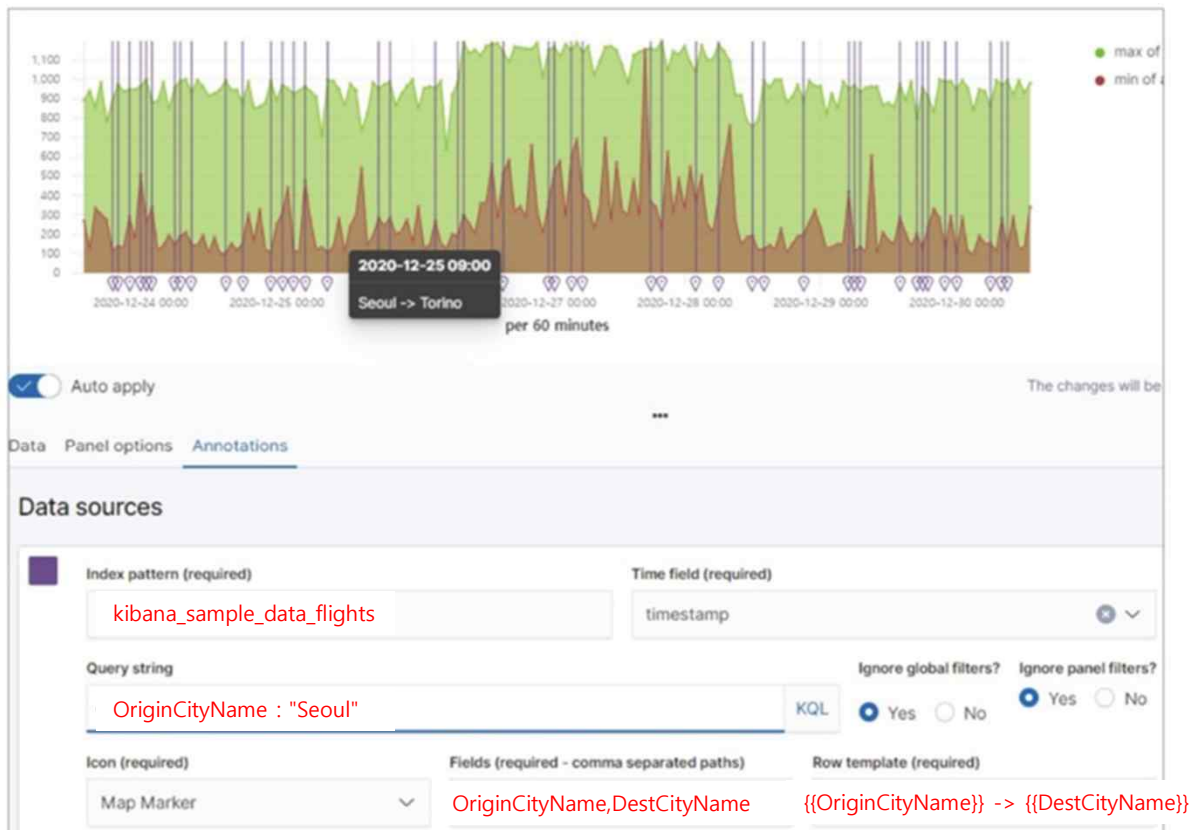
- +버튼으로 메트릭을 추가해 여러 데이터를 함께 볼 수 있다.

This block shows a detailed view of the TSVB configuration interface for two metrics. The top section is for 'max of avg ticket'. It has a 'Metrics' tab and an 'Options' tab. Under 'Metrics', there's a table with columns 'Aggregation' and 'Field'. The 'Aggregation' is set to 'Max' and the 'Field' is 'AvgTicketPrice'. Below this, there's a 'Group by' dropdown set to 'Everything'. The bottom section is for 'min of avg ticket'. It also has a 'Metrics' tab and an 'Options' tab. Under 'Metrics', there's a table with columns 'Aggregation' and 'Field'. The 'Aggregation' is set to 'Min' and the 'Field' is 'AvgTicketPrice'. Below this, there's a 'Group by' dropdown set to 'Everything'.

시각화

3. TSVB(Time Series Visual Builder)

- 시계열 데이터를 분석하는 데 특화된 어노테이션 기능



- Add Data sources 클릭
- Index pattern: kibana_sample_data_flights
- Time field: timestamp
- Query string: OriginCityName : "Seoul"
- Icon: Map Marker
- Fields, Row template은 어노테이션에 마우스를 올렸을 때 보여주는 문자들
 - Fields: OriginCityName, DestCityName
 - Row template: {{OriginCityName}} -> {{DestCityName}}

그림 8.31 TSVB 주석 설정

4 대시보드

- 시각화 타입들을 한 페이지에 모아 볼 수 있는 기능.
- 한 화면에서 다양한 관점으로 데이터를 보면서 분석 가능.
- 포괄적으로 데이터를 볼 수 있기 때문에 중요한 의사결정을 할 수 있게 도와준다.
- 자동 새로고침 기능을 이용해 대시보드를 통해 시스템을 실시간으로 모니터링 할 수 있음.

대시보드

1. 대시보드 만들기

- 시각화에서 만들었던 다양한 타입들을 잘 조합하고 배치해 제작.
- Dashboard 메뉴를 클릭하고 +Create dashboard 클릭
- 상단 Add 클릭 후 [Flights] Flight Count and Average Ticket Price, [Flights] Airline Carrier, [Flights] Delays & Cancellations, [Flights] Origin and Destination Flight Time 선택

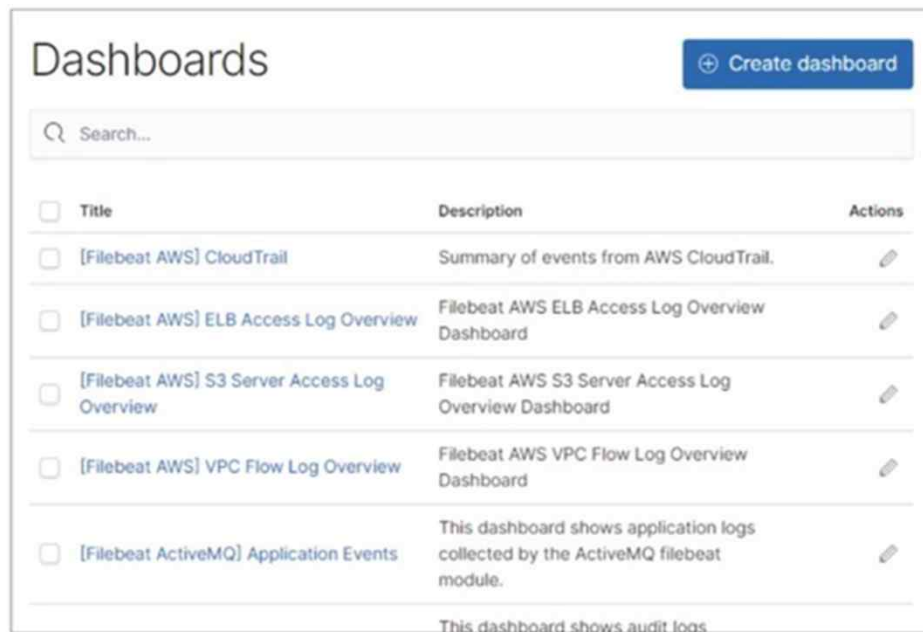


그림 8.32 새로운 대시보드 생성

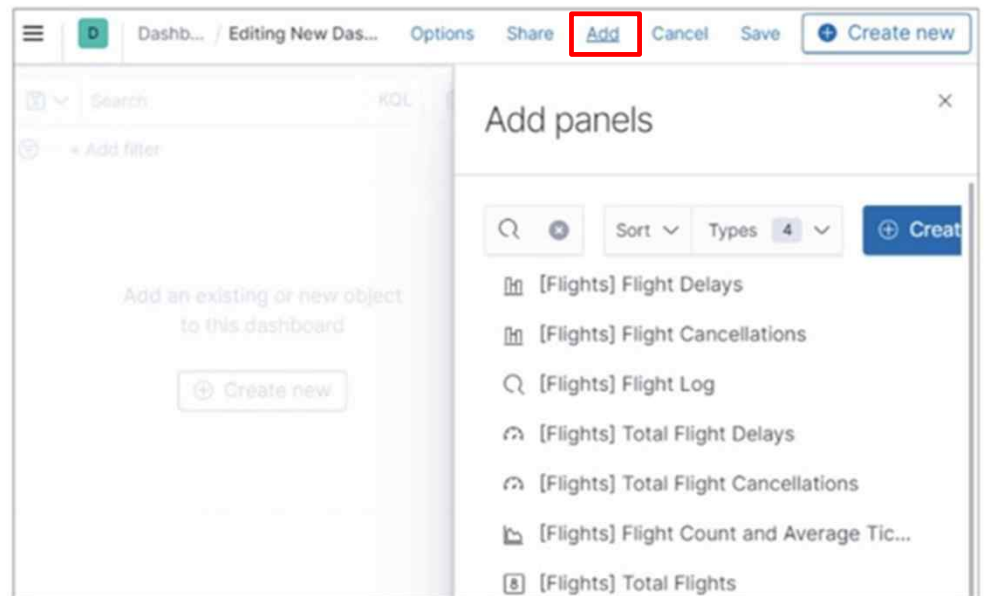


그림 8.33 대시보드에 사용할 패널들 선택

대시보드

1. 대시보드 만들기

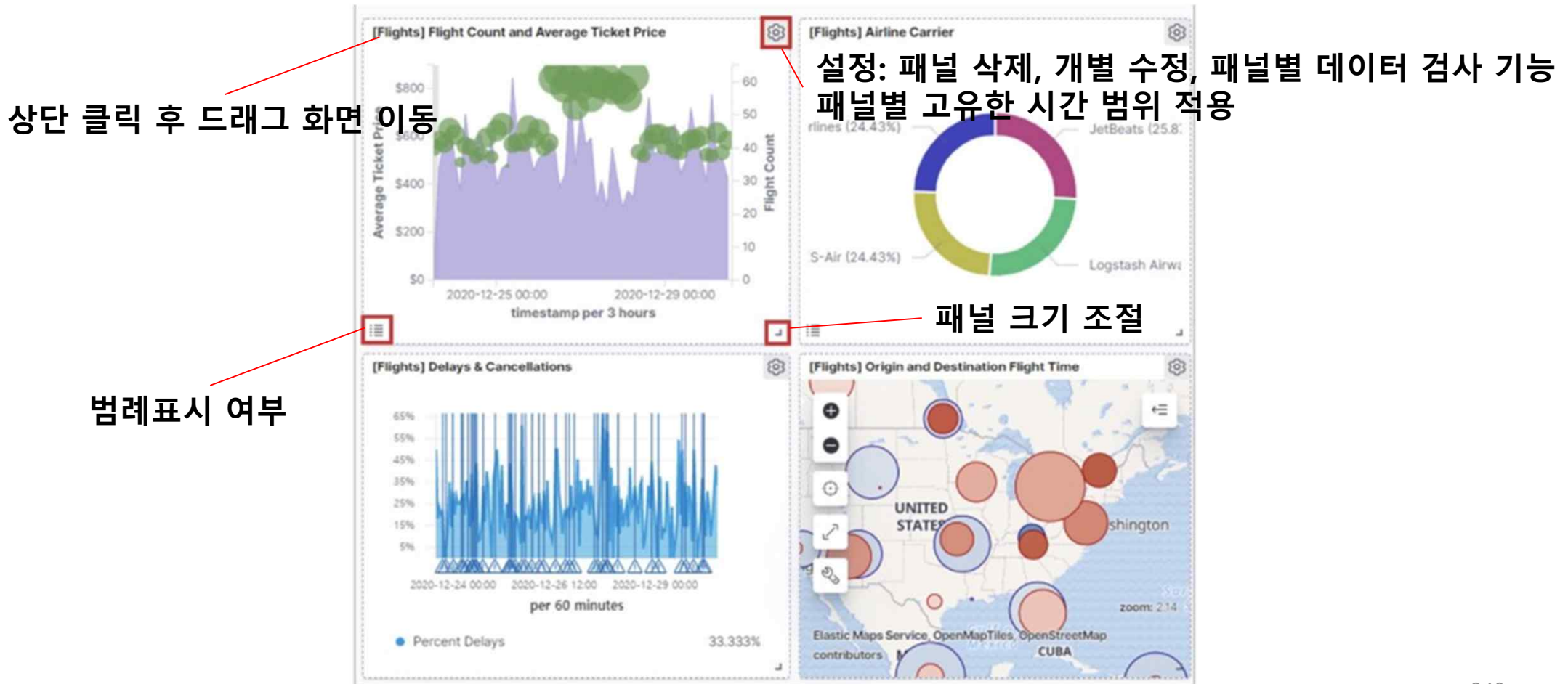


그림 8.34 대시보드 화면

대시보드

1. 대시보드 만들기

- 대시보드의 전체적인 시간 설정은 타임피커에서 하지만, 패널마다 개별적인 시간 설정은 패널 설정 메뉴에서 가능
- 필터 적용 타임피커 변경

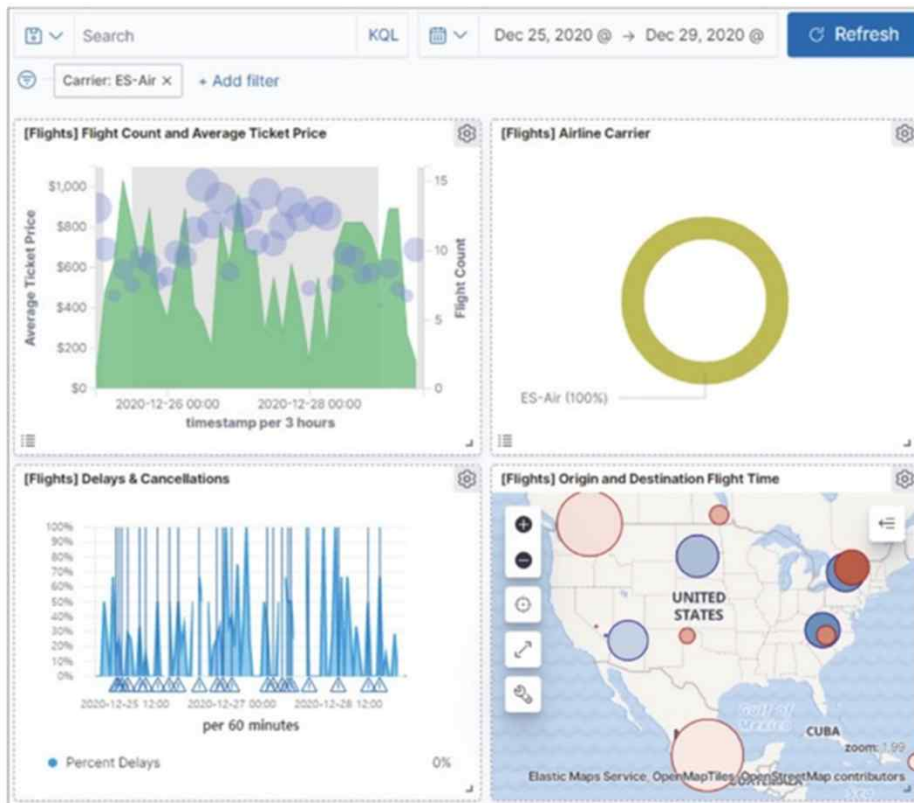


그림 8.35 대시보드에서 필터 적용

- [Flights] Airline Carrier에서 ES-Air를 클릭 하면 ES-Air 필터가 자동으로 생성
- [Flights] Flight Count and Average Ticket Price에서 차트 일정영역을 드래드 앤 드롭으로 선택 할 수 있다.
- 아이프레임 형태로 다른 웹 페이지에 임베딩하거나 링크를 통해 공유할 수 있다.

5 캔버스

- 캔버스는 인포그래픽 형태로 프리젠테이션할 수 있게 해주는 툴
- 캔버스는 간단한 UI 조작으로 사용 가능.
- 모든 것이 객체로 구성.

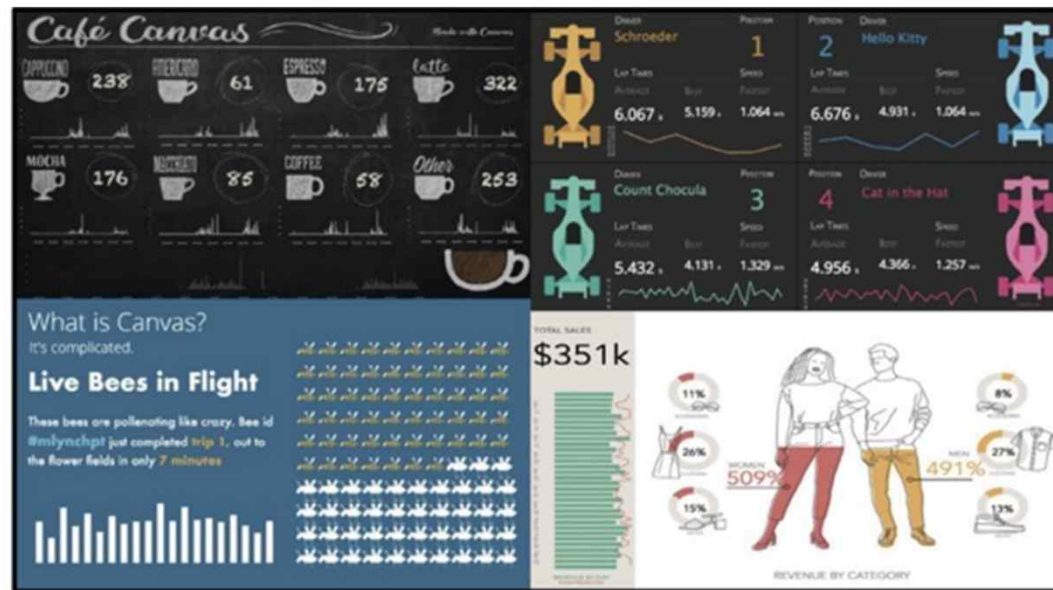


그림 8.36 캔버스를 이용한 인포그래픽²

캔버스

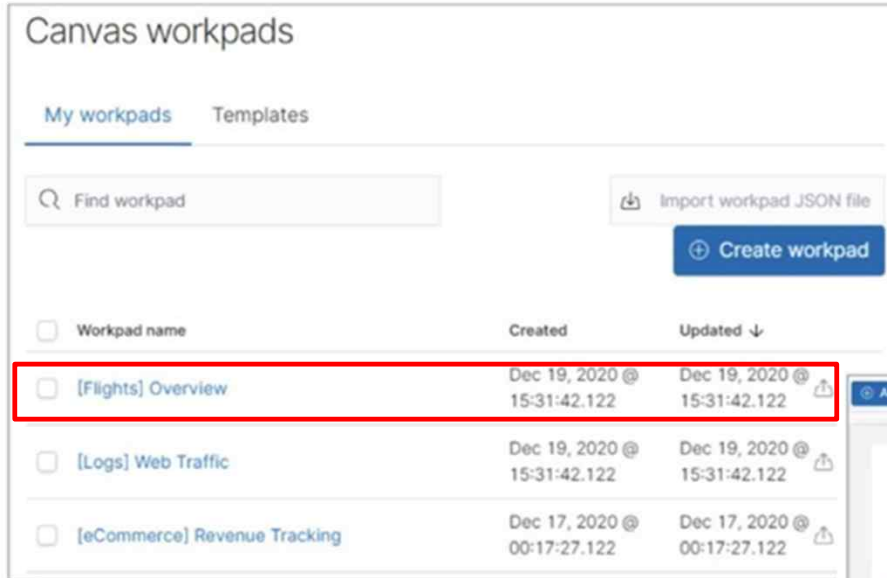


그림 8.37 캔버스 첫 화면

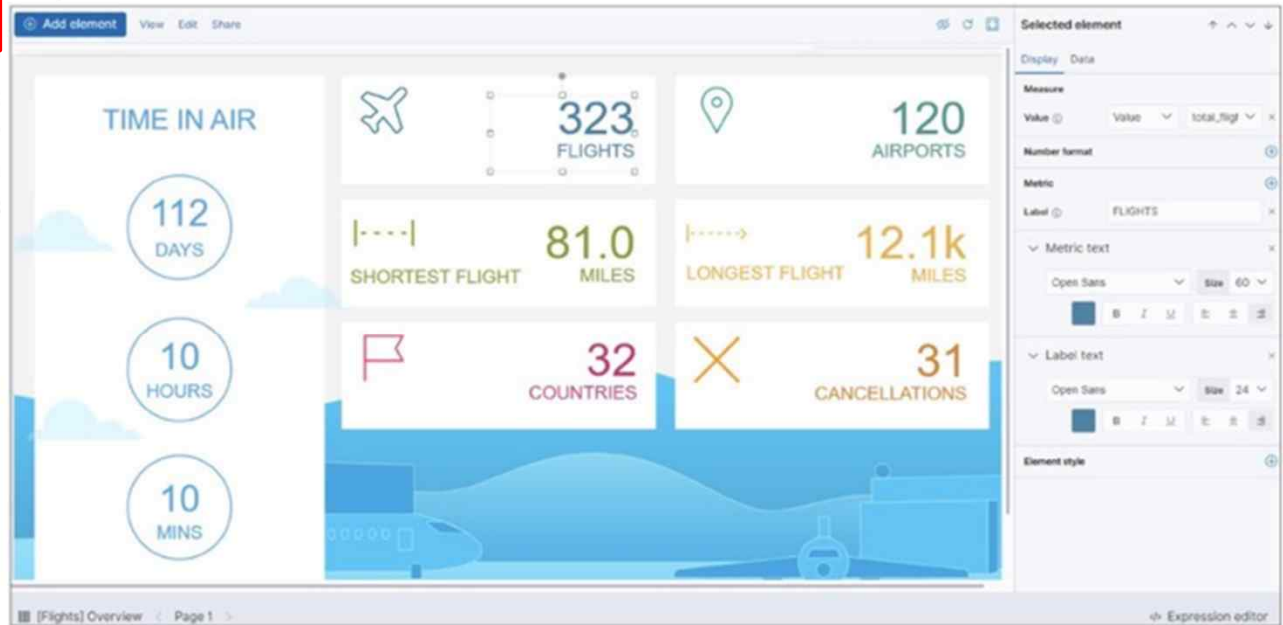


그림 8.38 캔버스 편집 화면

캔버스

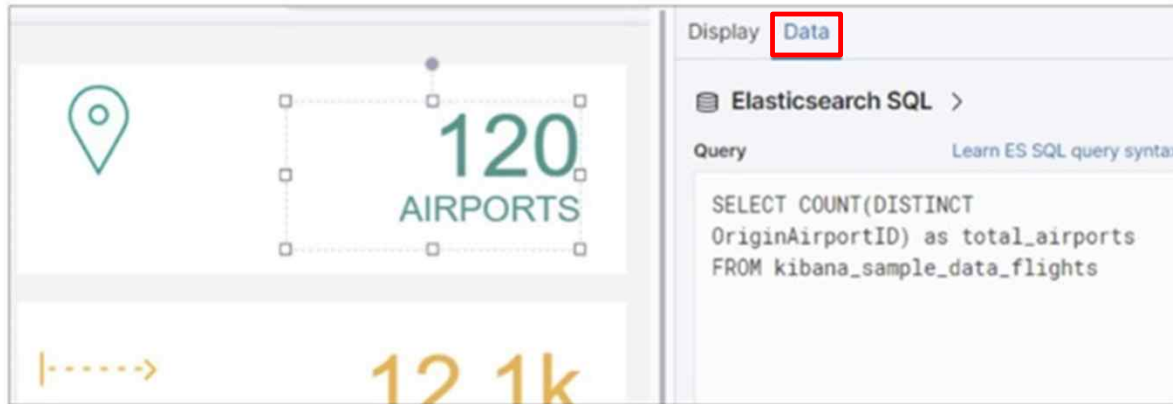


그림 8.39 엘라스틱서치 SQL을 이용한 동적 데이터 처리

표 8.5 데이터 소스 종류

데이터 소스	설명
Timelion	Timelion 문법을 이용해 시계열 데이터를 시각화한다.
Demo Data	프로젝트에 상관없이 데모로 제공하는 데이터를 사용해볼 수 있다.
Elasticsearch SQL	엘라스틱서치 쿼리 구문을 사용한다.
Elasticsearch raw documents	엘라스틱서치 데이터를 집계 과정 없이 직접 가져온다.

1. 엘라스틱서치 SQL

- 엘라스틱서치는 관계형 데이터베이스가 아니기 때문에 SQL 문법이 쉽지 않음.
- 기본적인 저장 구조나 동작 방식이 관계형 데이터베이스와 다르기 때문.
- SQL 쿼리로 요청을 하면 내부적으로 DSL 형태로 변형하여 동작.
- 6.3 버전 부터 지원

리스트 8.5 엘라스틱서치 SQL 문의 SELECT 문법

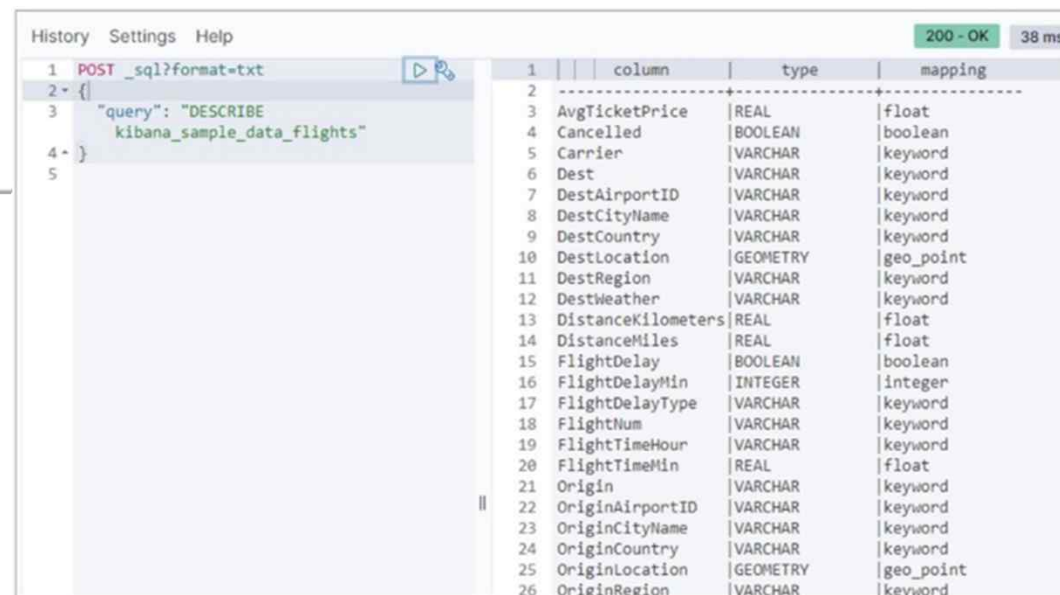
```
SELECT select_expr [, ...]  
[ FROM table_name ]  
[ WHERE condition ]  
[ GROUP BY grouping_element [, ...] ]  
[ HAVING condition]  
[ ORDER BY expression [ ASC | DESC ] [, ...] ]  
[ LIMIT [ count ] ]  
[ PIVOT ( aggregation_expr FOR column IN ( value [ [ AS ] alias ] [, ...] ) ) ]
```

1. 엘라스틱서치 SQL

리스트 8.6 간단한 엘라스틱서치 SQL 요청

```
POST _sql?format=txt
{
  "query": "DESCRIBE kibana_sample_data_flights"
}
```

- 엘라스틱서치는 SQL문 지원을 위해 _sql이라는 API를 제공
- Format=txt 옵션을 이용하면 SQL 쿼리 결과를 JSON이 아닌 텍스트 형태로 볼 수 있음.



	column	type	mapping
1	AvgTicketPrice	REAL	float
2	Cancelled	BOOLEAN	boolean
3	Carrier	VARCHAR	keyword
4	Dest	VARCHAR	keyword
5	DestAirportID	VARCHAR	keyword
6	DestCityName	VARCHAR	keyword
7	DestCountry	VARCHAR	keyword
8	DestLocation	GEOMETRY	geo_point
9	DestRegion	VARCHAR	keyword
10	DestWeather	VARCHAR	keyword
11	DistanceKilometers	REAL	float
12	DistanceMiles	REAL	float
13	FlightDelay	BOOLEAN	boolean
14	FlightDelayMin	INTEGER	integer
15	FlightDelayType	VARCHAR	keyword
16	FlightNum	VARCHAR	keyword
17	FlightTimeHour	VARCHAR	keyword
18	FlightTimeMin	REAL	float
19	Origin	VARCHAR	keyword
20	OriginAirportID	VARCHAR	keyword
21	OriginCityName	VARCHAR	keyword
22	OriginCountry	VARCHAR	keyword
23	OriginLocation	GEOMETRY	geo_point
24	OriginRegion	VARCHAR	keyword

그림 8.40 리스트 8.6 실행 결과

1. 엘라스틱서치 SQL

리스트 8.7 복잡한 엘라스틱서치 SQL

```
POST _sql?format=txt
```

```
{
```

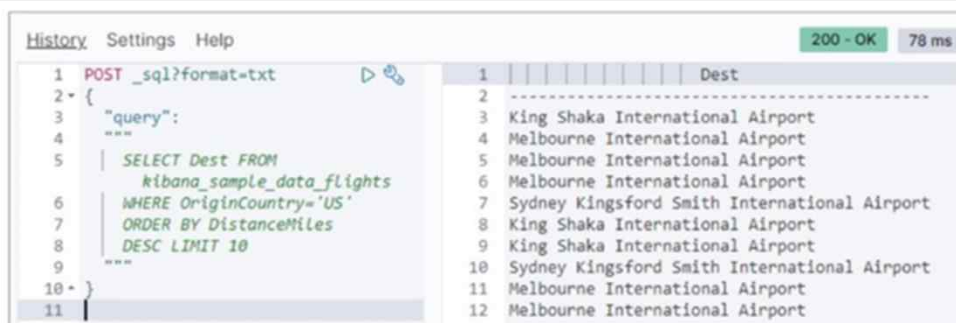
```
  "query":
```

```
  ""
```

```
    SELECT Dest FROM kibana_sample_data_flights
    WHERE OriginCountry='US'
    ORDER BY DistanceMiles
    DESC LIMIT 10
```

```
  ""
```

```
}
```



	Dest
1	King Shaka International Airport
2	Melbourne International Airport
3	Melbourne International Airport
4	Melbourne International Airport
5	Sydney Kingsford Smith International Airport
6	King Shaka International Airport
7	King Shaka International Airport
8	Sydney Kingsford Smith International Airport
9	Melbourne International Airport
10	Melbourne International Airport
11	Melbourne International Airport
12	Melbourne International Airport

그림 8.41 리스트 8.7 실행 결과

리스트 8.8 엘라스틱서치 SQL 문을 엘라스틱서치 DSL로 변환

```
GET kibana_sample_data_flights/_search
{
  "size" : 10,
  "query" : {
    "term" : {
      "OriginCountry" : {
        "value" : "US",
        "boost" : 1.0
      }
    }
  },
  "_source" : false,
  "stored_fields" : "_none_",
  "docvalue_fields" : [
    {
      "field" : "Dest"
    }
  ],
  "sort" : [
    {
      "DistanceMiles" : {
        "order" : "desc",
        "missing" : "_first",
        "unmapped_type" : "float"
      }
    }
  ]
}
```

6 맵스

- 위치 정보가 포함된 데이터를 지도에 올려 시각화 가능

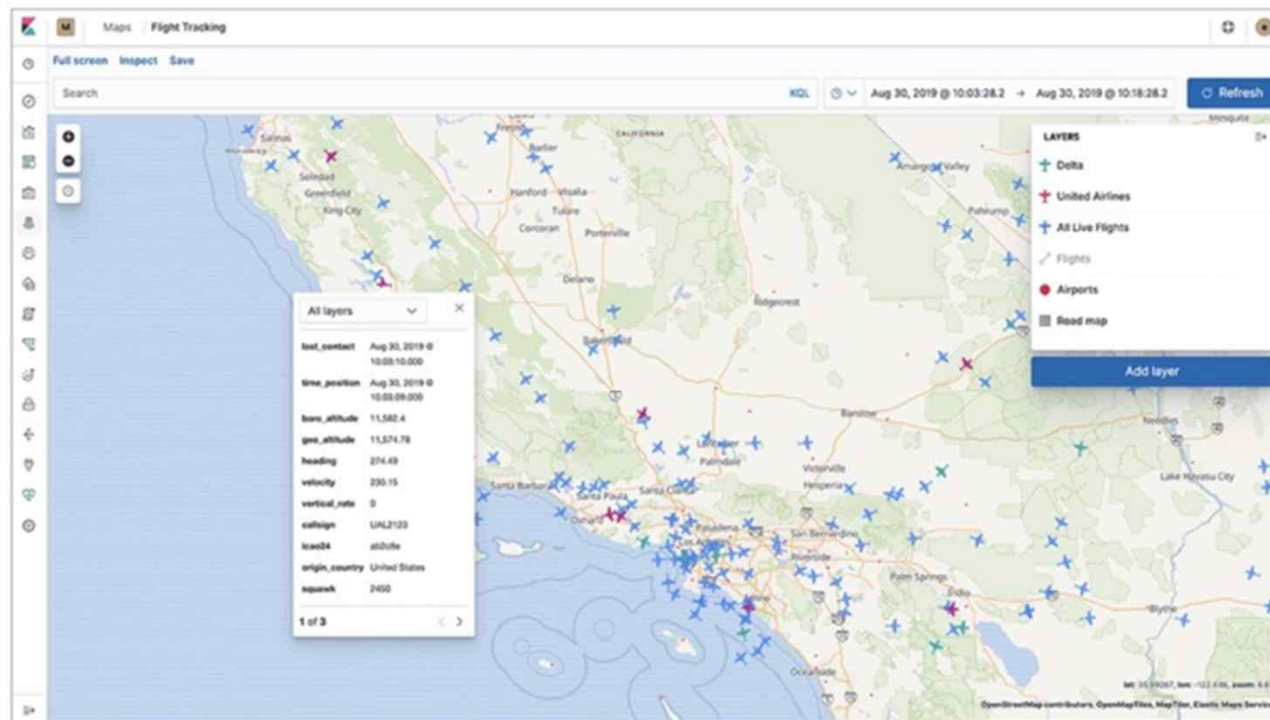


그림 8.42 키바나 맵스 화면⁶

맵스

- 멀티 레이어를 지원하고 선택한 항목에 대한 세부 정보도 확인.
- 기본적으로 위치/지역 데이터가 있는 도큐먼트만 시각화가 가능.

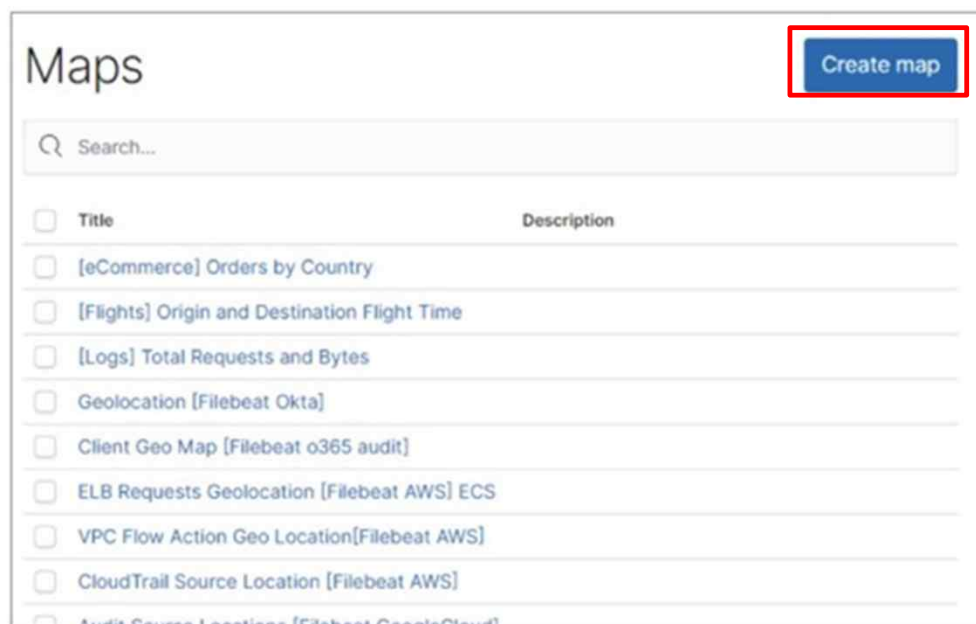


그림 8.43 맵스 첫 화면

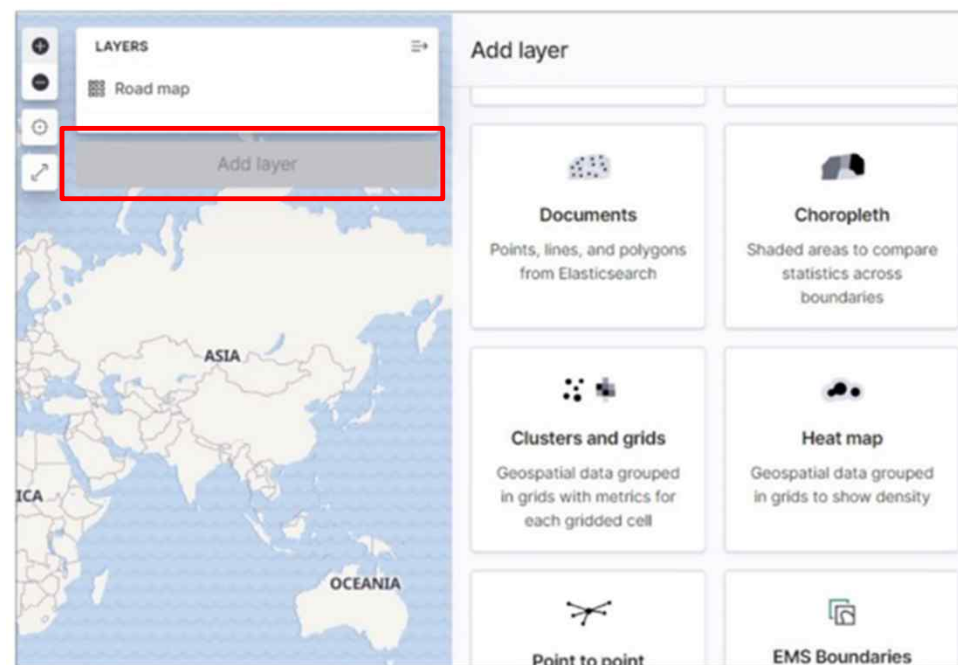


그림 8.44 지도 레이어 종류